Ed: I've designed each separately, and they're pretty simple operations. So I thought it might be a good idea to combine all of the display operations into just one that's called *displayCamera()*—it'll show the ID, the view, and the zoom. Whaddaya think?

Jamie (grimacing): Not sure that's such a good idea.

Ed (frowning): Why? All of these little ops can cause headaches.

Jamie: The problem with combining them is we lose cohesion, you know, the *displayCamera()* op won't be single-minded.

Ed (mildly exasperated): So what? The whole thing will be less than 100 source lines, max. It'll be easier to implement, I think.

Jamie: And what if marketing decides to change the way that we represent the view field?

Ed: I just jump into the *displayCamera()* op and make the mod.

Jamie: What about side effects?

Ed: Whaddaya mean?

Jamie: Well, say you make the change but inadvertently create a problem with the ID display.

Ed: I wouldn't be that sloppy.

Jamie: Maybe not, but what if some support person 2 years from now has to make the mod? He might not understand the op as well as you do, and, who knows, he might be sloppy.

Ed: So you're against it?

Jamie: You're the designer . . . it's your decision . . . just be sure you understand the consequences of low cohesion.

Ed (thinking a moment): Maybe we'll go with separate display ops.

Jamie: Good decision.

11.2.4 Coupling

In earlier discussions of analysis and design, we noted that communication and collaboration are essential elements of any object-oriented system. There is, however, a darker side to this important (and necessary) characteristic. As the amount of communication and collaboration increases (i.e., as the degree of "connectedness" between classes increases), the complexity of the system also increases. And as complexity increases, the difficulty of implementing, testing, and maintaining software grows.

Coupling is a qualitative measure of the degree to which classes are connected to one another. As classes (and components) become more interdependent, coupling increases. An important objective in component-level design is to keep coupling as low as possible.

Class coupling can manifest itself in a variety of ways. Lethbridge and Laganiére [Let04] define a spectrum of coupling categories. For example, *content coupling* occurs when one component "surreptitiously modifies data that is internal to another component" [Let04]. This violates information hiding—a basic design concept. *Control coupling* occurs when operation *A()* invokes operation *B()* and passes a control flag to *B*. The control flag then "directs" logical flow within *B*. The problem with this form of coupling is that an unrelated change in *B* can result in the necessity to change the meaning of the control flag that *A* passes. If this is overlooked, an error will result. *External coupling* occurs when a component communicates or collaborates with infrastructure components (e.g., operating system functions, database capability, telecommunication functions). Although this type of coupling is necessary, it should be limited to a small number of components or classes within a system.

SAFEHOME



Coupling in Action

The scene: Shakira's cubicle.

The players: Vinod and Shakira, members of the *SafeHome* software team who are working on the security function.

The conversation:

Shakira: I had what I thought was a great idea . . . then I thought about it a little, and it seemed like a not-so-great idea. I finally rejected it, but I just thought I'd run it by you.

Vinod: Sure. What's the idea?

Shakira: Well, each of the sensors recognizes an alarm condition of some kind, right?

Vinod (smiling): That's why we call them sensors, Shakira.

Shakira (exasperated): Sarcasm, Vinod, you've got to work on your interpersonal skills.

Vinod: You were saying?

Shakira: Okay, anyway, I figured . . . why not create an operation within each sensor object called *makeCall()* that would collaborate directly with the **OutgoingCall** component, well, with an interface to the **OutgoingCall** component.

Vinod (pensive): You mean rather than having that collaboration occur out of a component like **ControlPanel** or something?

Shakira: Yeah . . . but then, I said to myself, that means that every sensor object will be connected to the **OutgoingCall** component, and that means that it's indirectly coupled to the outside world and . . . well, I just thought it made things complicated.

Vinod: I agree. In this case, it's a better idea to let the sensor interface pass info to the **ControlPanel** and let it initiate the outgoing call. Besides, different sensors might result in different phone numbers. You don't want the sensor to store that information because if it changes . . .

Shakira: It just didn't feel right.

Vinod: Design heuristics for coupling tell us it's not right.

Shakira: Whatever . . .

Software must communicate internally and externally. Therefore, coupling is a fact of life. However, a designer should work to reduce coupling whenever possible and understand the ramifications of high coupling when it cannot be avoided.

11.3 CONDUCTING COMPONENT-LEVEL DESIGN

Earlier in this chapter we noted that component-level design is elaborative in nature. You must transform information from requirements and architectural models into a design representation that provides sufficient detail to guide the construction (coding and testing) activity. The following steps represent a typical task set for component-level design, when it is applied for an object-oriented system.

Step 1. Identify all design classes that correspond to the problem domain. Using the requirements and architectural model, each analysis class and architectural component is elaborated as described in Section 11.1.1.

Step 2. Identify all design classes that correspond to the infrastructure domain. These classes are not described in the requirements model and are often missing from the architecture model, but they must be described at this point. As we

have noted earlier, classes and components in this category include GUI components (often available as reusable components), operating system components, and object and data management components.

Step 3. Elaborate all design classes that are not acquired as reusable components. Elaboration requires that all interfaces, attributes, and operations necessary to implement the class be described in detail. Design heuristics (e.g., component cohesion and coupling) must be considered as this task is conducted.

Step 3a. Specify message details when classes or components collaborate. The requirements model makes use of a collaboration diagram to show how analysis classes collaborate with one another. As component-level design proceeds, it is sometimes useful to show the details of these collaborations by specifying the structure of messages that are passed between objects within a system. Although this design activity is optional, it can be used as a precursor to the specification of interfaces that show how components within the system communicate and collaborate.

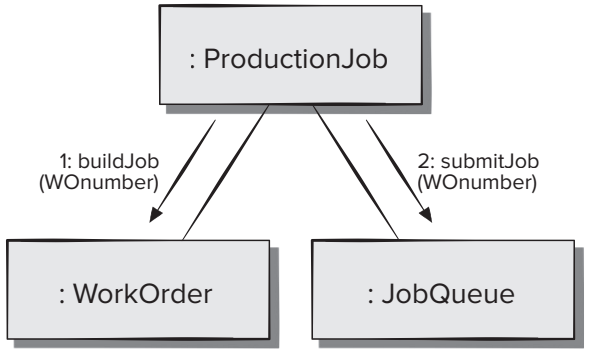
Figure 11.6 illustrates a simple collaboration diagram for the printing system discussed earlier. Three objects, **ProductionJob, WorkOrder,** and **JobQueue,** collaborate to prepare a print job for submission to the production stream. Messages are passed between objects as illustrated by the arrows in the figure. During requirements modeling the messages are specified as shown in the figure. However, as design proceeds, each message is elaborated by expanding its syntax in the following manner [Ben10a]:

```
[guard condition] sequence expression (return value) :=
message name (argument list)
```

where a `[guard condition]` is written in Object Constraint Language (OCL)[5] and specifies any set of conditions that must be met before the message can be sent; `sequence expression` is an integer value (or other ordering indicator, e.g., 3.1.2) that indicates the sequential order in which a message is sent; `(return value)` is the name of the information that is returned by the operation invoked by the message; `message name` identifies the operation that is to be invoked; and `(argument list)` is the list of attributes that are passed to the operation.

FIGURE 11.6

Collaboration diagram with messaging



5 OCL is discussed briefly in Appendix 1.

Step 3b. Identify appropriate interfaces for each component. Within the context of component-level design, a UML interface is "a group of externally visible (i.e., public) operations. The interface contains no internal structure, it has no attributes, no associations . . ." [Ben10a]. Stated more formally, an interface is the equivalent of an abstract class that provides a controlled connection between design classes. The elaboration of interfaces is illustrated in Figure 11.1. The operations defined for the design class are categorized into one or more abstract classes. Every operation within the abstract class (the interface) should be cohesive; that is, it should exhibit processing that focuses on one limited function or subfunction.

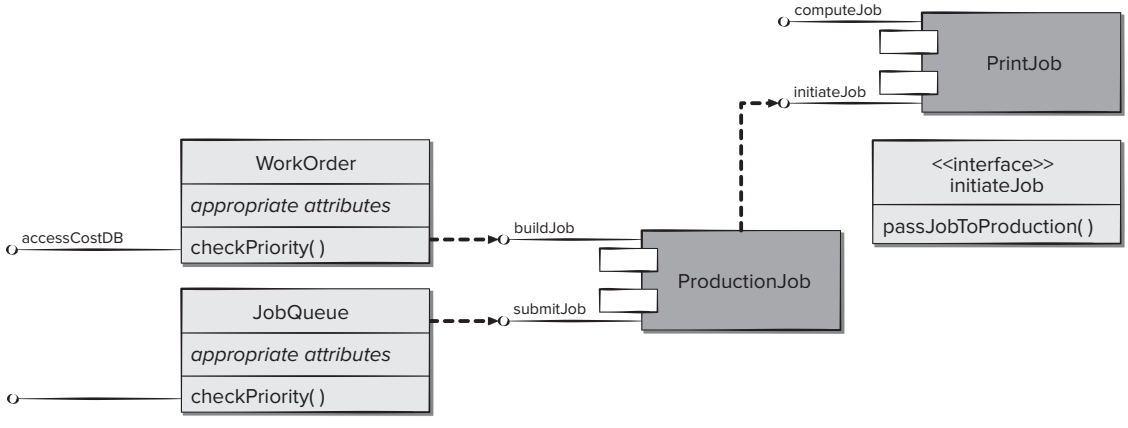
Referring to Figure 11.1, it can be argued that the interface *initiateJob* does not exhibit sufficient cohesion. It performs three different subfunctions—building a work order, checking job priority, and passing a job to production. The interface design should be refactored. One approach might be to reexamine the design classes and define a new class **WorkOrder** that would take care of all activities associated with the assembly of a work order. The operation *buildWorkOrder()* becomes a part of that class. Similarly, we might define a class **JobQueue** that would incorporate the operation *checkPriority()*. A class **ProductionJob** would encompass all information associated with a production job to be passed to the production facility. The interface *initiateJob* would then take the form shown in Figure 11.7. The interface *initiateJob* is now cohesive, focusing on one function. The interfaces associated with **ProductionJob, WorkOrder,** and **JobQueue** are similarly single-minded.

Step 3c. Elaborate attributes and define data types and data structures required to implement them. In general, data structures and types used to define attributes are defined within the context of the programming language that is to be used for implementation. UML defines an attribute's data type using the following syntax:

```
name : type-expression = initial-value {property-string}
```

where `name` is the attribute name, `type expression` is the data type, `initial value` is the value that the attribute takes when an object is created, and `property-string` defines a property or characteristic of the attribute.

FIGURE 11.7 Refactoring interfaces and class definitions for PrintJob



During the first component-level design iteration, attributes are normally described by name. Referring once again to Figure 11.1, the attribute list for **PrintJob** lists only the names of the attributes. However, as design elaboration proceeds, each attribute is defined using the UML attribute format noted. For example, `paperType-weight` is defined in the following manner:

```
paperType-weight: string = "A" {contains 1 of 4 values – A, B, C, or D}
```

which defines `paperType-weight` as a string variable initialized to the value A that can take on one of four values from the set {A, B, C, D}.

If an attribute appears repeatedly across several design classes, and it has a relatively complex structure, it is best to create a separate class to accommodate the attribute.

Step 3d. Describe processing flow within each operation in detail. This may be accomplished using a programming language-based pseudocode or with a UML activity diagram. Each software component is elaborated through several iterations that apply the stepwise refinement concept (Chapter 9).

The first iteration defines each operation as part of the design class. In every case, the operation should be characterized in a way that ensures high cohesion; that is, the operation should perform a single targeted function or subfunction. The next iteration does little more than expand the operation name. For example, the operation *computePaperCost()* noted in Figure 11.1 can be expanded in the following manner:

```
computePaperCost (weight, size, color): numeric
```

This indicates that *computePaperCost()* requires the attributes `weight`, `size`, and `color` as input and returns a value that is numeric (actually a dollar value) as output.

If the algorithm required to implement *computePaperCost()* is simple and widely understood, no further design elaboration may be necessary. The software engineer who does the coding will provide the detail necessary to implement the operation. However, if the algorithm is more complex or arcane, further design elaboration is required at this stage. Figure 11.8 depicts a UML activity diagram for *computePaperCost()*. When activity diagrams are used for component-level design specification, they are generally represented at a level of abstraction that is somewhat higher than source code.

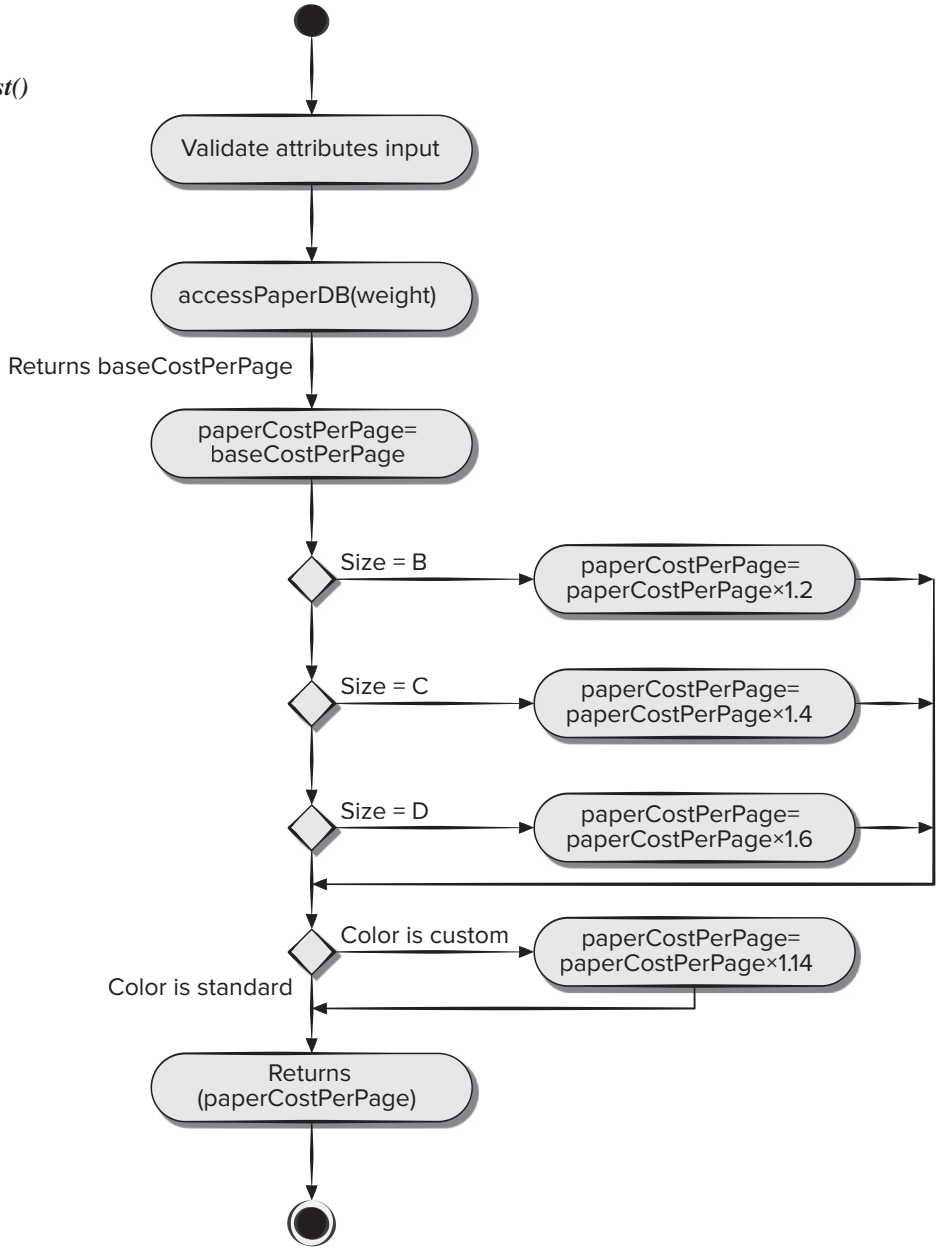
Step 4. Describe persistent data sources (databases and files) and identify the classes required to manage them. Databases and files normally transcend the design description of an individual component. In most cases, these persistent data stores are initially specified as part of architectural design. However, as design elaboration proceeds, it is often useful to provide additional detail about the structure and organization of these persistent data sources.

Step 5. Develop and elaborate behavioral representations for a class or component. UML state diagrams were used as part of the requirements model to represent the externally observable behavior of the system and the more localized behavior of individual analysis classes. During component-level design, it is sometimes necessary to model the behavior of a design class.

The dynamic behavior of an object (an instantiation of a design class as the program executes) is affected by events that are external to it and the current state (mode of behavior) of the object. To understand the dynamic behavior of an object, you

FIGURE 11.8

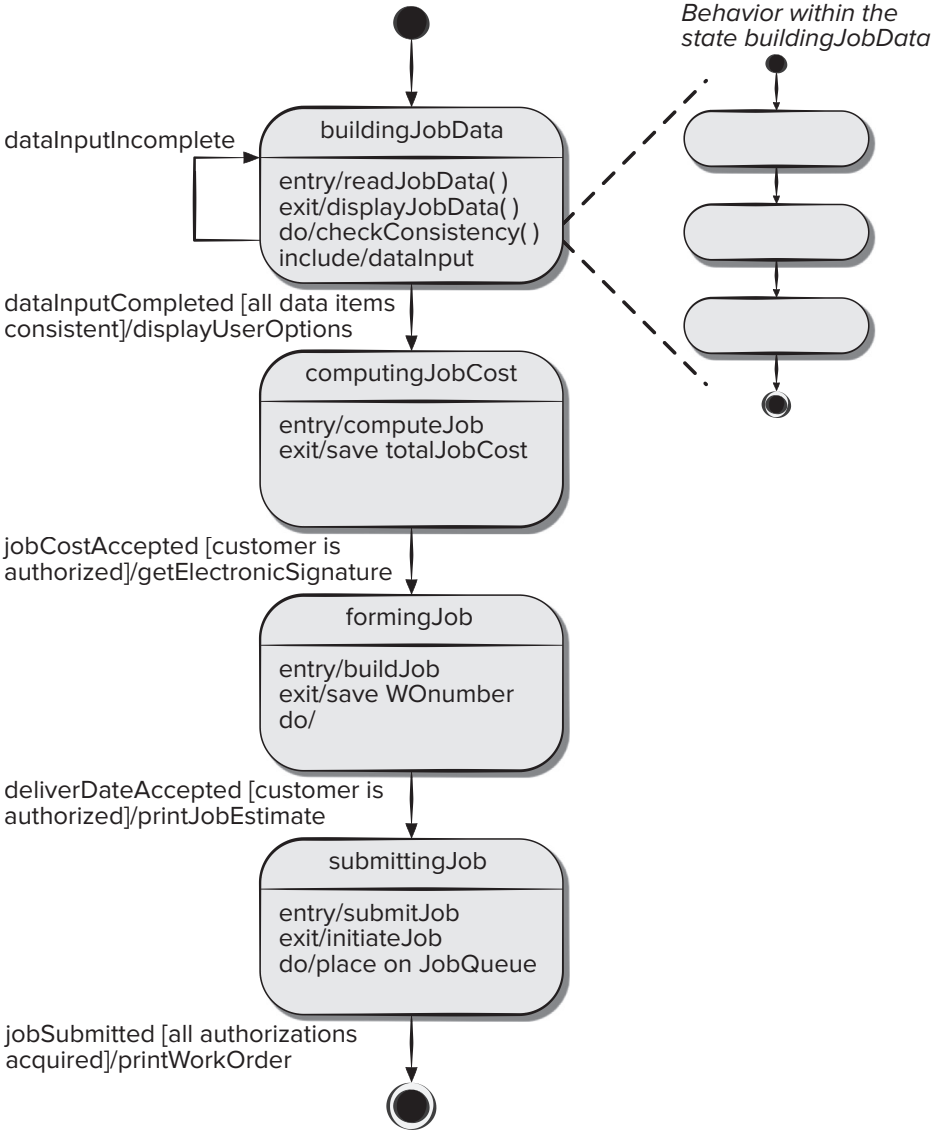
UML activity diagram for *computePaperCost()*



should examine all use cases that are relevant to the design class throughout its life. These use cases provide information that helps you to delineate the events that affect the object and the states in which the object resides as time passes and events occur. The transitions between states (driven by events) is represented using a UML statechart [Ben10a], as illustrated in Figure 11.9.

FIGURE 11.9

Statechart fragment for PrintJob class



The transition from one state (represented by a rectangle with rounded corners) to another occurs following an event that takes the form of:

```
Event-name (parameter-list) [guard-condition] / action expression
```

where `event-name` identifies the event, `parameter-list` incorporates data that are associated with the event, `guard-condition` is written in Object Constraint Language (OCL) and specifies a condition that must be met before the event can occur, and `action expression` defines an action that occurs as the transition takes place.

Referring to Figure 11.9, each state may define *entry/* and *exit/* actions that occur as transition into the state occurs and as transition out of the state occurs, respectively. In most cases, these actions correspond to operations that are relevant to the class that is being modeled. The *do/* indicator provides a mechanism for indicating activities that occur while in the state, and the *include/* indicator provides a means for elaborating the behavior by embedding more statechart detail within the definition of a state.

It is important to note that the behavioral model often contains information that is not immediately obvious in other design models. For example, careful examination of the statechart in Figure 11.9 indicates that the dynamic behavior of the **PrintJob** class is contingent upon two customer approvals as costs and schedule data for the print job are derived. Without approvals (the guard condition ensures that the customer is authorized to approve), the print job cannot be submitted because there is no way to reach the *submittingJob* state.

Step 6. Elaborate deployment diagrams to provide additional implementation detail. Deployment diagrams (Chapter 9) are used as part of architectural design and are represented in descriptor form. In this form, major system functions are represented (often as subsystems) within the context of the computing environment that will house them.

During component-level design, deployment diagrams can be elaborated to represent the location of key packages of components. However, components generally are not represented individually within a component diagram. The reason for this is to avoid diagrammatic complexity. In some cases, deployment diagrams are elaborated into instance form at this time. This means that the specific hardware and operating system environment(s) that will be used is (are) specified and the location of component packages within this environment is indicated.

Step 7. Refactor every component-level design representation and always consider alternatives. Throughout this book, we emphasize that design is an iterative process. The first component-level model you create will not be as complete, consistent, or accurate as the *n*th iteration you apply to the model. It is essential to refactor as design work is conducted.

In addition, you should not suffer from tunnel vision. There are always alternative design solutions, and the best designers consider all (or most) of them before settling on the final design model. Develop alternatives and consider each carefully, using the design principles and concepts presented in Chapter 9 and in this chapter.

11.4 SPECIALIZED COMPONENT-LEVEL DESIGN

There are many programming languages and many ways to create the components required to implement a software architectural design. The principles described in this chapter provide general advice for designing components. Many software products require the use of specialized program development environments to allow their deployment on targeted end user devices such as cell phones or digital assistants. In this section, we present overviews of some specialized component design techniques.

11.4.1 Component-Level Design for WebApps

The boundary between content and function is often blurred when Web-based systems and applications (WebApps) are considered. Therefore, it is reasonable to ask: What is a WebApp component?

In the context of this chapter, a WebApp component is (1) a well-defined cohesive function that manipulates content or provides computational or data processing for an end user or (2) a cohesive package of content and functionality that provides the end user with some required capability. Therefore, component-level design for WebApps often incorporates elements of content design and functional design.

Content design at the component level focuses on content objects and the ways they may be packaged for presentation to a WebApp end user. The formality of content design at the component level should be tuned to the characteristics of the WebApp to be built. In many cases, content objects need not be organized as components and can be manipulated individually. However, as the size and complexity (of the WebApp, content objects, and their interrelationships) grows, it may be necessary to organize content in a way that allows easier reference and design manipulation.[6] In addition, if content is highly dynamic (e.g., the content for an online auction site), it becomes important to establish a clear structural model that incorporates content components.

A good example of a component that might be part of an e-commerce WebApp is the "shopping cart." A shopping cart provides a convenient way for e-commerce customers to store and review their selected items prior to checking out. They can then pay for their selections with a single transaction at the end of their e-commerce session. A carefully designed shopping cart can be reused in several Web store applications by simply editing its content model.

WebApp functionality can be delivered as a series of components developed in parallel with the information architecture to ensure consistency. The shopping cart component described previously contains both content and algorithmic elements. You begin by considering both the requirements model and the initial information architecture. Next, you examine how functionality affects the user's interaction with the application, the information that is presented, and the user tasks that are conducted.

During architectural design, WebApp content and functionality are combined to create a functional architecture. A *functional architecture* is a representation of the functional domain of the WebApp and describes the key functional components in the WebApp and how these components interact with each other.

11.4.2 Component-Level Design for Mobile Apps

Mobile apps are typically structured using multilayered architectures, including a user interface layer, a business layer, and a data layer. If you are building a mobile app as a thin Web-based client, the only components residing on a mobile device are those required to implement the user interface. Some mobile apps may incorporate the components required to implement the business and/or data layers on the mobile device, subjecting these layers to the limitations of the physical characteristics of the device.

---

6 Content components can also be reused in other WebApps.

Considering the user interface layer first, it is important to recognize that a small display area requires the designer to be more selective in choosing the content (text and graphics) to be displayed. It may be helpful to tailor the content to a specific user group(s) and display only what each group needs. The business and data layers are often implemented by composing Web or cloud service components. If the components providing business and data services reside entirely on the mobile device, connectivity issues are not a significant concern. Intermittent (or missing) Internet connectivity must be considered when designing components that require access to current application data that reside on a networked server.

If a desktop application is being ported to a mobile device, the business-layer components may need to be reviewed to see if they meet nonfunctional requirements (e.g., security, performance, accessibility) required by the new platform. The target mobile device may lack the necessary processor speed, memory, or display real estate. The design of mobile applications is considered in greater detail in Chapter 13.

An example of a component in a mobile application might be the single-window full-screen user interface (UI) typically designed for phones and tablets. With careful design it may be possible to allow the mobile app to sense the display characteristics of the mobile device and adapt its appearance to ensure that text, graphics, and UI controls function correctly on many different screen types. This allows the mobile app to function in similar ways on all platforms, without having to be reprogrammed.

11.4.3 Designing Traditional Components

The foundations of component-level design for traditional software components were formed in the early 1960s and were solidified with the work of Edsger Dijkstra ([Dij65], [Dij76b]) and others (e.g., [Boh66]). A traditional software component implements an element of processing that addresses a function or subfunction in the problem domain or some capability in the infrastructure domain. Often these traditional components are called functions, modules, procedures, or subroutines. Traditional components do not encapsulate data in the same way that object-oriented components do. Most programmers make frequent use of function libraries and data structure templates when developing new software products.

In the late 1960s, Dijkstra and others proposed the use of a set of constrained logical constructs from which any program could be formed. The constructs emphasized "maintenance of functional domain." That is, each construct had a predictable logical structure and was entered at the top and exited at the bottom, enabling a reader to follow procedural flow more easily.

The constructs are sequence, condition, and repetition. *Sequence* implements processing steps that are essential in the specification of any algorithm. *Condition* provides the facility for selected processing based on some logical occurrence, and *repetition* allows for looping. These three constructs are fundamental to *structured programming*—an important component-level design technique.

The structured constructs were proposed to limit the procedural design of software to a small number of predictable logical structures. Complexity metrics (Chapter 23) indicate that the use of the structured constructs reduces program complexity and thereby enhances readability, testability, and maintainability. The use of a limited number of logical constructs also contributes to a human understanding process that psychologists call *chunking*. To understand this process, consider the way in which

you are reading this page. You do not read individual letters but rather recognize patterns or chunks of letters that form words or phrases. The structured constructs are logical chunks that allow a reader to recognize procedural elements of a module, rather than reading the design or code line by line. Understanding is enhanced when readily recognizable logical patterns are encountered.

Any program, regardless of application area or technical complexity, can be designed and implemented using only the three structured constructs. It should be noted, however, that dogmatic use of only these constructs can sometimes cause practical difficulties.

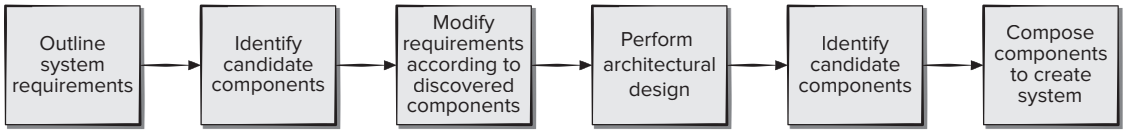
11.4.4 Component-Based Development

In software engineering, reuse is an idea both old and new. Programmers have reused ideas, abstractions, and processes since the earliest days of computing, but the early approach to reuse was ad hoc. Today, complex, high-quality computer-based systems must be built in very short time periods and demand a more organized approach to reuse.

Component-based software engineering (CBSE) is a process that emphasizes the design and construction of computer-based systems using reusable software components (Figure 11.10). Considering this description, many questions arise. Is it possible to construct complex systems by assembling them from a catalog of reusable software components? Can this be accomplished in a cost- and time-effective manner? Can appropriate incentives be established to encourage software engineers to reuse rather than reinvent? Is management willing to incur the added expense associated with creating reusable software components? Can a library of components necessary to accomplish reuse be created in a way that makes it accessible to those who need it? Can existing components be found by those who need them? Increasingly, the answer to each of these questions is yes.

Figure 11.10 shows the principle steps in CBSE. You start with the system requirements and refine them to the point that needed components can be identified. The developers would then search the repository to see if any of the components already exist. Each component has its own postconditions and preconditions. Components whose postconditions match a system requirement are identified, and the preconditions of each component are checked. If the preconditions are satisfied, the component is selected for inclusion in the current build. When no components can be selected, the developers must decide whether to modify the requirements or modify a component that most closely matches the original requirements. This is often an iterative process that continues until the architecture design can be implemented, using a combination of existing or newly created components.

FIGURE 11.10 **Component-based software design**



Consider the task of developing autonomous vehicles, either in real life or a video game. The software for these complex systems is typically created by combing several reusable components, where the components provide distinct modular services. Typically they would include many software components: a component that manages obstacle detection, a planning or navigation component, an artificial intelligence component to manage decision making, and a component of some type controlling vehicle movement or braking. Because these types of software modules have the potential to be used in many different vehicles, it would be desirable to be able to house them in a library of components.

Because CBSE makes use of existing components, it can shorten development time and increase quality. Practitioners [Vit03] often attribute the following advantages to CBSE:

- **Reduced lead time.** It is faster to build complete applications from a pool of existing components.
- **Greater return on investment (ROI).** Sometimes savings can be realized by purchasing components rather than redeveloping the same functionality in-house.
- **Leveraged costs of developing components.** Reusing components in multiple applications allows the costs to be spread over multiple projects.
- **Enhanced quality.** Components are reused and tested in many different applications.
- **Maintenance of component-based applications.** With careful engineering, it can be relatively easy to replace obsolete components with new or enhanced components.

Use of components in CBSE is not without risks. Several of these include the following [Kau11]:

- **Component selection risks.** It is difficult to predict component behavior for black-box components, or there may be poor mapping of user requirements to the component architectural design.
- **Component integration risks.** There is a lack of interoperability standards between components; this often requires the creation of "wrapper code" to interface components.
- **Quality risks.** Unknown design assumptions made for the components makes testing more difficult, and this can affect system safety, performance, and reliability.
- **Security risks.** A system can be used in unintended ways, and system vulnerabilities can be caused by integrating components in untested combinations.
- **System evolution risks.** Updated components may be incompatible with user requirements or contain additional undocumented features.

One of the challenges facing widespread component reuse is *architectural mismatch* [Gar09a]—incompatibilities between assumptions made about components and their operating environments.[7] These assumptions often focus on the component

7 This can be a result of several forms of coupling that should be avoided whenever possible.

control model, the nature of the component connections (interfaces), the architectural infrastructure itself, and the nature of the construction process.

Early detection of architectural mismatch can occur if stakeholder assumptions are explicitly documented. In addition, the use of a risk-driven process model emphasizes the definition of early architectural prototypes and points to areas of mismatch. Repairing architectural mismatch is often very difficult without making use of mechanisms like wrappers or adapters.[8] Sometimes it is necessary to completely redesign a component interface or the component itself to remove coupling issues.

11.5 COMPONENT REFACTORING

Design concepts such as abstraction, hiding, functional independence, refinement, and structured programming, along with object-oriented methods, testing, and software quality assurance (SQA) all contribute to the creation of software components that will be easier to refactor. Most developers would agree that refactoring components to improve quality is a good practice. It is often hard to convince management that it is important to expend resources fixing components that are working correctly instead of adding new functionality to them.

In this book, we focus on the incremental design and delivery of system components. Although there is no quantifiable relationship describing the effects of code changes on architectural quality, most software engineers agree that over time large numbers of changes to a system can lead to the creation of problematic structures in the code base. Failing to address these problems increases the amount of technical debt (Chapter 9) associated with the software system. Reducing this technical debt often involves architectural refactoring, which is generally perceived by developers as both costly and risky. You cannot simply break up large components into smaller components and expect to see an automatic increase in cohesion and a reduction in coupling that will reduce technical debt.

Large software systems may have thousands of components. Making use of data-mining techniques to identify refactoring opportunities can be very beneficial to this work. Automated tools can analyze the source code of system components and make refactoring recommendations to developers, based on generic design rules known to be associated with architectural problems. But it is still up to the developers and their managers to decide which changes to accept and which to ignore [Lin16].

It turns out that many of the error-prone components in a software system are architecturally connected to one another. These flawed architectural connections tend to propagate defects among themselves and accumulate high maintenance costs. If it was possible to automatically identify the technical debt present in the system and the associated maintenance costs, it would be easier to convince developers and managers to spend time refactoring these components. Accomplishing this type of

8 An *adapter* is a software device that allows a client with an incompatible interface to access a component by translating a request for service into a form that can access the original interface.

work requires examining the change histories of the system components [Xia16]. For example, if two or three components are always checked out of the code repository for modification at the same time, it may suggest the components share a common design defect.

11.6 SUMMARY

The component-level design process encompasses a sequence of activities that slowly reduces the level of abstraction with which software is represented. Component-level design ultimately depicts the software at a level of abstraction that is close to code.

Three different views of component-level design may be taken, depending on the nature of the software to be developed. The object-oriented view focuses on the elaboration of design classes that come from both the problem and infrastructure domain. The traditional view refines three different types of components or modules: control modules, problem domain modules, and infrastructure modules. In both cases, basic design principles and concepts that lead to high-quality software are applied. When considered from a process viewpoint, component-level design draws on reusable software components and design patterns that are pivotal elements of component-based software engineering.

Several important principles and concepts guide the designer as classes are elaborated. Ideas encompassed in the open-closed principle and the dependency inversion principle, along with concepts such as coupling and cohesion, guide the software engineer in building testable, implementable, and maintainable software components. To conduct component-level design in this context, classes are elaborated by specifying messaging details, identifying appropriate interfaces, elaborating attributes, and defining data structures to implement them, describing processing flow within each operation, and representing behavior at a class or component level. In every case, design iteration (refactoring) is an essential activity.

Traditional component-level design requires the representation of data structures, interfaces, and algorithms for a program module in sufficient detail to guide in the generation of programming language source code. To accomplish this, the designer uses one of several design notations that represent component-level detail in either graphical, tabular, or text-based formats.

Component-level design for WebApps considers both content and functionality as a Web-based system will deliver it. Content design at the component level focuses on content objects and the ways they may be packaged for presentation to a WebApp end user. Functional design for WebApps focuses on processing functions that manipulate content, perform computations, process database queries, and establish interfaces with other systems. All component-level design principles and guidelines apply.

Component-level design for mobile apps makes use of a multilayered architecture that includes a user interface layer, a business layer, and a data layer. If the mobile app requires the design of components that implement the business and/or data layers on the mobile device, the limitations of the physical characteristics of the device become important constraints on the design.

Structured programming is a procedural design philosophy that constrains the number and type of logical constructs used to represent algorithmic detail. The intent of structured programming is to assist the designer in defining algorithms that are less complex and therefore easier to read, test, and maintain.

Component-based software engineering identifies, constructs, catalogs, and disseminates a set of software components for an application domain. These components are then qualified, adapted, and integrated for use in a new system. Reusable components should be designed within an environment that establishes standard data structures, interface protocols, and program architectures for each application domain.

PROBLEMS AND POINTS TO PONDER

11.1. The term *component* is sometimes a difficult one to define. First provide a generic definition, and then provide more explicit definitions for object-oriented and traditional software. Finally, pick three programming languages with which you are familiar and illustrate how each defines a component.

11.2. Why are control components necessary in traditional software and generally not required in object-oriented software?

11.3. Describe the OCP in your own words. Why is it important to create abstractions that serve as an interface between components?

11.4. Describe the DIP in your own words. What might happen if a designer depends too heavily on concretions?

11.5. Select three components that you have developed recently, and assess the types of cohesion that each exhibits. If you had to define the primary benefit of high cohesion, what would it be?

11.6. Select three components that you have developed recently, and assess the types of coupling that each exhibits. If you had to define the primary benefit of low coupling, what would it be?

11.7. Develop (1) an elaborated design class, (2) interface descriptions, (3) an activity diagram for one of the operations within the class, and (4) a detailed statechart diagram for one of the *SafeHome* classes that we have discussed in earlier chapters.

11.8. What is a WebApp component?

11.9. Select the code from a small software component and represent it using an activity diagram.

11.10. Why is "chunking" important during the component-level design review process?

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

12 User Experience Design

We live in a world of high-technology products, and virtually all of them—consumer electronics, industrial equipment, automobiles, corporate systems, military systems, mobile apps, WebApps, video games, and virtual reality simulations—require human interaction. If a product is to be successful, it must provide a positive user experience (UX). The product needs to exhibit good *usability*—a qualitative measure of the ease and efficiency with which a human can employ the functions and features offered by the high-technology product. A product should incorporate *accessibility* considerations such as assistive technologies when its specified users include people with a range of disabilities within a specified context of use.

KEY CONCEPTS

accessibility 237
accessibility guidelines 259
command labeling 260
customer journey map 244
error handling 260
golden rules 234
help facilities 260
information architecture 235
interface analysis 243
interface consistency 240
interface design 250
internationalization 260
memory load 239
response time 259
storyboard 261
task analysis 247
usability guidelines 257
user experience analysis 243
user interaction design 236
user scenario 261
visual design 237



QUICK LOOK

What is it? User experience (UX) design is the process of enhancing user satisfaction with a product by creating a usable, accessible, and pleasurable interaction between product and its users.

Who does it? A software engineer designs the user experience and user interface assisted by knowledgeable stakeholders.

Why is it important? If software is difficult to use, if it forces you into mistakes, or if it frustrates your efforts to accomplish your goals, you won't like it, regardless of the computational power it exhibits, the content it delivers, or the functionality it offers. The user experience has to be right because it molds a user's perception of the software.

What are the steps? User interface design begins with the identification of user, task, and environmental requirements. These form the basis for the creation of a screen layout and navigation pathways through the information architecture.

What is the work product? User persona and scenarios are created based on the desired customer journey. Low-fidelity prototypes and digital interface prototypes are developed, evaluated, and modified in an iterative fashion.

How do I ensure that I've done it right? An interface prototype is "test driven" by the users, and feedback from the test drive is used for the next iterative modification of the prototype.

For the first three decades of the computing era, usability was not a dominant concern among those who built software. In his classic book on design, Donald Norman [Nor88] argued that it was time for a change in attitude: "To make technology that fits human beings, it is necessary to study human beings. But now we tend to study only the technology. As a result, people are required to conform to technology. It is time to reverse this trend, time to make technology that conforms to people."

As technologists studied human interaction, two dominant issues arose. First, a set of *golden rules* (discussed in Section 12.2) were identified. These applied to all human interaction with technology products. Second, a set of *interaction mechanisms* were defined to enable software designers to build systems that properly implemented the golden rules. These interaction mechanisms, collectively called the *user interface,* have eliminated some of the many egregious problems associated with human interfaces. But even today, we all encounter user interfaces that are difficult to learn, difficult to use, confusing, counterintuitive, unforgiving, and in many cases, totally frustrating. Yet, someone spent time and energy building each of these interfaces, and it is not likely that the builder created these problems purposely.

User experience design is a set of incremental process activities that help the development team and the project stakeholders focus on providing a positive experience for users of the software product. UX design is broader than user interface design and usability or accessibility engineering. It must begin early in the project life cycle if it is to be effective. Developers waiting until the end of a project to add user interface functionality are unlikely to provide a pleasurable experience for users.

In this chapter we will focus on user interface design issues in the context of user experience design. Readers wishing a more detailed coverage of UX should examine books by Shneiderman [Shn16], Nielsen [Nei93], and Norman [Nor13].

12.1 User Experience Design Elements

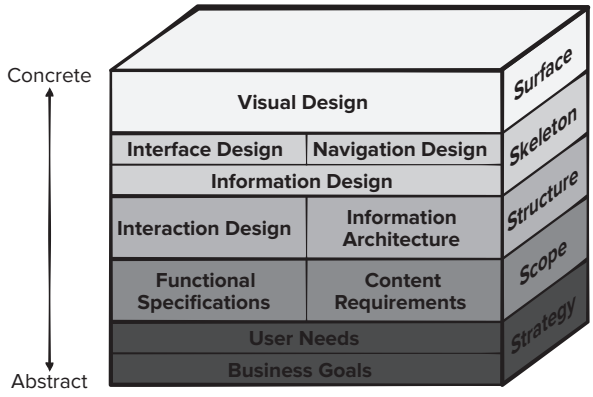
User experience design tries to ensure that no aspect of your software appears in the final release candidate without the explicit decision of the development team and other stakeholders to include it. This means taking into account every reasonable user action and expectation during every step of the development process. To make the task of crafting a positive user experience more manageable, Garret [Gar10] suggests breaking it down into component elements: strategy, scope, structure, skeleton, and surface. The relationships among these component elements and subcomponents are show in Figure 12.1.

The Garrett's organization of UX design can be interpreted as follows for software product development:

- **Strategy.** Identifies user needs and customer business goals that form the basis for all UX design work (Section 12.4)
- **Scope.** Includes both the functional and content (e.g., information, media, services) requirements needed to realize a feature set consistent with the project strategy

FIGURE 12.1

User experience design elements



- **Structure.** Consists of the interaction design [e.g., how the system reacts in response to user actions (Section 12.1.2)] and information architecture [e.g., the organization of the content elements (Section 12.1.1)]
- **Skeleton.** Comprised of three components: information design (e.g., presentation of content in a way to make it understandable to the user), interface design [e.g., arranging interface screen objects to allow the user to work with the system functionality (Section 12.5)], navigation design (e.g., the set of screen elements that allow users to traverse the information architecture)
- **Surface.** Presents visual design or the appearance of the finished project to its users (Section 12.1.4)

Several cross-cutting aspects of UX design are of particular interest to software engineers: information architecture, user interaction design, usability engineering, and visual design.

12.1.1 Information Architecture

As an architectural designer, you must identify information (content) architecture and software architecture. The term *information architecture* is used to connote structures that lead to better organization, labeling, navigation, and searching of content objects. Content architecture focuses on the manner in which content objects (or composite objects such as screens or widgets) are structured for presentation and navigation. Software architecture addresses the manner in which the application is structured to manage user interaction, handle internal processing tasks, effect navigation, and present content.

Architectural design (Chapter 10) is tied to the goals established for a software product, the content to be presented, the users who will visit, and the navigation philosophy that has been established. In most cases, architecture design is conducted in parallel with interface design, aesthetic design, and content design. Because the software architecture may have a strong influence on navigation, the decisions made during this design action will influence work conducted during navigation design. In many cases, a subject matter expert is needed to help project stakeholders organize the content items for efficient assimilation and traversal by product users.

12.1.2 User Interaction Design

Interaction design focuses on the interface between a product and its user. Not too many years ago, the only way for a user to interact with a computer system was typing input on a keyboard and reading output on a display screen of some kind. Today the modes of input and output are quite varied and may include voice input, computer speech generation, touch input, 3D printed output, immersive augmented reality experiences, and sensor tracking of users within their environment. Devices like this are needed to give users ways to control a computer system. Oftentimes these interaction devices interfere with the product providing a natural and pleasurable user experience.

User interaction should be defined by the stakeholders in the user stories (Chapter 7) created to describe how users can accomplish their goals using the software product. This suggests that user interaction design should also include a plan for how information should be presented within such a system and how to enable the user to understand that information. It is important to recall that the purpose of the user interface is to present just enough information to help the users decide what their next action should be to accomplish their goal and how to perform it.

We will describe the user interface design process in more detail in Section 12.5, but initially, there are important questions user interaction designers must ask when devising user interfaces:[1]

- What can users do with a mouse, finger, or stylus to interact with the interface directly?
- What about the appearance (e.g., color, shape, size) gives users clues about how the user interaction functions?
- What information do you provide to let users know what will happen before they perform an action?
- Are there any constraints put in place to help prevent errors?
- Do error messages provide a way for users to correct a problem or explain why an error occurs?
- What feedback do users get once an action is performed?
- Are the interface elements a reasonable size to facilitate interaction?
- What familiar or standard formats should be used to display information and accept input?

12.1.3 Usability Engineering

Usability engineering is part of UX design work that defines the specification, design, and testing of the human-computer interaction portion of a software product. This software engineering action focuses on devising human-computer interfaces that have high usability. Usability engineering provides structured methods for achieving efficiency and elegance in interface design. Terms like *user friendliness* do not provide much guidance here since this is often a very subjective judgment. If developers focus on making a product easy to learn, easy to use, and easy to remember over time, usability can be measured quantitatively and tested for improvements in usability.

1 See https://www.usability.gov/what-and-why/interaction-design.html.

Accessibility is another aspect of usability engineering that should be considered when designing user interactions with the software. *Accessibility* is the degree to which people with special needs (e.g., sight impaired, deaf, elderly, cognitively impaired) are provided with a means to perceive, understand, navigate, and interact with computer products. The goal of accessibility design is to provide hardware or software tools that can remove barriers that may prevent users from successfully completing tasks supported by the software. Usability and accessibility are discussed in greater detail in Section 12.7.

12.1.4 Visual Design

Visual design, also called aesthetic design or graphic design, is an artistic endeavor that complements the technical aspects of the user experience design. Without it, a software product may be functional, but unappealing. With it, a product draws its users into a world that embraces them on a visceral, as well as an intellectual level.

But what is aesthetic? There is an old saying, "Beauty exists in the eye of the beholder." This is particularly appropriate when aesthetic design for many games or mobile apps is considered. To perform effective aesthetic design, you should return to the user hierarchy developed as part of the requirements model (Chapter 8) and ask, "Who are the product's users and what 'look' do they desire?"

Graphic design considers every aspect of the look and feel of a web or mobile app. The graphic design process begins with screen layout and proceeds into a consideration of global color schemes; type fonts, sizes, and styles; the use of supplementary media (e.g., audio, video, animation); and all other aesthetic elements of an application. Not every software engineer has artistic talent. If you fall into this category, hire an experienced graphic designer for aesthetic design work.

SAFEHOME



Graphic Design

The scene: Doug Miller's office after the first *SafeHome* room layout interface prototype review.

The players: Doug Miller, SafeHome software engineering project manager, and Vinod Raman, member of the *SafeHome* software engineering team.

The conversation:

Doug: What's your impression of the new room layout design?

Vinod: I like it, but more importantly, our customers like it.

Doug: How much help did you get from the graphic designer we borrowed from marketing?

Vinod: A great deal actually. Marg has a great eye for page layout and suggested an awesome graphic theme for the app screens. Much better than what we came up with on our own.

Doug: That's good. Any issues?

Vinod: We still have to create an alternate screen to take accessibility issues into account for some of our visually impaired users. But we would've had to do that for any app design we had.

Doug: Can we use Marg for that work as well?

Vinod: Yeah, she has a good understanding of usability and accessibility.

Doug: OK, I'll set it up with marketing and borrow her a little longer.

12.2 The Golden Rules

In his book on interface design, Theo Mandel [Man97] coins three *golden rules:*

1. Place the user in control.
2. Reduce the user's memory load.
3. Make the interface consistent.

These golden rules actually form the basis for a set of user interface design principles that guide this important aspect of software design.

12.2.1 Place the User in Control

During a requirements gathering session for a major new information system, a key user was asked about the attributes of the window-oriented graphical interface.

"What I really would like," said the user solemnly, "is a system that reads my mind. It knows what I want to do before I need to do it and makes it very easy for me to get it done. That's all, just that."

Your first reaction might be to shake your head and smile, but pause for a moment. There was absolutely nothing wrong with the user's request. She wanted a system that reacted to her needs and helped her get things done. She wanted to control the computer, not have the computer control her.

Most interface constraints and restrictions that are imposed by a designer are intended to simplify the mode of interaction. But for whom?

As a designer, you may be tempted to introduce constraints and limitations to simplify the implementation of the interface. The result may be an interface that is easy to build, but frustrating to use. Mandel [Man97] defines a number of design principles that allow the user to maintain control:

Define interaction modes in a way that does not force a user into unnecessary or undesired actions. An interaction mode is the current state of the interface. For example, if *autocorrect* is selected in a text messaging app menu, the software performs autocorrect continually. There is no reason to force the user to remain in autocorrect mode. The user should be able to enter and exit the mode with little or no effort.

Provide for flexible interaction. Because different users have different interaction preferences, choices should be provided. For example, software might allow a user to interact via keyboard commands, mouse movement, a digitizer pen, a multitouch screen, or voice recognition commands. But every action is not amenable to every interaction mechanism. Consider, for example, the difficulty of using a keyboard command (or voice input) to draw a complex shape.

Allow user interaction to be interruptible and undoable. Even when involved in a sequence of actions, the user should be able to interrupt the sequence to do something else (without losing the work that had been done). The user should also be able to "undo" any action or any linear sequence of actions.

Streamline interaction as skill levels advance, and allow the interaction to be customized. Users often find that they perform the same sequence of interactions repeatedly. It is worthwhile to design a "macro" mechanism that enables an advanced user to customize the interface to facilitate interaction.

Hide technical internals from the casual user. The user interface should move the user into the virtual world of the application. The user should not be aware of the operating system, file management functions, or other arcane computing technology.

Design for direct interaction with objects that appear on the screen. The user feels a sense of control when able to manipulate the objects that are necessary to perform a task in a manner similar to what would occur if the object were a physical thing. For example, an application interface that allows a user to drag a document into the "trash" is an implementation of direct manipulation.

12.2.2 Reduce the User's Memory Load

A well-designed user interface does not tax a user's memory because the more a user has to remember, the more error-prone the interaction will be. Whenever possible, the system should "remember" pertinent information and assist the user with an interaction scenario that assists recall. Mandel [Man97] defines design principles that enable an interface to reduce the user's memory load:

Reduce demand on short-term memory. When users are involved in complex tasks, the demand on short-term memory can be significant. The interface should be designed to reduce the requirement to remember past actions, inputs, and results. This can be accomplished by providing visual cues that enable a user to recognize past actions, rather than having to recall them.

Establish meaningful defaults. The initial set of defaults should make sense for the average user, but a user should be able to specify individual preferences. However, a "reset" option should be available, enabling the redefinition of original default values.

Define shortcuts that are intuitive. When mnemonics are used to accomplish a system function (e.g., ctrl-C to invoke the *copy* function), the mnemonic should be tied to the action in a way that is easy to remember (e.g., first letter of the task to be invoked).

The visual layout of the interface should be based on a real-world metaphor. For example, a bill payment system should use a checkbook and check register metaphor to guide the user through the bill paying process. A room layout application should allow users to drag furniture from a visual catalog and arrange it on the screen using a touch interface. This enables the user to rely on well-understood visual cues, rather than memorizing an arcane interaction sequence.

Disclose information in a progressive fashion. The interface should be organized hierarchically. That is, information about a task, an object, or some behavior should be presented first at a high level of abstraction. More detail should be presented after the user indicates interest.

SAFEHOME



Violating a User Interface Golden Rule

The scene: Vinod's cubicle, as user interface design begins.

The players: Vinod and Jamie, members of the *SafeHome* software engineering team.

The conversation:

Jamie: I've been thinking about the surveillance function interface.

Vinod (smiling): Thinking is good.

Jamie: I think maybe we can simplify matters some.

Vinod: Meaning?

Jamie: Well, what if we eliminate the floor plan entirely? It's flashy, but it's going to take serious development effort. Instead, we just ask the user to specify the camera he wants to see and then display the video in a video window.

Vinod: How does the homeowner remember how many cameras are set up and where they are?

Jamie (mildly irritated): He's the homeowner; he should know.

Vinod: But what if he doesn't?

Jamie: He should.

Vinod: That's not the point . . . what if he forgets?

Jamie: Uh, we could provide a list of operational cameras and their locations.

Vinod: That's possible, but why should he have to ask for a list?

Jamie: Okay, we provide the list whether he asks or not.

Vinod: Better. At least he doesn't have to remember stuff that we can give him.

Jamie (thinking for a moment): But you like the floor plan, don't you?

Vinod: Uh-huh. Especially since we are creating the room layout application in a related product.

Jamie: Which one will marketing like, do you think?

Vinod: You're kidding, right?

Jamie: No.

Vinod: Duh . . . the one with the flash . . . they love sexy product features . . . they're not interested in which is easier to build.

Jamie (sighing): Okay, maybe I'll create paper prototypes of both.

Vinod: Good idea . . . then we let the customer decide.

12.2.3 Make the Interface Consistent

The interface should present and acquire information in a consistent fashion. This implies that (1) all visual information is organized according to design rules that are maintained throughout all screen displays, (2) input mechanisms are constrained to a limited set that is used consistently throughout the application, and (3) mechanisms for navigating from task to task are consistently defined and implemented. Mandel [Man97] defines a set of design principles that help make the interface consistent:

Allow the user to put the current task into a meaningful context. Many interfaces implement complex layers of interactions with dozens of screen

images. It is important to provide indicators (e.g., window titles, graphical icons, consistent color coding) that enable the user to know the context of the work at hand. In addition, the user should be able to determine where he has come from and what alternatives exist for a transition to a new task.

Maintain consistency across a complete product line. A family of applications (i.e., a product line) should implement the same design rules so that consistency is maintained for all interaction.

If past interactive models have created user expectations, do not make changes unless there is a compelling reason to do so. Once a particular interactive sequence has become a de facto standard (e.g., the use of alt-S to save a file), the user expects this in every application encountered. A change (e.g., using alt-S to invoke scaling) will cause confusion.

The interface design principles discussed in this and the preceding sections provide you with basic guidance. In the sections that follow, you'll learn about the user experience design process itself.

12.3 User Interface Analysis and Design

Although UX design work is not only about the user interface, design of the user interface is a good place to start understanding the UX process. The overall process for analyzing and designing a user interface begins with the creation of different models of system function (as perceived from the outside). You begin by delineating the human- and computer-oriented tasks that are required to achieve system function and then considering the design issues that apply to all interface designs. Tools are used to prototype and ultimately implement the design model, and the result is evaluated by end users for quality.

12.3.1 Interface Analysis and Design Models

Four different models come into play when a user interface is to be analyzed and designed. A human engineer (or the software engineer) establishes a *user model,* the software engineer creates a *design model,* the end user develops a mental image that is often called the user's *mental model* or the *system perception,* and the implementers of the system create an *implementation model*. Unfortunately, each of these models may differ significantly. Your role, as an interface designer, is to reconcile these differences and derive a consistent representation of the interface.

The user model establishes the profile of end users of the system. To build an effective user interface, "all design should begin with an understanding of the intended users, including profiles of their age, gender, physical abilities, education, cultural or ethnic background, motivation, goals and personality" [Shn16]. In addition, users can be categorized as novices; knowledgeable, intermittent users; or knowledgeable frequent users. Many UX designers like to build user profiles or personas (Section 12.4.2) as way of capturing what is known about each class of users.

The user's *mental model* (system perception) is the image of the system that end users carry in their heads. For example, if the user of a mobile app that rates restaurants were asked to describe its operation, the system perception would guide the

response. The accuracy of the description will depend on the user's profile (e.g., novices would provide a sketchy response at best) and overall familiarity with software in the application domain. A user who understands restaurant rating apps fully but has worked with the specific app only a few times might actually be able to provide a more complete description of its function than the novice who has spent days trying to apply the app effectively.

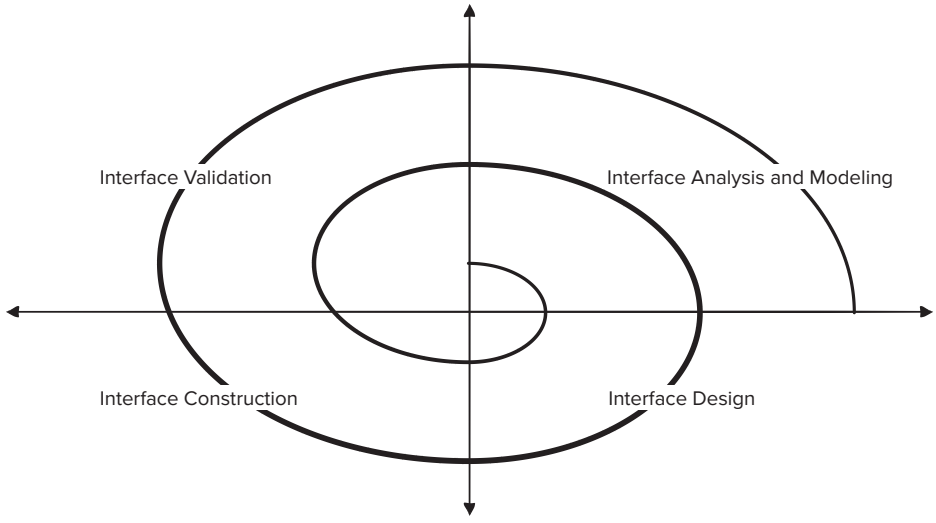
The *implementation model* combines the outward manifestation of the computer-based system (the look and feel of the interface), coupled with all supporting information (books, manuals, videotapes, help files) that describes interface syntax and semantics. When the implementation model and the user's mental model are coincident, users generally feel comfortable with the software and use it effectively. To accomplish this "melding" of the models, the design model must have been developed to accommodate the information contained in the user model, and the implementation model must accurately reflect syntactic and semantic information about the interface.

12.3.2 The Process

The analysis and design process for user interfaces is iterative and can be represented using a process model similar to the one discussed in Chapter 4. Referring to Figure 12.2, the user interface analysis and design process begins at the interior of the spiral and encompasses four distinct framework activities [Man97]: (1) interface analysis and modeling, (2) interface design, (3) interface construction, and (4) interface validation. The spiral shown in Figure 12.2 implies that each of these tasks will occur more than once, with each pass around the spiral representing additional elaboration of requirements and the resultant design. In most cases, the construction activity involves prototyping—the only practical way to validate what has been designed.

FIGURE 12.2

The user interface design process



Interface analysis focuses on the profile of the users who will interact with the system. Skill level, business understanding, and general receptiveness to the new system are recorded; and different user categories are defined. For each user category, requirements are elicited. In essence, you work to understand the system perception (Section 12.4.2) for each class of users.

Once general requirements have been defined, a more detailed *task analysis* is conducted. Those tasks that the user performs to accomplish the goals of the system are identified, described, and elaborated (over a number of iterative passes through the spiral). Task analysis is discussed in more detail in Section 12.4.3. Finally, analysis of the user environment focuses on the characteristics of the physical work environment (e.g., location, lighting, position constraints).

The information gathered as part of the analysis action is used to create an analysis model for the interface. Using this model as a basis, the design activity commences.

The goal of *interface design* is to define a set of interface objects and actions (and their screen representations) that enable a user to perform all defined tasks in a manner that meets every usability goal defined for the system. Interface design is discussed in more detail in Section 12.5.

Interface construction normally begins with the creation of a prototype that enables usage scenarios to be evaluated. As the iterative design process continues, a user interface tool kit may be used to complete the construction of the interface.

Interface validation focuses on (1) the ability of the interface to implement every user task correctly, to accommodate all task variations, and to achieve all general user requirements; (2) the degree to which the interface is easy to use and easy to learn, and (3) the user's acceptance of the interface as a useful tool in her work.

As we have already noted, the activities described in this section occur iteratively. Therefore, there is no need to attempt to specify every detail (for the analysis or design model) on the first pass. Subsequent passes through the process elaborate task detail, design information, and the operational features of the interface.

12.4 USER EXPERIENCE ANALYSIS[2]

A key tenet of all software engineering process models is this: *Understand the problem before you attempt to design a solution.* In the case of user experience design, understanding the problem means understanding (1) the people (end users) who will interact with the system through the interface, (2) the tasks that end users must perform to do their work, (3) the content that is presented as part of the interface, and (4) the environment in which these tasks will be conducted. In the sections that follow, we examine each of these elements of UX analysis with the intent of establishing a solid foundation for the interface design tasks that follow.

2 It is reasonable to argue that this section should be placed in Chapter 8, since requirements analysis issues are discussed there. It has been positioned here because user experience analysis and design are intimately connected to one another, and the boundary between the two is often fuzzy.

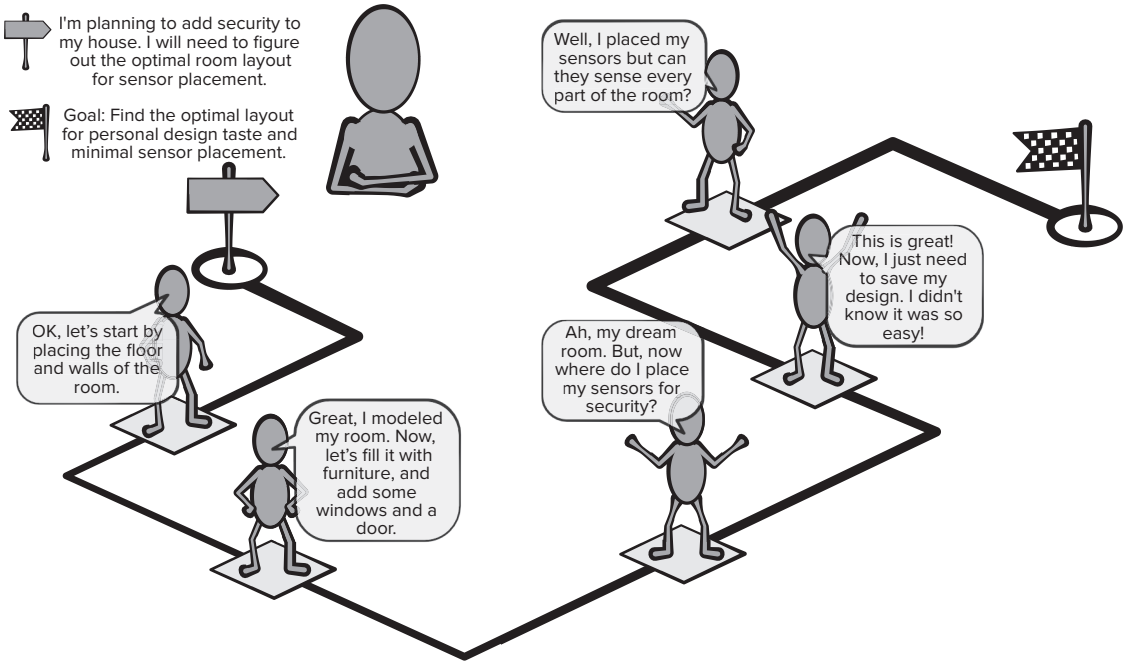
12.4.1 User Research

The phrase *user interface* is probably all the justification needed to spend some time understanding the user before worrying about technical matters. Earlier we noted that each user has a mental image of the software that may be different from the mental image developed by other users. In addition, the user's mental image may be vastly different from the software engineer's design model. The only way that you can get the mental image and the design model to converge is to work to understand the users themselves as well as how these people will use the system. Information from a broad array of sources (user interviews, sales input, marketing input, support input) can be used to accomplish this.

Many UX developers like to create a *customer journey map* (Figure 12.3) as a means of outlining their goals and plans for the software product. The customer journey map shows how the users will experience the software product as if they were traveling on a physical trip with touchpoints (milestones), obstacles, and ways to monitor their progress. Christensen [Chr13] suggests the following steps to create a customer journey map:

1. **Gather stakeholders.** Locate all affected parties needed to ensure diverse viewpoints are included in the customer journey map.
2. **Conduct research.** Collect all information you can about all the things (thoughts, feelings, actions, motivations, expectations, goals, needs,

FIGURE 12.3 Customer journey map



pain points, barriers, questions) users may experience as they use the software product and define your customer phases. The customer phases will become your touchpoints and are shown as labeled squares in Figure 12.3.

3. **Build the model.** Create a visualization of the touch points (any interaction between the user and product), channels (interaction devices or information streams), and actions taken by the customer (user).
4. **Refine the design.** Recruit a designer to make the deliverable visually appealing and ensure that customer phases are identified clearly.
5. **Identify gaps.** Note any gaps in the customer experience or points of friction or pain (places with information overlap or poor transition between phases).
6. **Implement your findings.** Assign responsible parties to bridge the gaps and resolve pain points found.

12.4.2 User Modeling

In previous chapters, you learned that the user story describes the manner in which an actor (in the context of user interface design, an actor is always a person) interacts with a system. When used as part of task analysis, the user story is refined to become a formal use case to show how an end user performs some specific work-related task. In most instances, the use case is written in an informal style (a simple paragraph) in the first person. For example, assume that a small software company wants to build a computer-aided design system explicitly for interior designers. To get a better understanding of how they do their work, actual interior designers are asked to describe a specific design function. When asked, "How do you decide where to put furniture in a room?" an interior designer writes the following informal user story:

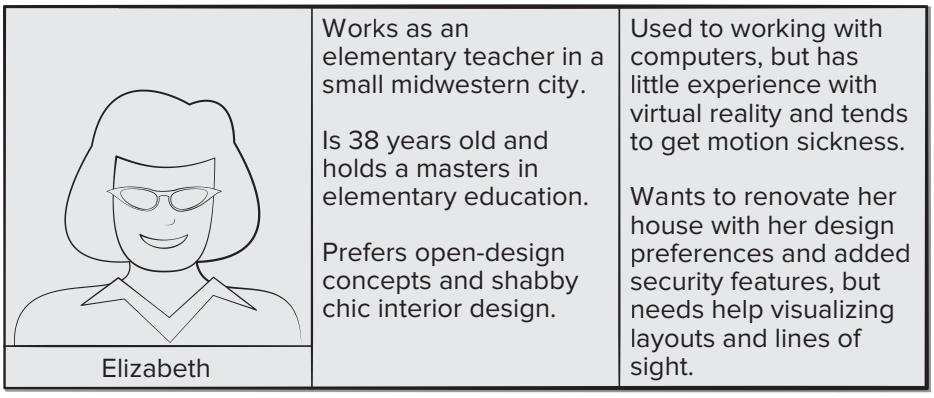
> I begin by sketching the floor plan of the room, the dimensions and the location of windows and doors. I'm very concerned about light as it enters the room, about the view out of the windows (if it's beautiful, I want to draw attention to it), about the running length of an unobstructed wall, about the flow of movement through the room. I then look at the list of furniture my customer and I have chosen. . . . Then, I draw a rendering (a 3-D picture) of the room to give my customer a feel for what it'll look like.

This user story provides a basic description of one important work task for the computer-aided design system from the perspective of one of its users. From it, you can extract tasks, objects, and the overall flow of the interaction. In addition, other features of the system that would please the interior designer might also be conceived. For example, a digital photo could be taken looking out each window in a room. When the room is rendered, the actual outside view could be represented through each window. However, if there is more than one type of user, it may be important to define more than one set of user goals for the system described by the user stories.

User experience designers often create fictional user personas to summarize the assumptions made for the different types of users. A *user persona* is a representation of the goals and behavior of a hypothesized group of users. Personas are often

FIGURE 12.4

Persona example



synthesized from data collected during interviews with users. Figure 12.4 shows an example of a user persona. Personas are often used to improve product designers' abilities to see through the eyes of target users [Hil17].

Lene Nielsen [Nie13] describes four tasks that occur during the general process of creating and using personas to guide the user experience design process:

- **Data collection and analysis.** Stakeholders collect as much information about the proposed product users as possible to determine the user groups and begin to emphasize what each group needs.
- **Describe personas.** The developers need to decide how many personas it is reasonable to create, if they will create more than one, and which persona will be their primary focus. The developers create and name each persona including details about their education, lifestyle, values, goals, needs, limitations, desires, attitudes, and behavior patterns.
- **Develop scenarios.** Scenarios are user stories about how personas will use the product being developed. They may focus on the touchpoints and obstacles described in the customer journey. They should show how personas would overcome problems using the system resources if they were the actual users of the system.
- **Acceptance by stakeholders.** Often this is done by validating the scenarios using a review technique or demonstration called *cognitive walkthrough*.[3] Stakeholders assume the role defined by the persona and work through a scenario using a system prototype.

3 A cognitive walkthrough is a method that evaluates whether the order of cues and prompts in a system supports the way people process tasks and anticipates the "next steps" of a system by having users verbalize their decision-making process while using a system representation to complete a user goal.

SAFEHOME



Use Cases for User Interface Design

The scene: Vinod's cubicle, as user interface design continues.

The players: Vinod and Jamie, members of the *SafeHome* software engineering team.

The conversation:

Jamie: I pinned down our marketing contact and had her write a user story for the surveillance interface.

Vinod: From whose point of view?

Jamie: The homeowner, who else is there?

Vinod: There's also the system administrator role. Even if it's the homeowner playing the role, it's a different point of view. The administrator sets the system up, configures stuff, lays out the floor plan, places the cameras . . .

Jamie: All I had her do was play the role of the homeowner when he wants to see video.

Vinod: That's okay. It's one of the major behaviors of the surveillance function interface. But we're going to have to examine the system administrator's behavior as well.

Jamie (irritated): You're right.

(Jamie leaves to find the marketing person. She returns a few hours later.)

Jamie: I was lucky. I found her, and we worked through the administrator's user story together using the homeowner persona. Basically, we're going to define "administration" as one function that's applicable to all other *SafeHome* functions. Here's what we came up with.

(Jamie shows the user story to Vinod.)

User story: I want to be able to set up or edit the system layout at any time. When I set up the system, I select an administration function. It asks me whether I want to do a new setup or whether I want to edit an existing setup. If I select a new setup, the system displays a drawing screen that will enable me to draw the floor plan onto a grid. There will be icons for walls, windows, and doors so that drawing is easy. I just stretch the icons to their appropriate lengths. The system will display the lengths in feet or meters (I can select the measurement system). I can select from a library of sensors and cameras and place them on the floor plan. I get to label each, or the system will do automatic labeling. I can establish settings for sensors and cameras from appropriate menus. If I select edit, I can move sensors or cameras, add new ones or delete existing ones, edit the floor plan, and edit the setting for cameras and sensors. In every case, I expect the system to do consistency checking and to help me avoid mistakes.

Vinod (after reading the scenario): Okay, there are probably some useful design patterns [Chapter 14] or reusable components for GUIs for drawing programs. I'll betcha 50 bucks we can implement some or most of the administrator interface using them.

Jamie: Agreed. I'll check it out.

12.4.3 Task Analysis

The user's goal is to accomplish one or more tasks via the software product. To accomplish this, the software user interface must provide mechanisms that allow the user to achieve her goal. The goal of task or scenario analysis is to answer the following questions:

- What work will the user perform in specific circumstances?
- What tasks and subtasks will be performed as the user does the work?

- What specific problem domain objects will the user manipulate as work is performed?
- What is the sequence of work tasks—the workflow?
- What is the hierarchy of tasks?

To answer these questions, you must draw upon techniques that we have discussed earlier in this book, but in this instance, these techniques are applied to the user interface.

In Chapter 8, we discussed stepwise elaboration (also called functional decomposition or stepwise refinement) as a mechanism for refining the processing tasks that are required for software to accomplish some desired function. Task analysis for interface design uses an elaborative approach that assists in understanding the human activities the user interface must accommodate.

First, you should define and classify the human tasks that are required to accomplish the goal of the system or app. For example, let's reconsider the computer-aided design system for interior designers discussed earlier. By observing an interior designer at work, you notice that interior design comprises a number of major activities: furniture layout (note the user story discussed earlier), fabric and material selection, wall and window coverings selection, presentation (to the customer), costing, and shopping. Each of these major tasks can be elaborated into subtasks. For example, using information contained in the use case, furniture layout can be refined into the following tasks: (1) draw a floor plan based on room dimensions, (2) place windows and doors at appropriate locations, (3a) use furniture templates to draw scaled furniture outlines on the floor plan, (3b) use accents templates to draw scaled accents on the floor plan, (4) move furniture outlines and accent outlines to get the best placement, (5) label all furniture and accent outlines, (6) draw dimensions to show location, and (7) draw a perspective-rendering view for the customer. A similar approach could be used for each of the other major tasks.

Subtasks 1 to 7 can each be refined further. Subtasks 1 to 6 will be performed by manipulating information and performing actions within the user interface. On the other hand, subtask 7 can be performed automatically in software and will result in little direct user interaction.[4] The design model of the interface should accommodate each of these tasks in a way that is consistent with the user model (the profile of a "typical" interior designer) and system perception (what the interior designer expects from an automated system).

12.4.4 Work Environment Analysis

Hackos and Redish [Hac98] discuss work environment analysis this way: "People do not perform their work in isolation. They are influenced by the activity around them, the physical characteristics of the workplace, the type of equipment they are using, and the work relationships they have with other people." In some applications the user interface for a computer-based system is placed in a "user-friendly location"

4 However, this may not be the case. The interior designer might want to specify the perspective to be drawn, the scaling, the use of color, and other information. The use case related to drawing perspective renderings would provide the information you need to address this task.

(e.g., proper lighting, good display height, easy keyboard access), but in others (e.g., a factory floor or an airplane cockpit), lighting may be suboptimal, noise may be a factor, a keyboard or mouse or touch screen may not be an option, or display placement may be less than ideal. The interface designer may be constrained by factors that mitigate against ease of use.

In addition to physical environmental factors, the workplace culture also comes into play. Will system interaction be measured in some manner (e.g., time per transaction or accuracy of a transaction)? Will two or more people have to share information before an input can be provided? How will support be provided to users of the system? These and many related questions should be answered before the interface design commences.

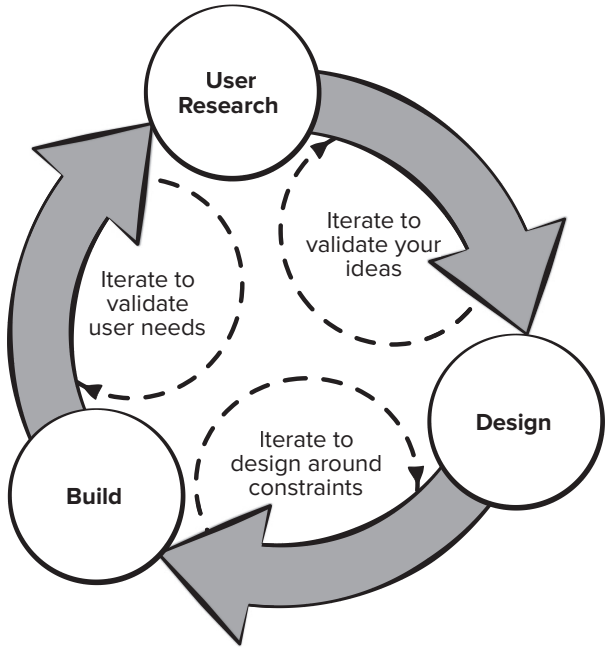
12.5 User Experience Design

As with all iterative processes, there is not a clean division between analysis and design. Figure 12.5 illustrates the practice of cycling between user research and design, as well as cycling between design and construction. The emphasis is on creating incremental prototypes and by testing it with actual users.

Google has defined a 5-day sprint for doing UX design [Kna16], [Goo18], [DXL18]. The steps are outlined below with one day allocated to each:

- **Understand.** Encompasses the user research activities in which the team gathers the information on the problem to be solved (user needs and business goals) for the software product; one way of doing this is to have a series of

FIGURE 12.5
Iterative UX design process



lightning talks (10- to 15-minute presentations) by domain experts on topics such as the business case, competing products, and user profiles. This information is captured on whiteboards (e.g., as customer journey maps, personas, or user task workflow) and remains posted for easy reference throughout the sprint.

- **Sketch.** Individual team members (including all stakeholders) are given the time and space needed to brainstorm solutions to the problems discovered in the understand phase. It is best to do this on paper as quick visual images. Paper drawings and notes are easy to generate, easy to modify, and quite inexpensive. This phase generates lots of ideas because participant ideas are not restricted when creating their solutions.
- **Decide.** Each stakeholder presents his solution sketch, and the team votes to determine the solutions that should be tackled in the prototyping activities that will follow. If there is not a clear consensus following the voting, the development team may decide to consider assumptions that involve constraints posed by the budget, user profiles, available resources (both human and technological), and product business goals.
- **Prototype.** The prototype created during this phase may be a minimally viable product based on the solution selected from the sketch phase, or it may be based on the portions of the customer journey map or storyboard you want to evaluate with potential users in the validate phase. Think of your prototype as an experiment developed to test a hypothesis. That means the team should be developing test cases based on the user stories as the prototype is being built. There is no need to create a fully functional backend for this user interface prototype. It would be best to build a digital prototype using a simple tool (e.g., Keynote[5]). In some cases it may be desirable to create a paper prototype using one of the developers to provide the screen sequences to the users if a prototyping tool is not available.
- **Validate.** Watching users try out your prototype is the best way to discover major issues with its UX design, which in turn lets you start iterating immediately. In a UX design sprint everyone on the development team observes the validation sessions, not just the UX expert or test case designer. This is critical to capturing potential learning opportunities by exposing product decision makers to user feedback in real time. We will talk more about prototype reviews and user testing in Section 12.7.

12.6 User Interface Design

The definition of interface objects and the actions that are applied to them is an important step in interface design. To accomplish this, user scenarios are parsed in much the same way as described in Chapter 9. That is, a use case is written. Nouns (objects) and verbs (actions) are isolated to create a list of screen objects and systems actions.

5 See https://keynotopia.com/.

Once the objects and actions have been defined and elaborated iteratively, they are categorized by type. Target, source, and application objects are identified. A *source object* (e.g., a sensor icon) is dragged and dropped onto a *target object* (e.g., a screen location). The implication of this action is to place a sensor on the room floor plan. An *application object* represents application-specific data that are not directly manipulated as part of screen interaction. For example, the code used to simulate the coverage of the room sensors allow for testing of the sensor placement. The code would be associated with each sensor through some type of logical link when the object is placed on the screen, but the code itself is not dragged and dropped via user interaction.

When you are satisfied that all important objects and actions have been defined (for one design iteration), screen layout is performed. Like other interface design activities, screen layout is an interactive process in which graphical design and placement of icons, definition of descriptive screen text, specification and titling for windows, and definition of major and minor menu items are conducted. If a real-world metaphor is appropriate for the application, it is specified at this time, and the layout is organized in a manner that complements the metaphor.

12.6.1 Applying Interface Design Steps

To provide a brief illustration of how design might proceed to create a prototype of a user interface, let's consider a user scenario for the *SafeHome* system (discussed in earlier chapters). A preliminary user story (written by the homeowner) for the interface follows:

> **Preliminary user story:** I want to gain access to my *SafeHome* system from any remote location via the Internet. Using browser software operating on my mobile device (while I'm at work or traveling), I can determine the status of the alarm system, arm or disarm the system, reconfigure security zones, and view different rooms within the house via preinstalled video cameras.
>
> To access *SafeHome* from a remote location, I provide an identifier and a password. These define levels of access (e.g., all users may not be able to reconfigure the system) and provide security. Once validated, I can check the status of the system and change the status by arming or disarming *SafeHome.* I can reconfigure the system by displaying a floor plan of the house, viewing each of the security sensors, displaying each currently configured zone, and modifying zones as required. I can view the interior of the house via strategically placed video cameras. I can pan and zoom each camera to provide different views of the interior.

Based on this user story, the following homeowner tasks, objects, and data items are identified:

- *Accesses* the *SafeHome* system
- *Enters* an **ID** and **password** to allow remote access
- *Checks* **system status**
- *Arms* or *disarms SafeHome* system
- *Displays* **floor plan** and **sensor locations**
- *Displays* **zones** on floor plan
- *Changes* **zones** on floor plan
- *Displays* **video camera locations** on floor plan

- *Selects* **video camera** for viewing
- *Views* **video images** (four frames per second)
- *Pans* or *zooms* the **video camera**

Objects (boldface) and actions (italics) are extracted from this list of homeowner tasks. The majority of objects noted are application objects. However, **video camera location** (a source object) is dragged and dropped onto **video camera** (a target object) to create a **video image** (a window with video display).

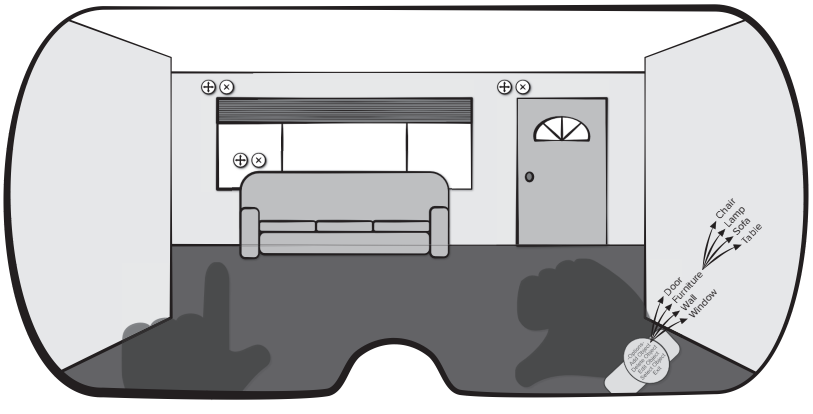
A preliminary sketch of the screen layout for video monitoring is created (Figure 12.6).[6] To invoke the video image, a video camera location icon, *C,* located in the floor plan displayed in the monitoring window is selected. In this case a camera location in the living room (LR) is then dragged and dropped onto the video camera icon in the upper-left-hand portion of the screen. The video image window appears, displaying streaming video from the camera located in the LR. The zoom and pan control slides are used to control the magnification and direction of the video image. To select a view from another camera, the user simply drags and drops a different camera location icon into the camera icon in the upper-left-hand corner of the screen.

The layout sketch shown would have to be supplemented with an expansion of each menu item within the menu bar, indicating what actions are available for the video monitoring mode (state). A complete set of sketches for each homeowner task noted in the user scenario would be created during the interface design.

12.6.2 User Interface Design Patterns

Graphical user interfaces have become so common that a wide variety of user interface design patterns has emerged. A *design pattern* is an abstraction that prescribes a design solution to a specific, well-bounded design problem.

FIGURE 12.6
Preliminary screen layout



6 Note that this differs somewhat from the implementation of these features in earlier chapters. This might be considered a first-draft design based on the new room layout app.

As an example of a commonly encountered interface design problem, consider a situation in which a user must enter one or more calendar dates, sometimes months in advance. There are many possible solutions to this simple problem, and a number of different patterns that might be proposed. Laakso [Laa00] suggests a pattern called **CalendarStrip** that produces a continuous, scrollable calendar in which the current date is highlighted and future dates may be selected by picking them from the calendar. The calendar metaphor is well known to every user and provides an effective mechanism for placing a future date in context.

A vast array of interface design patterns has been proposed over the past few decades.[7] A more detailed discussion of user interface design patterns is presented in Chapter 14. In addition, Punchoojit [Pun17] provides a systematic review of many mobile device user interface design patterns (e.g., zooming, lateral access, revealing information in context, control, and conformation).

12.7 Design Evaluation

Once you create an operational user interface prototype, it must be evaluated to determine whether it meets the needs of the user. Evaluation can span a formality spectrum that ranges from an informal "test drive," in which a user provides impromptu feedback to a formally designed study that uses statistical methods for the evaluation of questionnaires completed by a population of end users.

The user interface evaluation cycle takes the form shown in Figure 12.7. After the design model has been completed, a first-level prototype is created. The prototype is evaluated by the user,[8] who provides you with direct comments about the efficacy of the interface. In addition, if formal evaluation techniques are used (e.g., questionnaires, rating sheets), you can extract information from these data (e.g., 80 percent of all users did not like the mechanism for saving data files). Design modifications are made based on user input, and the next level prototype is created. The evaluation cycle continues until no further modifications to the interface design are necessary. We discuss specialized prototype review and testing techniques for graphical user interfaces in Chapter 21.

12.7.1 Prototype Review

The prototyping approach is effective, but is it possible to evaluate the quality of a user interface before a prototype is built?[9] If you identify and correct potential problems early, the number of loops through the evaluation cycle will be reduced and development time will shorten. If a design model (user stories, storyboard, personas,

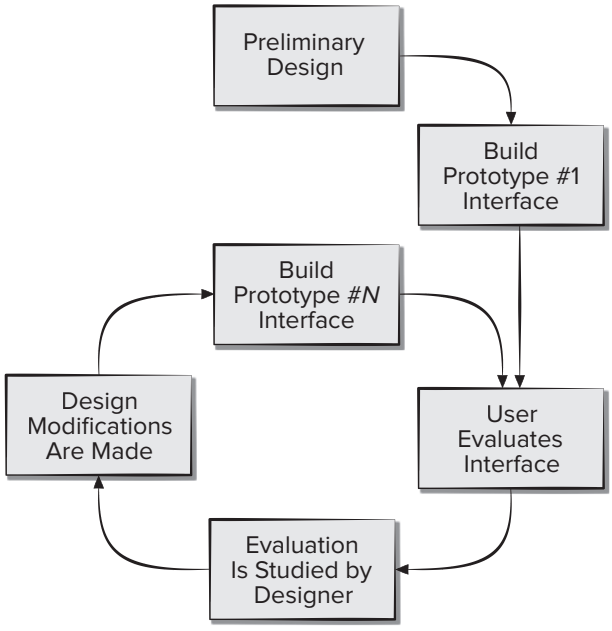
7 Useful suggestions for sites that address user interface patterns can be found at https://www.interaction-design.org/literature/article/10-great-sites-for-ui-design-patterns.

8 It is important to note that experts in ergonomics and interface design may also conduct reviews of the interface. These reviews are called *heuristic evaluations* or *cognitive walk-throughs.*

9 Some software engineers prefer to develop a low-fidelity mockup of the user interface (UI) called a paper prototype to allow stakeholders to test the UI concept before committing any programming resources. The process is described here: http://www.paperprototyping.com/what_examples.html.

FIGURE 12.7

The interface design evaluation cycle



etc.) of the interface has been created, a number of evaluation criteria [Mor81] can be applied during early design reviews:

1. The length and complexity of the requirements model or written specification of the system and its interface provide an indication of the amount of learning required by users of the system.
2. The number of user tasks specified and the average number of actions per task provide an indication of interaction time and the overall efficiency of the system.
3. The number of actions, tasks, and system states indicated by the design model imply the memory load on users of the system.
4. Interface style, help facilities, and error-handling protocol provide a general indication of the complexity of the interface and the degree to which it will be accepted by the user.

SAFEHOME



Interface Design Review

The scene: Doug Miller's office.

The players: Doug Miller, manager of the *SafeHome* software engineering group, and Vinod Raman, a member of the *SafeHome* product software engineering team.

The conversation:

Doug: Vinod, have you and the team had a chance to review the **SafeHomeAssured.com** e-commerce interface prototype?

Vinod: Yeah . . . we all went through it from a technical point of view, and I have a bunch of notes. I e-mailed 'em to Sharon [manager of the WebApp team for the outsourcing vendor for the *SafeHome* e-commerce website] yesterday.

Doug: You and Sharon can get together and discuss the small stuff . . . give me a summary of the important issues.

Vinod: Overall, they've done a good job. Nothing ground breaking, but it's a typical e-commerce interface, decent aesthetics, reasonable layout; they've hit all the important functions . . .

Doug (smiling ruefully): But?

Vinod: Well, there are a few things. . . .

Doug: Such as?

Vinod (showing Doug a sequence of storyboards for the interface prototype): Here's the major functions menu that's displayed on the home page:

Learn about *SafeHome*.

Describe your home.

Get *SafeHome* component recommendations.

Purchase a *SafeHome* system.

Get technical support.

The problem isn't with these functions. They're all okay, but the level of abstraction isn't right.

Doug: They're all major functions, aren't they?

Vinod: They are, but here's the thing . . . you can purchase a system by inputting a list of components . . . no real need to describe the house if you don't want to. I'd suggest only four menu options on the home page:

Learn about *SafeHome*.

Specify the *SafeHome* system you need.

Purchase a *SafeHome* system.

Get technical support.

When you select **Specify the *SafeHome* system you need**, you'll then have the following options:

Select *SafeHome* components.

Get *SafeHome* component recommendations.

If you're a knowledgeable user, you'll select components from a set of categorized pull-down menus for sensors, cameras, control panels, and more. If you need help, you'll ask for a recommendation and that will require that you describe your house. I think it's a bit more logical.

Doug: I agree. Have you talked with Sharon about this?

Vinod: No, I want to discuss this with marketing first; then I'll give her a call.

12.7.2 User Testing

Once the first interactive prototype is built, you can collect a variety of qualitative and quantitative data that will assist in evaluating the interface. To collect qualitative data, questionnaires that allow users to assess the interface prototype can be distributed. If quantitative data are desired, a form of time-study analysis can be conducted. Users are observed during interaction, and data—such as number of tasks correctly completed over a standard time period, frequency of actions, sequence of actions, time spent "looking" at the display, number and types of errors, error recovery time, time spent using help, and number of help references per standard time period—are collected and used as a guide for interface modification.

We discuss the task of testing virtual environments in more detail in Chapter 21. However, a complete discussion of user interface evaluation methods is beyond the scope of this book. For further information, see [Gao14], [Hus15], [Hac98], and [Sto05].

12.8 USABILITY AND ACCESSIBILITY

Every user interface—whether it is designed for the Web, a mobile device, a traditional software application, a consumer product, or an industrial device—should exhibit the usability characteristics that are described in the sidebar on usability. Dix [Dix99] argues that mobile interfaces should answer three primary questions: *Where am I? What can I do now? Where have I been and where can I go?* Answers to these questions allow a user to understand context and navigate more effectively through the app.

INFO



Usability

In an insightful paper on usability, Larry Constantine [Con95] asks a question that has significant bearing on the subject: "What do users want, anyway?" He answers this way:

> What users really want are good tools. All software systems, from operating systems and languages to data entry and decision support applications, are just tools. End users want from the tools we engineer for them much the same as we expect from the tools we use. They want systems that are easy to learn and that help them do their work. They want software that doesn't slow them down, that doesn't trick or confuse them, that does make it easier to make mistakes or harder to finish the job.

Constantine argues that usability is not derived from aesthetics, state-of-the-art interaction mechanisms, or built-in interface intelligence. Rather, it occurs when the architecture of the interface fits the needs of the people who will be using it.

A formal definition of usability is somewhat illusive. Donahue and his colleagues [Don99] define it in the following manner: "Usability is a measure of how well a computer system . . . facilitates learning; helps learners remember what they've learned; reduces the likelihood of errors; enables them to be efficient, and makes them satisfied with the system."

The only way to determine whether "usability" exists within a system you are building is to conduct usability assessment or testing. Watch users interact with the system and answer the following questions [Con95]:

- Is the system usable without continual help or instruction?
- Do the rules of interaction help a knowledgeable user to work efficiently?
- Do interaction mechanisms become more flexible as users become more knowledgeable?
- Has the system been tuned to the physical and social environment in which it will be used?
- Is the user aware of the state of the system? Does the user know where she is at all times?
- Is the interface structured in a logical and consistent manner?
- Are interaction mechanisms, icons, and procedures consistent across the interface?
- Does the interaction anticipate errors and help the user correct them?
- Is the interface tolerant of errors that are made?
- Is the interaction simple?

If each of these questions is answered yes, it is likely that usability has been achieved.

Among the many measurable benefits derived from a usable system are [Don99]: increased sales and customer satisfaction, competitive advantage, better reviews in the media, better word of mouth, reduced support costs, improved end-user productivity, reduced training costs, reduced documentation costs, and reduced likelihood of litigation from unhappy customers.

12.8.1 Usability Guidelines

The user interface of a software product is its "first impression." Regardless of the value of its content, the sophistication of its processing capabilities and services, and the overall benefit of the application itself, a poorly designed interface will disappoint the potential user and may, in fact, cause the user to go elsewhere. Because of the sheer volume of competing Web and mobile apps in virtually every subject area, the interface must "grab" a potential user immediately.

There are, of course, important differences between conventional and mobile apps. By virtue of the physical constraints imposed by small mobile devices (e.g., smartphones), the mobile interface designer must compress interaction in a focused manner. However, the basic principles discussed in this section continue to apply.

Bruce Tognozzi [Tog01] defines a set of fundamental design principles that lead to better usability:[10]

Anticipation. *An application should be designed so that it anticipates the user's next move.* For example, a user has requested a content object that presents information about a printer driver for a new version of an operating system. The designer of the WebApp should anticipate that the user might request a download of the driver and should provide navigation facilities that allow this to happen directly.

Communication. *The interface should communicate the status of any activity initiated by the user.* Communication can be obvious (e.g., a text message) or subtle (e.g., an image of a sheet of paper moving through a printer to indicate that printing is under way).

Consistency. *The use of navigation controls, menus, icons, and aesthetics (e.g., color, shape, layout) should be consistent throughout.* For example, if a mobile app uses a set of four icons (to represent major functions) across the bottom of the display, these icons should appear on every screen and should not be moved to the top of the display. The meaning of the icons should be self-evident within the context of the app.

Controlled Autonomy. *The interface should facilitate user movement throughout the application, but it should do so in a manner that enforces navigation conventions that have been established for the application.* For example, navigation to content requiring controlled access should be controlled by userID and password, and there should be no navigation mechanism that enables a user to circumvent these controls.

Efficiency. *The design of the application and its interface should optimize the user's work efficiency, not the efficiency of the developer who designs and builds it or the client-server environment that executes it.* Tognozzi [Tog01] discusses this when he writes: "This simple truth is why it is so important for everyone . . . to appreciate the importance of making user productivity goal one and to understand the vital difference between building an efficient [application] and empowering an efficient user."

---

10 Tognozzi's original principles have been adapted and extended for use in this book. See [Tog01] for further discussion of these principles.

Flexibility. *The interface should be flexible enough to enable some users to accomplish tasks directly and others to explore the application in a somewhat random fashion.* In every case, it should enable the user to understand where he is and provide the user with functionality that can undo mistakes and retrace poorly chosen navigation paths.

Focus. *The interface (and the content it presents) should stay focused on the user task(s) at hand.* This concept is particularly important for mobile apps that can become very cluttered in the designer's attempts to do too much.

Human Interface Objects. *A vast library of reusable human interface objects has been developed for both Web and mobile apps. Use them.* Any interface object that can be "seen, heard, touched or otherwise perceived" [Tog01] by an end user can be acquired from any one of a number of object libraries.

Latency Reduction. *Rather than making the user wait for some internal operation to complete (e.g., downloading a complex graphical image), the application should use multitasking in a way that lets the user proceed with work as if the operation has been completed.* In addition to reducing latency, delays must be acknowledged so that the user understands what is happening. This includes (1) providing audio feedback when a selection does not result in an immediate action by the application, (2) displaying an animated clock or progress bar to indicate that processing is under way, and (3) providing some entertainment (e.g., an animation or text presentation) while lengthy processing occurs.

Learnability. *An application interface should be designed to minimize learning time and, once learned, to minimize relearning required when the app is revisited.* In general, the interface should emphasize a simple, intuitive design that organizes content and functionality into categories that are obvious to the user.

Metaphors. *An interface that uses an interaction metaphor is easier to learn and easier to use, as long as the metaphor is appropriate for the application and the user.* Metaphors are an excellent idea because they mirror real-world experience. Just be sure that the metaphor you choose is well known to end users. A metaphor should call on images and concepts from the user's experience, but it does not need to be an exact reproduction of a real-world experience.

Readability. *All information presented through the interface should be readable by young and old.* The interface designer should emphasize readable type styles, user-controllable font sizes, and color background choices that enhance contrast.

Track State. *When appropriate, the state of the user interaction should be tracked and stored so that a user can log off and return later to pick up where he left off.* In general, cookies can be designed to store state information. However, cookies are a controversial technology, and other design solutions may be more palatable for some users.

Visible Navigation. *A well-designed interface provides "the illusion that users are in the same place, with the work brought to them"* [Tog01]. When this approach is used, navigation is not a user concern. Rather, the user retrieves the content object and selects functions that are displayed and executed through the interface.

Nielsen and Wagner [Nie96] suggest a few pragmatic "don'ts" for interface design (based on their redesign of a major WebApp). These provide a nice complement to the principles suggested earlier in this section.

- Don't force the user to read voluminous amounts of text, particularly when the text explains the operation of the WebApp or assists in navigation.
- Don't make users scroll unless it is absolutely unavoidable.
- Don't rely on browser functions to assist in navigation.
- Don't allow aesthetics to supersede functionality.
- Don't force the user to search the display to determine how to link to other content or services.

A well-designed interface improves the user's perception of the content or services provided by the site. It need not necessarily be flashy, but it should always be well structured and ergonomically sound. Additional advice on usability evaluation can be found in [Gao14] and [Hus15}.

12.8.2 Accessibility Guidelines

As the design of a user interface evolves, four common design issues almost always surface: system response time, user help facilities, error information handling, and command labeling. All these can lead to accessibility issues for all users, not just those with special needs. Unfortunately, many designers do not address these issues until relatively late in the design process (sometimes the first inkling of a problem doesn't occur until an operational prototype is available). Unnecessary iteration, project delays, and end-user frustration often result. It is far better to establish each as a design issue to be considered at the beginning of software design, when changes are easy and costs are low.

Application Accessibility. As computing applications become ubiquitous, software engineers must ensure that interface design encompasses mechanisms that enable easy access for those with special needs. *Accessibility* for users (and software engineers) that may be physically challenged is an imperative for ethical, legal, and business reasons. A variety of accessibility guidelines (e.g., [W3C18])—many designed for Web applications but often applicable to all types of software—provide detailed suggestions for designing interfaces that achieve varying levels of accessibility. Others (e.g., [App13], [Mic13a], and [Zan18]) provide specific guidelines for "assistive technology" that addresses the needs of those with visual, hearing, mobility, speech, and learning impairments.

Response Time. System response time has two important characteristics: length and variability. If system response is too long, user frustration and stress are inevitable. *Variability* refers to the deviation from average response time, and in many ways, it is the most important response time characteristic. Low variability enables the user to establish an interaction rhythm, even if response time is relatively long. For example, a 1-second response to a command will often be preferable to a response that varies from 0.1 to 2.5 seconds. When variability is significant, the user is always off balance, always wondering whether something "different" has occurred behind the scenes.

Help Facilities. Almost every user of an interactive, computer-based system requires help now and then. Modern software should provide online help facilities that enable a user to get a question answered or resolve a problem without leaving the interface.

Error Handling. In general, every error message or warning produced by an interactive system should have the following characteristics:

1. Describes the problem in jargon that the user can understand
2. Provides constructive advice for recovering from the error
3. Indicates any negative consequences of the error (e.g., potentially corrupted data files) so that the user can check to ensure that they have not occurred (or correct them if they have)
4. Be accompanied by an audible or visual cue
5. Should never place blame for the error on the user

Menu and Command Labeling. The typed command was once the most common mode of interaction between user and system software and was commonly used for applications of every type. Today, the use of window-oriented, point-and-pick interfaces has reduced reliance on typed commands, but some power-users continue to prefer a command-oriented mode of interaction. A number of design issues arise when typed commands or menu labels are provided as a mode of interaction:

- Will every menu option have a corresponding command?
- What form will commands take? Options include a control sequence (e.g., alt-P), function keys, or a typed word.
- How difficult will it be to learn and remember the commands? What can be done if a command is forgotten?
- Can commands be customized or abbreviated by the user?
- Are menu labels self-explanatory within the context of the interface?
- Are submenus consistent with the function implied by a master menu item?
- Have appropriate conventions for command usage been established across a family of applications?

Internationalization. Software engineers and their managers invariably underestimate the effort and skills required to create user interfaces that accommodate the needs of different locales and languages. Too often, interfaces are designed for one locale and language and then jury-rigged to work in other countries. The challenge for interface designers is to create "globalized" software. That is, user interfaces should be designed to accommodate a generic core of functionality that can be delivered to all who use the software. *Localization* features enable the interface to be customized for a specific market.

A variety of internationalization guidelines (e.g., [IBM13]) are available to software engineers. These guidelines address broad design issues (e.g., screen layouts may differ in various markets) and discrete implementation issues (e.g., different alphabets may create specialized labeling and spacing requirements). The *Unicode* standard

[Uni03] has been developed to address the daunting challenge of managing dozens of natural languages with hundreds of characters and symbols.

12.9 Conventional Software UX and Mobility

Earlier in this chapter, we noted that all user interface design begins with the identification of user, task, and environmental requirements. Once user tasks have been identified, user scenarios (use cases) are created and analyzed to define a set of interface objects and actions.

Information contained within the requirements model forms the basis for the creation of a screen layout that depicts graphical design and placement of icons, definition of descriptive screen text, specification and titling for windows, and specification of major and minor menu items. Tools are then used to prototype and ultimately implement the interface design model.

When designing for mobility, developers need to pay more attention to differences in screen sizes and user interaction devices. Mobile users of a software product are more likely to expect the product to be easily customized to their preferences and to take advantage of changes in the user's location when they are actively using the app. We will focus on designing for mobile devices in Chapter 13.

12.10 Summary

The user interface is arguably the most important element of a computer-based system or product. If the interface is poorly designed, the user's ability to tap the computational power and informational content of an application may be severely hindered. In fact, a weak interface may cause an otherwise well-designed and solidly implemented application to fail.

Three important principles guide the design of effective user interfaces: (1) place the user in control, (2) reduce the user's memory load, and (3) make the interface consistent. To achieve an interface that abides by these principles, an organized design process must be conducted.

The development of a user interface begins with a series of analysis tasks. User analysis defines personas for the profiles of various end users and is gathered from a variety of business and technical sources. The user analysis allows the developers to create a customer journey map as a visual representation of the goals for the product. Task analysis defines user tasks and actions using either an elaborative or object-oriented approach, applying use cases, task and object elaboration, workflow analysis, and hierarchical task representations to fully understand the human-computer interaction. Environmental analysis identifies the physical and social structures in which the interface must operate.

After analyzing usage scenarios, interface objects and actions are created and provide a basis for the creation of a screen layout that depicts graphical design and placement of icons, definition of descriptive screen text, specification and titling for windows, and specification of major and minor menu items. A storyboard may be created to illustrate the navigation through screens developed for the product to

accomplish specific user tasks. Design issues such as response time, command and action structure, error handling, and help facilities are considered as the design model is refined. A variety of implementation tools are used to build a prototype for evaluation by the user.

Like interface design for conventional software, the design mobile app interface describes the structure and organization of the user interface and includes a representation of screen layout, a definition of the modes of interaction, and a description of navigation mechanisms. A set of interface design principles and an interface design workflow guide the mobile app designer when layout and interface control mechanisms are designed.

The user interface is the window into the software. In many cases, the interface molds a user's perception of the quality of the system. If the "window" is smudged, wavy, or broken, the user may reject an otherwise powerful computer-based system. Usability and accessibility issues in the interface may also cause users to find an alternative product that better meets their needs and expectations.

PROBLEMS AND POINTS TO PONDER

12.1. Describe the worst interface that you have ever worked with and critique it relative to the concepts introduced in this chapter. Describe the best interface that you have ever worked with and critique it relative to the concepts introduced in this chapter.

12.2. Consider one of the following interactive applications (or an application assigned by your instructor):

a. A desktop publishing system
b. A computer-aided design system
c. An interior design system (as described in Section 12.4.2)
d. An automated course registration system for a university
e. A library management system
f. An Internet-based polling booth for public elections
g. A home banking system
h. An interactive application assigned by your instructor

Develop a user model, design model, mental model, and an implementation model, for any one of these systems.

12.3. Perform a detailed task analysis for any one of the systems listed in Problem 12.2.

12.4. Create a customer journey map for one of the systems listed in Problem 12.2.

12.5. Continuing Problem 12.2, define interface objects and actions for the application you have chosen. Identify each object type.

12.6. Develop a set of screen layouts and organize them into a storyboard for the system you chose in Problem 12.2.

12.7. Use a prototyping tool like Keynote to create an interactive prototype for the storyboard you created in Problem 12.6.

12.8. Describe your approach to user help facilities for the task analysis design model and task analysis you performed as part of Problems 12.3, 12.4, and 12.5.

12.9. Provide a few examples that illustrate why response time variability can be an issue.

12.10. Develop an approach that would automatically integrate error messages and a user help facility. That is, the system would automatically recognize the error type and provide a help window with suggestions for correcting it. Perform a reasonably complete software design that considers appropriate data structures and algorithms.

12.11. Develop an interface evaluation questionnaire that contains 20 generic questions that would apply to most interfaces. Have 10 classmates complete the questionnaire for an interactive system that you all use. Summarize the results, and report them to your class.

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

13 Design for Mobility

Mobile devices—smartphones, tablets, wearable devices, handheld gaming devices, and other specialized products—have become the new face of computing. According to Pew Research Center [Pew18], in the United States 77 percent of people own a smartphone and 50 percent of people own a tablet computer of some kind. Mobile computing has become a dominant force.

Key Concepts

aesthetic design 277
architecture design 278
challenges 265
cloud computing 273
component-level design 282
content architecture 279
content design 277
content objects 277
context-aware apps 274
design
  best practices 285
  mistakes 272
  pyramid 275
  quality 282
graphic design 277
mobile architectures 273
mobile development life cycle 268
model-view-controller 279
navigation design 280
quality checklist 285
technical considerations 266
user interface design 270
WebApp architecture 279

Quick Look

What is it? Mobile design encompasses technical and nontechnical activities that include: establishing the look and feel of the mobile application (including mobile apps, WebApps, virtual reality, and games), creating the aesthetic layout of the user interface, establishing the rhythm of user interaction, defining the overall architectural structure, developing the content and functionality that reside within the architecture, and planning the navigation that occurs within the mobile product.

Who does it? Software engineers, graphic designers, content developers, security specialists, and other stakeholders all participate in the creation of a mobile design model.

Why is it important? Design allows you to create a model that can be assessed for quality and improved before content and code are generated, tests are conducted, and end users become involved in large numbers. Design is the place where mobile app quality is established.

What are the steps? Mobile design encompasses six major steps that are driven by information obtained during requirements modeling and are described in this chapter.

What is the work product? A design model that encompasses content, aesthetics, architecture, interface, navigation, and component-level design issues is the primary work product that is produced during mobile design.

How do I ensure that I've done it right? Each element of the design model is reviewed in an effort to uncover errors, inconsistencies, or omissions. In addition, alternative solutions are considered, and the degree to which the current design model will lead to effective implementation on a variety of software platforms and devices is also assessed.

In his authoritative book on Web design, Jakob Nielsen [Nie00] states: "There are essentially two basic approaches to design: the artistic ideal of expressing yourself and the engineering ideal of solving a problem for a customer." During the first decade of mobile development, the artistic idea was the approach that many developers chose. Design occurred in an ad hoc manner and was usually conducted as HTML was generated. Design evolved out of an artistic vision that evolved as Web page construction occurred.

Even today, many developers use mobile apps as poster children for "limited design." They argue that immediacy and volatility of the mobile market mitigate against formal design; that design evolves as an application is built (coded), and that relatively little time should be spent on creating a detailed design model. This argument has merit, but only for relatively simple apps. When content and function are complex; when the size of the mobile app encompasses hundreds or thousands of content objects, functions, and analysis classes; and when the success of the app will have a direct impact on the success of the business, design cannot and should not be taken lightly. This reality leads us to Nielsen's second approach—"the engineering ideal of solving a problem for a customer."

13.1 THE CHALLENGES

Although mobile devices have many features in common with each other, their users often have very different perceptions of what features they expect to be bundled in each. Some users expect the same features that are provided on their personal computers. Others focus on the freedom that portable devices give them and gladly accept the reduced functionality in the mobile version of a familiar software product. Still others expect unique experiences not possible on traditional computing or entertainment devices. The user's perception of "goodness" might be more important than any of the technical quality dimensions of the mobile product itself.

13.1.1 Development Considerations

Like all computing devices, mobile platforms are differentiated by the software they deliver—a combination of operating system (e.g., Android or iOS) and a small subset of the hundreds of thousands of mobile software products that provide a very wide range of functionality. New tools allow individuals with little formal training to create and sell apps alongside other apps developed by large teams of software developers.

Even though apps can be developed by amateurs, many software engineers think that MobileApps are among the most challenging software systems being built today [Voa12]. Mobile platforms are very complex. Both the Android and iOS operating systems contain over 12 million lines of code. Mobile devices often have mini browsers that will not display the full set of content available on a Web page. Different mobile devices use different operating systems and platform-dependent development environments. Mobile devices tend to have smaller screen sizes and more varied screen sizes than personal computers. This may require greater attention to user interface design issues, including decisions to limit display of some content. MobileApps

must be designed to take intermittent connectivity outages into account, limitations on battery life, and other device constraints[1] [Whi08].

System components in mobile computing environments are likely to change their locations while their apps are running. To maintain connectivity in nomadic networks,[2] coordination mechanisms for discovering devices, exchanging information, maintaining security and communication integrity, and synchronizing actions must be developed. There is always a trade-off between security and other elements of the mobile product design.

In addition, software engineers must identify the proper design trade-offs between the expressive power of the MobileApp and stakeholder security concerns. Developers must seek to discover algorithms (or adapt existing algorithms) that are energy efficient to conserve battery power when possible. Middleware may have to be created to allow different types of mobile devices to communicate with each other in the same mobile networks [Gru00].

Software engineers should craft a user experience that takes advantage of device characteristics and context-aware applications. The nonfunctional requirements (e.g., security, performance, usability) are a bit different from those for either WebApps or desktop software applications. There is always a trade-off between security and other elements of the mobile design. Testing mobile software products (Chapter 21) provides additional challenges because users expect that they will work in a large number of physically different environments. Portability is another challenge for software engineers as there are several popular device platforms. It is expensive to develop and support applications for multiple device platforms [Was10].

13.1.2 Technical Considerations

The low cost of adding Web capabilities to everyday devices such as phones, cameras, and TVs is transforming the way people access information and use network services [Sch11]. Among the many technical considerations that MobileApps should address are the following:

Multiple hardware and software platforms. It is not at all unusual for a mobile product to run on many different platforms (both mobile and stationary) with a range of differing levels of functionality. The reasons for these differences are in part because the hardware and software available are quite different from device to device. This increases both development cost and time. It also can make configuration management (Chapter 22) more difficult.

Many development frameworks and programming languages. Mobile products are currently being written in several distinct programming or scripting languages (e.g., HTML5, JavaScript, Java, Swift, and C#) for a multitude of popular development frameworks (e.g., Android, iOS, Xamarin, Windows, AngularJS). Very few mobile devices allow direct development on a device itself. Instead, mobile developers typically use emulators running on desktop

1 Available at http://www.devx.com/SpecialReports/Article/37693.

2 *Nomadic networks* have changing connections to mobile devices or servers.

development systems. These emulators may or may not accurately reflect the limitations of the device itself. Thin-client applications are often easier to port to multiple devices than applications designed to run exclusively on the mobile device.

Many app stores with different rules and tools. Each mobile platform has its own app store and its own standards for accepting apps (e.g., Apple,[3] Google,[4] Microsoft,[5] and Amazon[6] publish their own standards). Development of a mobile product for multiple platforms must proceed separately, and each version of the app needs its own standards expert.

Very short development cycles. The marketplace for mobile products is very competitive, and software engineers are likely to make use of agile development processes when building MobileApps in an effort to reduce development time [Was10].

User interface limitations and complexities of interaction with sensors and cameras. Mobile devices have smaller screen sizes than personal computers and a richer set of interaction possibilities (touch, gesture, camera, etc.) and usage scenarios based on context awareness. The style and appearance of the user interface is often dictated by the nature of platform-specific development tools [Rot02]. Allowing smart devices to interact with smart spaces offers the potential to create personalized, networked, high-fidelity application platforms such as those seen by merging smartphones and car infotainment systems.[7]

Effective use of context. Users expect MobileApps to deliver personalized user experiences based on the physical location of a device in relation to the available network features. User interface design and context-aware applications are discussed in greater detail in Section 13.4.

Power management. Battery life is often one of the most limiting constraints on many mobile devices. Backlighting, reading and writing to memory, using wireless connections, making use of specialized hardware, and processor speed all impact power usage and need to be considered by software developers [Mei09].

Security and privacy models and policies. Wireless communication is difficult to protect from eavesdropping. Preventing *man-in-the-middle-attacks*[8] in automotive applications can be critical to the safety of the users [Bos11]. Data stored on a mobile device are subject to theft if a device is lost or a malicious app is downloaded. Software policies that increase the level of confidence in the security and privacy of a MobileApp often reduce the usability of the app and the spontaneity of the communication among users [Rot02].

3 https://developer.apple.com/appstore/guidelines.html.
4 http://developer.android.com/distribute/googleplay/publish/preparing.html.
5 http://msdn.microsoft.com/en-us/library/ff941089%28v=vs.92%29.aspx.
6 https://developer.amazon.com/apps-and-games/app-submission/android.
7 When used in an automotive setting, smart devices should be able to restrict access to services that may distract the driver and allow hands-free operation when a vehicle is moving [Bos11].
8 These attacks involve a third party intercepting communications between two trusted sources and impersonating one or both of the parties.

Computational and storage limitations. There is great interest in using mobile devices to control home environmental and security services. When MobileApps are allowed to interact with devices and services in their environment, it is easy to overwhelm the mobile device (storage, processing speed, power consumed) with the sheer volume of information [Spa11]. Developers may need to look for programming shortcuts and means of reducing the demands made on processor and memory resources.

Applications that depend on external services. Building thin mobile clients suggests the need to rely on Web service providers and cloud storage facilities. This increases concerns for both data or service accessibility and security [Rot02].

Testing complexity. Mobile products that run entirely on the device can be tested using traditional software testing methods (Chapters 19 and 20) or using emulators running on personal computers. Thin-client MobileApps are particularly challenging to test. They exhibit many of the same testing challenges found in WebApps, but they have the additional concerns associated with transmission of data through Internet gateways and telephone networks [Was10]. Testing of mobile software products will be discussed in Chapter 21.

13.2 Mobile Development Life Cycle

Burns [Bur16] and her Microsoft colleagues describe a recommendation for an iterative mobile SDLC that contains five major stages:

Inception. Goals, features, and functions of the mobile product are identified to determine the scope and the size of the first increment or feasibility prototype. Developers and stakeholders must be conscious of human, social, cultural, and organizational activities that may reveal hidden aspects of the users' needs and affect the business targets and functionality of the proposed mobile product.

Design. The design includes architectural design, navigation design, interface design, content design. Developers define the app user experience using screen mockups and paper prototypes to help create a proper user interface design that will take different screen sizes and capabilities into account as well as the capabilities of each targeted platform.

Development. Mobile software is coded, functional and nonfunctional. Test cases are created and executed, and usability and accessibility evaluations are conducted as the product evolves.

Stabilization. Most mobile products go through a series of prototypes: feasibility prototype, intended as a proof of concept with perhaps only one complete logic path through the application; alpha prototype, which contains the functionality for minimum viable product; beta prototype, which is largely complete and contains most tested functionality; and lastly the release candidate, which contains all required functionality, for which all scheduled tests have been completed, and which is ready for review by the product owner.

Deployment. Once stabilized, a mobile product is reviewed by a commercial app store and made available for sale and download. For apps intended for internal company use only, a product owner review may be all that is required before deployment.

Mobile development makes use of an agile, spiral engineering process model. The stages are not completed in order like they would be if mobile development was done using the waterfall model. The stages described above are visited repeatedly as developers and stakeholders gain better understanding of the user needs and product business goals.

SAFEHOME



Formulating Mobile Device Requirements

The scene: A meeting room. The first meeting to identify requirements for a mobile version of the *SafeHome* WebApp.

The players: Jamie Lazar, software team member; Vinod Raman, software team member; Ed Robbins, software team member; Doug Miller, software engineering manager; three members of marketing; a product engineering representative; and a facilitator.

The conversation:

Facilitator (pointing at whiteboard): So that's the current list of objects and services for the home security function present in the WebApp.

Vinod (interrupting): My understanding is that people want *SafeHome* functionality to be accessible from mobile devices as well . . . including the home security function?

Marketing person: Yes, that's right . . . we'll have to add that functionality and try to make it context aware to help personalize the user experience.

Facilitator: Context aware in what sense?

Marketing person: People might want to use a smartphone instead of the control panel and avoid logging on to a website when they are in the driveway at home. Or they might not want all family members to have access to the master control dashboard for the system from their phones.

Facilitator: Do you have specific mobile devices in mind?

Marketing person: Well, all smartphones would be nice. We will have a Web version done, so won't the MobileApp run on all of them?

Jamie: Not quite. If we took a mobile phone browser approach, we might be able to reuse a lot of our WebApp functionality. But remember, smartphone screen sizes vary, and they may or may not all have the same touch capabilities. So, at the very least we would have to create a mobile website that takes the features of each device into account.

Ed: Perhaps we should build the mobile version of the website first.

Marketing person: OK, but a mobile website solution wasn't what we had in mind.

Vinod: Each mobile platform seems to have its own unique development environment, too.

Production rep: Can we restrict MobileApp development to only one or two types of smartphones?

Marketing person: I think that might work. Unless I'm mistaken, the smartphone market is dominated by two smartphone platforms right now.

Jamie: There's also security to worry about. We better make sure an outsider can't hack into the system, disarm it, and rob the place or worse. Also, a phone could get lost or stolen more easily than a laptop.

Doug: Very true.

Marketing: But we still need the same level of security . . . just also be sure to stop an outsider from getting in with a stolen phone.

Ed: That's easier said than done and . . .

Facilitator (interrupting): Let's not worry about those details yet.

(Doug, serving as the recorder for the meeting, makes an appropriate note.)

Facilitator: As a starting point, can we identify which elements of WebApp security function are needed in the MobileApp and which will need to be newly created? Then we can decide how many mobile platforms we can support and when we can move forward on this project.

(The group spends the next 20 minutes refining and expanding the details of the home security function.)

13.2.1 User Interface Design

Mobile device users expect that minimal learning time will be required to master a MobileApp. To achieve this, MobileApp designers use consistent icon representations and placement across multiple platforms. In addition, designers must be sensitive to the user's expectation of privacy with regard to the display of personal information on the screen of the mobile device. Touch and gesture interfaces, along with sophisticated voice input and facial recognition, are maturing rapidly [Shu12] and have already become part of the user interface designer's toolbox.

Legal and ethical pressure to provide for access by all persons suggests that mobile device interfaces need to account for brand differences, cultural differences, differences in computing experience, elderly users, and users with disabilities (e.g., visual, aural, mobility). The effects of poor usability may mean that users cannot complete their tasks or will not be satisfied with the results. This suggests the importance of user-centered design activities in each of the usability areas (user interface, external accessory interface, and service interface). Accessibility is an important design issue and must be considered when user-centered design is applied.

In trying to meet stakeholder usability expectations, MobileApp developers should attempt to answer these questions to assess the out-of-the-box readiness of the device:

- Is the user interface consistent across applications?
- Is the device interoperable with different network services?
- Is the device acceptable in terms of stakeholder values[9] in the target market area?

Eisenstein [Eis01] claims that the use of abstract, platform-neutral models to describe a user interface greatly facilitates the development of consistent, usable multiplatform user interfaces for mobile devices. Three models in particular are useful. A *platform model* describes the constraints imposed by each platform to be supported. A *presentation model* describes the appearance of the user interface. The *task model* is a structured representation of the tasks a user needs to perform to meet her task goals. In the best case, model-based design (Chapter 9) involves the creation of databases that contain the

9 Brand, ethical preferences, moral preferences, cognitive beliefs.

models and has tool support for generating user interfaces for multiple devices automatically. Utilizing model-based design techniques can also help designers recognize and accommodate the unique contexts and context changes that are present in mobile computing. Without an abstract description of a user interface, the development of mobile user interfaces can be error prone and time consuming.

INFO



Mobile User Interface Design Considerations

Design choices affect performance and should be examined early in the user interface design process. Ivo Weevers [Wee11] posted several mobile user–interface design practices that have proven to be helpful when designing mobile applications:

- **Define user interface brand signatures.** Differentiate the app from its competitors. Make the core signature elements of the brand the most responsive, because users will use them over and over.
- **Focus the portfolio of products.** Target the platforms that are most important to the success of the app and the company. Not all platforms have the same number of users.
- **Identify the core user stories.** Make use of techniques that require stakeholders to prioritize their needs as a way to reduce a lengthy list of requirements and to consider the constrained resources available on mobile devices.
- **Optimize user interface flows and elements.** Users do not like to wait. Identify potential bottlenecks in user work flow and make sure the user is given an indication of progress when delays occur. Make sure that the time to display screen elements is justified in terms of user benefits.
- **Define scaling rules.** Determine the options that will be used when information to be displayed is too large to fit on the screen. Managing functionality, aesthetics, usability, and performance is a continual balancing act.
- **User performance dashboard.** The dashboard is used to communicate the product's current state of completion (e.g., number of use stories implemented), its performance relative to its targets, and perhaps comparisons to its competitors.
- **Champion-dedicated user interface engineering skills.** It is important to understand that the implementation of layout, graphics, and animation has performance implications. Techniques to interleave rendering of display items and program execution can be helpful.

13.2.2 Lessons Learned

de Sá and Carriço [Des08] contend that there are important differences between developing conventional software and developing mobile applications. Software engineers cannot continue to use the same conventional techniques they have used and expect them to be successful. They suggest three approaches for the design of mobile applications:

Usage Scenarios. Described in Chapter 12, usage scenarios must consider context variables (location, user, and device) and transitions between contextual scenarios (e.g., user moves from bedroom to kitchen or switches from stylus to a finger). de Sá and Carriço have identified a set of scenario-variable types that should be considered in developing the user scenarios—locations and settings, movement and posture, devices and usages, workloads and distractions, user preferences.

Ethnographic Observation.[10] This is a widely used method for gathering information about representative users of a software product as it is being designed. It is often difficult to observe users as they change contexts, because the observer must follow users for long periods of time, something that could raise privacy concerns.[11] A complicating factor is that users seem to complete tasks differently in private settings than in social settings. The same users may need to be observed performing tasks in multiple contexts while monitoring transitions, as well as recording user reactions to the changes.

Low-Fidelity Paper Prototypes (e.g., cards or Post-it notes). This is a cost-effective usability assessment approach in user interface design that can be used before any programming takes place. It is important for these prototypes to be similar in size and weight and for their use to be allowed in a variety of contexts. It is also important that the sketches or text displays be true to size and for the final product to be of high quality. Placement and size of user interface widgets (e.g., buttons or scrollbars) must be designed so that they will not disappear when users extend their screens by zooming. The interaction type (e.g., stylus, joy stick, touch screen) needs to be emulated in the low-fidelity prototype (e.g., colored pen or push pin) to check placement and ease of use. Later prototypes may then be created to run on the targeted mobile devices once the layout and placement issues have been resolved.

INFO



MobileApp Design Mistakes

Joh Koester [Koe12] posts several examples of mobile design practices that should be avoided:

- **Kitchen sink.** Avoid adding too many features to the app and too many widgets on the screen. Simple is understandable. Simple is marketable.
- **Inconsistency.** To avoid this, set standards for page navigation, menu use, buttons, tabs, and other user-interface elements. Stick to a uniform look and feel.
- **Overdesigning.** Be ruthless when designing apps. Remove unnecessary elements and wasteful graphics. Do not be tempted to add elements just because you think you should.
- **Lack of speed.** Users do not care about device constraints—they want to view things quickly. Preload what you can. Eliminate what is not needed.
- **Verbiage.** Unnecessarily long, wordy menus and screen displays are indications of a mobile product that has not been tested with users and developers who have not spent enough time understanding the user's task.
- **Nonstandard interaction.** One reason for targeting a platform is to take advantage of the user's experience with the way things are done on that platform. Where standards exist use them. This needs to be balanced with the need to have an application appear and behave the same way on multiple devices when possible.
- **Help-and-FAQ-itis.** Adding online help is not the way to repair a poorly designed user interface. Make sure you have tested your app with your targeted users and repaired the identified defects.

10 *Ethnographic observation* is a means determining the nature of user tasks by watching users in their work environment.

11 Asking users to fill out anonymous questionnaires may have to suffice when direct observation is not possible.

13.3 MOBILE ARCHITECTURES

Services computing[12] and cloud computing[13] enable the rapid development of large-scale distributed applications based on innovative architectural designs [Yau11]. These two computing paradigms have made it easier and more economical to create applications on many different mobile devices (e.g., laptop computers, smartphones, and tablets). These two paradigms allow resource outsourcing and transfer of information technology management to service providers while at the same time mitigating the impact of resource limitations on some mobile devices. A service-oriented architecture provides the architectural style (e.g., REST),[14] standard protocols (e.g., XML[15] and SOAP[16]), and interfaces (e.g., WSDL)[17] needed for mobile development. Cloud computing enables convenient, on-demand network access to a shared pool of configurable computing resources (servers, storage, applications, and services).

Service computing allows mobile developers to avoid the need to integrate service source code into the client running on a mobile device. Instead, the service runs out of the provider's server and is loosely coupled with the applications that make use of it through messaging protocols. A service typically provides an application programming interface (API) to allow it to be treated like an abstract black box.

Cloud computing lets the client (either a user or program) request computing capabilities as needed, across network boundaries anywhere or any time. The cloud architecture has three layers, each of which can be called as a service (Figure 13.1). The *software as service* layer consists of software components and applications hosted by third-party service providers. The *platform as service* layer provides a collaborative development platform to assist with design, implementation, and testing by geographically distributed team members. *Infrastructure as a service* provides virtual computing resources (storage, processing power, network connectivity) on the cloud.

Mobile devices can access cloud services from any location at any time. The risks of identity theft and service hijacking require providers of mobile services and cloud computing to employ rigorous security engineering techniques (Chapter 18) to protect their users.

Taivalsaari [Tai12] points out that making use of cloud storage can allow any mobile device or software features to be updated easily on millions of devices worldwide. In fact, it is possible to virtualize the entire mobile user experience so that all applications are downloaded from the cloud.

12 *Services computing* focuses on architectural design and enables application development through service discovery and composition.

13 *Cloud computing* focuses on the effective delivery of services to users through flexible and scalable resource virtualization and loading balancing.

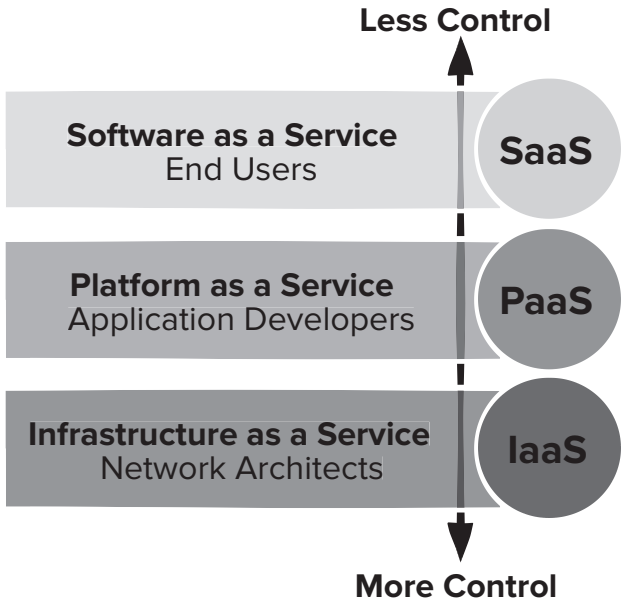
14 *Representation State Transfer* describes a networked Web architectural style where the resource representation (e.g., a Web page) places the client in a new state. The client changes or transfers state with each resource representation.

15 *Extensible Markup Language* (XML) is designed to store and transport data, while HTML is designed to display data.

16 *Simple Object Access Protocol* is a specification for exchanging structured information in the implementation of Web services in computer networks.

17 *Web Services Description Language* is an XML-based language for describing Web services and how to access them.

FIGURE 13.1
Cloud computing layers



13.4 CONTEXT-AWARE APPS

Context allows the creation of new applications based on the location of the mobile device and the functionality to be delivered by the device. Context can also help tailor personal computer applications for mobile devices (e.g., downloading patient information to a device carried by a home health care worker as he arrives at the patient's house).

Using highly adaptive, contextual interfaces is a good way to deal with device limitations (e.g., screen size and memory). To facilitate the development of context-aware user interaction requires the support of corresponding software architectures.

In an early discussion of context-aware applications, Rodden [Rod98] points out that mobile computing merges the real and virtual world by providing functionality that allows a device to be aware of its location, time, and other objects in its surroundings. The device could be in a fixed location like an alarm sensor, embedded in an autonomous device, or be carried around by a human. Because the device can be designed to be used by individuals, groups, or the public, it must detect the presence and identity of the user, as well as the attributes of the context that are relevant or permitted for that user (even if the user is another device).

To achieve context awareness, mobile systems must produce reliable information in the presence of uncertain and rapidly changing data from a variety of heterogeneous sources. Extracting relevant context information by combing data from several sensors proves challenging because of problems with noise, miscalibration, wear and tear, and weather. Event-based communication is preferable to the management of continuous streams of high-abstraction-level data in context-aware applications [Kor03].

In ubiquitous computing environments, multiple users work with a wide range of different devices. The configuration of the devices should be flexible enough to change frequently because of the demands of mobile work practices. It is important for the software infrastructure to support different styles of interaction (e.g., gestures, voice, and pen) and store them in abstractions that can be shared easily.

There are times when one user may desire to work with more than one device simultaneously on the same product (e.g., use a touch-screen device to edit a document image and a personal keyboard to edit document text). It is challenging to integrate mobile devices that are not always connected to the network and have a variety of device constraints [Tan01]. Networked, multiplayer games have had to deal with these problems by storing the game state on each device and sharing change information among other game players' devices in real time.

13.5 WEB DESIGN PYRAMID

What is design in the context of Web engineering? This simple question is more difficult to answer than one might believe. Pressman and Lowe [Pre08] discuss this when they write:

> The creation of an effective design will typically require a diverse set of skills. Sometimes, for small projects, a single developer may need to be multi-skilled. For larger projects, it may be advisable and/or feasible to draw on the expertise of specialists: Web engineers, graphic designers, content developers, programmers, database specialists, information architects, network engineers, security experts, and testers. Drawing on these diverse skills allows the creation of a model that can be assessed for quality and improved *before* content and code are generated, tests are conducted, and end-users become involved in large numbers. If analysis is where *WebApp quality is established,* then design is where the *quality is truly embedded.*

The appropriate mix of design skills will vary depending upon the nature of the WebApp. Figure 13.2 depicts a design pyramid for WebApps. Each level of the pyramid represents a design action that is described in the sections that follow.

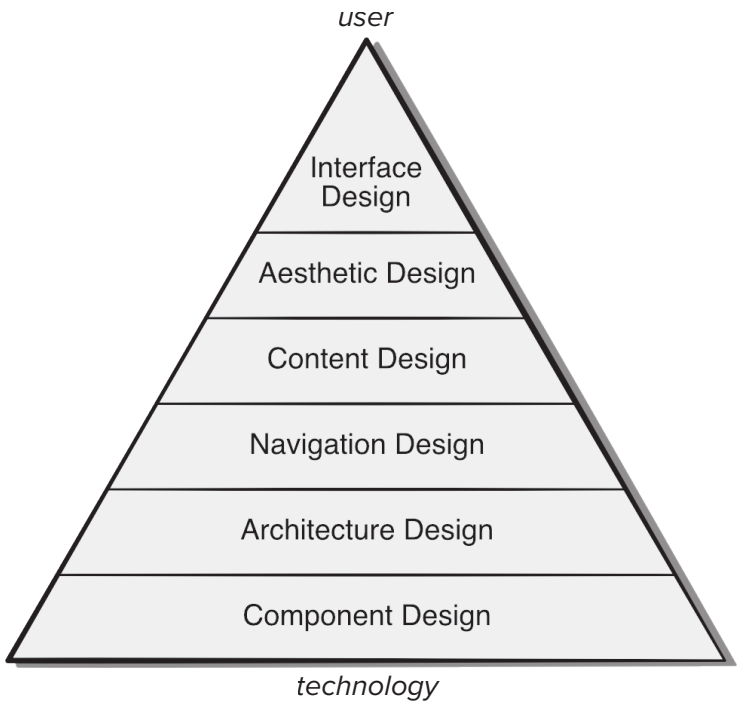
13.5.1 WebApp Interface Design

When a user interacts with a computer-based system, a set of fundamental principles and overriding design guidelines apply. These have been discussed in Chapter 12.[18] Although WebApps present a few special user interface design challenges, the basic principles and guidelines are applicable.

One of the challenges of interface design for WebApps is the indeterminate nature of the user's entry point. That is, the user may enter the WebApp at a "home" location (e.g., the home page) or may be linked into some lower level of the WebApp architecture. In some cases, the WebApp can be designed in a way that reroutes the user to a home location, but if this is undesirable, the WebApp design must provide interface navigation features that accompany all content objects and are available regardless of how the user enters the system.

18 Section 12.1 is dedicated to the user interface design portion of user experience design. If you have not already done so, read it at this time.

FIGURE 13.2
A design pyramid for WebApps



The objectives of a WebApp interface are to: (1) establish a consistent window into the content and functionality provided by the interface, (2) guide the user through a series of interactions with the WebApp, and (3) organize the navigation options and content available to the user. To achieve a consistent interface, you should first use visual design (Section 12.1) to establish a coherent "look." This encompasses many characteristics but must emphasize the layout and form of navigation mechanisms. To guide user interaction, you may draw on an appropriate metaphor[19] that enables the user to gain an intuitive understanding of the interface. To implement navigation options, you can select *navigation menus* positioned consistently on Web pages, *graphic icons* represented in a manner that enable a user to recognize that the icon is a navigation element, and/or *graphic images* that provide a link to a content object or WebApp functionality. It is important to note that one or more of these navigation mechanisms should be provided at every level of the content hierarchy.

Every Web page has a limited amount of "real estate" that can be used to support nonfunctional aesthetics, navigation features, informational content, and user-directed functionality. The development of this real estate is planned during aesthetic design.

19 In this context, a *metaphor* is a representation (drawn from the user's real-world experience) that can be modeled within the context of the interface. A simple example might be a slider switch that is used to control the auditory volume of an .mp4 file.

13.5.2 Aesthetic Design

Aesthetic design, also called visual design or *graphic design,* is an artistic endeavor that complements the technical aspects of WebApp design. We discussed visual design in Section 12.1.4. Page layout is one aspect of aesthetic design that can affect the usefulness (and usability) of a WebApp.

There are no absolute rules when a Web page layout is designed. However, a number of general layout guidelines are worth considering:

Don't be afraid of open space. It is inadvisable to pack every square inch of a Web page with information. The resulting clutter makes it difficult for the user to identify needed information or features and create visual chaos that is not pleasing to the eye.

Emphasize content. After all, that's the reason the user is there. Nielsen [Nie00] suggests that the typical Web page user should be 80 percent content with the remaining real estate dedicated to navigation and other features.

Organize layout elements from top left to bottom right. The vast majority of users will scan a Web page in much the same way as they scan the page of a book—top left to bottom right.[20] If layout elements have specific priorities, high-priority elements should be placed in the upper-left portion of the page real estate.

Group navigation, content, and function geographically within the page. Humans look for patterns in virtually all things. If there are no discernible patterns within a Web page, user frustration is likely to increase (owing to unnecessary searching for needed information).

Don't extend your real estate with the scrolling bar. Although scrolling is often necessary, most studies indicate that users would prefer not to scroll. It is often better to reduce page content or to present necessary content on multiple pages.

Consider resolution and browser window size when designing layout. Rather than defining fixed sizes within a layout, the design should specify all layout items as a percentage of available space [Nie00]. With the growing use of mobile devices with different screen sizes, this concept becomes increasingly important.

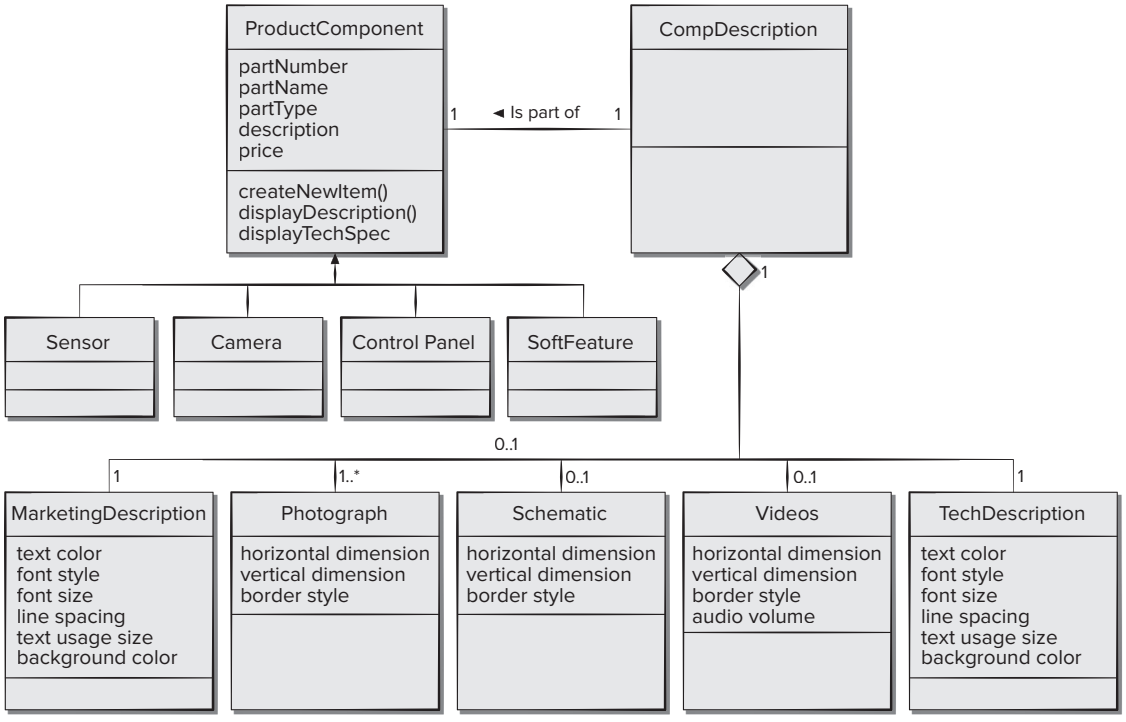
13.5.3 Content Design

We introduced content design in Section 12.1.1. In WebApp design, a content object is more closely aligned with a data object for traditional software. A *content object* has attributes that include content-specific information (normally defined during WebApp requirements modeling) and implementation-specific attributes that are specified as part of design.

As an example, consider an analysis class, **ProductComponent**, developed for the *SafeHome* e-commerce system. The analysis class attribute, `description`, is represented

20 There are exceptions that are cultural and language based, but this rule holds for most users.

FIGURE 13.3 Design representation of content objects



as a design class named **CompDescription** composed of five content objects: **MarketingDescription, Photograph, TechDescription, Schematic,** and **Videos** shown as the bottom row of shaded objects noted in Figure 13.3. Information contained within the content object is noted as attributes. For example, **Photograph** (a jpg image) has the attributes `horizontal dimension`, `vertical dimension`, and `border style`.

UML association and an aggregation[21] may be used to represent relationships between content objects. For example, the UML association shown in Figure 13.3 indicates that one **CompDescription** is used for each instance of the **ProductComponent** class. **CompDescription** is composed on the five content objects shown. However, the multiplicity notation shown indicates that **Schematic** and **Videos** are optional (zero occurrences are possible), one **MarketingDescription** and one **TechDescription** are required, and one or more instances of **Photograph** are used.

13.5.4 Architecture Design

Architecture design is tied to the goals established for a WebApp, the content to be presented, the users who will visit, and the navigation philosophy that has been established. As an architectural designer, you must identify content architecture and

21 Both of these representations are discussed in Appendix 1.

WebApp architecture. *Content architecture*[22] focuses on the manner in which content objects (or composite objects such as Web pages) are structured for presentation and navigation. *WebApp architecture* addresses the manner in which the application is structured to manage user interaction, handle internal processing tasks, effect navigation, and present content.

In most cases, architecture design is conducted in parallel with interface design, aesthetic design, and content design. Because the WebApp architecture may have a strong influence on navigation, the decisions made during this design action will influence work conducted during navigation design.

WebApp architecture describes an infrastructure that enables a Web-based system or application to achieve its business objectives. Jacyntho and his colleagues [Jac02b] describe the basic characteristics of this infrastructure in the following manner:

> Applications should be built using layers in which different concerns are taken into account; in particular, application data should be separated from the page's contents (navigation nodes) and these contents, in turn, should be clearly separated from the interface look-and-feel (pages).

The authors suggest a three-layer design architecture that decouples the interface from navigation and from application behavior. They argue that keeping the interface, application, and navigation separate simplifies implementation and enhances reuse.

The *Model-View-Controller* (MVC) architecture [Kra88][23] is a popular WebApp architectural model that decouples the user interface from the WebApp functionality and information content. The *model* (sometimes referred to as the "model object") contains all application-specific content and processing logic, including all content objects, access to external data/information sources, and all processing functionality that is application specific. The *view* contains all interface-specific functions and enables the presentation of content and processing logic, including all content objects, access to external data and information sources, and all processing functionality required by the end user. The *controller* manages access to the model and the view and coordinates the flow of data between them. In a WebApp, "the view is updated by the controller with data from the model based on user input" [WMT02]. A schematic representation of the MVC architecture is shown in Figure 13.4.

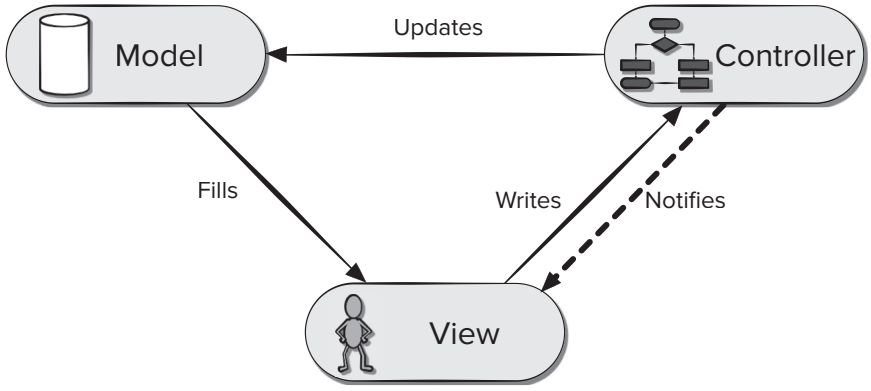
Referring to the figure, user requests or data are handled by the controller. The controller also selects the view object that is applicable based on the user request. Once the type of request is determined, a behavior request is transmitted to the model, which implements the functionality or retrieves the content required to accommodate the request. The model object can access data stored in a corporate database, as part of a local data store, or as a collection of independent files. The data developed by the model must be formatted and organized by the appropriate view object and then transmitted from the application server back to the client-based browser for display on the customer's machine.

---

22 The term *information architecture* is also used to connote structures that lead to better organization, labeling, navigation, and searching of content objects.

23 It should be noted that MVC is actually an architectural design pattern developed for the Smalltalk environment (see www.smalltalk.org) and can be used for any interactive application.

FIGURE 13.4
The MVC architecture



In many cases, WebApp architecture is defined within the context of the development environment in which the application is to be implemented. If you have further interest, see [Fow03] for a discussion of development environments and their role in the design of Web application architectures.

13.5.5 Navigation Design

Once the WebApp architecture has been established and the components (pages, scripts, applets, and other processing functions) of the architecture have been identified, you must define navigation pathways that enable users to access WebApp content and functions. To accomplish this, identify the semantics of navigation for different users of the site, and define the mechanics (syntax) of achieving the navigation.

Like many WebApp design actions, navigation design begins with a consideration of the user hierarchy and related use cases (Chapter 8) developed for each category of user (actor). Each actor may use the WebApp somewhat differently and therefore have different navigation requirements. In addition, the use cases developed for each actor will define a set of classes that encompass one or more content objects or WebApp functions. As each user interacts with the WebApp, she encounters a series of *navigation semantic units* (NSUs)—"a set of information and related navigation structures that collaborate in the fulfillment of a subset of related user requirements" [Cac02]. An NSU describes the navigation requirements for each use case. In essence, the NSU shows how an actor moves between content objects or WebApp functions.

An NSU is composed of a set of navigation elements called *ways of navigating* (WoN) [Gna99]. A WoN represents the best navigation pathway to achieve a navigational goal for a specific type of user. Each WoN is organized as a set of *navigational nodes* (NN) that are connected by navigational links. In some cases, a navigational link may be another NSU. Therefore, the overall navigation structure for a WebApp may be organized as a hierarchy of NSUs.

To illustrate the development of an NSU, consider the use case **Select SafeHome Components:**

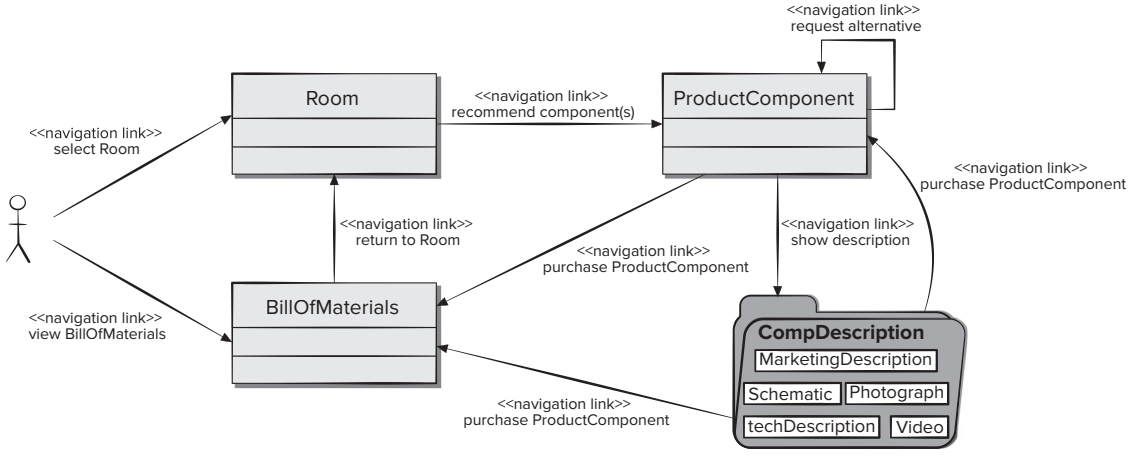
Use Case: *Select SafeHome Components*

The WebApp will recommend product components (e.g., control panels, sensors, cameras) and other features (e.g., PC-based functionality implemented in software) for each room and exterior entrance. If I request alternatives, the WebApp will provide them, if they exist. I will be able to get descriptive and pricing information for each product component. The WebApp will create and display a bill-of-materials as I select various components. I'll be able to give the bill-of-materials a name and save it for future reference (see use case **Save Configuration**).

The underlined items in the use case description represent classes and content objects that will be incorporated into one or more NSUs that will enable a new customer to perform the scenario described in the **Select SafeHome Components** use case.

Figure 13.5 depicts a partial semantic analysis of the navigation implied by the **Select SafeHome Components** use case. Using the terminology introduced earlier, the figure also represents a way of navigating (WoN) for the **SafeHomeAssured.com** WebApp. Important problem domain classes are shown, along with selected content objects (in this case the package of content objects named `CompDescription`, an attribute of the **ProductComponent** class). These items are navigation nodes. Each of the arrows represents a navigation link[24] and is labeled with the user-initiated action that causes the link to occur.

FIGURE 13.5 Creating an NSU



24 These are sometimes referred to as *navigation semantic links* (NSLs) [Cac02].

You can create an NSU for each use case associated with each user role. For example, a **new customer** for **SafeHomeAssured.com** may have three different use cases, all resulting in access to different information and WebApp functions. An NSU is created for each goal.

During the initial stages of navigation design, the WebApp content architecture is assessed to determine one or more WoN for each use case. As noted earlier, a WoN identifies navigation nodes (e.g., content) and then links that enable navigation between them. The WoN are then organized into NSUs.

As design proceeds, your next task is to define the mechanics of navigation. Most websites make use of one or more of the following navigation options for implementing each NSU: individual navigation links, horizontal or vertical navigation bars (lists), tabs, or access to a complete site map. If a site map is defined, it should be accessible from every page. The map itself should be organized so that the structure of WebApp information is readily apparent.

In addition to choosing the mechanics of navigation, you should also establish appropriate navigation conventions and aids. For example, icons and graphical links should look "clickable" by beveling the edges to give the image a three-dimensional look. Audio or visual feedback should be designed to provide the user with an indication that a navigation option has been chosen. For text-based navigation, color should be used to indicate navigation links and to provide an indication of links already traveled. These are but a few of dozens of design conventions that make navigation user-friendly.

13.6 COMPONENT-LEVEL DESIGN

Mobile apps deliver increasingly sophisticated processing functions that (1) perform localized processing to generate content and navigation capability in a dynamic fashion, (2) provide computation or data processing capability that are appropriate for the app's business domain, (3) provide sophisticated database query and access, and (4) establish data interfaces with external corporate systems. To achieve these (and many other) capabilities, you must design and construct program components that are identical in form to software components for traditional software.

The design methods discussed in Chapters 11 and 12 apply to mobile components with little, if any, modification. The implementation environment, programming languages, and design patterns, frameworks, and software may vary somewhat, but the overall design approach remains the same. To be cost conscious, you can design mobile components in such a way that they can be used without modification on several different mobile platforms.

13.7 MOBILITY AND DESIGN QUALITY

Every person has an opinion about what makes a "good" mobile app. Individual viewpoints vary widely. Some users enjoy flashy graphics; others want simple text. Some demand copious information; others desire an abbreviated presentation. Some like sophisticated analytical tools or database access; others like to keep it simple.

In fact, the user's perception of "goodness" (and the resultant acceptance or rejection of a mobile app as a consequence) might be more important than any technical discussion of mobile app quality. Mobile design quality attributes are virtually the same as WebApp quality characteristics.

But how is mobile quality perceived? What attributes must be exhibited to achieve goodness in the eyes of end users and at the same time exhibit the technical characteristics of quality that will enable you to correct, adapt, enhance, and support the mobile product over the long term?

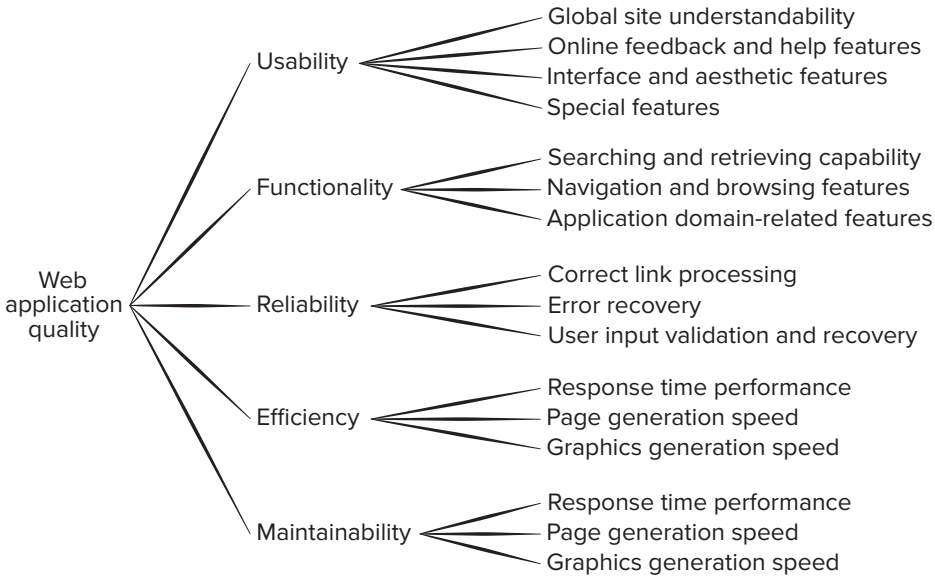
In reality, all the technical characteristics of design quality discussed in Chapter 12 and the generic quality attributes presented in Chapter 19 apply to mobile apps. However, the most relevant of these generic attributes—usability, functionality, reliability, efficiency, and maintainability—provide a useful basis for assessing the quality of mobile systems as well. Andreou [And05] suggests that end-user satisfaction with a mobile app is dictated by the same important quality factors—functionality, reliability, usability, efficiency, and maintainability—but adds portability to the list.

Olsina and his colleagues [Ols99] have prepared a "quality requirement tree" that identifies a set of technical attributes—usability, functionality, reliability, efficiency, and maintainability—that lead to high-quality mobile products.[25] Figure 13.6 summarizes their work. The criteria noted in the figure are of particular interest if you design, build, and maintain mobile products over the long term.

FIGURE 13.6

Quality requirements tree

Source: Olsina, Luis, Lafuente, Guillermo and Rossi, Gustavo, "Specifying Quality Characteristics and Attributes for Web Sites," Proceedings of the 1st International Conference on Software Engineering Workshop on Web Engineering, ACM, Los Angeles, May 1999.



25 These quality attributes are quite similar to those presented in Chapters 9 and 15. The implication is that quality characteristics are universal for all software.

Offutt [Off02] extends the five major quality attributes noted in Figure 13.6 by adding the following attributes:

Security. Mobile products have become heavily integrated with critical corporate and government databases. E-commerce applications extract and then store sensitive customer information. For these and many other reasons, mobile security is paramount in many situations. The key measure of security is the ability of the mobile app and its server environment to rebuff unauthorized access and/or thwart an outright malevolent attack. Security engineering is discussed in Chapter 18. For additional information on WebApp and mobile app security, see [Web13], [Pri10], [Vac06], and [Kiz05].

Availability. Even the best mobile product will not meet users' needs if it is unavailable. In a technical sense, availability is the measure of the percentage of time that a Web-based mobile resource is available for use. But Offutt [Off02] suggests that "using features available on only one browser or one platform" makes the mobile product unavailable to those with a different browser or platform configuration. The user will invariably go elsewhere.

Scalability. Can the mobile product and its server environment be scaled to handle 100, 1,000, 10,000, or 100,000 users? Will the app and the systems with which it is interfaced handle significant variation in volume, or will responsiveness drop dramatically (or cease altogether)? It is important to design a mobile environment that can accommodate the burden of success (i.e., significantly more end users) and become even more successful.

Time to Market. Although time to market is not a true quality attribute in the technical sense, it is a measure of quality from a business point of view. The first mobile product to address a specific market segment often captures a disproportionate number of end users.

Content Quality. Billions of Web pages are available for those in search of information. Even well-targeted Web searches result in an avalanche of content. With so many sources of information to choose from, how does the user assess the quality (e.g., veracity, accuracy, completeness, timeliness) of the content that is presented within a mobile product? This is part of what data science tries to address. The basics of data science are introduced in Appendix 2 of this book.

Tillman [Til00] suggests a useful set of criteria for assessing the quality of content: Can the scope and depth of content be easily determined to ensure that it meets the user's needs? Can the background and authority of the content's authors be easily identified? Is it possible to determine the currency of the content, the last update, and what was updated? Are the content and its location stable (i.e., will they remain at the referenced URL)? Is content credible? Is content unique? That is, does the mobile product provide some unique benefit to those who use it? Is content valuable to the targeted user community? Is content well organized? Indexed? Easily accessible? These questions represent only a small sampling of the issues that should be addressed as the design of a mobile product evolves.

INFO



Mobile Product—Quality Checklist

The following checklist provides a set of questions that will help both software engineers and end users assess overall mobile product quality:

- Can content, function, and/or navigation options be tailored to the user's preferences?
- Can content and/or functionality be customized to the bandwidth at which the user communicates? Does the app account for weak or lost signals in an acceptable manner?
- Can content, function, and/or navigation options be made context aware according to the user's preferences?
- Has adequate consideration been given to the power availability on the target device(s)?
- Have graphics, media (audio, video), and other Web or cloud services been used appropriately?
- Is the overall page design easy to read and navigate? Does the app take screen-size differences into account?
- Does the user interface conform to the display and interaction standards adopted for the targeted mobile device(s)?
- Does the app conform to the reliability, security, and privacy expectations of its users?
- What provisions have been made to ensure an app remains current?
- Has the mobile product been tested in all targeted user environments and for all targeted devices?

13.8 MOBILITY DESIGN BEST PRACTICES

There are several guidelines for developing mobile products[26] and for developing apps for specific platforms like Apple's iOS[27] or Google's Android.[28] Schumacher [Sch09] has collected many best practice ideas and has posted several specially adapted to the design of mobile applications and Web pages. Some important considerations when designing mobile touch-screen applications listed by Schumacher include:

- **Identify your audience.** The application must be written with the expectations and backgrounds of its users in mind. Experienced users want to do things quickly. Less experienced users will appreciate a handholding approach when they are first using the app.
- **Design for context of use.** It is important to consider how the user will interact with the real world while using the mobile product. Watching a movie on an airplane calls for a different user interface than checking the weather before you leave the office.
- **There is a fine line between simplicity and laziness.** Creating an intuitive user interface on a mobile device is much harder than simply removing features found in the user interface for the application running on a larger device. The user interface should provide all the information that enables a user to make her next decision.

26 See http://www.w3.org/TR/mwabp/.
27 See https://developer.apple.com/design/human-interface-guidelines/.
28 See http://developer.android.com/guide/components/index.html.

- **Use the platform as an advantage.** Touch-screen navigation is not intuitive and must be learned by all new users. This learning task will be easier if the user interface designers adhere to standards that have been set for the platform.
- **Make scrollbars and selection highlighting more salient.** Scrollbars are often hard to locate on touch devices because they are too small. Make sure that menu or icon borders are wide enough for color changes to catch the users' attention. When color coding is used, make sure there is sufficient contrast between foreground and background colors to allow them to be distinguishable by any color-blind users.
- **Increase discoverability of advanced functionality.** Hot keys and other shortcuts are sometimes included in mobile products to allow experienced users to complete their tasks more quickly. You can increase the discoverability of features like these by including visual design clues in the user interface.
- **Use clear and consistent labels.** Widget labels should be recognized by all app users, regardless of standards used by specific platforms. Use abbreviations cautiously and avoid them if possible.
- **Clever icons should never be developed at the expense of user understanding.** Icons often only make sense to their designers. Users must be able to learn their meaning quickly. It is hard to guarantee that icons are meaningful across all languages and user groups. A good strategy to enhance recognition is to add a text label beneath a novel icon.
- **Support user expectations for personalization.** Mobile device users expect to be able to personalize everything. At the very least, developers should try to allow users to set their location (or detect it automatically) and select content options that may be available at that location. It is important to indicate to users what features can be personalized and how users can personalize them.
- **Long scrolling forms trump multiple screens on mobile devices.** Experienced mobile device users want all information on a single input screen even if this requires scrolling. Novice users often become experienced quickly and will grow tired of multiple input screens.

Developing native applications for multiple device platforms can be costly and time consuming. Development costs can be reduced by using technologies familiar to Web developers (e.g., JavaScript, CSS, and HTML) to create mobile products that will be accessed using a Web browser on a mobile device.

There are no guarantees that a desktop program or a WebApp can be easily adapted for implementation as a mobile product. However, many of the agile software engineering practices (Chapter 3) used to create desktop computer applications can be used to create stand-alone apps or mobile client software, and many of the practices used to create quality WebApps apply to the creation of Web services used by mobile products.

The most important architectural design decision is often whether to build a thin or fat client. The model-view-controller (MVC) architecture (Section 13.3) is commonly used in mobile products. Because the mobile architecture has a strong influence on navigation, the decisions made during this design action will influence work conducted during navigation design. The architectural design must take device resources

into account (storage, processor speed, and network connectivity). The design should include provisions for discoverable services and movable devices.

Usability testing and deployment testing take place during each prototype development cycle. Code reviews that focus on security issues should be included as part of the implementation activities. These code reviews should be based on the appropriate security objectives and threats identified in the system design activities. Security testing is a routine part of system testing (Chapter 21).

13.9 SUMMARY

The quality of a mobile product—defined in terms of functionality, reliability, usability, efficiency, security, maintainability, scalability, and portability—is introduced during design. A good mobile product should be based on the following design goals: simplicity, ubiquity, personalization, flexibility, and localization.

Interface design describes the structure and organization of the user interface and includes a representation of screen layout, a definition of the modes of interaction, and a description of navigation mechanisms. In addition, the interface for a good mobile product will promote the brand signature and focus on its targeted device platform(s). A set of core user stories is used to trim unnecessary features from the app to manage its resource requirements. Context-aware devices make use of discoverable services to help personalize the user experience.

Content design is critically important and takes the screen and other limitations of mobile devices into account. Aesthetic design, also called graphic design, describes the "look and feel" of the mobile product and includes color schemes, graphic layout, the use of graphics, and related aesthetic decisions. Aesthetic design must also take device limitations into account.

Architecture design identifies the overall hypermedia structure for the mobile product and encompasses both content architecture and mobile architecture. It is critical to determine how much of the mobile functionality will reside on the mobile device and how much will be provided by Web or cloud services.

Navigation design represents the navigational flow between content objects and for all mobile functions. Navigation syntax is defined by the widgets available on the targeted mobile device(s), and the semantics are often determined by the mobile platform. Content chunking must take intermittent service interruptions and user demands for fast performance into account.

Component design develops the detailed processing logic required to implement the components that are used to build a complete MobileApp function. The design techniques described in Chapter 12 may be applicable for the engineering of mobile components.

PROBLEMS AND POINTS TO PONDER

13.1. Explain why deciding to develop a MobileApp for several devices can be a costly design decision. Is there a way to mitigate the risks of supporting the wrong platform?

13.2. In this chapter we listed many quality attributes for mobile products. Select the three that you believe are most important, and make an argument that explains why each should be emphasized in mobile design work.

13.3. You are a MobileApp designer for *Project Planning Corporation*, a company that builds productivity software. You want to implement the equivalent of a digital three-ring binder that allows tablet users to organize and categorize electronic documents of several types under user-defined tabs. For example, a kitchen remodeling project might require a pdf catalog, a jpg or layout drawing, an MS Word proposal, and an Excel spreadsheet stored under a Cabinetry tab. Once defined, the binder and its tab content can be stored either on the tablet or on some cloud storage. The application needs to provide five key functions: binder and tab definition, digital document acquisition from a Web location or the device, binder management functions, page display functions, and a notes function to allow a Post-it note to be added to any page. Develop an interface design for the three-ring application, and implement it as a paper prototype.

13.4. What is the most aesthetically pleasing MobileApp you have ever used and why?

13.5. Create user stories for the three-ring application described in Problem 13.3.

13.6. What might be considered to make the three-ring application a context-aware MobileApp?

13.7. Reconsidering the *ProjectPlanning* three-ring application described in Problem 13.3, select a development platform for the first working prototype. Discuss why you made the choice.

13.8. Do a bit of additional research on the MVC architecture and decide whether it would be an appropriate MobileApp architecture for the three-ring discussed in Problem 13.3.

13.9. Describe three context-aware features that would be desirable to add to a *SafeHome* MobileApp.

13.10. You are a WebApp designer for FutureLearning Corporation, a distance learning company. You intend to implement an Internet-based "learning engine" that will enable you to deliver course content to a student. The learning engine provides the basic infrastructure for delivering learning content on any subject (content designers will prepare appropriate content). Develop a prototype interface design for the learning engine.

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

14 PATTERN-BASED DESIGN

Each of us has encountered a design problem and silently thought: *I wonder if anyone has developed a solution for this?* The answer is almost always *yes!* The problem is finding the solution; ensuring that it does, in fact, fit the problem you've encountered; understanding the constraints that may restrict the ways in which the solution is applied; and finally, translating the proposed solution into your design environment.

KEY CONCEPTS

anti-patterns 302
architectural patterns 299
behavioral patterns 292
component-level design patterns 300
creational patterns 292
design mistakes 298
design patterns 290
frameworks 293
kinds of patterns 291
machine learning 294
pattern languages 297
pattern-organizing table 298
structural patterns 292
system of forces 290
user interface design patterns 304



QUICK LOOK

What is it? Pattern-based design creates a new application by finding a set of proven solutions to a clearly delineated set of problems. Each problem and its solution are described by a design pattern that has been cataloged and vetted by other software engineers who have encountered the problem and implemented the solution while designing other applications.

Who does it? A software engineer examines each problem encountered for a new application and then attempts to find a relevant solution by searching one or more pattern repositories.

Why is it important? Have you ever heard the phrase "reinventing the wheel"? It happens all the time in software development, and it's a waste of time and energy. By using existing design patterns, you can acquire a proven solution for a specific problem. As each pattern is applied, solutions are integrated and the application to be built moves closer to a complete design.

What are the steps? The requirements model is examined to isolate the hierarchical set of problems to be solved. The problem space is partitioned so that subsets of problems associated with specific software functions and features can be identified. Problems can also be organized by type: architectural, component-level, algorithmic, user interface, and so forth. Once a subset of problems is defined, one or more pattern repositories are searched to determine if a design pattern, represented at an appropriate level of abstraction, exists.

What is the work product? A design model that depicts the architectural structure, user interface, and component-level detail is developed.

How do I ensure that I've done it right? As each design pattern is translated into some element of the design model, work products are reviewed for clarity, correctness, completeness, and consistency with requirements and with one another.

But what if the solution were codified in some manner? What if there was a standard way of describing a problem (so you could look it up), and an organized method for representing the solution to the problem? It turns out that software problems have been codified and described using a standardized template, and solutions to them (along with constraints) have been proposed. Called *design patterns,* this codified method for describing problems and their solution allows the software engineering community to capture design knowledge in a way that enables it to be reused.

The early history of software patterns begins not with a computer scientist but a building architect, Christopher Alexander, who recognized that a recurring set of problems was encountered whenever a building was designed. He characterized these recurring problems and their solutions as *patterns,* describing them in the following manner [Ale77]: "Each pattern describes a problem that occurs repeatedly in our environment and then describes the core of the solution to that problem in such a way that you can use the solution a million times over without ever doing it the same way twice." Alexander's ideas were first translated into the software world in books by Gamma [Gam95], Buschmann [Bus07], and their many colleagues.[1] Today, dozens of pattern repositories exist, and pattern-based design can be applied in many different application domains.

14.1 DESIGN PATTERNS

A *design pattern* can be characterized as "a three-part rule which expresses a relation between a certain context, a problem, and a solution" [Ale79]. For software design, *context* allows the reader to understand the environment in which the problem resides and what solution might be appropriate within that environment. A set of requirements, including limitations and constraints, acts as a *system of forces* that influences how the problem can be interpreted within its context and how the solution can be effectively applied.

It is reasonable to argue that most problems have multiple solutions, but that a solution is effective only if it is appropriate within the context of the existing problem. It is the system of forces that causes a designer to choose a specific solution. The intent is to provide a solution that best satisfies the system of forces, even when these forces are contradictory. Finally, every solution has consequences that may have an impact on other aspects of the software and may themselves become part of the system of forces for other problems to be solved within the larger system.

An effective design pattern: (1) captures a specific solution to a bounded problem, (2) provides a solution that has been proven in practice, (3) identifies an approach to a problem that is not obvious, (4) identifies the relationship(s) between the design and other architectural elements, and (5) is elegant in its approach and its utility.

1 Earlier discussions of software patterns do exist, but these two classic books were the first cohesive treatments of the subject.

A design pattern saves you from "reinventing the wheel," or worse, inventing a "new wheel" that is slightly out of round, too small for its intended use, and too narrow for the ground it will roll over. Design patterns, if used effectively, will invariably make you a better software designer.

14.1.1 Kinds of Patterns

One of the reasons that software engineers are interested in (and intrigued by) design patterns is that human beings are inherently good at pattern recognition. If we weren't, we'd be frozen in space and time—unable to learn from experience, unwilling to venture forward because of our inability to recognize situations that might lead to high risk, unhinged by a world that seems to have no regularity or logical consistency. Luckily, none of this occurs because we do recognize patterns in virtually every aspect of our lives.

In the real world, the patterns we recognize are learned over a lifetime of experience. We recognize them instantly and inherently understand what they mean and how they might be used. Some of these patterns provide us with insight into recurring phenomenon. For example, you're on your way home from work on the interstate when your navigation system (or car radio) informs you that a serious accident has occurred on the interstate in the opposing direction. You're 4 miles from the accident, but already you begin to see traffic slowing, recognizing a pattern that we'll call **RubberNecking.** People in the travel lanes moving in your direction are slowing at the sight of the accident to get a better view of what happened on the opposite side of the highway. The **RubberNecking** pattern yields remarkably predictable results (a traffic jam), but it does nothing more than describe a phenomenon. In patterns jargon, it might be called a *nongenerative* pattern because it describes a context and a problem, but it does not provide any clear-cut solution.

When software design patterns are considered, we strive to identify and document *generative* patterns. That is, we identify a pattern that describes an important and repeatable aspect of a system and that provides us with a way to build that aspect within a system of forces that are unique to a given context. In an ideal setting, a collection of generative design patterns could be used to "generate" an application or computer-based system whose architecture enables it to adapt to change. Sometimes called *generativity,* "the successive application of several patterns, each encapsulating its own problem and forces, unfolds a larger solution which emerges indirectly as a result of the smaller solutions" [App00].

Design patterns span a broad spectrum of abstraction and application. *Architectural patterns* describe broad-based design problems that are solved using a structural approach. *Data patterns* describe recurring data-oriented problems and the data modeling solutions that can be used to solve them. *Component patterns* (also referred to as *design patterns*) address problems associated with the development of subsystems and components, the way they communicate with one another, and their placement within a larger architecture. *Interface design patterns* describe common user interface problems and their solutions with a system of forces that includes the specific characteristics of end users. *WebApp patterns* address a problem set that is encountered

when building WebApps and often incorporates many of the other pattern categories just mentioned. *Mobile patterns* describe commonly encountered problems when developing solutions for mobile platforms. At a lower level of abstraction, *idioms* describe how to implement all or part of a specific algorithm or data structure for a software component within the context of a specific programming language. Don't force a pattern, even if it addresses the problem at hand. If the context and forces are wrong, look for another pattern.

In their seminal book on design patterns, Gamma and his colleagues[2] [Gam95] focus on three types of patterns that are particularly relevant to object-oriented design: creational patterns, structural patterns, and behavioral patterns.

INFO



Creational, Structural, and Behavioral Patterns

A wide variety of design patterns that fit into creational, structural, and behavioral categories have been proposed and can be found on the Web. Here is a sampling of patterns for each type. Comprehensive descriptions of each of these patterns can be obtained via links at **www.wikipedia.org.**

Creational Patterns

- **Abstract factory pattern.** Centralize decision of what factory to instantiate.
- **Factory method pattern.** Centralize creation of an object of a specific type, choosing one of several implementations.
- **Builder pattern.** Separate the construction of a complex object from its representation so that the same construction process can create different representations.

Structural Patterns

- **Adapter pattern.** "Adapts" one interface for a class into one that a client expects.
- **Aggregate pattern.** A version of the composite pattern with methods for aggregation of children.
- **Composite pattern.** A tree structure of objects where every object has the same interface.
- **Container pattern.** Create objects for the sole purpose of holding other objects and managing them.
- **Proxy pattern.** A class functioning as an interface to another thing.
- **Pipes and filters.** A chain of processes where the output of each process is the input of the next.

Behavioral Patterns

- **Chain of responsibility pattern.** Command objects are handled or passed on to other objects by logic-containing processing objects.
- **Command pattern.** Command objects encapsulate an action and its parameters.
- **Iterator pattern.** Iterators are used to access the elements of an aggregate object sequentially without exposing its underlying representation.
- **Mediator pattern.** Provides a unified interface to a set of interfaces in a subsystem.
- **Visitor pattern.** A way to separate an algorithm from an object.
- **Hierarchical visitor pattern.** Provides a way to visit every node in a hierarchical data structure such as a tree.

2 Gamma and his colleagues [Gam95] are often referred to as the "Gang of Four" (GoF) in patterns literature.

Creational patterns focus on the "creation, composition, and representation" of objects and provide mechanisms that make the instantiation of objects easier within a system and enforce "constraints on the type and number of objects that can be created within a system" [Maa07]. *Structural patterns* focus on problems and solutions associated with how classes and objects are organized and integrated to build a larger structure. *Behavioral patterns* address problems associated with the assignment of responsibility between objects and the way communication is affected between objects.

14.1.2 Frameworks

Patterns themselves may not be sufficient to develop a complete design. In some cases, it may be necessary to provide an implementation-specific skeletal infrastructure, called a *framework,* for design work. A *framework* is a reusable "mini-architecture" that serves as a foundation from which other design patterns can be applied. That is, you can select a "*reusable mini-architecture* that provides the generic structure and behavior for a family of software abstractions, along with a context . . . which specifies their collaboration and use within a given domain" [Amb98].

A framework is not an architectural pattern, but rather a skeleton with a collection of "plug points" (also called *hooks* and *slots*) that enable it to be adapted to a specific problem domain. The plug points enable you to integrate problem-specific classes or functionality within the skeleton. In an object-oriented context, a framework is a collection of cooperating classes.

Gamma and his colleagues [Gam95] note that patterns are more abstract than frameworks. A framework can be "embodied in code," while a pattern is generally code independent. A framework often encompasses more than a single pattern and is therefore a larger architectural element than a pattern. Finally, a framework resides within a specific application domain, but patterns can be applied in any domain in which the problem to be addressed is encountered.

The designer of a framework will argue that one, reusable, mini-architecture is applicable to all software to be developed within a limited domain of application. To be most effective, frameworks are applied with no changes. Additional design elements may be added, but only via the plug points that allow the designer to flesh out the framework skeleton.

14.1.3 Describing a Pattern

Pattern-based design begins with the recognition of patterns within the application you intend to build, continues with a search to determine whether others have addressed the pattern, and concludes with the application of an appropriate pattern to the problem at hand. The second of these three tasks is often the most difficult. How do you find patterns that fit your needs?

An answer to this question must rely on effective communication of the problem the pattern addresses, the context in which the pattern resides, the system of forces that mold the context, and the solution that is proposed. To communicate this information unambiguously, a standard form or template for pattern descriptions is required. Although several different pattern templates have been proposed, almost all contain a major

subset of the content suggested by Gamma and his colleagues [Gam95]. A simplified pattern template is shown in the sidebar.

INFO



Design Pattern Template

Pattern name. Describes the essence of the pattern in a short but expressive name.

Problem. Describes the problem that the pattern addresses.

Motivation. Provides an example of the problem.

Context. Describes the environment in which the problem resides, including the application domain.

Forces. Lists the system of forces that affect the way the problem must be solved; includes a discussion of limitation and constraints that must be considered.

Solution. Provides a detailed description of the solution proposed for the problem.

Intent. Describes the pattern and what it does.

Collaborations. Describes how other patterns contribute to the solution.

Consequences. Describes the potential trade-offs that must be considered when the pattern is implemented and the consequences of using the pattern.

Implementation. Identifies special issues that should be considered when implementing the pattern.

Known uses. Provides examples of actual uses of the design pattern in real applications.

Related patterns. Cross-references related design patterns.

The names of design patterns should be chosen with care. One of the key technical problems in pattern-based design is the inability to find existing patterns when hundreds or thousands of candidate patterns exist. The search for the "right" pattern is aided immeasurably by a meaningful pattern name.

A pattern template provides a standardized means for describing a design pattern. Each of the template entries represents characteristics of the design pattern that can be searched (e.g., via a database) so that the appropriate pattern can be found.

14.1.4 Machine Learning and Pattern Discovery

Software patterns can be described as best practice solutions to known problems. Information about where design patterns have been implemented in a software design is useful information for developers to have when creating or maintaining a software system. Sadly, this information is lost due to poor documentation practices of the original developers. In recent years, there has been significant interest in making use of automatic techniques to identify new patterns present, but not documented, in existing software products [Alh12].

One way to do this is to create an artificial intelligence (AI) system capable of recognizing design patterns after examining many similar software systems. The same software pattern may be implemented in many ways. *Machine learning*[3] techniques

3 Machine learning is an AI technique that uses statistical techniques to allow a system to learn from examples and improve its performance without being explicitly programmed. For those readers interested in pursuing this subject in greater detail, see [Kub17].

can provide a means of teaching a system to recognize the presence of a pattern in the software source code. The machine learning system repeatedly examines a training set containing both good and bad examples of software patterns using specific quantitative criteria of "good" and "bad" examples. This process continues until the system has learned to recognize most of the good patterns in the training set. Often these training sets are derived from large open-source software systems available on the Internet [Zan15].

Once trained, the tool can be used to locate software patterns in new systems outside the training set. To be useful, the software patterns are collected in a repository. Ideally, this repository can be searched for software patterns applicable to the problems developers need to solve [Amp13].

14.2 PATTERN-BASED SOFTWARE DESIGN

The best designers in any field have an uncanny ability to see patterns that characterize a problem and the corresponding patterns that can be combined to create a solution. Throughout the design process, you should look for every opportunity to apply existing design patterns (when they meet the needs of the design) rather than creating new ones.

14.2.1 Pattern-Based Design in Context

Pattern-based design is not used in a vacuum. The concepts and techniques discussed for architectural, component-level, and user interface design (Chapter 10 through 12) are all used in conjunction with a pattern-based approach.

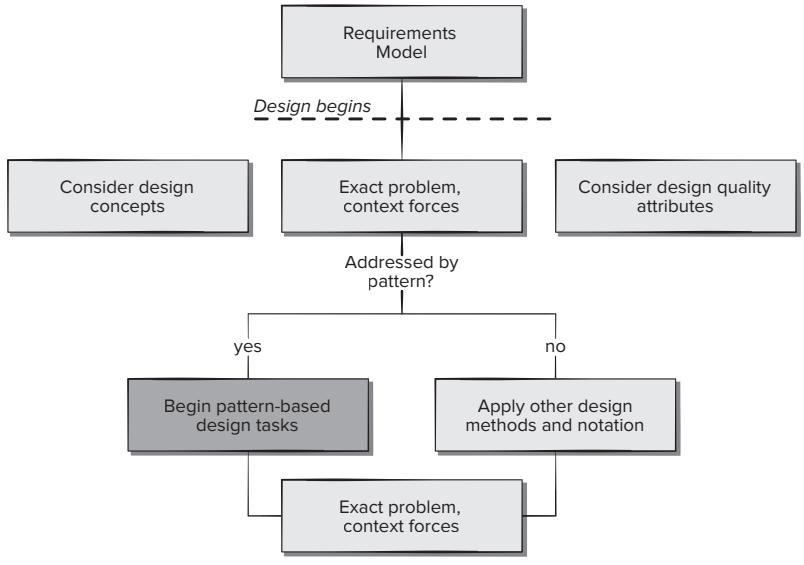
In Chapter 9, we noted that a set of quality guidelines and attributes serve as the basis for all software design decisions. The decisions themselves are influenced by a set of fundamental design concepts (e.g., separation of concerns, stepwise refinement, functional independence) that are achieved using heuristics that have evolved over many decades, and best practices (e.g., techniques, modeling notation) that have been proposed to make design easier to perform and more effective as a basis for construction.

The role of pattern-based design in all of this is illustrated in Figure 14.1. A software designer begins with a requirements model (either explicit or implied) that presents an abstract representation of the system. The requirements model describes the problem set, establishes the context, and identifies the system of forces that hold sway. It may imply the design in an abstract manner, but the requirements model does little to represent the design explicitly.

As you begin your work as a designer, it's always important to keep quality attributes (Chapter 9) in mind. These attributes establish a way to assess software quality but do little to help you achieve it. Therefore, you should apply proven techniques for translating the abstractions contained in the requirements model into a more concrete form that is the software design. To accomplish this, you'll use the methods and modeling tools available for architectural, component-level, and interface design. But, only when you're faced with a problem, context, and system of forces that have not been solved before. If a solution already exists, use it! And that means applying a pattern-based design approach.

FIGURE 14.1

Pattern-based design in context



14.2.2 Thinking in Patterns

Pattern-based design implies a "new way of thinking" [Sha05] that begins by considering context—the big picture. As context is evaluated, you extract a hierarchy of problems that must be solved. Some of these problems will be global in nature, while others will address specific features and functions of the software. All will be affected by a system of forces that will influence the nature of the solution that is proposed.

Shalloway and Trott [Sha05] suggest the following approach[4] that enables a designer to think in patterns:

1. Be sure you understand the big picture—the context in which the software to be built resides. The requirements model should communicate this to you.
2. Examining the big picture, extract the patterns that are present at that level of abstraction.
3. Begin your design with "big picture" patterns that establish a context or skeleton for further design work.
4. "Work inward from the context" [Sha05], looking for patterns at lower levels of abstraction that contribute to the design solution.
5. Repeat steps 1 to 4 until the complete design is fleshed out.
6. Refine the design by adapting each pattern to the specifics of the software you're trying to build.

It's important to note that patterns are not independent entities. Design patterns that are present at a high level of abstraction will invariably influence the ways other patterns are applied at lower levels of abstraction. In addition, patterns often collaborate

4 Based on the work of Christopher Alexander [Ale79].

with one another. The implication is that when you select an architectural pattern, it may very well influence the component-level design patterns you choose. Likewise, when you select a specific interface design pattern, you are sometimes forced to use other patterns that collaborate with it.

To illustrate, consider the **SafeHomeAssured.com** WebApp. If you consider the big picture, the WebApp must address how to provide information about *SafeHome* products and services, how to sell *SafeHome* products and services to customers, and how to establish Internet-based monitoring and control of an installed security system. Each of these fundamental problems can be further refined into a set of subproblems.

For example, *How to sell* via the Internet implies an **e-commerce** pattern that itself implies many patterns at lower levels of abstraction. The **e-commerce** pattern (likely, an architectural pattern) implies mechanisms for setting up a customer account, displaying the products to be sold, selecting products for purchase, and so forth. Hence, if you think in patterns, it is important to determine whether a pattern for setting up an account exists. If **SetUpAccount** is available as a viable pattern for the problem context, it may collaborate with other patterns such as **BuildInputForm, ManageFormsInput,** and **ValidateFormsEntry.** Each of these patterns delineates problems to be solved and solutions that may be applied.

14.2.3 Design Tasks

The following design tasks are applied when a pattern-based design philosophy is used:

1. **Examine the requirements model and develop a problem hierarchy.** Describe each problem and subproblem by isolating the problem, the context, and the system of forces that apply. Work from broad problems (high level of abstraction) to smaller subproblems (at lower levels of abstraction).
2. **Determine if a reliable pattern language has been developed for the problem domain.** A *pattern language* encompasses a collection of patterns, each described using a standardized template (Section 14.1.3) and interrelated to show how these patterns collaborate to solve problems across an application domain. The *SafeHome* software team would look for a pattern language developed specifically for home security products. If that level of pattern language specificity could not be found, the team would partition the *SafeHome* software problem into a series of generic problem domains (e.g., digital device monitoring problems, user interface problems, digital video management problems) and search for appropriate pattern languages.
3. **Beginning with a broad problem, determine whether one or more architectural patterns are available for it.** If an architectural pattern is available, be certain to examine all collaborating patterns. If the pattern is appropriate, adapt the design solution proposed and build a design model element that adequately represents it. For example, a broad problem for the **SafeHomeAssured.com** WebApp is addressed with an **e-commerce** pattern (Section 14.2.2). This pattern will suggest a specific architecture for addressing e-commerce requirements.
4. **Using the collaborations provided for the architectural pattern, examine subsystem or component-level problems and search for appropriate patterns to address them.** It may be necessary to search through other pattern repositories as

well as the list of patterns that corresponds to the architectural solution. If an appropriate pattern is found, adapt the design solution proposed and build a design model element that adequately represents it. Be certain to apply step 7.

5. **Repeat steps 2 to 4 until all broad problems have been addressed.** The implication is to begin with the big picture and elaborate to solve problems at increasingly more detailed levels.
6. **If user interface design problems have been isolated (this is almost always the case), search the many user interface design pattern repositories for appropriate patterns.** Proceed in a manner similar to steps 3 to 5.
7. **Regardless of its level of abstraction, if a pattern language and/or patterns repository or individual pattern shows promise, compare the problem to be solved against the existing pattern(s) presented.** Be certain to examine context and forces to ensure that the pattern does, in fact, provide a solution that is amenable to the problem.
8. **Be certain to refine the design as it is derived from patterns using design quality criteria as a guide.**

Although this design approach is fundamentally top-down, Gillis [Gil06] suggests that "it's more organic than that, more inductive than deductive, more bottom-up than top-down." In addition, the pattern-based approach must be used in conjunction with other software design concepts and techniques.

14.2.4 Building a Pattern-Organizing Table

As pattern-based design proceeds, you may encounter trouble organizing and categorizing candidate patterns from multiple pattern languages and repositories. To help organize your evaluation of candidate patterns, Microsoft [Mic13b] suggests the creation of a *pattern-organizing table* that takes the general form shown in Figure 14.2.

A pattern-organizing table can be implemented as a spreadsheet model using the form shown in the figure. An abbreviated list of problem statements, organized by data and content, architecture, component level, and user interface issues, is presented in the left-hand (shaded) column. Four pattern types—database, application, implementation, and infrastructure—are listed across the top row. The names of candidate patterns are noted in the cells of the table.

To provide entries for the organizing table, you'll search through pattern languages and repositories for patterns that address an individual problem statement. When one or more candidate patterns is found, it is entered in the row corresponding to the problem statement and the column that corresponds to the pattern type. The name of the pattern is entered as a hyperlink to the URL of the Web address that contains a complete description of the pattern.

14.2.5 Common Design Mistakes

Several common mistakes may occur when pattern-based design is used. In some cases, not enough time has been spent to understand the underlying problem and its context and forces, and you may select a pattern that looks right but is inappropriate for the solution required. Once the wrong pattern is selected, you refuse to see your error and force-fit the pattern. In other cases, the problem has forces that are not

FIGURE 14.2

A pattern-organizing table
Source: Adapted from Microsoft, "Prescriptive Architecture: Integration and Patterns," MSDN, May 2004.

	Database	Application	Implementation	Infrastructure
Data/Content				
Problem statement ...	PatternName(s)		PatternName(s)	
Problem statement ...		PatternName(s)		PatternName(s)
Problem statement ...	PatternName(s)			PatternName(s)
Architecture				
Problem statement ...		PatternName(s)		
Problem statement ...		PatternName(s)		PatternName(s)
Problem statement ...				
Component-level				
Problem statement ...		PatternName(s)	PatternName(s)	
Problem statement ...				PatternName(s)
Problem statement ...		PatternName(s)	PatternName(s)	
User interface				
Problem statement ...		PatternName(s)	PatternName(s)	
Problem statement ...		PatternName(s)	PatternName(s)	
Problem statement ...		PatternName(s)	PatternName(s)	

considered by the pattern you've chosen, resulting in a poor or erroneous fit. Sometimes a pattern is applied too literally and the required adaptations for your problem space are not implemented.

Can these mistakes be avoided? In most cases, the answer is yes. Every good designer looks for a second opinion and welcomes review of her work. The review techniques discussed in Chapter 16 can help to ensure that the pattern-based design you've developed will result in a high-quality solution for the software problem to be solved.

14.3 ARCHITECTURAL PATTERNS

If a house builder decides to construct a center-hall colonial, there is a single architectural style that can be applied. The details of the style (e.g., number of fireplaces, façade of the house, placement of doors and windows) can vary considerably, but once the decision on the overall architecture of the house is made, the style is imposed on the design.[5]

Architectural patterns are a bit different. For example, every house (and every architectural style for houses) employs a **Kitchen** pattern. The **Kitchen** pattern and

5 This implies that there will be a central foyer and hallway, that rooms will be placed to the left and right of the foyer, that the house will have two (or more) stories, that the bedrooms of the house will be upstairs, and so on. These "rules" are imposed once the decision is made to use the center-hall colonial style.

patterns it collaborates with address problems associated with the storage and preparation of food, the tools required to accomplish these tasks, and rules for placement of these tools relative to workflow in the room. In addition, the pattern might address problems associated with countertops, lighting, wall switches, a central island, flooring, and so on. Obviously, there is more than a single design for a kitchen, often dictated by the context and system of forces. But every design can be conceived within the context of the "solution" suggested by the **Kitchen** pattern.

A software architecture may have several architectural patterns that address issues such as concurrency, persistence, and distribution. Before a representative architectural pattern can be chosen in a specific domain, it must be assessed for its appropriateness for the application and the overall architectural style, as well as the context and system of forces that it specifies.

14.4 COMPONENT-LEVEL DESIGN PATTERNS

Component-level design patterns provide you with proven solutions that address one or more subproblems extracted from the requirements model. In many cases, design patterns of this type focus on some functional element of a system. For example, the **SafeHomeAssured.com** application must address the following design subproblem: *How do I get product specifications and related information for any* SafeHome *device?*

Having stated the subproblem that must be solved, you should now consider context and the system of forces that affect the solution. Examining the appropriate requirements model use case, you learn that the consumer uses the specification for a *SafeHome* device (e.g., a security sensor or camera) for informational purposes. However, other information that is related to the specification (e.g., pricing) may be used when e-commerce functionality is selected.

The solution to the subproblem involves a search. Because searching is a very common problem, it should come as no surprise that there are many search-related patterns. Looking through several patterns repositories, you find the following patterns, along with the problem that each solves:

AdvancedSearch. Users must find a specific item in a large collection of items.

HelpWizard. Users need help on a certain topic related to the website or when they need to find a specific page within the site.

SearchArea. Users must find a page.

SearchTips. Users need to know how to control the search engine.

SearchResults. Users must process a list of search results.

SearchBox. Users must find an item or specific information.

For **SafeHomeAssured.com**, the number of products is not particularly large, and each has a relatively simple categorization, so **AdvancedSearch** and **HelpWizard** are probably not necessary. Similarly, the search is simple enough not to require **SearchTips**. The description of **SearchBox**, however, is given (in part) as:

Search Box

(*Adapted from* www.welie.com/patterns/showPattern.php?patternID=search)

Problem:	The users need to find an item or specific information.
Motivation:	Any situation in which a keyword search is applied across a collection of content objects organized as Web pages.
Context:	Rather than using navigation to acquire information or content, the user wants to do a direct search through content contained on multiple Web pages. Any website that already has primary navigation. User may want to search for an item in a category. User might want to further specify a query.
Forces:	The website already has primary navigation. Users may want to search for an item in a category. Users might want to further specify a query using simple Boolean operators.
Solution:	Offer search functionality consisting of a search label, a keyword field, a filter if applicable, and a "go" button. Pressing the return key has the same function as selecting the go button. Also provide Search Tips and examples in a separate page. A link to that page is placed next to the search functionality. The edit box for the search term is large enough to accommodate three typical user queries (typically around 20 characters). If the number of filters is more than 2, use a combo box for filters selection, otherwise a radio button. The search results are presented on a new page with a clear label containing at least "Search results" or similar. The search function is repeated in the top part of the page with the entered keywords, so that the users know what the keywords were.

The pattern description continues with other entries, as described in Section 14.1.3.

The pattern goes on to describe how the search results are accessed, presented, matched, and so on. Based on this, the **SafeHomeAssured.com** team can design the components required to implement the search or (more likely) acquire existing reusable components.

SAFEHOME



Applying Patterns

The scene: Informal discussion during the design of a software increment that implements sensor control via the Internet for **SafeHomeAssured.com.**

The players: Jamie, responsible for design, and Vinod, **SafeHomeAssured.com** chief system architect.

The conversation:

Vinod: So how is the design of the camera control interface coming along?

Jamie: Not too bad. I've designed most of the capability to connect to the actual sensors without too many problems. I've also started thinking about the interface for the users to move,

pan, and zoom the cameras from a remote device, but I'm not sure I've got it right yet.

Vinod: What have you come up with?

Jamie: Well, the requirements are that the camera control needs to be highly interactive—as the user moves the control, the camera should move as soon as possible. So, I was thinking of having a set of buttons laid out like a normal camera, but when the user clicks them, it controls the camera.

Vinod: Hmmm. Yeah, that would work, but I'm not sure it's right—for each click of a control you need to wait for the whole client-server communication to occur, and so you won't get a good sense of quick feedback.

Jamie: That's what I thought—and why I wasn't very happy with the approach, but I'm not sure how else I might do it.

Vinod: Well, why not just use the **Interactive-DeviceControl** pattern?

Jamie: Uhmmm—what's that? I haven't heard of it.

Vinod: It's basically a pattern for exactly the problem you are describing. The solution it proposes is basically to create a control connection to the server with the device, through which control commands can be sent. That way you don't need to send normal HTTP requests. And the pattern even shows how you can implement this using some simple AJAX techniques. You have some simple client-side JavaScript that communicates directly with the server and sends the commands as soon as the user does anything.

Jamie: Cool! That's just what I needed to solve this thing. Where do I find it?

Vinod: It's available in an online repository. Here's the URL.

Jamie: I'll go check it out.

Vinod: Yep—but remember to check the consequences field for the pattern. I seem to remember that there was something in there about needing to be careful about issues of security. I think it might be because you are creating a separate control channel and so bypassing the normal Web security mechanisms.

Jamie: Good point. I probably wouldn't have thought of that! Thanks.

14.5 Anti-Patterns

Design patterns provide you with proven solutions that address one or more problems extracted from the requirements model. *Anti-patterns* describe commonly used solutions to design problems that usually have negative effects on software quality. In other words, they describe bad solutions to the design problems or at the very least describe the consequence of applying a design pattern in the wrong context. Anti-patterns can provide tools to help developers recognize when these problems exist and may provide detailed plans for reversing the underlying problem causes and implementing better solutions to these problems [Bro98]. Anti-patterns can provide valuable guidance to developers when they are looking for ways to refactor software products to improve their quality. In addition, they can be used by technical reviewers (Chapter 16) to uncover area of concern.

Brown and his colleagues [Bro98] make the following comparisons between pattern and anti-pattern descriptions. Design patterns are usually written from the bottom up. A design pattern description starts with a recurring solution to a problem and then adds forces, symptoms, and context elements of the situation in which the solution is to be applied. Anti-patterns are written from the top down.

An anti-pattern description takes a recurring design problem, or a bad development practice, and then lists its symptoms and negative consequences. It is then possible to include a recommended procedure for reducing the consequences documented in the anti-pattern. **The Blob** anti-pattern is presented in the following example.

The Blob

(*Adapted from* http://antipatterns.com/briefing/sld024.htm)

Problem:

Single class with large number of attributes, operations, or both.

Symptoms and Consequences:

- A disparate collection of unrelated attributes and operations encapsulated in a single class.
- An overall lack of cohesiveness of the attributes and operations is typical of **The Blob**.
- A single controller class with associated simple, data-object classes.
- A single controller class encapsulates the entire functionality like a procedural main program.
- **The Blob** limits the ability to modify the system without affecting the functionality of other objects.
- Modifications to other objects in the system are also likely to impact **The Blob's** class.
- **The Blob** class is typically too complex for reuse and testing.
- **The Blob** class may be expensive to load into memory and uses excessive resources.

Typical Causes:

- Lack of an object-oriented architecture.
- The team may be lacking appropriate abstraction skills.
- Lack of defined software architecture.
- Lack of programming language support for architectural design.
- In iterative projects, developers tend to add little pieces of functionality to existing classes.
- Defining system architecture during requirements analysis often leads to **The Blob**.

Solution:

- The solution involves a form of re-factoring.
- The key is to move behavior away from **The Blob**.
- Reallocate behavior to other data objects in ways that make **The Blob** less complex.

A complete discussion of anti-patterns is beyond the scope of this book. Several anti-patterns with colorful and descriptive names appear in the sidebar that follows. We suspect you will recognize many of the anti-pattern names as development practices you were told to avoid when you learned to program.

INFO



Selected Anti-Patterns

A wide variety of anti-design patterns have been identified to assist developers in making refactoring decisions. Comprehensive descriptions of each of these patterns can be obtained via links at **https://en.wikipedia.org/wiki/Anti-pattern.**

- **Big ball of mud.** A system with no recognizable structure
- **Stovepipe system.** A barely maintainable assemblage of ill-related components
- **Boat anchor.** Retaining a part of a system that no longer has any use
- **Lava flow.** Retaining undesirable (redundant or low-quality) code because removing it is too expensive or has unpredictable consequences
- **Spaghetti code.** Program whose structure is barely comprehensible, especially because of misuse of code structures
- **Copy and paste programming.** Copying existing code several times rather than creating generic solutions
- **Silver bullet.** Assume that a favorite technical solution will always solve a larger process or problem
- **Programming by permutation.** Trying to approach a solution by successively modifying the code to see if it works

14.6 USER INTERFACE DESIGN PATTERNS

Hundreds of user interface (UI) patterns have been proposed in recent years. Most fall within one of 10 categories of patterns as described by Tidwell [Tid11] and vanWelie [Wel01]. A few representative categories (discussed with a simple example[6]) follow:

Whole UI. Provide design guidance for top-level structure and navigation throughout the entire interface.

Pattern:	**Top-level navigation**
Brief description:	Used when a site or application implements several major functions. Provides a top-level menu, often coupled with a logo or identifying graphic, that enables direct navigation to any of the system's major functions.
Details:	Major functions (generally limited to between four and seven function names) are listed across the top of the display (vertical column formats are also possible) in a horizontal line of text. Each name provides a link to the appropriate function or information source. Often used with the **BreadCrumbs** pattern discussed later.
Navigation elements:	Each function/content name represents a link to the appropriate function or content.

Page layout. Address the general organization of pages (for websites) or distinct screen displays (for interactive applications).

6 An abbreviated pattern template is used here. Full pattern descriptions (along with dozens of other patterns) can be found at [Tid11] and [Wel01].

Pattern:	**Card stack**
Brief description:	Used when several specific subfunctions or content categories related to a feature or function must be selected in random order. Provides the appearance of a stack of tabbed cards, each selectable with a mouse click and each representing specific subfunctions or content categories.
Details:	Tabbed cards are a well-understood metaphor and are easy for the user to manipulate. Each tabbed card (divider) may have a slightly different format. Some may require input and have buttons or other navigation mechanisms; others may be informational. May be combined with other patterns such as **drop-down list, fill-in-the-blanks**, and others.
Navigation elements:	A mouse click on a tab causes the appropriate card to appear. Navigation features within the card may also be present, but in general, these should initiate a function that is related to card data, not cause an actual link to some other display.

E-commerce. Specific to websites, these patterns implement recurring elements of e-commerce applications.

Pattern:	**Shopping cart**
Brief description:	Provides a list of items selected for purchase.
Details:	Lists item, quantity, product code, availability (in stock, out of stock), price, delivery information, shipping costs, and other relevant purchase information. Also provides ability to edit (e.g., remove, change quantity).
Navigation elements:	Contains ability to proceed with shopping or go to checkout.

Each of the preceding example patterns (and all patterns within each category) would also have a complete component-level design, including design classes, attributes, operations, and interfaces. If you have further interest, see [Cho16], [Gas17], [Kei18], [Tid11], [Hoo12], and [Wel01] for further information. The role of anti-patterns and their effects on UX design can be found in [Sch15] and [Gra18].

14.7 MOBILITY DESIGN PATTERNS

Throughout this chapter you've learned that there are different types of patterns and how they can be categorized. By their nature, mobile applications are all about the interface. In many cases, mobile UI patterns [Mob12] are represented as a collection of "best of breed" screen images for apps in a variety of different categories. Typical examples might include:

Check-in screens. How do I check in from a specific location, make a comment, and share comments with friends and followers on a social network?

Maps. How do I display a map within the context of an app that addresses some other subject? For example, review a restaurant and represent its location within a city.

Popovers. How do I represent a message or information (from the app or another user) that arises in real time or as the consequence of a user action?

Sign-up flows. How do I provide a simple way to sign in or register for information or functionality?

Custom tab navigation. How do I represent a variety of different content objects in a manner that enables the user to select the one she wants?

Invitations. How do I inform the user that he must participate in some action or dialog? Typical examples might include making use of a dialog box, a tool tip, or a self-playing video demo.

Additional information about mobile UI patterns can be found in [Nei14], [Hoo12], [McT16], [Abr17], and [Pun17]. In addition to UI patterns, Meier and his colleagues [Mei12] propose a variety of more general pattern descriptions for mobile apps.

14.8 SUMMARY

Design patterns provide a codified mechanism for describing problems and their solution in a way that allows the software engineering community to capture design knowledge for reuse. A pattern describes a problem, indicates the context enabling the user to understand the environment in which the problem resides, and lists a system of forces that indicate how the problem can be interpreted within its context and how the solution can be applied. In software engineering work, we identify and document generative patterns. These patterns describe an important and repeatable aspect of a system and then provide us with a way to build that aspect within a system of forces that is unique to a given context.

Architectural patterns describe broad-based design problems that are solved using a structural approach. Data patterns describe recurring data-oriented problems and the data modeling solutions that can be used to solve them. Component patterns (also referred to as design patterns) address problems associated with the development of subsystems and components, the ways in which they communicate with one another, and their placement within a larger architecture. Software anti-patterns describe commonly occurring solutions to a problem that lead to the creation of software products with poor quality. Interface design patterns describe common user interface problems and their solution with a system of forces that includes the specific characteristics of end users. Mobile patterns address the unique nature of the mobile interface and functionality and control elements that are specific to mobile platforms.

A framework provides an infrastructure in which patterns may reside and idioms describe programming language–specific implementation detail for all or part of a specific algorithm or data structure. A standard form or template is used for pattern descriptions. A pattern language encompasses a collection of patterns, each described using a standardized template and interrelated to show how these patterns collaborate to solve problems across an application domain.

Pattern-based design is used in conjunction with architectural, component-level, and user interface design methods. The design approach begins with an examination of the requirements model to isolate problems, define context, and describe the system of forces. Next, pattern languages for the problem domain are searched to determine if patterns exist for the problems that have been isolated. Once appropriate patterns have been found, they are used as a design guide.

PROBLEMS AND POINTS TO PONDER

14.1. Discuss the three "parts" of a design pattern, and provide a concrete example of each from some field other than software.

14.2. What is the difference between a pattern and an anti-pattern?

14.3. How do architectural patterns differ from component patterns?

14.4. What is a framework, and how does it differ from a pattern? What is an idiom, and how does it differ from a pattern?

14.5. Using the design pattern template presented in Section 14.1.3, develop a complete pattern description for a pattern suggested by your instructor.

14.6. Develop a skeletal pattern language for a sport with which you are familiar. You can begin by addressing the context, the system of forces, and the broad problems that a coach and team must solve. You need only specify pattern names and provide a one-sentence description for each pattern.

14.7. When Christopher Alexander says, "good design cannot be achieved simply by adding together performing parts," what do you think he means?

14.8. Using the pattern-based design tasks noted in Section 14.2.3, develop a skeletal design for the "interior design system" described in Section 12.3.2.

14.9. Build a pattern-organizing table for the patterns you used in Problem 14.8.

14.10. Using the design pattern template presented in Section 14.1.3, develop a complete pattern description for the **Kitchen** pattern mentioned in Section 14.3.

Design element: Quick Look icon magnifying glass: © Roger Pressman

PART

Three

QUALITY AND SECURITY

In this part of *Software Engineering: A Practitioner's Approach,* you'll learn about the principles, concepts, and techniques that are applied to manage and control software quality. These questions are addressed in the chapters that follow:

- What are the generic characteristics of high-quality software?
- How do we review quality, and how are effective reviews conducted?
- What is software quality assurance?
- What strategies are applicable for software testing?
- What methods are used to design effective test cases?
- Are there realistic methods that will ensure that software is correct?
- How can we manage and control changes that always occur as software is built?
- What measures and metrics can be used to assess the quality of requirements and design models, source code, and test cases?
- How do we ensure the security concerns for a product are being addressed during the software life cycle?

Once these questions are answered, you'll be better prepared to ensure that high-quality software has been produced.

CHAPTER

15 QUALITY CONCEPTS

The drumbeat for improved software quality began in earnest as software became increasingly integrated in every facet of our lives. By the 1990s, major corporations recognized that billions of dollars each year were being wasted on software that didn't deliver the features and functionality that were promised. Worse, both government and industry became increasingly concerned that a major software fault might cripple important infrastructure, costing tens of billions more. By the turn of the century, *CIO Magazine* trumpeted the headline, "Let's Stop Wasting $78 Billion a Year," lamenting the fact that "American businesses spend billions for software that doesn't do what it's supposed to do" [Lev01]. Sadly, at least one survey of the state of software quality practices done in 2014 suggests that maintenance and software evolution activities make up as much as 90 percent of the total software development costs [Nan14]. Poor software quality caused by a rush to release products without adequate testing continues to plague the software industry.

KEY CONCEPTS

cost of quality 317
good enough 316
liability 320
machine learning. 322
management actions 321
quality 311
quality dilemma. 315
quality dimensions 314
quality factors 312
quantitative quality assessment. 315
risks 319
security 320

QUICK LOOK

What is it? The answer isn't as easy as you might think. You know quality when you see it, and yet, it can be an elusive thing to define. But for computer software, quality is something that we must define, and that's what we'll do in this chapter.

Who does it? Everyone—software engineers, managers, all stakeholders—involved in the software process is responsible for quality.

Why is it important? You can do it right, or you can do it over again. If a software team stresses quality in all software engineering activities, it reduces the amount of rework that it must do. That results in lower costs, and more importantly, improved time to market.

What are the steps? To achieve high-quality software, four activities must occur: proven software engineering process and practice, solid project management, comprehensive quality control, and the presence of a quality assurance infrastructure.

What is the work product? Software that meets its customer's needs, performs accurately and reliably, and provides value to all who use it.

How do I ensure that I've done it right? Track quality by examining the results of all quality control activities, and measure quality by examining errors before delivery and defects released to the field.

Today, software quality remains an issue, but who is to blame? Customers blame developers, arguing that sloppy practices lead to low-quality software. Developers blame customers (and other stakeholders), arguing that irrational delivery dates and a continuing stream of changes force them to deliver software before it has been fully validated. Who's right? *Both*—and that's the problem. In this chapter, we consider software quality as a concept and examine why it's worthy of serious consideration whenever software engineering practices are applied.

15.1 What Is Quality?

In his mystical book *Zen and the Art of Motorcycle Maintenance,* Robert Persig [Per74] commented on the thing we call *quality*:

> Quality . . . you know what it is, yet you don't know what it is. But that's self-contradictory. But some things are better than others; that is, they have more quality. But when you try to say what the quality is, apart from the things that have it, it all goes poof! There's nothing to talk about . . . Obviously some things are better than others . . . but what's the betterness? . . . So round and round you go, spinning mental wheels and nowhere finding anyplace to get traction. What the hell is Quality? What is it?

Indeed—what is it?

At a somewhat more pragmatic level, David Garvin [Gar84] of the Harvard Business School suggests that "quality is a complex and multifaceted concept" that can be described from five different points of view. The *transcendental view* argues (like Persig) that quality is something you immediately recognize but cannot explicitly define. The *user view* sees quality in terms of an end user's specific goals. If a product meets those goals, it exhibits quality. The *manufacturer's view* defines quality in terms of the original specification of the product. If the product conforms to the spec, it exhibits quality. The *product view* suggests that quality can be tied to inherent characteristics (e.g., functions and features) of a product. Finally, the *value-based view* measures quality based on how much a customer is willing to pay for a product. In reality, quality encompasses all these views and more.

Quality of design refers to the characteristics that designers specify for a product. The grade of materials, tolerances, and performance specifications all contribute to the quality of design. As higher-grade materials are used, tighter tolerances and greater levels of performance are specified, and the design quality of a product increases if the product is manufactured according to specifications.

In software development, quality of design encompasses the degree to which the design meets the functions and features specified in the requirements model. *Quality of conformance* focuses on the degree to which the implementation follows the design and the resulting system meets its requirements and performance goals.

But are quality of design and quality of conformance the only issues that software engineers must consider? Robert Glass [Gla98] argues that a more "intuitive" relationship is in order:

$$\text{user satisfaction} = \text{compliant product} + \text{good quality} + \text{delivery within budget and schedule}$$

At the bottom line, Glass contends that quality is important, but if the user isn't satisfied, nothing else really matters. DeMarco [DeM98] reinforces this view when he states: "A product's quality is a function of how much it changes the world for the better." This view of quality contends that if a software product provides substantial benefit to its end users, they may be willing to tolerate occasional reliability or performance problems. A modern view of software quality requires attention to customer satisfaction as well as conformance to the product requirements [Max16].

15.2 SOFTWARE QUALITY

Even the most jaded software developers will agree that high-quality software is an important goal. But how do we define *software* quality? In the most general sense, software quality can be defined as: *An effective software process applied in a manner that creates a useful product that provides measurable value for those who produce it and those who use it.*

There is little question that the preceding definition could be modified or extended and debated endlessly. For the purposes of this book, the definition serves to emphasize three important points:

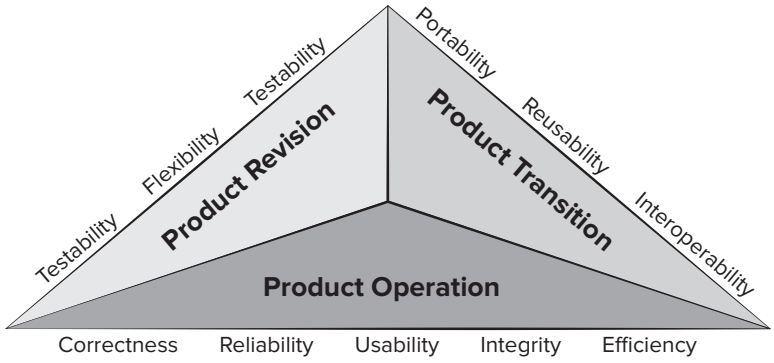
1. An *effective software process* establishes the infrastructure that supports any effort at building a high-quality software product. The management aspects of process create the checks and balances that help avoid project chaos—a key contributor to poor quality. Software engineering practices allow the developer to analyze the problem and design a solid solution—both critical to building high-quality software. Finally, umbrella activities such as change management and technical reviews have as much to do with quality as any other part of software engineering practice.
2. A *useful product* delivers the content, functions, and features that the end user desires, but as important, it delivers these assets in a reliable, error-free way. A useful product always satisfies those requirements that have been explicitly stated by stakeholders. In addition, it satisfies a set of implicit requirements (e.g., ease of use) that are expected of all high-quality software.
3. By *adding value for both the producer and user* of a software product, high-quality software provides benefits for the software organization and the end-user community. The software organization gains added value because high-quality software requires less maintenance effort, fewer bug fixes, and reduced customer support. This enables software engineers to spend more time creating new applications and less on rework. The user community gains added value because the application provides a useful capability in a way that expedites some business process. The end result is (1) greater software product revenue, (2) better profitability when an application supports a business process, and/or (3) improved availability of information that is crucial for the business.

15.2.1 Quality Factors

Several software quality models and standards have been proposed in the software engineering literature. David Garvin [Gar84] writes that quality is a multifaceted

FIGURE 15.1

McCall's software quality factors



phenomena and requires the use of multiple perspectives to assess it. McCall and Walters [McC77] proposed a useful way to think about and organize factors affecting software quality. Their software quality factors (shown in Figure 15.1) focus on three software product aspects: its operation characteristics, its ability to undergo change, and its adaptability to new environments. McCall's quality factors provide a basis for engineering software that provides high levels of user satisfaction by focusing on the overall user experience delivered by the software product. This cannot be done unless developers ensure that the requirements specification is correct and that defects are removed early in the software development process [Max16].

The ISO 25010 quality model is the newest standard (created in 2011 and revised in 2017).[1] This standard defines two quality models. The *quality in use* model describes five characteristics that are appropriate when considering using the product in a particular context (e.g., using the product on a specific platform by a human). The *product quality* model describes eight characteristics that focus on both the static and dynamic nature of computer systems.

- **Quality in Use Model**
  - **Effectiveness.** Accuracy and completeness with which users achieve goals
  - **Efficiency.** Resources expended to achieve user goals completely with desired accuracy
  - **Satisfaction.** Usefulness, trust, pleasure, comfort
  - **Freedom from risk.** Mitigation of economic, health, safety, and environmental risks
  - **Context coverage.** Completeness, flexibility
- **Product Quality**
  - **Functional suitability.** Complete, correct, appropriate
  - **Performance efficiency.** Timing, resource utilization, capacity
  - **Compatibility.** Coexistence, interoperability

1 The ISO 25010 can be found at https://www.iso.org/standard/35733.html.

- **Usability.** Appropriateness, learnability, operability, error protection, aesthetics, accessibility
- **Reliability.** Maturity, availability, fault tolerance, recoverability
- **Security.** Confidentiality, integrity, accountability, authenticity
- **Maintainability.** Modularity, reusability, modifiability, testability
- **Portability.** Adaptability, installability, replaceability

The addition of the quality in use model helps to emphasize the importance of customer satisfaction in the assessment of software quality. The product quality model points out the importance of assessing both the functional and nonfunctional requirements for the software product [Max16].

15.2.2 Qualitative Quality Assessment

The quality dimensions and factors presented in Section 15.2.1 focus on the complete software product and can be used as a generic indication of the quality of an application. Your software team can develop a set of quality characteristics and associated questions that would probe the degree to which each factor has been satisfied.[2] For example, ISO 25010 identifies *usability* as an important quality factor. If you were asked to evaluate a user interface and assess its usability, how would you proceed?

Although it's tempting to develop quantitative measures for the quality factors noted in Section 15.2.1, you can also create a simple checklist of attributes that provide a solid indication that the factor is present. You might start with the sub-characteristics suggested for usability in ISO 25010: appropriateness, learnability, operability, error protection, aesthetics, and accessibility. You and your team might decide to create a user questionnaire and a set of structured tasks for users to perform. You might observe the users while they perform these tasks and have them complete the questionnaire when they finish. We will discuss usability testing in more detail in Chapter 21.

To conduct your assessment, you and your team will need to address specific, measurable (or at least, recognizable) attributes of the interface. Your tasks might be focused on answering the following questions:

- How quickly can users determine whether the software product can be used to help them complete their task or not? (appropriateness)
- How long does it take users to learn how to use the system functions needed to complete their task? (learnability)
- Is the user able to recall how to use system functions in subsequent testing sessions without having to relearn them? (learnability)
- How long does it take users to complete tasks using the system? (operability)
- Does the system try to prevent users from making errors? (error protection)
- Does the system allow users to undo operations that may have resulted in errors? (error protection)
- Do answers give favorable responses to questions about the appearance of the user interface? (aesthetics)

2 These characteristics and questions would be addressed as part of a software review (Chapter 16).

- Does the interface conform to the expectations set forth by the golden rules from Chapter 12? (accessibility)
- Does the user interface conform to the accessibility checklist items required for the intended users? (accessibility)

As the interface design is developed, the software team would review the design prototype and ask the questions noted. If the answer to most of these questions is yes, it is likely that the user interface exhibits high quality. A collection of questions similar to these would be developed for each quality factor to be assessed. In the case of usability, it is always important to observe representative users interact with the system. For some other quality factors it may be important to test the software in the wild (or at least in the production environment).

15.2.3 Quantitative Quality Assessment

In the preceding subsections, we have presented a set of qualitative factors for the "measurement" of software quality. The software engineering community strives to develop precise measures for software quality and is sometimes frustrated by the subjective nature of the activity. Cavano and McCall [Cav78] discuss this situation:

> Subjectivity and specialization . . . apply to determining software quality. To help solve this problem, a more precise definition of software quality is needed as well as some way to derive quantitative measurements of software quality for objective analysis . . . Since there is no such thing as absolute knowledge, one should not expect to measure software quality exactly, for every measurement is partially imperfect.

Several software design defects can be detected using software metrics. The process consists of finding code fragments that suggest the presence of things like high coupling or unnecessary levels of complexity. Internal code attributes can be described quantitatively using software metrics. Any time software metric values computed for a code fragment fall outside the range of acceptable values, it signals the existence of a quality problem that should be investigated [Max16].

In Chapter 23, we'll present a set of software metrics that can be applied to the quantitative assessment of software quality. In all cases, the metrics represent indirect measures; that is, we never really measure *quality* but rather some manifestation of quality. The complicating factor is the precise relationship between the variable that is measured and the quality of software.

15.3 The Software Quality Dilemma

In an interview [Ven03] published on the Web, Bertrand Meyer discusses what we call the *quality dilemma:*

> If you produce a software system that has terrible quality, you lose because no one will want to buy it. If on the other hand you spend infinite time, extremely large effort, and huge sums of money to build the absolutely perfect piece of software, then it's going to take so long to complete and it will be so expensive to produce that you'll be out of business anyway. Either you missed the market window, or you simply exhausted all your resources. So people in industry try to get to that magical middle ground where the product is good enough not to be rejected right away, such as during evaluation, but also not the object of so much perfectionism and so much work that it would take too long or cost too much to complete.

It's fine to state that software engineers should strive to produce high-quality systems. It's even better to apply good practices in your attempt to do so. But the situation discussed by Meyer is real life and represents a dilemma for even the best software engineering organizations. When you're faced with the quality dilemma (and everyone is faced with it at one time or another), try to achieve balance—enough effort to produce acceptable quality without burying the project.

15.3.1 "Good Enough" Software

Stated bluntly, if we are to accept the argument made by Meyer, is it acceptable to produce "good enough" software? The answer to this question must be yes, because software companies do it every day [Rod17]. They create software with known bugs and deliver it to a broad population of end users. They recognize that some of the functions and features delivered in version 1.0 may not be of the highest quality and plan for improvements in version 2.0. They do this knowing that some customers will complain, but they recognize that time to market may trump better quality as long as the delivered product is "good enough."

Exactly what is "good enough"? Good enough software delivers high-quality functions and features that end users desire, but at the same time it delivers other more obscure or specialized functions and features that contain known bugs. The software vendor hopes that the vast majority of end users will overlook the bugs because they are so happy with other application functionality.

This idea may resonate with many readers. If you're one of them, we can only ask you to consider some of the arguments against "good enough." It is true that "good enough" may work in some application domains and for a few major software companies. After all, if a company has a large marketing budget and can convince enough people to buy version 1.0, it has succeeded in locking them in. As we noted earlier, it can argue that it will improve quality in subsequent versions. By delivering a good enough version 1.0, it has cornered the market.

If you work for a small company, be wary of this philosophy. When you deliver a good enough (buggy) product, you risk permanent damage to your company's reputation. You may never get a chance to deliver version 2.0 because bad buzz may cause your sales to plummet and your company to fold.

If you work in certain application domains (e.g., real-time embedded software) or build application software that is integrated with hardware (e.g., automotive software, telecommunications software), delivering software with known bugs can be negligent and open your company to expensive litigation. In some cases, it can even be criminal. No one wants good enough aircraft avionics software! Ebert writes that the software process model should provide clear criteria to guide developers to determine what is really "good enough" for the intended application domain [Ebe14].

So, proceed with caution if you believe that "good enough" is a shortcut that can solve your software quality problems. It can work, but only for a few and only in a limited set of application domains.[3]

3 A worthwhile discussion of the pros and cons of "good enough" software can be found in [Bre02].

15.3.2 The Cost of Quality

The argument goes something like this: *We know that quality is important, but it costs us time and money—too much time and money to get the level of software quality we really want.* On its face, this argument seems reasonable (see Meyer's comments earlier in this section). There is no question that quality has a cost, but lack of quality also has a cost—not only to end users who must live with buggy software, but also to the software organization that has built and must maintain it. The real question is this: *Which cost should we be worried about?* To answer this question, you must understand both the cost of achieving quality and the cost of low-quality software.

The *cost of quality* includes all costs incurred in the pursuit of quality or in performing quality-related activities and the downstream costs of lack of quality. To understand these costs, an organization should collect metrics to provide a baseline for the current cost of quality, identify opportunities for reducing these costs, and provide a normalized basis of comparison. The cost of quality can be divided into costs associated with prevention, appraisal, and failure.

Prevention costs include (1) the cost of management activities required to plan and coordinate all quality control and quality assurance activities, (2) the cost of added technical activities to develop complete requirements and design models, (3) test planning costs, and (4) the cost of all training associated with these activities. Don't be afraid to incur significant prevention costs. Rest assured that your investment will provide an excellent return.

Appraisal costs include activities to gain insight into product condition the "first time through" each process. Examples of appraisal costs include: (1) the cost of conducting technical reviews (Chapter 16) for software engineering work products, (2) the cost of data collection and metrics evaluation (Chapter 23), and (3) the cost of testing and debugging (Chapters 19 through 21).

Failure costs are those that would disappear if no errors appeared before shipping a product to customers. Failure costs may be subdivided into internal failure costs and external failure costs. *Internal failure costs* are incurred when you detect an error in a product prior to shipment. Internal failure costs include: (1) the cost required to perform rework (repair) to correct an error, (2) the cost that occurs when rework inadvertently generates side effects that must be mitigated, and (3) the costs associated with the collection of quality metrics that allow an organization to assess the modes of failure. *External failure costs* are associated with defects found after the product has been shipped to the customer. Examples of external failure costs are complaint resolution, product return and replacement, help line support, and labor costs associated with warranty work. A poor reputation and the resulting loss of business is another external failure cost that is difficult to quantify but nonetheless very real. Bad things happen when low-quality software is produced.

In an indictment of software developers who refuse to consider external failure costs, Cem Kaner [Kan95] states:

> Many of the external failure costs, such as goodwill, are difficult to quantify, and many companies therefore ignore them when calculating their cost-benefit tradeoffs. Other external failure costs can be reduced (e.g. by providing cheaper, lower-quality, post-sale support, or by charging customers for support) without increasing customer satisfaction. By ignoring the costs to our customers of bad products, quality engineers encourage quality-related decision-making that victimizes our customers, rather than delighting them.

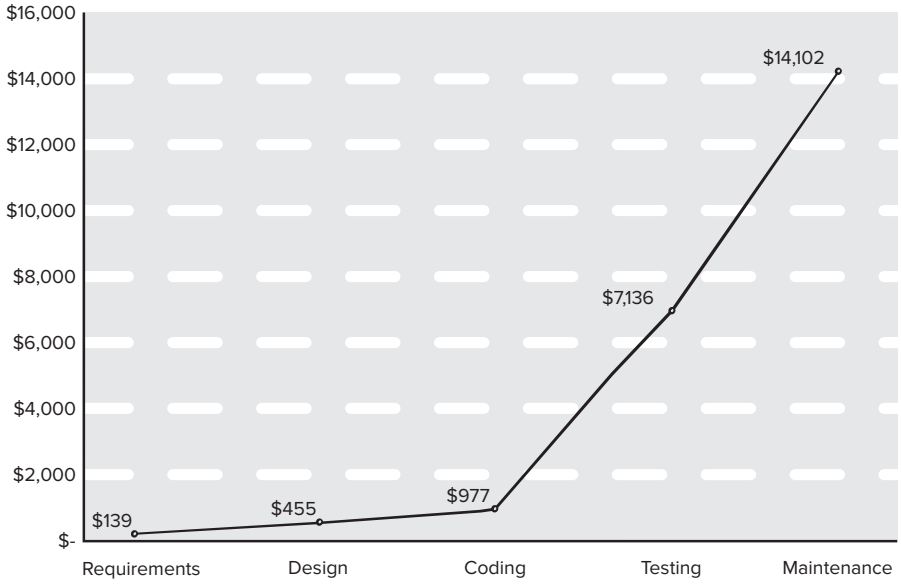


FIGURE 15.2

Relative cost of correcting errors and defects

Source: Boehm, Barry and Basili, Victor R., "Software Defect Reduction Top 10 List," IEEE Computer, vol. 34, no. 1, January 2001.

As expected, the relative costs to find and repair an error or defect increase dramatically as we go from prevention to detection to internal failure to external failure costs. Figure 15.2, based on data collected by Boehm and Basili [Boe01b] and illustrated by Cigital Inc. [Cig07], illustrates this phenomenon.

The industry average cost to correct a defect during code generation is approximately $977 per error. The industry average cost to correct the same error if it is discovered during system testing is $7,136 per error. Cigital Inc. [Cig07] considers a large application that has 200 errors introduced during coding.

> According to industry average data, the cost of finding and correcting defects during the coding phase is $977 per defect. Thus, the total cost for correcting the 200 "critical" defects during this phase (200 × $977) is approximately $195,400.
>
> Industry average data shows that the cost of finding and correcting defects during the system testing phase is $7,136 per defect. In this case, assuming that the system testing phase revealed approximately 50 critical defects (or only 25% of those found by Cigital in the coding phase), the cost of finding and fixing those defects (50 × $7,136) would have been approximately $356,800. This would also have resulted in 150 critical errors going undetected and uncorrected. The cost of finding and fixing these remaining 150 defects in the maintenance phase (150 × $14,102) would have been $2,115,300. Thus, the total cost of finding and fixing the 200 defects after the coding phase would have been $2,472,100 ($2,115,300 + $356,800).

Even if your software organization has costs that are half of the industry average (most have no idea what their costs are!), the cost savings associated with early quality control and assurance activities (conducted during requirements analysis and design) are compelling.

SAFEHOME



Quality Issues

The scene: Doug Miller's office as the *SafeHome* software project begins.

The players: Doug Miller, manager of the *SafeHome* software engineering team, and other members of the product software engineering team.

The conversation:

Doug: I was looking at an industry report on the costs of repairing software defects. It's pretty sobering.

Jamie: We're already working on developing test cases for each functional requirement.

Doug: That's good, but I was noticing that it costs eight times as much to repair a defect that is discovered in testing than it does if the defect is caught and repaired during coding.

Vinod: We're using pair programming, so we should be able to catch most of the defects during coding.

Doug: I think you're missing the point. Quality is more than simply removing coding errors. We need to look at the project quality goals and ensure that the evolving software products are meeting them.

Jamie: Do you mean things like usability, security, and reliability?

Doug: Yes, I do. We need to build checks into the software process to monitor our progress toward meeting our quality goals.

Vinod: Can't we finish the first prototype and then check it for quality?

Doug: I'm afraid not. We must establish a culture of quality early in the project.

Vinod: What do you want us to do, Doug?

Doug: I think we will need to find a technique that will allow us to monitor the quality of the *SafeHome* products. Let's think about this and revisit this again tomorrow.

15.3.3 Risks

In Chapter 1 of this book, we wrote, "people bet their jobs, their comforts, their safety, their entertainment, their decisions, and their very lives on computer software. It better be right." The implication is that low-quality software increases risks for both the developer and the end user.[4] In the preceding subsection, we discussed one of these risks (cost). But the downside of poorly designed and implemented applications does not always stop with dollars and time. An extreme example [Gag04] might serve to illustrate.

Throughout the month of November 2000 at a hospital in Panama, 28 patients received massive overdoses of gamma rays during treatment for a variety of cancers. In the months that followed, 5 of these patients died from radiation poisoning and 15 others developed serious complications. What caused this tragedy? A software package, developed by a U.S. company, was modified by hospital technicians to compute modified doses of radiation for each patient.

4 In an article titled "And the 'Most Shocking Software Failure' Award Goes To . . ." Chelsea Frischnecht provides a few brief examples of what can go wrong. The article can be found at https://www.tricentis.com/blog/2017/03/01/software-fail-awards/.

The three Panamanian medical physicists, who tweaked the software to provide additional capability, were charged with second-degree murder. The U.S. company was faced with serious litigation in two countries. Gage and McCormick comment:

> This is not a cautionary tale for medical technicians, even though they can find themselves fighting to stay out of jail if they misunderstand or misuse technology. This also is not a tale of how human beings can be injured or worse by poorly designed or poorly explained software, although there are plenty of examples to make the point. This is a warning for any creator of computer programs: that software quality matters, that applications must be foolproof, and that—whether embedded in the engine of a car, a robotic arm in a factory or a healing device in a hospital—poorly deployed code can kill.

Poor quality leads to risks, some of them very serious.[5]

15.3.4 Negligence and Liability

The story is all too common. A governmental or corporate entity hires a major software developer or consulting company to analyze requirements and then design and construct a software-based "system" to support some major activity. The system might support a major corporate function (e.g., pension management) or some governmental function (e.g., health care administration or homeland security).

Work begins with the best of intentions on both sides, but by the time the system is delivered, things have gone bad. The system is late, fails to deliver desired features and functions, is error-prone, and does not meet with customer approval. Litigation ensues.

In most cases, the customer claims that the developer has been negligent (in the manner in which it has applied software practices) and is therefore not entitled to payment. The developer often claims that the customer has repeatedly changed its requirements and has subverted the development partnership in other ways. In every case, the quality of the delivered system comes into question.

15.3.5 Quality and Security

As the criticality of Web-based and mobile systems grows, application security has become increasingly important. Stated simply, software that does not exhibit high quality is easier to hack, and as a consequence, low-quality software can indirectly increase the security risk with all its attendant costs and problems.

In an interview in *ComputerWorld,* author and security expert Gary McGraw comments [Wil05]:

> Software security relates entirely and completely to quality. You must think about security, reliability, availability, dependability—at the beginning, in the design, architecture, test, and coding phases, all through the software life cycle [process]. Even people aware of the software security problem have focused on late life-cycle stuff. The earlier you find the software problem, the better. And there are two kinds of software problems. One is bugs, which are implementation problems. The other is software flaws—architectural problems in the design. People pay too much attention to bugs and not enough on flaws.

To build a secure system, you must focus on quality, and that focus must begin during design. The concepts and methods discussed in Part Two of this book lead to

5 In early 2019, an error in the flight control software produced by a major aircraft manufacturer was directly linked to two crashes and the deaths of 346 people.

a software architecture that reduces "flaws." A more detailed discussion of security engineering is presented in Chapter 18.

15.3.6 The Impact of Management Actions

Software quality is often influenced as much by management decisions as it is by technology decisions. Even the best software engineering practices can be subverted by poor business decisions and questionable project management actions.

In Part Four of this book we discuss project management within the context of the software process. As each project task is initiated, a project leader will make decisions that can have a significant impact on product quality.

Estimation Decisions. A software team is rarely given the luxury of providing an estimate for a project *before* delivery dates are established and an overall budget is specified. Instead, the team conducts a "sanity check" to ensure that delivery dates and milestones are rational. In many cases there is enormous time-to-market pressure that forces a team to accept unrealistic delivery dates. As a consequence, shortcuts are taken, activities that lead to higher-quality software may be skipped, and product quality suffers. If a delivery date is irrational, it is important to hold your ground. Explain why you need more time, or alternatively, suggest a subset of functionality that can be delivered (with high quality) in the time allotted.

Scheduling Decisions. When a software project schedule is established (Chapter 25), tasks are sequenced based on dependencies. For example, because component **A** depends on processing that occurs within components **B, C,** and **D,** component **A** cannot be scheduled for testing until components **B, C,** and **D** are fully tested. A project schedule would reflect this. But if time is very short, and **A** must be available for further critical testing, you might decide to test **A** without its subordinate components (which are running slightly behind schedule), so that you can make it available for other testing that must be done before delivery. After all, the deadline looms. As a consequence, **A** may have defects that are hidden, only to be discovered much later. Quality suffers.

Risk-Oriented Decisions. Risk management (Chapter 26) is one of the key attributes of a successful software project. You really do need to know what might go wrong and establish a contingency plan if it does. Too many software teams prefer blind optimism, establishing a development schedule under the assumption that nothing will go wrong. Worse, they don't have a way of handling things that do go wrong. As a consequence, when a risk becomes a reality, chaos reigns, and as the degree of craziness rises, the level of quality invariably falls.

The software quality dilemma can best be summarized by stating Meskimen's law: *There's never time to do it right, but always time to do it over again.* Our advice: Taking the time to do it right is almost never the wrong decision.

15.4 ACHIEVING SOFTWARE QUALITY

Software quality doesn't just appear. It is the result of good project management and solid software engineering practice. Management and practice are applied within the context of four broad activities that help a software team achieve high software quality: software engineering methods, project management techniques, quality control actions, and software quality assurance.

15.4.1 Software Engineering Methods

If you expect to build high-quality software, you must understand the problem to be solved. You must also be capable of creating a design that conforms to the problem while at the same time exhibiting characteristics that lead to software that exhibits the quality dimensions and factors discussed in Section 15.2.

In Part Two of this book, we presented a wide array of concepts and methods that can lead to a reasonably complete understanding of the problem and a comprehensive design that establishes a solid foundation for the construction activity. If you apply those concepts and adopt appropriate analysis and design methods, the likelihood of creating high-quality software will increase substantially.

15.4.2 Project Management Techniques

The impact of poor management decisions on software quality has been discussed in Section 15.3.6. The implications are clear: If (1) a project manager uses estimation to verify that delivery dates are achievable, (2) schedule dependencies are understood and the team resists the temptation to use shortcuts, (3) risk planning is conducted so problems do not breed chaos, software quality will be affected in a positive way.

In addition, the project plan should include explicit techniques for quality and change management. Techniques that lead to good project management practices are discussed in Part Four of this book.

15.4.3 Machine Learning and Defect Prediction

Defect prediction [Mun17] is an important part of identifying software components that may have quality concerns. Defect prediction models use statistical techniques to examine the relationships among combinations of software metrics and software components containing known software defects. They can be an efficient and effective way for software developers to quickly identify defect-prone classes. This can reduce costs and development times [Mal16].

Machine learning is an application of artificial intelligence (AI) techniques that provide systems with the ability to learn and improve from experience without being explicitly programmed. Stated another way, machine learning focuses on the development of computer programs that can access data and use the data to learn for themselves. Machine learning techniques can be used to automate the process of discovering predictive relationships between software metrics and defective components [Ort17], [Li16], [Mal16].

Machine learning systems process large data sets containing representative combinations of metrics for defective and nondefective software components. These data are used to tune classification algorithms. Once the system has built a prediction model though this type of training, it can be used for quality assessment and defect prediction on data associated with future software products. Building these types of classifiers is a big part of what modern data scientists do. More discussion on the use of data science and software engineering appears in Appendix 2 of this book.

15.4.4 Quality Control

Quality control encompasses a set of software engineering actions that help to ensure that each work product meets its quality goals. Models are reviewed to ensure that

they are complete and consistent. Code may be inspected to uncover and correct errors before testing commences. A series of testing steps is applied to uncover errors in processing logic, data manipulation, and interface communication. A combination of measurement and feedback allows a software team to tune the process when any of these work products fails to meet quality goals. Quality control activities are discussed in detail throughout the remainder of Part Three of this book.

15.4.5 Quality Assurance

Quality assurance establishes the infrastructure that supports solid software engineering methods, rational project management, and quality control actions—all pivotal if you intend to build high-quality software. In addition, quality assurance consists of a set of auditing and reporting functions that assess the effectiveness and completeness of quality control actions. The goal of quality assurance is to provide management and technical staff with the data necessary to be informed about product quality, thereby gaining insight and confidence that actions to achieve product quality are working. Of course, if the data provided through quality assurance identifies problems, it is management's responsibility to address the problems and apply the necessary resources to resolve quality issues. Software quality assurance is discussed in detail in Chapter 17.

15.5 SUMMARY

Concern for the quality of the software-based systems has grown as software becomes integrated into every aspect of our daily lives. But it is difficult to develop a comprehensive description of software quality. In this chapter, quality has been defined as an effective software process applied in a manner that creates a useful product that provides measurable value for those who produce it and those who use it.

A wide variety of software quality dimensions and factors has been proposed over the years. All try to define a set of characteristics that, if achieved, will lead to high software quality. McCall's and the ISO 25010 quality factors establish characteristics such as reliability, usability, maintainability, functionality, and portability as indicators that quality exists.

Every software organization is faced with the software quality dilemma. In essence, everyone wants to build high-quality systems, but the time and effort required to produce "perfect" software are simply unavailable in a market-driven world. The question becomes, Should we build software that is "good enough"? Although many companies do just that, there is a significant downside that must be considered.

Regardless of the approach that is chosen, quality does have a cost that can be discussed in terms of prevention, appraisal, and failure. Prevention costs include all software engineering actions that are designed to prevent defects in the first place. Appraisal costs are associated with those actions that assess software work products to determine their quality. Failure costs encompass the internal price of failure and the external effects that poor quality precipitates.

Software quality is achieved through the application of software engineering methods, solid management practices, and comprehensive quality control—all supported by a software quality assurance infrastructure. In the chapters that follow, quality control and assurance are discussed in some detail.

PROBLEMS AND POINTS TO PONDER

15.1. Describe how you would assess the quality of a university before applying to it. What factors would be important? Which would be critical?

15.2. Using the definition of software quality proposed in Section 15.2, do you think it's possible to create a useful product that provides measurable value without using an effective process? Explain your answer.

15.3. Using the subattributes noted for the ISO 25010 quality factor "maintainability" in Section 15.2.1, develop a set of questions that explore whether or not these attributes are present. Follow the example shown in Section 15.2.2.

15.4. Describe the software quality dilemma in your own words.

15.5. What is "good enough" software? Name a specific company and specific products that you believe were developed using the good enough philosophy.

15.6. Considering each of the four aspects of the cost of quality, which do you think is the most expensive and why?

15.7. Do a Web search and find three other examples of "risks" to the public that can be directly traced to poor software quality. Consider beginning your search at http://catless.ncl.ac.uk/risks.

15.8. Are *quality* and *security* the same thing? Explain.

15.9. Explain why it is that many of us continue to live by Meskimen's law. What is it about the software business that causes this?

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

16

REVIEWS—A RECOMMENDED APPROACH

Software reviews are a "filter" for the software process work flow. Too few, and the flow is "dirty." Too many, and the flow slows to a trickle. Reviews are applied at various points during software engineering and serve to uncover errors and defects. Software reviews "purify" software engineering work products, including requirements and design models, code, and testing data. Using metrics, you can determine which reviews work and emphasize them and at the same time remove ineffective reviews from the flow to accelerate the process.

KEY CONCEPTS

bugs 326
cost effectiveness 329
defect amplification 327
defects 326
error density 328
errors 326
informal reviews 331
record keeping 333
review reporting 333
technical reviews 326

QUICK LOOK

What is it? You'll make mistakes as you develop software engineering work products. There's no shame in that—as long as you try hard, very hard, to find and correct the mistakes before they are delivered to end users. Technical reviews are the most effective mechanism for finding mistakes early in the software process.

Who does it? Software engineers perform technical reviews, also called peer reviews, with their colleagues. As we discussed in Chapters 3 and 4, sometimes it is wise to include other stakeholders in these reviews.

Why is it important? If you find an error early in the process, it is less expensive to correct. In addition, errors have a way of amplifying as the process proceeds. So a relatively minor error left untreated early in the process can be amplified into a major set of errors later in the project. Finally, reviews save time by reducing the amount of rework that will be required late in the project.

What are the steps? Your approach to reviews will vary depending on the type of review you select. In general, six steps are employed, although not all are used for every type of review: planning, preparation, structuring the meeting, noting errors, making corrections (done outside the review), and verifying that corrections have been performed properly.

What is the work product? The output of a review is a list of issues and/or errors that have been uncovered. In addition, the technical status of the work product is also indicated.

How do I ensure that I've done it right? First, select the type of review that is appropriate for your development culture. Follow the guidelines that lead to successful reviews. If the reviews that you conduct lead to higher-quality software, you've done it right.

Many different types of reviews can be conducted as part of software engineering. Each has its place. An informal meeting around the coffee machine is a form of review, if technical problems are discussed. A formal presentation of software architecture to an audience of customers, management, and technical staff is also a form of review. In this book, however, we focus on *technical* or *peer reviews,* exemplified by *casual reviews, walkthroughs,* and *inspections.* A technical review (TR) is the most effective filter from a quality control standpoint. Conducted by software engineers and other stakeholders for all project team members, the TR is an effective means for uncovering errors and improving software quality.

16.1 Cost Impact of Software Defects

Within the context of the software process, the terms *defect* and *fault* are synonymous. Both imply a quality problem that is discovered *after* the software has been released to end users (or to another framework activity in the software process). In earlier chapters, we used the term *error* to depict a quality problem that is discovered by software engineers (or others) *before* the software is released to the end user (or to another framework activity in the software process).

INFO



Bugs, Errors, and Defects

The goal of software quality control, and in a broader sense, quality management in general, is to remove quality problems in the software. These problems are referred to by various names—*bugs, faults, errors,* or *defects,* to name a few. Are these terms synonymous, or are there subtle differences between them?

In this book we make a clear distinction between an *error* (a quality problem found *before* the software is released to other stakeholders or end users) and a *defect* (a quality problem found only *after* the software has been released to end users or other stakeholders).[1] We make this distinction because errors and defects have very different economic, business, psychological, and human impacts. As software engineers, we want to find and correct as many errors as possible before the customer and/or end user encounter them. We want to avoid defects—because defects (justifiably) make software people look bad.

It is important to note, however, that the temporal distinction made between errors and defects in this book is *not* mainstream thinking. The general consensus within the software engineering community is that defects and errors, faults, and bugs are synonymous. That is, the point in time that the problem was encountered has no bearing on the term used to describe the problem. Part of the argument in favor of this view is that it is sometimes difficult to make a clear distinction between pre- and postrelease (e.g., consider an incremental process used in agile development).

Regardless of how you choose to interpret these terms, recognize that the point in time at which a problem is discovered does matter and that software engineers should try hard—*very* hard—to find problems before their customers and end users encounter them.

1 If software process improvement is considered, a quality problem that is propagated from one process framework activity (e.g., **modeling**) to another (e.g., **construction**) can also be called a "defect" because the problem should have been found before a work product (e.g., a design model) was "released" to the next activity.

The primary objective of a formal technical review (FTR) is to find errors before they are passed on to another software engineering activity or released to the end user. The obvious benefit of technical reviews is the early discovery of errors so that they do not propagate to the next step in the software process.

A number of industry studies indicate that design activities introduce between 50 and 65 percent of all errors (and ultimately, all defects) during the software process. However, review techniques have been shown to be up to 75 percent effective [Jon86] in uncovering design flaws. By detecting and removing a large percentage of these errors, the review process substantially reduces the cost of subsequent activities in the software process. We have known this for decades, and yet there are still many developers who do not believe the time spent on reviews is almost always less than the time required to rewrite bad code [Yad17].

16.2 Defect Amplification and Removal

Defect amplification is a concept originally proposed almost four decades ago [IBM81]. It helps to justify the effort expended on software reviews. In essence, defect amplification makes the following argument—an error introduced early in the software engineering work flow (e.g., during requirement modeling) and undetected, can and often will be amplified into multiple errors during design. If those errors are not uncovered (using effective reviews), they themselves may be further amplified into still more errors during coding. A single error introduced early and not uncovered and corrected can amplify into multiple errors later in the process. *Defect propagation* is a term used to describe the impact an undiscovered error has on future development activities or product behavior [Vit17].

As a development team moves deeper into the software process, the cost of finding and fixing an error grows. This simple reality is exacerbated by defect amplification and propagation because a single error may become multiple errors downstream. The cost of finding and fixing a single error can be signficant, but the cost to find and fix multiple errors propagated by a single earlier error is substantially more significant.

To conduct reviews, you must expend time and effort, and your development organization must spend money. However, the reality of defect amplification and propagation leaves little doubt that you can pay now or pay much more later. This is what *technical debt* (Chapter 11) is all about [Xia16] [Vit17].

16.3 Review Metrics and Their Use

Technical reviews are one of many actions that are required as part of good software engineering practice. Each action requires dedicated human effort. Because available project effort is finite, it is important for a software engineering organization to understand the effectiveness of each action by defining a set of metrics (Chapter 23) that can be used to assess their efficacy.

Although many metrics can be defined for technical reviews, a relatively small subset can provide useful insight. The following review metrics can be collected for each review that is conducted:

- **Preparation effort,** E_p. The effort (in person-hours) required to review a work product prior to the actual review meeting
- **Assessment effort,** E_a. The effort (in person-hours) that is expended during the actual review
- **Rework effort,** E_r. The effort (in person-hours) that is dedicated to the correction of those errors uncovered during the review
- **Review effort,** E_{review}. Represents the sum of effort measures for reviews:

  $$E_{\text{review}} = E_p + E_a + E_r$$

- **Work product size (WPS).** A measure of the size of the work product that has been reviewed (e.g., the number of UML models, the number of document pages, or the number of lines of code)
- **Minor errors found,** $\text{Err}_{\text{minor}}$. The number of errors found that can be categorized as minor (requiring less than some prespecified effort to correct)
- **Major errors found,** $\text{Err}_{\text{major}}$. The number of errors found that can be categorized as major (requiring more than some prespecified effort to correct)
- **Total errors found,** Err_{tot}. Represents the sum of the errors found:

  $$\text{Err}_{\text{tot}} = \text{Err}_{\text{minor}} + \text{Err}_{\text{major}}$$

- **Error density.** Represents the errors found per unit of work product reviewed:

  $$\text{Error density} = \frac{\text{Err}_{\text{tot}}}{\text{WPS}}$$

How might these metrics be used? As an example, consider a requirements model that is reviewed to uncover errors, inconsistencies, and omissions. It would be possible to compute the error density in several different ways. Assume the requirements model contains 18 UML diagrams as part of 32 overall pages of descriptive materials. The review uncovers 18 minor errors and 4 major errors. Therefore, $\text{Err}_{\text{tot}} = 22$. Error density is 1.2 errors per UML diagram, or 0.68 errors per requirements model page.

If reviews are conducted for a number of different types of work products (e.g., requirements model, design model, code, test cases), the percentage of errors uncovered for each review can be computed against the total number of errors found for all reviews. In addition, the error density for each work product can be computed.

Once data are collected for many reviews conducted across many projects, average values for error density enable you to estimate the number of errors to be found in a new document before it is reviewed. For example, if the average error density for a requirements model is 0.68 errors per page, and a new requirements model is 40 pages long, a rough estimate suggests that your software team will find around 27 errors during the review of the document. If you find only 9 errors, you've either done an extremely good job in developing the requirements model or your review approach was not thorough enough.

It is difficult to measure the cost effectiveness of any technical review in real time. A software engineering organization can assess the effectiveness of reviews and their cost benefit only after reviews have been completed, review metrics have been collected, average data have been computed, and then the downstream quality of the software is measured (via testing).

Returning to the previous example, the average error density for requirements models was determined to be 0.68 per page. The effort required to correct a minor model error (immediately after the review) has been found to require 4 person-hours. The effort required for a major requirement error has been found to be 18 person-hours. Examining the review data collected, you find that minor errors occur about six times more frequently than major errors. Therefore, you can estimate that the average effort to find and correct a requirements error during review is about 6 person-hours.

Requirements-related errors uncovered during testing require an average of 45 person-hours to find and correct (no data are available on the relative severity of the error). Using the averages noted, we get:

$$\begin{aligned}\text{Effort saved per error} &= E_{\text{testing}} - E_{\text{reviews}} \\ &= 45 - 6 = 39 \text{ person-hours/error}\end{aligned}$$

Because 22 errors were found during the review of the requirements model, a savings of about 858 person-hours of testing effort would be achieved. And that's just for requirements-related errors. Errors associated with design and code would add to the overall benefit.

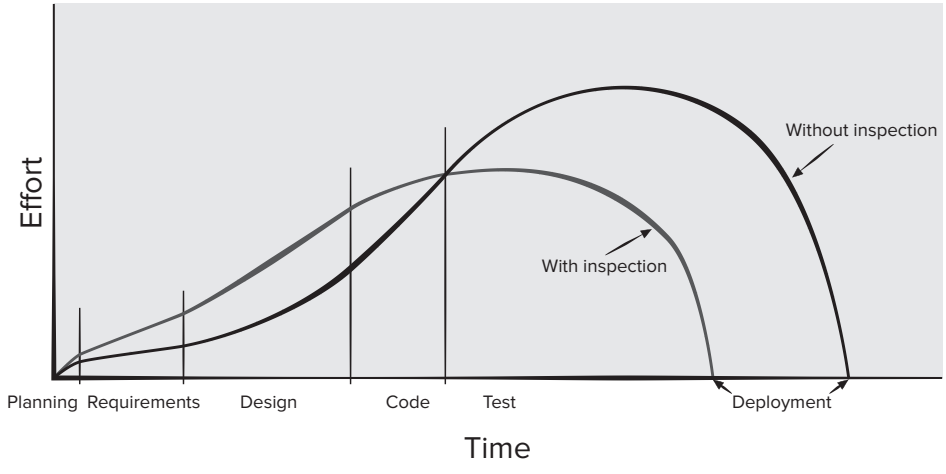
The bottom line—effort saved leads to shorter delivery cycles and improved time to market. The example presented in this section suggests this may be true. More importantly, industry data for software reviews has been collected for more than three decades and is summarized qualitatively using the graphs shown in Figure 16.1.

Referring to the figure, the effort expended when reviews are used does increase early in the development of a software increment, but this early investment for reviews pays dividends because testing and corrective effort is reduced. It is important to note

FIGURE 16.1

Effort expended with and without reviews

Source: Fagan, Michael E., "Advances in Software Inspections," IEEE Transactions on Software Engineering, vol. SE-12, no. 7, July 1986, 744–751.



that the deployment date for development with reviews is sooner than the deployment date without reviews. Reviews don't take time; they save it!

16.4 Criteria for Types of Reviews

Technical reviews can be classified as either formal or informal or somewhere in between these two extremes. The level of formality is chosen to match the type of product to be built, the project time line, and the people who are doing the work. Figure 16.2 depicts a reference model for technical reviews [Lai02] that identifies four characteristics that contribute to the formality with which a review is conducted.

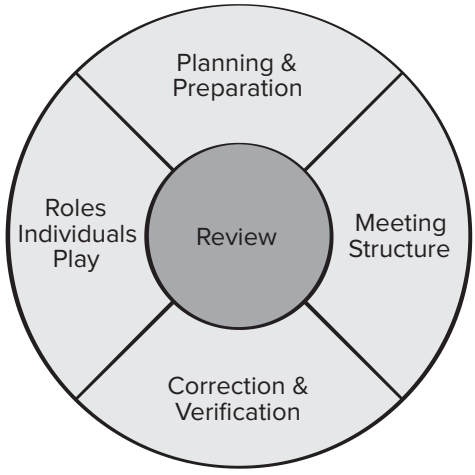
Each of the reference model characteristics helps to define the level of review formality. The formality of a review increases when (1) distinct roles are explicitly defined for the reviewers, (2) there is a sufficient amount of planning and preparation for the review, (3) a distinct structure for the review (including tasks and internal work products) is defined, and (4) follow-up by the reviewers occurs for any corrections that are made.

An element that is not presented in this model is the frequency of the reviews themselves. If you are using an agile prototyping model (Chapter 4) that contains relatively short sprints, your team may opt for less formal reviews because the reviews are happening fairly often. This usually means that defects are caught sooner and more often.

To understand the reference model, let's assume that you've decided to review the interface design for **SafeHomeAssured.com.** You can do this in a variety of different ways that range from relatively casual to extremely rigorous. If you decide that the casual approach is most appropriate, you ask a few colleagues (peers) to examine the interface prototype in an effort to uncover potential problems. All of you decide that there will be no advance preparation, but that you will evaluate the prototype in a reasonably structured way—looking at layout first, aesthetics next, navigation options after that, and so on. As the designer, you decide to take a few notes, but nothing formal.

FIGURE 16.2

Reference model for technical reviews



But what if the interface is pivotal to the success of the entire project? What if human lives depended on an interface that was ergonomically sound? You might decide that a more rigorous approach was necessary. A review team would be formed. Each person on the team would have a specific role to play—leading the team, recording findings, presenting the material, and so on. Each reviewer would be given access to the work product (in this case, the interface prototype) before the review and would spend time looking for errors, inconsistencies, and omissions. A set of specific tasks would be conducted based on an agenda that was developed before the review occurred. The results of the review would be formally recorded, and the team would decide on the status of the work product based on the outcome of the review. Members of the review team might also verify that the corrections made were done properly.

In this book we consider two broad categories of technical reviews: informal reviews and more formal technical reviews. Within each category, a number of different approaches can be chosen. These are presented in the sections that follow.

16.5 INFORMAL REVIEWS

Informal reviews include a simple desk check of a software engineering work product with a colleague, a casual meeting (involving more than two people) for the purpose of reviewing a work product, or the review-oriented aspects of pair programming (Chapter 3).

A simple *desk check* or a *casual meeting* conducted with a colleague is a review. However, because there is no advance planning or preparation, no agenda or meeting structure, and no follow-up on the errors that are uncovered, the effectiveness of such reviews is considerably lower than more formal approaches. But a simple desk check can and does uncover errors that might otherwise propagate further into the software process.

One way to improve the efficacy of a desk check review is to develop a set of simple review checklists[2] for each major work product produced by the software team. The questions posed within the checklist are generic, but they will serve to guide the reviewers as they check the work product. For example, let's reexamine a desk check of the interface prototype for **SafeHomeAssured.com.** Rather than simply playing with the prototype at the designer's workstation, the designer and a colleague examine the prototype using a checklist for interfaces:

- Is the layout designed using standard conventions? Left to right? Top to bottom?
- Does the presentation need to be scrolled?
- Are color and placement, typeface, and size used effectively?
- Are all navigation options or functions represented at the same level of abstraction?
- Are all navigation choices clearly labeled?

2 Literally hundreds of technical review checklists can be found via a web search. For example, a useful code review checklist can be downloaded from https://courses.cs.washington.edu/courses/cse403/12wi/sections/12wi_code_review_checklist.pdf.

and so on. Any errors or issues noted by the reviewers are recorded by the designer for resolution at a later time. Desk checks may be scheduled in an ad hoc manner, or they may be mandated as part of good software engineering practice. In general, the amount of material to be reviewed is relatively small and the overall time spent on a desk check spans little more than 1 or 2 hours.

In Chapter 3, we described *pair programming* in the following manner: XP recommends that two people work together at one computer workstation to create code for a story. This provides a mechanism for real-time problem solving (two heads are often better than one) and real-time quality assurance.

Pair programming (Section 3.5.1) can be characterized as a continuous desk check. Rather than scheduling a review at some point in time, pair programming encourages continuous review as a work product (design or code) is created. The benefit is immediate discovery of errors and better work product quality.

Some software engineers argue that the inherent redundancy built into pair programming is wasteful of resources. After all, why assign two people to a job that one person can accomplish? The answer to this question can be found in Section 16.3. If the quality of the work product produced as a consequence of pair programming is significantly better than the work of an individual, the quality-related savings can more than justify the "redundancy" implied by pair programming.

16.6 FORMAL TECHNICAL REVIEWS

A *formal technical review* (FTR) is a software quality control activity performed by software engineers (and others). The objectives of an FTR are: (1) to uncover errors in function, logic, or implementation for any representation of the software; (2) to verify that the software under review meets its requirements; (3) to ensure that the software has been represented according to predefined standards; (4) to achieve software that is developed in a uniform manner; and (5) to make projects more manageable. In addition, the FTR serves as a training ground, enabling junior engineers to observe different approaches to software analysis, design, and implementation. The FTR also serves to promote backup and continuity because a number of people become familiar with parts of the software that they may not have otherwise seen.

The FTR is actually a class of reviews that includes *walkthroughs* and *inspections*. Each FTR is conducted as a meeting and will be successful only if it is properly planned, controlled, and attended. In the sections that follow, guidelines similar to those for a walkthrough are presented as a representative formal technical review. If you have interest in software inspections, as well as additional information on walkthroughs, see [Rad02], [Wie02], or [Fre90].

16.6.1 The Review Meeting

Regardless of the FTR format that is chosen, every review meeting should abide by the following constraints:

- Between three and five people (typically) should be involved in the review.
- Advance preparation should occur but should require no more than 2 hours of work for each person.
- The duration of the review meeting should be less than 2 hours.

Given these constraints, it should be obvious that an FTR focuses on a specific (and small) part of the overall software. For example, rather than attempting to review an entire design, walkthroughs are conducted for each component or small group of components. By narrowing the focus, the FTR has a higher likelihood of uncovering errors.

The focus of the FTR is on a work product (e.g., a self-contained portion of a requirements model, a detailed component design, or source code for a component). The individual who has developed the work product—the *producer*—informs the project leader that the work product is complete and that a review is required. The project leader contacts a *review leader,* who evaluates the product for readiness, generates copies of product materials, and distributes them to two or three *reviewers* for advance preparation. Each reviewer is expected to spend between 1 and 2 hours reviewing the product, making notes, and otherwise becoming familiar with the work. Concurrently, the review leader also reviews the product and establishes an agenda for the review meeting, which is typically scheduled for the next day.

The review meeting is attended by the review leader, all reviewers, and the producer. One of the reviewers takes on the role of a *recorder,* that is, the individual who records (in writing) all important issues raised during the review. The FTR begins with an introduction of the agenda and a brief introduction by the producer. The producer then proceeds to "walk through" the work product, explaining the material, while reviewers raise issues based on their advance preparation. When valid problems or errors are discovered, the recorder notes each. Don't point out errors too harshly. One way to be gentle is to ask a question that enables the producer to discover the error.

At the end of the review, all attendees of the FTR must decide whether to: (1) accept the product without further modification, (2) reject the product due to severe errors (once corrected, another review must be performed), or (3) accept the product provisionally (minor errors have been encountered and must be corrected, but no additional review will be required). After the decision is made, all FTR attendees complete a sign-off, indicating their participation in the review and their concurrence with the review team's findings.

16.6.2 Review Reporting and Record Keeping

During the FTR, a reviewer (the recorder) actively records all issues that have been raised. These are summarized at the end of the review meeting, and a *review issues list* is produced. In addition, a *formal technical review summary report* is completed. A review summary report answers three questions:

1. What was reviewed?
2. Who reviewed it?
3. What were the findings and conclusions?

The review summary report is a single-page form (with possible attachments). It becomes part of the project historical record and may be distributed to the project leader and other interested parties.

The review issues list serves two purposes: (1) to identify problem areas within the product and (2) to serve as an action item checklist that guides the producer as corrections are made. An issues list is normally attached to the summary report.

You should establish a follow-up procedure to ensure that items on the issues list have been properly corrected. Unless this is done, it is possible that issues raised can "fall between the cracks." One approach is to assign the responsibility for follow-up to the review leader.

16.6.3 Review Guidelines

Guidelines for conducting formal technical reviews must be established in advance, distributed to all reviewers, agreed upon, and then followed. A review that is uncontrolled can often be worse than no review at all. The following represents a minimum set of guidelines for formal technical reviews:

1. **Review the product, not the producer.** An FTR involves people and egos. Conducted properly, the FTR should leave all participants with a warm feeling of accomplishment. Conducted improperly, the FTR can take on the aura of an inquisition. Errors should be pointed out gently; the tone of the meeting should be loose and constructive; the intent should not be to embarrass or belittle. The review leader should conduct the review meeting to ensure that the proper tone and attitude are maintained and should immediately halt a review that has gotten out of control.
2. **Set an agenda and maintain it.** One of the key maladies of meetings of all types is drift. An FTR must be kept on track and on schedule. The review leader is chartered with the responsibility for maintaining the meeting schedule and should not be afraid to nudge people when drift sets in.
3. **Limit debate and rebuttal.** When an issue is raised by a reviewer, there may not be universal agreement on its impact. Rather than spending time debating the question, the issue should be recorded for further discussion off-line.
4. **Enunciate problem areas, but don't attempt to solve every problem noted.** A review is not a problem-solving session. The solution of a problem can often be accomplished by the producer alone or with the help of only one other individual. Problem solving should be postponed until after the review meeting.
5. **Take written notes.** It is sometimes a good idea for the recorder to make notes on a wall board, so that wording and priorities can be assessed by other reviewers as information is recorded.
6. **Limit the number of participants and insist upon advance preparation.** Two heads are better than one, but 14 are not necessarily better than 4. Keep the number of people involved to the necessary minimum. However, all review team members must prepare in advance.
7. **Develop a checklist for each product that is likely to be reviewed.** A checklist helps the review leader to structure the FTR meeting and helps each reviewer to focus on important issues. Checklists should be developed for analysis, design, code, and even testing work products.
8. **Allocate resources and schedule time for FTRs.** For reviews to be effective, they should be scheduled as a task during the software process. In addition, time should be scheduled for the inevitable modifications that will occur as the result of an FTR.

9. **Conduct meaningful training for all reviewers.** The training should stress both process-related issues and the human psychological side of reviews.
10. **Review your early reviews.** Debriefing can be beneficial in uncovering problems with the review process itself. The very first product to be reviewed should be the review guidelines themselves.

Because many variables (e.g., number of participants, type of work products, timing and length, specific review approach) have an impact on a successful review, a software organization should experiment to determine what approach works best in a local context.

In an ideal setting, every software-engineering work product would undergo a formal technical review. In the real world of software projects, resources are limited and time is short. As a consequence, reviews are often skipped, even though their value as a quality control mechanism is recognized. However, full FTR resources should be used on those work products that are likely to be error prone.

SAFEHOME



Quality Issues

The scene: Doug Miller's office as the *SafeHome* software project begins.

The players: Doug Miller, manager of the *SafeHome* software engineering team, and other members of the product software engineering team.

The conversation:

Doug: I know we didn't spend time developing a quality plan for this project, but we're already into it and we have to consider quality . . . right?

Jamie: Sure. We've already decided that as we develop the requirements model [Chapter 8], Ed has committed to develop a testing procedure for each requirement.

Doug: That's really good, but we're not going to wait until testing to evaluate quality, are we?

Vinod: No! Of course not. We've got reviews scheduled into the project plan for this software increment. We'll begin quality control with the reviews.

Jamie: I'm a bit concerned that we won't have enough time to conduct all the reviews. In fact, I know we won't.

Doug: Hmmm. So what do you propose?

Jamie: I say we select those elements of the requirements and design model that are most critical to *SafeHome* and review them.

Vinod: But what if we miss something in a part of the model we don't review?

Jamie: Maybe . . . but I'm not sure we even have time to review every element of the models.

Vinod: What do you want us to do, Doug?

Doug: Let's steal something from Extreme Programming [Chapter 3]. We'll develop the elements of each model in pairs—two people—and conduct an informal review of each as we go. We'll then target "critical" elements for a more formal team review, but keep those reviews to a minimum. That way, everything gets looked at by more than one set of eyes, but we still maintain our delivery dates.

Jamie: That means we're going to have to revise the schedule.

Doug: So be it. Quality trumps schedule on this project.

16.7 Postmortem Evaluations

Many lessons can be learned if a software team takes the time to evaluate the results of a software project after the software has been delivered to end users. Baaz and his colleagues [Baa10] suggest the use of a *postmortem evaluation* (PME) as a mechanism to determine what went right and what went wrong when software engineering process and practice is applied in a specific project.

Unlike an FTR that focuses on a specific work product, it is more like a Scrum retrospective (Section 3.4.5). A PME examines the entire software project, focusing on both "*excellences* (that is, achievements and positive experiences) and *challenges* (problems and negative experiences)" [Baa10]. Often conducted in a workshop format, a PME is attended by members of the software team and stakeholders. The intent is to identify excellences and challenges and to extract lessons learned from both. The objective is to suggest improvements to both process and practice going forward. Many software engineers regard PME documents as some of the most valuable documents to save in the project archive.

16.8 Agile Reviews

It's not surprising that some software engineers balk at the thought of including reviews of any kind in agile development processes. Yet failing to catch defects early can be costly in terms of time and resources. Ignoring technical debt does not make it go away. Agile developers have the same need to find defects early (and often) that all software developers have. Admittedly, it may be somewhat more difficult to get agile developers to make use of metrics, but many of them can be collected unobtrusively.

If we take a closer look at the Scrum framework (Section 3.4), there are several places where informal and formal reviews take place. During the sprint planning meeting, user stories are reviewed and ordered according to priority, before selecting the user stories to include in the next sprint. The daily Scrum meeting is an informal way to ensure that the team members are all working on the same priorities and to catch any defects that may prevent completing the sprint on time. Agile developers often use pair programming, another informal review technique. The sprint review meeting is often conducted using guidelines similar to those discussed for a formal technical review. The code producers walk through the user stories selected for the sprint and demonstrate to the product owner that all functionality is present. Unlike the FTR, the product owner has the final word on whether to accept the sprint prototype or not.

If we look at the evaluate prototype portion of our recommended process model (Section 4.5), this task is also likely to be conducted as a formal technical review with development risk assessment added to it. We mentioned previously (Section 16.7) that the sprint retrospective meeting is really similar to the project postmortem meeting in that the development team is trying to capture its lessons learned. A major aspect of software quality assurance is being able to repeat your successes and avoid repeating your mistakes.

16.9 SUMMARY

The intent of every technical review is to find errors and uncover issues that would have a negative impact on the software to be deployed. The sooner an error is uncovered and corrected, the less likely that error will propagate to other software engineering work products and amplify itself, resulting in significantly more effort to correct it.

To determine whether quality control activities are working, a set of metrics should be collected. Review metrics focus on the effort required to conduct the review and the types and severity of errors uncovered during the review. Once metrics data are collected, they can be used to assess the efficacy of the reviews you do conduct. Industry data indicate that reviews provide a significant return on investment.

A reference model for review formality identifies the roles people play, planning and preparation, meeting structure, correction approach, and verification as the characteristics that indicate the degree of formality with which a review is conducted. Informal reviews are casual in nature but can still be used effectively to uncover errors. Formal reviews are more structured and have the highest probability of leading to high-quality software.

Informal reviews are characterized by minimal planning and preparation and little record keeping. Desk checks and pair programming fall into the informal review category.

A formal technical review is a stylized meeting that has been shown to be extremely effective in uncovering errors. Formal technical reviews establish defined roles for each reviewer, encourage planning and advance preparation, require the application of defined review guidelines, and mandate record keeping and status reporting.

PROBLEMS AND POINTS TO PONDER

16.1. Explain the difference between an *error* and a *defect.*

16.2. Why can't we just wait until testing to find and correct all software errors?

16.3. Assume that 10 errors have been introduced in the requirements model and that each error will be amplified by a factor of 2:1 into design and an additional 20 design errors are introduced and then amplified 1.5:1 into code where an additional 30 errors are introduced. Assume further that all unit testing will find 30 percent of all errors, integration will find 30 percent of the remaining errors, and validation tests will find 50 percent of the remaining errors. No reviews are conducted. How many errors will be released to the end users?

16.4. Reconsider the situation described in Problem 16.3, but now assume that requirements, design, and code reviews are conducted and are 60 percent effective in uncovering all errors at that step. How many errors will be released to the field?

16.5. Reconsider the situation described in Problems 16.3 and 16.4. If each of the errors released to the field costs $4800 to find and correct and each error found in review costs $240 to find and correct, how much money is saved by conducting reviews?

16.6. Describe the meaning of Figure 16.1 in your own words.

16.7. Can you think of a few instances in which a desk check might create problems rather than provide benefits?

16.8. A formal technical review is effective only if everyone has prepared in advance. How do you recognize a review participant who has not prepared? What do you do if you're the review leader?

16.9. How is technical debt addressed in agile process models?

16.10. Consider the review guidelines presented in Section 16.6.3. Which do you think is most important and why?

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

17 SOFTWARE QUALITY ASSURANCE

The software engineering approach described in this book works toward a single goal: to produce on-time, high-quality software. Yet many readers will be challenged by the question: "What is software quality?"

KEY CONCEPTS

Bayesian inference 351
elements of software quality assurance 341
formal approaches 347
genetic algorithms 352
goals 346
ISO 9001:2015 standard 354
Six Sigma 349
software reliability 350
software safety 352
SQA plan 354
SQA tasks 343
statistical software quality assurance 347



QUICK LOOK

What is it? It's not enough to talk the talk by saying that software quality is important. You have to (1) explicitly define what is meant when you say "software quality," (2) create a set of activities that will help ensure that every software engineering work product exhibits high quality, (3) perform quality control and assurance activities on every software project, (4) use metrics to develop strategies for improving your software process and, as a consequence, the quality of the end product.

Who does it? Everyone involved in the software engineering process is responsible for quality.

Why is it important? You can do it right, or you can do it over again. If a software team stresses quality in all software engineering activities, it reduces the amount of rework that it must do. That results in lower costs, and more importantly, improved time to market.

What are the steps? Before software quality assurance (SQA) activities can be initiated, it is important to define *software quality* at several different levels of abstraction. Once you understand what quality is, a software team must identify a set of SQA activities that will filter errors out of work products before they are passed on.

What is the work product? A Software Quality Assurance Plan is created to define a software team's SQA strategy. During modeling and coding, the primary SQA work product is the output of technical reviews (Chapter 16). During testing (Chapters 19 through 21), test plans and procedures are produced. Other work products associated with process improvement may also be generated.

How do I ensure that I've done it right? Find errors before they become defects! That is, work to improve your defect removal efficiency (Chapter 23), thereby reducing the amount of rework that your software team must perform.

Philip Crosby [Cro79], in his landmark book on quality, provides a wry answer to this question:

> The problem of quality management is not what people don't know about it. The problem is what they think they do know . . .
>
> In this regard, quality has much in common with sex. Everybody is for it. (Under certain conditions, of course.) Everyone feels they understand it. (Even though they wouldn't want to explain it.) Everyone thinks execution is only a matter of following natural inclinations. (After all, we do get along somehow.) And, of course, most people feel that problems in these areas are caused by other people. (If only they would take the time to do things right.)

Indeed, quality is a challenging concept—one that we addressed in some detail in Chapter 15.[1]

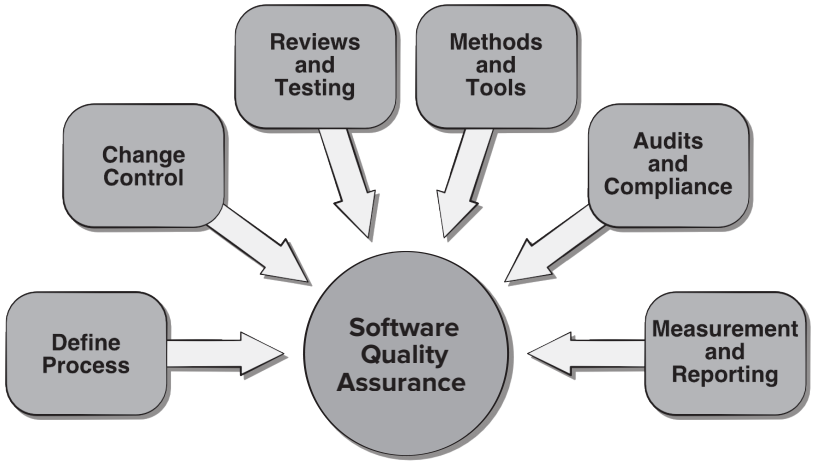
Some software developers continue to believe that software quality is something you begin to worry about after code has been generated. Nothing could be further from the truth! *Software quality assurance* (often called *quality management*) is an umbrella activity (Chapter 2) that is applied throughout the software process.

Software quality assurance (SQA) encompasses (Figure 17.1): (1) an SQA process, (2) specific quality assurance and quality control tasks (including technical reviews and a multitiered testing strategy), (3) effective software engineering practice (methods and tools), (4) control of all software work products and the changes made to them (Chapter 22), (5) a procedure to ensure compliance with software development standards (when applicable), and (6) measurement and reporting mechanisms.

In this chapter, we focus on the management issues and the process-specific activities that enable a software organization to ensure that it does "the right things at the right time in the right way."

FIGURE 17.1

Software quality assurance



1 If you have not read Chapter 15, you should do so now.

17.1 Background Issues

Quality control and assurance are essential activities for any business that produces products to be used by others. Prior to the twentieth century, quality control was the sole responsibility of the craftsperson who built a product. As time passed and mass production techniques became commonplace, quality control became an activity performed by people other than the ones who built the product.

The first formal quality assurance and control function was introduced at Bell Labs in 1916 and spread rapidly throughout the manufacturing world. During the 1940s, more formal approaches to quality control were suggested. These relied on measurement and continuous process improvement [Dem86] as key elements of quality management.

The history of quality assurance in software development parallels the history of quality in hardware manufacturing. During the early days of computing (1950s and 1960s), quality was the sole responsibility of the programmer. Standards for quality assurance for software were introduced in military contract software development during the 1970s and have spread rapidly into software development in the commercial world [IEE17]. Extending the definition presented earlier, software quality assurance is a "planned and systematic pattern of actions" [Sch01] that are required to ensure high quality in software. The scope of quality assurance responsibility might best be characterized by paraphrasing a once-popular automobile commercial: "Quality Is Job #1." The implication for software is that many different constituencies have software quality assurance responsibility—software engineers, project managers, customers, salespeople, and the individuals who serve within an SQA group.

The SQA group serves as the customer's in-house representative. That is, the people who perform SQA must look at the software from the customer's point of view. Does the software adequately meet the quality factors noted in Chapter 15? Have software engineering practices been conducted according to preestablished standards? Have technical disciplines properly performed their roles as part of the SQA activity? The SQA group attempts to answer these and other questions to ensure that software quality is maintained.

17.2 Elements of Software Quality Assurance

Software quality assurance encompasses a broad range of concerns and activities that focus on the management of software quality. These can be summarized in the following manner [Hor03]:

Standards. The IEEE, ISO, and other standards organizations have produced a broad array of software engineering standards and related documents. Standards may be adopted voluntarily by a software engineering organization or imposed by the customer or other stakeholders. The job of SQA is to ensure that standards that have been adopted are followed and that all work products conform to them.

Reviews and audits. Technical reviews are a quality control activity performed by software engineers for software engineers (Chapter 16). Their

intent is to uncover errors. Audits are a type of review performed by SQA personnel with the intent of ensuring that quality guidelines are being followed for software engineering work. For example, an audit of the review process might be conducted to ensure that reviews are being performed in a manner that will lead to the highest likelihood of uncovering errors.

Testing. Software testing (Chapters 19 through 21) is a quality control function that has one primary goal—to find errors. The job of SQA is to ensure that testing is properly planned and efficiently conducted so that it has the highest likelihood of achieving its primary goal.

Error/defect collection and analysis. The only way to improve is to measure how you're doing. SQA collects and analyzes error and defect data to better understand how errors are introduced and what software engineering activities are best suited to eliminating them.

Change management. Change is one of the most disruptive aspects of any software project. If it is not properly managed, change can lead to confusion, and confusion almost always leads to poor quality. SQA ensures that adequate change management practices (Chapter 22) have been instituted.

Education. Every software organization wants to improve its software engineering practices. A key contributor to improvement is education of software engineers, their managers, and other stakeholders. The SQA organization takes the lead in software process improvement (Chapter 28) and is a key proponent and sponsor of educational programs.

Vendor management. Three categories of software are acquired from external software vendors—*shrink-wrapped packages* (e.g., Microsoft Office), a *tailored shell* [Hor03] that provides a basic skeletal structure that is custom tailored to the needs of a purchaser, and *contracted software* that is custom designed and constructed from specifications provided by the customer organization. The job of the SQA organization is to ensure that high-quality software results by suggesting specific quality practices that the vendor should follow (when possible) and incorporating quality mandates as part of any contract with an external vendor.

Security management. With the increase in cyber crime and new government regulations regarding privacy, every software organization should institute policies that protect data at all levels, establish firewall protection for mobile apps, and ensure that software has not been tampered with internally. SQA ensures that appropriate process and technology are used to achieve software security (Chapter 18).

Safety. Because software is almost always a pivotal component of human-rated systems (e.g., automotive or aircraft applications), the impact of hidden defects can be catastrophic. SQA may be responsible for assessing the impact of software failure and for initiating those steps required to reduce risk.

Risk management. Although the analysis and mitigation of risk (Chapter 26) is the concern of software engineers, the SQA organization ensures that risk management activities are properly conducted and that risk-related contingency plans have been established.

In addition to each of these concerns and activities, SQA works to ensure that software support activities (e.g., maintenance, help lines, documentation, and manuals) are conducted or produced with quality as a dominant concern.

17.3 SQA Processes and Product Characteristics

As we begin a discussion of software quality assurance, it's important to note that SQA procedures and approaches that work in one software environment may not work as well in another. Even within a company that adopts a consistent approach[2] to software engineering, different software products may exhibit different levels of quality [Par11].

The solution to this dilemma is to understand the specific quality requirements for a software product and then select the process and specific SQA actions and tasks that will be used to meet those requirements. The Software Engineering Institute's CMMI and ISO 9000 standards are the most commonly used process frameworks. Each proposes "a syntax and semantics" [Par11] that will lead to the implementation of software engineering practices that improve product quality. Rather than instantiating either framework in its entirety, a software organization can "harmonize" the two models by selecting elements of both frameworks and matching them to the quality requirements of an individual product.

17.4 SQA Tasks, Goals, and Metrics

Software quality assurance is composed of a variety of tasks associated with two different constituencies—the software engineers who do technical work and an SQA group that has responsibility for quality assurance planning, oversight, record keeping, analysis, and reporting.

Modern software quality assurance is often data driven, as shown in Figure 17.2. The product stakeholders define goals and quality measures, problem areas are identified, indicators are measured, and a determination is made as to whether or not process changes are needed. Software engineers address quality (and perform quality control activities) by applying solid technical methods and measures, conducting technical reviews, and performing well-planned software testing.

17.4.1 SQA Tasks

The charter of the SQA group is to assist the software team in achieving a high-quality end product. The Software Engineering Institute recommends a set of SQA activities that address quality assurance planning, oversight, record keeping, analysis, and reporting. These activities are performed (or facilitated) by an independent SQA group that:

Prepares an SQA plan for a project. The plan is developed as part of project planning and is reviewed by all stakeholders. Quality assurance activities performed by the software engineering team and the SQA group are governed by the plan. The plan identifies evaluations to be performed, audits and

2 For example, CMMI-defined process and practices (Chapter 28).

FIGURE 17.2
Software quality assurance



reviews to be conducted, standards that are applicable to the project, procedures for error reporting and tracking, work products that are produced by the SQA group, and feedback that will be provided to the software team.

Participates in the development of the project's software process description. The software team selects a process for the work to be performed. The SQA group reviews the process description for compliance with organizational policy, internal software standards, externally imposed standards (e.g., ISO-9001), and other parts of the software project plan.

Reviews software engineering activities to verify compliance with the defined software process. The SQA group identifies, documents, and tracks deviations from the process and verifies that corrections have been made.

Audits designated software work products to verify compliance with those defined as part of the software process. The SQA group reviews selected work products; identifies, documents, and tracks deviations; verifies that corrections have been made; and periodically reports the results of its work to the project manager.

Ensures that deviations in software work and work products are documented and handled according to a documented procedure. Deviations may be encountered in the project plan, process description, applicable standards, or software engineering work products.

Records any noncompliance, and reports to senior management. Noncompliance items are tracked until they are resolved.

In addition to these activities, the SQA group coordinates the control and management of change (Chapter 22) and helps to collect and analyze software metrics.

SAFEHOME



Software Quality Assurance

The scene: Doug Miller's office as the *SafeHome* software project begins.

The players: Doug Miller, manager of the *SafeHome* software engineering team, and other members of the product software engineering team.

The conversation:

Doug: How are things going with the informal reviews?

Jamie: We're conducting informal reviews of the critical project elements in pairs as we code but before testing. It's going faster than I thought.

Doug: That's good, but I want to have Bridget Thorton's SQA group conduct audits of our work products to ensure that we're following our processes and meeting our quality goals.

Venod: Aren't they already doing the bulk of the testing?

Doug: Yes, they are. But QA is more than testing. We need to be sure that our documents are evolving along with our code and that we're making sure we don't introduce errors as we integrate new components.

Jamie: I really don't want to be evaluated based on their findings.

Doug: No worries. The audits are focused on conformance of our work products to the requirements and the process activities we've defined. We'll only be using audit results to try to improve our processes as well as our software products.

Vinod: I must believe it's going to take more of our time.

Doug: In the long run it will save us time when we find defects earlier. It also costs less to fix defects if they're caught early.

Jamie: That sounds like a good thing then.

Doug: It's also important to identify the activities where defects were introduced and add review tasks to catch them in the future.

Vinod: That'll help us determine if we're sampling carefully enough with our review activities.

Doug: I think SQA activities will make us a better team in the long run.

17.4.2 Goals, Attributes, and Metrics

The SQA activities described in the preceding section are performed to achieve a set of pragmatic goals:

Requirements quality. The correctness, completeness, and consistency of the requirements model will have a strong influence on the quality of all work products that follow. SQA must ensure that the software team has properly reviewed the requirements model to achieve a high level of quality.

Design quality. Every element of the design model should be assessed by the software team to ensure that it exhibits high quality and that the design itself conforms to requirements. SQA looks for attributes of the design that are indicators of quality.

Code quality. Source code and related work products (e.g., other descriptive information) must conform to local coding standards and exhibit characteristics that will facilitate maintainability. SQA should isolate those attributes that allow a reasonable analysis of the quality of code.

Quality control effectiveness. A software team should apply limited resources in a way that has the highest likelihood of achieving a high-quality result. SQA analyzes the allocation of resources for reviews and testing to assess whether they are being allocated in the most effective manner.

Table 17.1 (adapted from [Hya96]) identifies the attributes that are indicators for the existence of quality for each of the goals discussed. Metrics that can be used to indicate the relative strength of an attribute are also shown.

TABLE 17.1

Software quality goals, attributes, and metrics

Source: Adapted from Hyatt, L., and Rosenberg, L., "A Software Quality Model and Metrics for Identifying Project Risks and Assessing Software Quality," NASA SATC, 1996.

Goal	Attribute	Metric
Requirement quality	Ambiguity	Number of ambiguous modifiers (e.g., many, large, human-friendly)
	Completeness	Number of TBA, TBD
	Understandability	Number of sections/subsections
	Volatility	Number of changes per requirement
		Time (by activity) when change is requested
	Traceability	Number of requirements not traceable to design/code
	Model clarity	Number of UML models
		Number of descriptive pages per model
		Number of UML errors
Design quality	Architectural integrity	Existence of architectural model
	Component completeness	Number of components that trace to architectural model
		Complexity of procedural design
	Interface complexity	Average number of pick to get to a typical function or content
		Layout appropriateness
	Patterns	Number of patterns used
Code quality	Complexity	Cyclomatic complexity
	Maintainability	Design factors
	Understandability	Percent internal comments
		Variable naming conventions
		Percent reused components
	Reusability	Percent reused component
	Documentation	Readability index
QC effectiveness	Resource allocation	Staff hour percentage per activity
	Completion rate	Actual vs. budgeted completion time
	Review effectiveness	See review metrics
	Testing effectiveness	Number of errors found and criticality
		Effort required to correct an error
		Origin of error

17.5 Formal Approaches to SQA

In the preceding sections, we have argued that software quality is everyone's job and that it can be achieved through competent software engineering practice as well as through the application of technical reviews, a multitiered testing strategy, better control of software work products and the changes made to them, and the application of accepted software engineering standards and process frameworks. In addition, quality can be defined in terms of a broad array of quality attributes and measured (indirectly) using a variety of indices and metrics.

Over the past three decades, a small, but vocal, segment of the software engineering community has argued that a more formal approach to software quality assurance is required. It can be argued that a computer program is a mathematical object. A rigorous syntax and semantics can be defined for every programming language, and a rigorous approach to the specification of software requirements is available. If the requirements model (specification) and the programming language can be represented in a rigorous manner, it should be possible to apply mathematic proof of correctness to demonstrate that a program conforms exactly to its specifications.

Attempts to prove programs correct are not new. Dijkstra [Dij76a] and Linger, Mills, and Witt [Lin79], among others, advocated proofs of program correctness and tied these to the use of structured programming concepts. Although formal methods are interesting to some software engineering researchers, commercial developers rarely make use of formal methods in 2018.

17.6 Statistical Software Quality Assurance

Statistical quality assurance reflects a growing trend throughout the industry to become more quantitative about quality. For software, statistical quality assurance implies the following steps:

1. Information about software errors and defects is collected and categorized.
2. An attempt is made to trace each error and defect to its underlying cause (e.g., nonconformance to specifications, design error, violation of standards, poor communication with the customer).
3. Using the Pareto principle (80 percent of the defects can be traced to 20 percent of all possible causes), isolate the 20 percent (the *vital few*).
4. Once the vital few causes have been identified, move to correct the problems that have caused the errors and defects.

This relatively simple concept represents an important step toward the creation of an adaptive software process in which changes are made to improve those elements of the process that introduce error.

17.6.1 A Generic Example

To illustrate the use of statistical methods for software engineering work, assume that a software engineering organization collects information on errors and defects for a period of 1 year. Some of the errors are uncovered as software is being developed. Other defects are encountered after the software has been released to its end users.

Although hundreds of different problems are uncovered, all can be tracked to one (or more) of the following causes:

- Incomplete or erroneous specifications (IES)
- Misinterpretation of customer communication (MCC)
- Intentional deviation from specifications (IDS)
- Violation of programming standards (VPS)
- Error in data representation (EDR)
- Inconsistent component interface (ICI)
- Error in design logic (EDL)
- Incomplete or erroneous testing (IET)
- Inaccurate or incomplete documentation (IID)
- Error in programming language translation of design (PLT)
- Ambiguous or inconsistent human/computer interface (HCI)
- Miscellaneous (MIS)

To apply statistical SQA, a table is built (see Table 17.2). The table indicates that IES, MCC, and EDR are the vital few causes that account for 53 percent of all errors. It should be noted, however, that IES, EDR, PLT, and EDL would be selected as the vital few causes if only serious errors are considered. Once the vital few causes are determined, the software engineering organization can begin corrective action. For example, to correct MCC, you might implement requirements gathering techniques (Chapter 7) to improve the quality of customer communication and specifications. To improve EDR, you might acquire tools for data modeling and perform more stringent data design reviews.

TABLE 17.2

Data collection for statistical SQA

	Total		Serious		Moderate		Minor	
Error	**No.**	**%**	**No.**	**%**	**No.**	**%**	**No.**	**%**
IES	205	22%	34	27%	68	18%	103	24%
MCC	156	17%	12	9%	68	18%	76	17%
IDS	48	5%	1	1%	24	6%	23	5%
VPS	25	3%	0	0%	15	4%	10	2%
EDR	130	14%	26	20%	68	18%	36	8%
ICI	58	6%	9	7%	18	5%	31	7%
EDL	45	5%	14	11%	12	3%	19	4%
IET	95	10%	12	9%	35	9%	48	11%
IID	36	4%	2	2%	20	5%	14	3%
PLT	60	6%	15	12%	19	5%	26	6%
HCI	28	3%	3	2%	17	4%	8	2%
MIS	56	6%	0	0%	15	4%	41	9%
Totals	942	100%	128	100%	379	100%	435	100%

It is important to note that corrective action focuses primarily on the vital few. As the vital few causes are corrected, new candidates pop to the top of the stack.

Statistical quality assurance techniques for software have been shown to provide substantial quality improvement (e.g., [Rya11], [Art97]). In some cases, software organizations have achieved a 50 percent reduction per year in defects after applying these techniques.

The application of the statistical SQA and the Pareto principle can be summarized in a single sentence: *Spend your time focusing on things that really matter, but first be sure that you understand what really matters!*

17.6.2 Six Sigma for Software Engineering

Six Sigma is the most widely used strategy for statistical quality assurance in industry today. Originally popularized by Motorola in the 1980s, the Six Sigma strategy "is a business-management strategy designed to improve the quality of process outputs by minimizing variation and causes of defects in processes. It is a subset of the Total Quality Management (TQM) methodology with a heavy focus on statistical applications used to reduce costs and improve quality" [Voe18]. The term *Six Sigma* is derived from six standard deviations—3.4 instances (defects) per million occurrences—implying an extremely high-quality standard. The Six Sigma methodology defines three core steps:

- *Define* customer requirements and deliverables and project goals via well-defined methods of customer communication.
- *Measure* the existing process and its output to determine current quality performance (collect defect metrics).
- *Analyze* defect metrics and determine the vital few causes.

If an existing software process is in place, but improvement is required, Six Sigma suggests two additional steps:

- *Improve* the process by eliminating the root causes of defects.
- *Control* the process to ensure that future work does not reintroduce the causes of defects.

These core and additional steps are sometimes referred to as the DMAIC (define, measure, analyze, improve, and control) method.

If an organization is developing a software process (rather than improving an existing process), the core steps are augmented as follows:

- *Design* the process to (1) avoid the root causes of defects and (2) to meet customer requirements.
- *Verify* that the process model will, in fact, avoid defects and meet customer requirements.

This variation is sometimes called the DMADV (define, measure, analyze, design, and verify) method.

A comprehensive discussion of Six Sigma is best left to resources dedicated to the subject. If you have further interest, see [Voe18], [Pyz14], and [Sne18].

17.7 SOFTWARE RELIABILITY

There is no doubt that the reliability of a computer program is an important element of its overall quality. If a program repeatedly and frequently fails to perform, it matters little whether other software quality factors are acceptable.

Software reliability, unlike many other quality factors, can be measured directly and estimated using historical and developmental data. *Software reliability* is defined in statistical terms as "the probability of failure-free operation of a computer program in a specified environment for a specified time" [Mus87]. To illustrate, program *X* is estimated to have a reliability of 0.999 over eight elapsed processing hours. In other words, if program *X* were to be executed 1000 times and require a total of 8 hours of elapsed processing time (execution time), it is likely to operate correctly (without failure) 999 times.

Whenever software reliability is discussed, a pivotal question arises: What is meant by the term *failure*? In the context of any discussion of software quality and reliability, failure is nonconformance to software requirements. Yet, even within this definition, there are gradations. Failures can be only annoying or catastrophic. One failure can be corrected within seconds, while another requires weeks or even months to correct. Complicating the issue even further, the correction of one failure may in fact result in the introduction of other errors that ultimately result in other failures.

17.7.1 Measures of Reliability and Availability

Early work in software reliability attempted to extrapolate the mathematics of hardware reliability theory to the prediction of software reliability. Most hardware-related reliability models are predicated on failure due to wear rather than failure due to design defects. In hardware, failures due to physical wear (e.g., the effects of temperature, corrosion, shock) are more likely than a design-related failure. Unfortunately, the opposite is true for software. In fact, all software failures can be traced to design or implementation problems; wear (see Chapter 1) is not a factor.

There has been an ongoing debate over the relationship between key concepts in hardware reliability and their applicability to software. Although an irrefutable link has yet to be established, it is worthwhile to consider a few simple concepts that apply to both system elements.

If we consider a computer-based system, a simple measure of reliability is *mean time between failure* (MTBF):[3]

$$\text{MTBF} = \text{MTTF} + \text{MTTR}$$

where the acronyms MTTF and MTTR are *mean time to failure* and *mean time to repair*,[4] respectively.

3 It is important to note that MTBF and related measures are based on CPU time, not wall clock time.

4 Although debugging (and related corrections) may be required following a failure, in many cases the software will work properly after a restart with no other change.

Many researchers argue that MTBF is a far more useful measure than other quality-related software metrics discussed in Chapter 23. Stated simply, an end user is concerned with failures, not with the total defect count. Because each defect contained within a program does not have the same failure rate, the total defect count provides little indication of the reliability of a system. For example, consider a program that has been in operation for 3000 processor hours without failure. Many defects in this program may remain undetected for tens of thousands of hours before they are discovered. The MTBF of such obscure errors might be 30,000 or even 60,000 processor hours. Other defects, not yet discovered, might have a failure rate of 4000 or 5000 hours. Even if every one of the first category of errors (those with long MTBF) is removed, the impact on software reliability is negligible.

However, MTBF can be problematic for two reasons: (1) it projects a time span between failures but does not provide us with a projected failure rate, and (2) MTBF can be misinterpreted to mean average life span even though this is *not* what it implies.

An alternative measure of reliability is *failures in time* (FIT)—a statistical measure of how many failures a component will have over 1 billion hours of operation. Therefore, 1 FIT is equivalent to one failure in every billion hours of operation.

In addition to a reliability measure, you should also develop a measure of availability. *Software availability* is the probability that a program is operating according to requirements at a given point in time and is defined as

$$\text{Availability} = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}} \times 100\%$$

The MTBF reliability measure is equally sensitive to MTTF and MTTR. The availability measure is somewhat more sensitive to MTTR, an indirect measure of the maintainability of software. Of course, some aspects of availability have nothing to do with failure. For example, scheduling downtime (for support functions) causes the software to be unavailable. For a comprehensive discussion of software reliability measures, see [Laz11].

17.7.2 Use of AI to Model Reliability

Some software engineers view data science as the application of artificial intelligence techniques to solve software engineering problems. One of the things artificial intelligence methods attempt to do is provide reasonable solutions to problems where the needed data may be incomplete. *Software reliability* is defined as the probability of failure-free software operation for a specified time period in a specified environment. This means that we can never know the exact moment when a software product will fail because we will never have the complete data needed to calculate the probability.

Software engineers have used statistical techniques based on Bayes' theorem[5] in quantitative decision-making situations for several years. *Bayesian inference* is a method of statistical inference in which Bayes' theorem is used to update the probability

5 Bayes' theorem for conditional probabilities is P(A|B) = (P(B|A) * P(A)) / P(B). For more details, see http://www.statisticshowto.com/bayes-theorem-problems/.

for a hypothesis (such as system reliability) as more evidence or information becomes available. Bayesian inference can be used to estimate probabilistic quantities using historic data even when some information is missing. Using Bayesian techniques has allowed for real-time solutions to probability estimation problems that are beyond human reasoning [Tos17].

Proactive failure prediction using machine learning was discussed briefly in Section 15.4.3. It would be nice to be able to predict system failures in subsequent sprints before you deliver the prototype being developed in the current sprint. Making use of predictive data analytics such as a regression model involving MTBF has been used to estimate where and what types of defects might occur in future prototypes [Bat18].

A *genetic algorithm* is a heuristic search method used in artificial intelligence and computing. It is used for finding near-optimal solutions to search problems based on the theory of natural selection and evolutionary biology. Genetic algorithms can be used to grow reliability models by discovering relationships in historic system data. These models have been used to identify software components that may fail in the future. Sometimes these models have been created based on metrics estimated from UML models before any codes has been written [Pad17]. This type of work is very important to developers interested in refactoring software products or reusing software components in other products.

17.7.3 Software Safety

Software safety is a software quality assurance activity that focuses on the identification and assessment of potential hazards that may affect software negatively and cause an entire system to fail. If hazards can be identified early in the software process, software design features can be specified that will either eliminate or control potential hazards.

A modeling and analysis process is conducted as part of software safety. Initially, hazards are identified and categorized by criticality and risk. For example, some of the hazards associated with a computer-based cruise control for an automobile might be: (1) causes uncontrolled acceleration that cannot be stopped, (2) does not respond to depression of brake pedal (by turning off), (3) does not engage when switch is activated, and (4) slowly loses or gains speed. Once these system-level hazards are identified, analysis techniques are used to assign severity and probability of occurrence.[6] To be effective, software must be analyzed in the context of the entire system. For example, a subtle user input error (people are system components) may be magnified by a software fault to produce control data that improperly positions a mechanical device. If and only if a set of external environmental conditions is met, the improper position of the mechanical device will cause a disastrous failure. Analysis techniques [Eri15] such as fault tree analysis, real-time logic, and Petri net models can be used to predict the chain of events that can cause hazards and the probability that each of the events will occur to create the chain.

6 This approach is similar to the risk analysis methods described in Chapter 26. The primary difference is the emphasis on technology issues rather than project-related topics.

Once hazards are identified and analyzed, safety-related requirements can be specified for the software. That is, the specification can contain a list of undesirable events and the desired system responses to these events. The role of software in managing undesirable events is then indicated.

Although software reliability and software safety are related to one another, it is important to understand the subtle difference between them. Software reliability uses statistical analysis to determine the likelihood that a software failure will occur. However, the occurrence of a failure does not necessarily result in a hazard or mishap. Software safety examines the ways in which failures result in conditions that can lead to a mishap. That is, failures are not considered in a vacuum but are evaluated in the context of an entire computer-based system and its environment.

A comprehensive discussion of software safety is beyond the scope of this book. If you have further interest in software safety and related system issues, see [Fir13], [Har12a], and [Lev12].

17.8 THE ISO 9000 QUALITY STANDARDS[7]

A *quality assurance system* may be defined as the organizational structure, responsibilities, procedures, processes, and resources for implementing quality management [ANS87]. Quality assurance systems are created to help organizations ensure their products and services satisfy customer expectations by meeting their specifications. These systems cover a wide variety of activities encompassing a product's entire life cycle including planning, controlling, measuring, testing, and reporting, and improving quality levels throughout the development and manufacturing process. ISO 9000 describes quality assurance elements in generic terms that can be applied to any business regardless of the products or services offered.

To become registered to one of the quality assurance system models contained in ISO 9000, a company's quality system and operations are scrutinized by third-party auditors for compliance to the standard and for effective operation. Upon successful registration, a company is issued a certificate from a registration body represented by the auditors. Semiannual surveillance audits ensure continued compliance to the standard.

The requirements delineated by ISO 9001:2015 address topics such as management responsibility, quality system, contract review, design control, document and data control, product identification and traceability, process control, inspection and testing, corrective and preventive action, control of quality records, internal quality audits, training, servicing, and statistical techniques. For a software organization to become registered to ISO 9001:2015, it must establish policies and procedures to address each of the requirements just noted (and others) and then be able to demonstrate that these policies and procedures are being followed. If you desire further information on ISO 9001:2015, see [ISO14].

7 This section, written by Michael Stovsky, has been adapted from *Fundamentals of ISO 9000*, a workbook developed for *Essential Software Engineering,* a video curriculum developed by R. S. Pressman & Associates, Inc. Reprinted with permission.

INFO



The ISO 9001:2015 Standard

The following outline defines the basic elements of the ISO 9001:2015 standard. Comprehensive information on the standard can be obtained from the International Organization for Standardization (**www.iso.ch**) and other Internet sources (e.g., **www.praxiom.com**).

Establish the elements of a quality management system.
- Develop, implement, and improve the system.
- Define a policy that emphasizes the importance of the system.

Document the quality system.
- Describe the process.
- Produce an operational manual.
- Develop methods for controlling (updating) documents.
- Establish methods for record keeping.

Support quality control and assurance.
- Promote the importance of quality among all stakeholders.
- Focus on customer satisfaction.
- Define a quality plan that addresses objectives, responsibilities, and authority.
- Define communication mechanisms among stakeholders.

Establish review mechanisms for the quality management system.
- Identify review methods and feedback mechanisms.
- Define follow-up procedures.

Identify quality resources including personnel, training, and infrastructure elements.

Establish control mechanisms.
- For planning.
- For customer requirements.
- For technical activities (e.g., analysis, design, testing).
- For project monitoring and management.

Define methods for remediation.
- Assess quality data and metrics.
- Define approach for continuous process and quality improvement.

17.9 THE SQA PLAN

The *SQA plan* provides a road map for instituting software quality assurance. Developed by the SQA group (or by the software team if an SQA group does not exist), the plan serves as a template for SQA activities that are instituted for each software project.

A standard for SQA plans has been published by the IEEE [IEE17]. The standard recommends a structure that identifies: (1) the purpose and scope of the plan, (2) a description of all software engineering work products (e.g., models, documents, source code) that fall within the purview of SQA, (3) all applicable standards and practices that are applied during the software process, (4) SQA actions and tasks (including reviews and audits) and their placement throughout the software process, (5) the tools and methods that support SQA actions and tasks, (6) software configuration management (Chapter 22) procedures, (7) methods for assembling, safeguarding, and maintaining all SQA-related records, and (8) organizational roles and responsibilities relative to product quality.

17.10 SUMMARY

Software quality assurance is a software engineering umbrella activity that is applied at each step in the software process. SQA encompasses procedures for the effective application of methods and tools, oversight of quality control activities such as technical reviews and software testing, procedures for change management, procedures for assuring compliance to standards, and measurement and reporting mechanisms.

To properly conduct software quality assurance, data about the software engineering process should be collected, evaluated, and disseminated. Statistical SQA helps to improve the quality of the product and the software process itself. Software reliability models extend measurements, enabling collected defect data to be extrapolated into projected failure rates and reliability predictions.

In summary, you should note the words of Dunn and Ullman [Dun82]: "Software quality assurance is the mapping of the managerial precepts and design disciplines of quality assurance onto the applicable managerial and technological space of software engineering." The ability to ensure quality is the measure of a mature engineering discipline. When the mapping is successfully accomplished, mature software engineering is the result.

PROBLEMS AND POINTS TO PONDER

17.1. Some people say that "variation control is the heart of quality control." Because every program that is created is different from every other program, what are the variations that we look for and how do we control them?

17.2. Is it possible to assess the quality of software if the customer keeps changing what it is supposed to do?

17.3. Quality and reliability are related concepts but are fundamentally different in a number of ways. Discuss the differences.

17.4. Can a program be correct and still not be reliable? Explain.

17.5. Can a program be correct and still not exhibit good quality? Explain.

17.6. Why is there often tension between a software engineering group and an independent software quality assurance group? Is this healthy?

17.7. You have been given the responsibility for improving the quality of software across your organization. What is the first thing that you should do? What's next?

17.8. Besides counting errors and defects, are there other countable characteristics of software that imply quality? What are they and can they be measured directly?

17.9. The MTBF concept for reliability is open to criticism. Explain why.

17.10. Consider two safety-critical systems that are controlled by computer. List at least three hazards for each that can be directly linked to software failures.

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

18 Software Security Engineering

Contributed by: Nancy Mead *Carnegie Mellon University Software Engineering Institute*

Stop and take a look around. Where do you see software being deployed? Sure, it's in your laptop, tablet, and cell phone. What about your appliances—refrigerator, dishwasher, and so forth? How about your car? Financial transactions—ATM, online banking, financial software, tax software? Your supplier of electricity? Definitely using software. Do you have any wearable devices on? A fit-bit? Maybe you have medical devices, like a pacemaker. The bottom line is: Software is all around us, on us, and sometimes in us. Every software product has the potential to be hacked, sometimes with dire consequences. This is the reason that we, as software engineers, need to be concerned about software security.

Key Concepts

attack patterns 363
attack surface 366
maturity models 370
measurement 368
misuse and abuse cases 363
requirements engineering 360
secure coding 367
secure development life-cycle activities 359
security life-cycle models 357
security process improvement 370
security risk analysis 364
software security engineering, importance of 357
threat modeling, prioritization, and mitigation 365

Quick Look

What is it? Software security engineering encompasses a set of techniques that can improve the security of software *while* it is under development.

Who does it? Although software engineers do not need to become security experts, they need to collaborate with security experts. The security experts may be members of the software team, on a separate specialized team, or they may be outside consultants.

Why is it important? The media continually reports instances of hacking—whether by gangsters, corporate competitors, hostile nations, or any other bad actor. The consequences for critical infrastructure, financial institutions, health care, and all aspects of modern life are significant.

What are the steps? There are a number of steps that can be taken to ensure that software is secure. We will discuss some of them here and provide pointers to resources for further exploration.

What is the work product? As you will see, there are many work products that are developed in the process of secure software engineering. The ultimate work product, or course, is the software that you have developed using secure software engineering practices.

How do I ensure that I've done it right? Everything that we will discuss as a method to improve software security, whether at the organizational or project level, can and should be reviewed by the interested parties. In addition, secure development processes can be improved, if they are found to be lacking.

18.1 Why Software Security Engineering Is Important

Software security is about more than just securing operational software with firewalls, strong passwords, and encryption. It's also about developing software in such a way that it is more secure from the outset. Techniques are now available that will help us develop software that is significantly more secure than it would be otherwise.

In this chapter, we're going to look as some of the models and techniques that can help us achieve better software security. We'll start by looking at security process models. Then we'll examine specific process activities, including such activities as requirements engineering, misuse or abuse cases, security risk analysis, threat modeling, attack surface, secure coding, and measurement. We'll also consider security process improvement models. Finally, we'll summarize and provide you with a list of references so that you can dive into any of these topics in more depth.

Software security engineering research is a very active area. In this book we're only providing an overview of methods and tools to support actual practice. There are many books (for example, [Mea16], [Shu13], and [Hel18]) and other resources devoted exclusively to software security engineering, and we will point you to some of them.

18.2 Security Life-Cycle Models

The Microsoft Security Development Lifecycle (SDL) [Mea16] [Mic18] is an industry-leading software security process. A Microsoft-wide initiative and a mandatory policy since 2004, the SDL enabled Microsoft to embed security and privacy in its software and culture. The SDL introduces security and privacy early and throughout all phases of the development process and is without question the most widely known and used security development life-cycle model.

Microsoft defined a collection of principles it calls *Secure by Design, Secure by Default, Secure in Deployment, and Communications* (SD3+C) to help determine where security efforts are needed. These are as follows [Mic10]:

Secure by Design

Secure architecture, design, and structure. Developers consider security issues part of the basic architectural design of software development. They review detailed designs for possible security issues, and they design and develop mitigations for all threats.

Threat modeling and mitigation. Threat models are created, and threat mitigations are present in all design and functional specifications.

Elimination of vulnerabilities. No known security vulnerabilities that would present a significant risk to the anticipated use of the software remain in the code after review. This review includes the use of analysis and testing tools to eliminate classes of vulnerabilities.

Improvements in security. Less secure legacy protocols and code are deprecated, and, where possible, users are provided with secure alternatives that are consistent with industry standards.

Secure by Default

Least privilege. All components run with the fewest possible permissions.

Defense in depth. Components do not rely on a single threat mitigation solution that leaves users exposed if it fails.

Conservative default settings. The development team is aware of the attack surface for the product and minimizes it in the default configuration.

Avoidance of risky default changes. Applications do not make any default changes to the operating system or security settings that reduce security for the host computer. In some cases, such as for security products, it is acceptable for a software program to strengthen (increase) security settings for the host computer. The most common violations of this principle are games that either open firewall ports without informing the user or instruct users to open firewall ports without informing users of possible risks.

Less commonly used services off by default. If fewer than 80 percent of a program's users use a feature, that feature should not be activated by default. Measuring 80 percent usage in a product is often difficult because programs are designed for many different personas. It can be useful to consider whether a feature addresses a core/primary use scenario for all personas. If it does, the feature is sometimes referred to as a P1 feature.

Secure in Deployment

Deployment guides. Prescriptive deployment guides outline how to deploy each feature of a program securely, including providing users with information that enables them to assess the security risk of activating non-default options (and thereby increasing the attack surface).

Analysis and management tools. Security analysis and management tools enable administrators to determine and configure the optimal security level for a software release.

Patch deployment tools. Deployment tools aid in patch deployment.

Communications

Security response. Development teams respond promptly to reports of security vulnerabilities and communicate information about security updates.

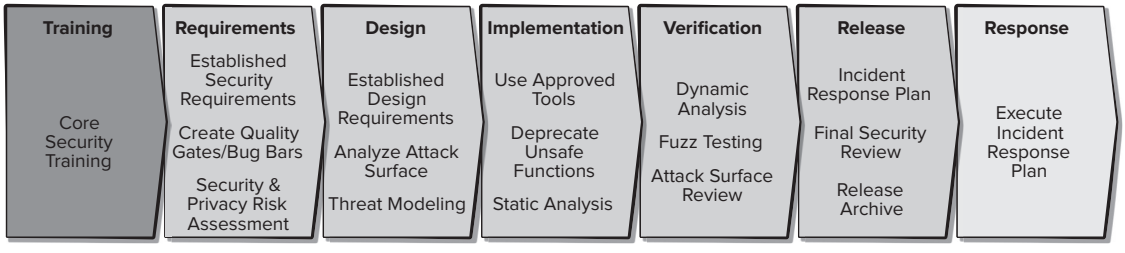
Community engagement. Development teams proactively engage with users to answer questions about security vulnerabilities, security updates, or changes in the security landscape.

The secure software development process model looks like the one shown in Figure 18.1.

The Microsoft SDL documentation describes what architects, designers, developers, and testers are required to do for each of the 16 recommended practices. The data that Microsoft collected after implementing the SDL shows a significant reduction in

FIGURE 18.1 **Secure Software Development Process Model at Microsoft**

Adapted from Shunn, A., et al. Strengths in Security Solutions, Software Engineering Institute, Carnegie Mellon University, 2013. Available at http://resources.sei.cmu.edu/library/asset-view.cfm?assetid=77878.



vulnerabilities, which led to a need for fewer patches, and thus a significant cost savings. We recommend that you browse the SDL website to learn more about these practices. Since the SDL was developed, there have been numerous papers, books, training, and so on, to go with the SDL model.[1]

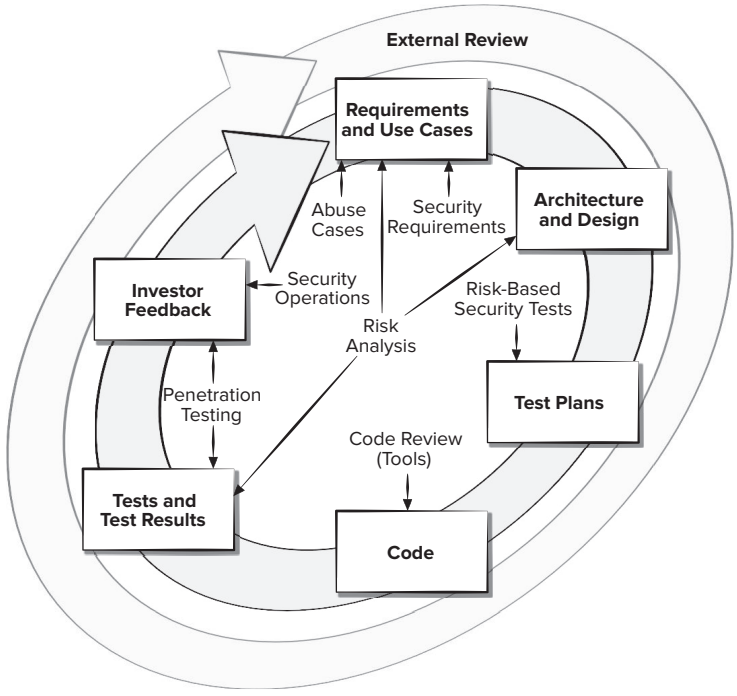
18.3 Secure Development Life-Cycle Activities

A different approach that is independent of a life-cycle model is the touchpoints for software security [McG06], which argues that the *activities* (touchpoints) are what matter, not the model. The activities can be incorporated into any life-cycle model, and thus are considered to be process agnostic. The touchpoints later formed the basis for BSIMM, a maturity model that will be discussed later in this chapter. Some organizations consider the touchpoints to be the minimum set of activities that should be performed in secure software development. A pictorial version of the touchpoints is shown in Figure 18.2. In this diagram, the recommended security activities appear above the corresponding software development activity, or life-cycle phase:

With the SDL and touchpoints in mind, we'll look at some of the important secure software development activities that are associated with them.

FIGURE 18.2

Software security touchpoints



1 More recently, Microsoft has shown how the SDL activities can be integrated with an agile development approach: https://www.microsoft.com/en-us/SDL/Discover/sdlagile.aspx.

18.4 SECURITY REQUIREMENTS ENGINEERING

Although security requirements are an important part of secure software development, in practice they are often neglected. When they exist, they are often an add-on, copied from a generic list of security features. The requirements engineering that is needed to get a better set of security requirements seldom takes place [All08].

Requirements engineering practice typically addresses desired user features. Therefore, attention is given to the functionality of the system from the user's perspective, but little attention is given to what the system should *not* do [Bis02]. Users expect systems to be secure, and these assumptions need to make their way into security requirements for software systems before they are developed, not after the fact. Often the users' assumptions about security are overlooked because system features are the primary focus.

In addition to security life-cycle models, there are many process models that are specific to security requirements. These include: core security requirements artifacts [Mof04], Software Cost Reduction (SCR) [Hei02], SQUARE (Security QUAlity Requirements Engineering) [Mea05], and Security Requirements Engineering Process (SREP) [Mel06]. For the remainder of this section, we'll consider SQUARE as a representative example of security life-cycle models.

18.4.1 SQUARE

SQUARE is a representative security requirements engineering process model, but it's important to keep in mind that if you already have a development process model, such as the one presented in Chapter 4, you can just pick up some of the SQUARE steps to enhance your existing model. There's no need to develop a whole new process to address security in your software development activities. We suggest that you add security definitions to your glossary; perform risk analysis, including identification of potential attacks via misuse cases or threat modeling; develop mitigation strategies; and perform categorization and prioritization of candidate security requirements.

The SQUARE process model provides for eliciting, categorizing, and prioritizing security requirements for software-intensive systems. Its focus is to build security concepts into the early stages of the development life cycle. It can also be used for fielded systems and those undergoing improvements and modifications. The process is shown in Table 18.1, followed by brief descriptions of each step.

18.4.2 The SQUARE Process

The SQUARE process is best applied by the project's requirements engineers and security experts with supportive executive management and stakeholders. Let's take a look at the steps.[2]

> **Step 1. Agree on definitions.** So that there is not semantic confusion, this step is needed as a prerequisite to security requirements engineering. On a

2 To dig deeper, see the resources available at the SEPA 9/e website.

TABLE 18.1 The SQUARE process

Number	Step	Input	Techniques	Participants	Output
1	Agree on definitions.	Candidate definitions from IEEE and other standards	Structured interviews, focus group	Stakeholders, requirements team	Agreed-to definitions
2	Identify assets and security goals.	Definitions, candidate assets and goals, business drivers, policies and procedures, examples	Facilitated work session, surveys, interviews	Stakeholders, requirements engineer	Assets and security goals
3	Develop artifacts to support security requirements definition.	Potential artifacts (e.g., scenarios, misuse cases, templates, forms)	Work session	Requirements engineer	Needed artifacts: scenarios, misuse cases, models, templates, forms
4	Perform (security) risk assessment.	Misuse cases, scenarios, security goals	Risk assessment method, analysis of anticipated risk against organizational risk tolerance, including threat analysis	Requirements engineer, risk expert, stakeholders	Risk assessment results
5	Select elicitation techniques.	Goals, definitions, candidate techniques, expertise of stakeholders, organizational style, culture, level of security needed, cost-benefit analysis, etc.	Work session	Requirements engineer	Selected elicitation techniques
6	Elicit security requirements.	Artifacts, risk assessment results, selected techniques	Accelerated Requirements Method, Joint Application Development, interviews, surveys, model-based analysis, checklists, lists of reusable requirements types, document reviews	Stakeholders facilitated by requirements engineer	Initial cut at security requirements
7	Categorize requirements as to level (system, software, etc.) and whether they are requirements or other kinds of constraints.	Initial requirements, architecture	Work session using a standard set of categories	Requirements engineer, other specialists as needed	Categorized requirements
8	Prioritize requirements.	Categorized requirements and risk assessment results	Prioritization methods such as Analytical Hierarchy Process (AHP), Triage, Win-Win, etc.	Stakeholders facilitated by requirements engineer	Prioritized requirements
9	Inspect requirements.	Prioritized requirements, candidate formal inspection technique	Inspection method such as Fagan and peer reviews	Inspection team	Initial selected requirements, documentation of decision-making process and rationale

given project, team members tend to have definitions in mind, based on their prior experience, but those definitions are often different from one another [Woo05]. Sources such as the Institute for Electrical and Electronics Engineers (IEEE) and the Software Engineering Body of Knowledge (SWEBOK) provide a range of definitions to select from or tailor [SWE14].

Step 2. Identify assets and security goals. This step occurs at the project's organizational level and is needed to support software development. Different stakeholders usually have concerns about different assets, and thus have different goals. For example, a stakeholder in human resources may be concerned about maintaining the confidentiality of personnel records, whereas a stakeholder in a research area may be concerned with ensuring that research project information is not accessed, modified, or stolen.

Step 3. Develop artifacts. This step is necessary to support all subsequent security requirements engineering activities. Often, organizations do not have key documents needed to support requirements definition, or they may not be up to date. This means that a lot of time may be spent backtracking to try to obtain documents, or the team will have to bring them up to date before going further.

Step 4. Perform risk assessment. This step requires an expert in risk assessment methods, the support of the stakeholders, and the support of a security requirements engineer. There are a number of risk assessment methods, but regardless of the one that you choose, the outcomes of risk assessment can help in identifying the high-priority security exposures.

Step 5. Select elicitation technique. This step becomes important when there are diverse stakeholders. A more formal elicitation technique, such as the Accelerated Requirements Method [Hub99], Joint Application Design [Woo89], or structured interviews, can be effective in overcoming communication issues when there are stakeholders with different cultural backgrounds. In other cases, elicitation may simply consist of sitting down with a primary stakeholder to try to understand that stakeholder's security requirements needs.

Step 6. Elicit security requirements. This step encompasses the actual elicitation process using the selected technique. Most elicitation techniques provide detailed guidance on how to perform elicitation. This builds on the artifacts that were developed in earlier steps.

Step 7. Categorize requirements. This step allows the security requirements engineer to distinguish among essential requirements, goals (desired requirements), and architectural constraints that may be present. This categorization also helps in the prioritization activity that follows.

Step 8. Prioritize requirements. This step depends on the prior step and may also involve performing a cost-benefit analysis to determine which security requirements have a high payoff relative to their cost. Of course prioritization may also depend on other consequences of security breaches, such as loss of life, loss of reputation, and loss of consumer confidence.

Step 9. Requirements inspection. This review activity can be accomplished at varying levels of formality, discussed in Chapter 16. Once inspection is complete, the project team should have an initial set of prioritized security requirements that can be revisited as needed later in the project.

18.5 Misuse and Abuse Cases and Attack Patterns

Misuse (or *abuse*) cases can help you view your software in the same way that attackers do. By thinking about negative events, you can better understand how to develop secure software. A misuse case can be thought of as a use case that the attacker initiates.

One of the goals of misuse cases [Sin00] is to decide up front how software should react to potential attacks. You can also use misuse and normal use cases together to conduct threat and hazard analysis [Ale03].

We suggest creating misuse cases through brainstorming. Teaming security experts with subject matter experts (SMEs) covers a lot of ground quickly. During brainstorming, software security experts ask many questions of developers to help identify the places where the system is likely to have weaknesses. This involves a careful look at all user interfaces and considers events that developers assume can't happen, but that attackers can actually cause to happen.

Here are some questions that need to be considered: How can the system distinguish between valid and invalid input data? Can it tell whether a request is coming from a legitimate application or a rogue application? Can an insider cause a system to malfunction? Trying to answer such questions helps developers to analyze their assumptions and allows them to fix problems up front.

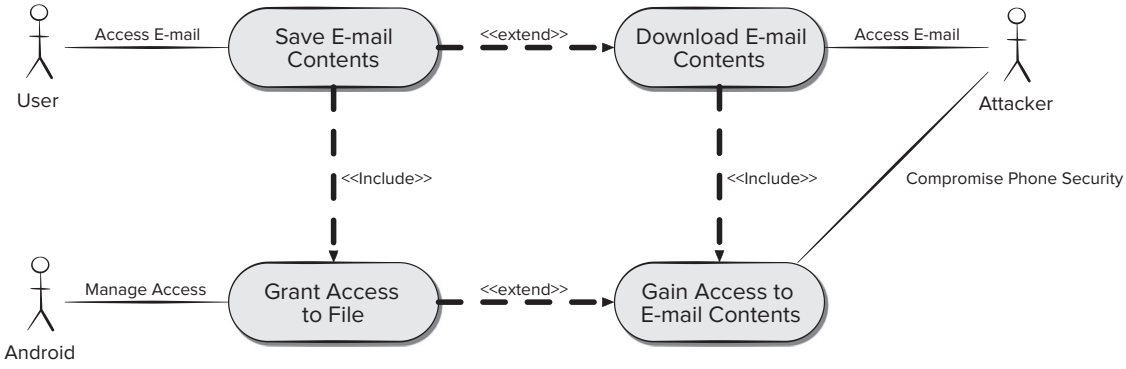
Misuse cases can be in table or diagram form. Figure 18.3 provides an example misuse case that shows how DroidCleaner malware can successfully attack a cell phone using an open-source e-mail application called K-9. This is extracted from a much larger report that you may wish to study [Ali14].

In this misuse case, the user keeps e-mail on the phone's external storage area. The attacker gains access to the phone's storage by compromising the operating system. A common way for the attacker to gain access to the phone is by tricking the user into installing a Trojan, to which the user unwittingly grants access to the drive during the install process. The attacker is then able to use the Trojan to download files, including the e-mail contents file.

Attack patterns can provide some help by providing a blueprint for creating an attack. For example, buffer overflow is one type of security exploitation. Attackers trying to capitalize on a buffer overflow make use of similar steps [OWA16]. Attack patterns can document these steps (e.g., timing, resources, techniques) as well as practices software developers can use to prevent or mitigate their success [Hog04]. When you're trying to develop misuse and abuse cases, attack patterns can help.

Misuse cases need to be prioritized as they are generated. In addition, they need to strike the right balance between cost and benefit. The project budget may not allow

FIGURE 18.3 **Misuse case (exploited by DroidCleaner): Data in an e-mail stored on the smartphone is stolen**



a software team to implement all defined mitigation strategies at once. In such cases, the strategies can be prioritized and implemented incrementally. The team can also exclude certain cases as being extremely unlikely.

Templates for misuse and abuse cases appear in a number of references. They can be text or diagrams and may be supported by tools. Good sources for templates are in materials by Sindre and Opdahl [Sin01] and Alexander [Ale02].

18.6 SECURITY RISK ANALYSIS

A wide variety of security risk assessment methods have been proposed. Typical examples include SEI CERT's Security Engineering Risk Analysis (SERA) method[3] and the NIST Risk Management Framework (RMF).[4]

RMF has emerged as an approach that is widely used, providing guidelines for the users. The RMF steps for security are:

- **Categorize** the information system and the information processed, stored, and transmitted by that system based on an impact analysis.
- **Select** an initial set of baseline security controls for the information system based on the security categorization; using an organizational assessment of risk and local conditions, tailor and supplement the security control baseline as needed.
- **Implement** the security controls, and describe how the controls are employed within the information system and its operational environment.
- **Assess** the security controls using appropriate assessment procedures to determine the extent to which the controls are implemented correctly, operating as intended, and producing the desired outcome with respect to meeting the security requirements for the system.

3 See https://resources.sei.cmu.edu/library/asset-view.cfm?assetid=485410.
4 See https://csrc.nist.gov/publications/detail/sp/800-37/rev-1/final.

- **Authorize** the information system operation based on a determination of the risk to organizational operations and assets, individuals, or other organizations (including national defense), from the operation of the information system that this risk is acceptable.
- **Monitor** the security controls in the information system on an ongoing basis including assessing control effectiveness, documenting changes to the system or its environment of operation, conducting security impact analyses of the associated changes, and reporting the security state of the system to designated organizational officials.

It's important to note that NIST also provides a set of security controls to select from, thus simplifying the risk assessment work. Recently, the RMF has been modified to include privacy concerns.

18.7 Threat Modeling, Prioritization, and Mitigation

A threat modeling method (TMM) is an approach for creating an abstraction of a software system, aimed at identifying attackers' abilities and goals, and using that abstraction to generate and catalog possible threats that the system must mitigate [Shu16].

STRIDE (an acronym for six threat categories) is representative of a number of threat modeling methods [Mea18] and is the most well-established TMM, representing the state of the practice. At its core, STRIDE requires breaking down a system into its various elements, assessing each of these elements for their vulnerability to threats, and then mitigating these threats [Her06]. In practice, a typical STRIDE implementation includes modeling a system with data flow diagrams (DFDs),[5] mapping the DFD elements to the six threat categories, determining the specific threats via checklists or threat trees, and documenting the threats and steps for their prevention [Sca15]. STRIDE can be implemented manually; however, a free Microsoft Secure Development Lifecycle (SDL) Threat Modeling Tool [Mic17] can also be used. Table 18.2 identifies the security property associated with each of the six threat categories.

TABLE 18.2

Threat categories and security properties

Threat	Security Property
Spoofing	Authentication
Tampering	Integrity
Repudiation	Nonrepudiation
Information disclosure	Confidentiality
Denial of service	Availability
Elevation of privilege	Authorization

5 A brief tutorial on data flow diagrams can be downloaded from https://ratandon.mysite.syr.edu/cis453/notes/DFD_over_Flowcharts.pdf.

TABLE 18.3
Threat categories of DFD system elements

Element	Spoofing	Tampering	Repudiation	Information Disclosure	Denial of Service	Elevation of Privilege
Data flows		X		X	X	
Data stores		X		X	X	
Processes	X	X	X	X	X	X
External entity	X		X			

DFDs are designed to show how a system works by using standard symbols to graphically represent the interaction between data stores (e.g., databases, files, registries), processes (e.g., DLLs, Web services), data flows (e.g., function calls, remote procedure calls), and external entities (e.g., people, other systems) [Sho14]. Once complete, each of these system elements in turn can be associated with one or more relevant threat categories, as depicted in Table 18.3.

In the next stage, the typical STRIDE user works through a checklist (that may be in the form of a threat tree) of specific threats that are associated with each match between a DFD element and threat category. Such checklists are accessible through STRIDE reference books or tools.

Once the threats have been identified, mitigation strategies can be developed and prioritized. Typically, prioritization is based on cost and value considerations. Considering the cost of implementing the mitigation strategy is important, but it's equally important to also consider the cost of *not* implementing it, which is reflected in value. Remember that risks that are realized result in costs that are not only expressed in terms of dollars, but could also reflect loss of reputation, loss of trust, and even loss of life.

18.8 ATTACK SURFACE

An *attack surface* can be defined[6] in the following manner:

> The attack surface describes all of the different points where an attacker could get into a system, and where they could get data out.
>
> The attack surface of an application is:
>
> 1. the sum of all paths for data/commands into and out of the application, and
> 2. the code that protects these paths (including resource connection and authentication, authorization, activity logging, data validation and encoding), and
> 3. all valuable data used in the application, including secrets and keys, intellectual property, critical business data, personal data and PII, and
> 4. the code that protects these data (including encryption and checksums, access auditing, and data integrity and operational security controls). [OWA18]

6 See https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/Attack_Surface_Analysis_Cheat_Sheet.md.

The OWASP Foundation [OWA18] states that attack surface analysis is:

> . . . targeted to be used by developers to understand and manage application security risks as they design and change an application, as well as by application security specialists doing a security risk assessment. The focus here is on protecting an application from external attack—it does not take into account attacks on the users or operators of the system (e.g., malware injection, social engineering attacks), and there is less focus on insider threats, although the principles remain the same. The internal attack surface is likely to be different to the external attack surface and some users may have a lot of access.
>
> Attack Surface Analysis is about mapping out what parts of a system need to be reviewed and tested for security vulnerabilities. The point of attack surface analysis is to understand the risk areas in an application, to make developers and security specialists aware of what parts of the application are open to attack, to find ways of minimizing this, and to notice when and how the Attack Surface changes and what this means from a risk perspective.

18.9 SECURE CODING

Secure coding is just what the name implies—coding in such a way that vulnerabilities are not introduced as a result of coding errors. It's not surprising that most software vulnerabilities occur because of sloppy and erroneous coding practices, many of which can be easily avoided.

For example, a condition known as *buffer overflow* results from one of the most well-known and common coding errors. OWASP[7] describes it as follows:

> A buffer overflow condition exists when a program attempts to put more data in a buffer than it can hold or when a program attempts to put data in a memory area past a buffer. In this case, a buffer is a sequential section of memory allocated to contain anything from a character string to an array of integers. Writing outside the bounds of a block of allocated memory can corrupt data, crash the program, or cause the execution of malicious code.

Buffer overflow is just one example of coding errors that can result in vulnerabilities. Fortunately, a number of coding standards now exist to provide guidance on secure coding. The SEI/CERT website[8] provides a list of the top-10 secure coding practices:

1. **Validate input.** Validate input from all untrusted data sources.
2. **Heed compiler warnings.** Compile code using the highest warning level available for your compiler and eliminate warnings by modifying the code.
3. **Architect and design for security policies.** Create a software architecture and design your software to implement and enforce security policies.
4. **Keep it simple.** Keep the design as simple and as small as possible.
5. **Default deny.** Base access decisions on permission rather than exclusion.

7 See https://www.owasp.org/index.php/Buffer_overflow_attack.

8 See https://wiki.sei.cmu.edu/confluence/display/seccode/Top+10+Secure+Coding+Practices.

6. **Adhere to the principle of least privilege.** Every process should execute with the least set of privileges necessary to complete the job.
7. **Sanitize data sent to other systems.** Sanitize all data passed to complex subsystems such as command shells, relational databases, and commercial off-the-shelf (COTS) components.
8. **Practice defense in depth.** Manage risk with multiple defensive strategies.
9. **Use effective quality assurance techniques.**
10. **Adopt a secure coding standard.**

SEI CERT and others also provide secure coding standards.[9] In addition to using a secure coding standard, you should inspect for coding errors that lead to vulnerabilities. This is just a natural add-on to your normal code inspection and review process (Chapter 16). *Static analysis tools*[10] are used to automatically analyze code and are another mechanism for detecting vulnerabilities due to coding errors.

18.10 MEASUREMENT

Developing adequate measures of software security is a difficult problem, and one for which there are differing viewpoints. On the one hand, you can look at the development processes followed and assess whether the resultant software is likely to be secure. On the other hand, you can look at the incidence of vulnerabilities and successful break-ins and measure those as a way of assessing software security. However, neither of these measurement approaches will allow you to say with 100 percent certainty that our software is secure. When you add supporting software such as operating systems and external interoperable systems, the measurement of software security becomes even more difficult. Nevertheless some progress has been made.

Measures of software quality can go a long way toward measuring software security. Specifically, vulnerabilities invariably point to software defects. Although not all software defects are security problems, vulnerabilities in software generally result from a defect of some kind, whether it is in the requirements, architecture, or code. Therefore, measures such as defect and vulnerability counts [Woo14] are useful. Microsoft uses measures such as attack surface analysis and tries to keep the attack surface (places where software can be compromised) to a minimum.

Just as use of maturity models such as CMMI (Chapter 28) suggests that higher-quality software will result, mature security development processes, such as those emphasized by BSIMM,[11] will result in more secure software. In some cases, organizations are encouraged to identify the unique set of security metrics that are relevant to them. BSIMM makes reference to this, as does SAMM.[12]

9 See https://wiki.sei.cmu.edu/confluence/display/seccode/SEI+CERT+Coding+Standards.

10 A list of commercially available tools can be found at https://en.wikipedia.org/wiki/List_of_tools_for_static_code_analysis.

11 See https://www.bsimm.com/.

12 See https://www.owasp.org/index.php/OWASP_SAMM_Project.

It's important to note that none of the measurement characteristics implied by various maturity models is perfect. If you follow good secure software development processes, does that guarantee that the software is secure? No! If you find a lot of vulnerabilities, does that mean that most of them have been found or that there are still more to be found because this is a particularly poor piece of software? There are no simple answers. However, to assess vulnerabilities and associated software security, we have to collect data so that patterns can be analyzed over time. If we don't collect data about software security, we will never be able to measure its improvement.

Tables 18.4 and 18.5 provide examples of how to assess software security during each life-cycle phase. The full tables and discussion can be found in [Mea17] and [Alb10].

TABLE 18.4

Examples of life-cycle-phase measures

Life-Cycle Phase	Example Software Security Measures
Requirements engineering	Percentage of relevant software security principles reflected in requirements-specific actions (assuming security principles essential for a given development project have been selected)
	Percentage of security requirements that have been subject to analysis (risk, feasibility, cost-benefit, performance trade-offs) prior to being included in the specification
	Percentage of security requirements covered by attack patterns, misuse and abuse cases, and other specified means of threat modeling and analysis
Architecture and design	Percentage of architectural and design components subject to attack surface analysis and measurement
	Percentage of architectural and design components subject to architectural risk analysis
	Percentage of high-value security controls covered by a security design pattern

TABLE 18.5

Example measures based on the seven principles of evidence

Principle	Description
Risk	Number of active and latent threats, categorized
	Incidents reported by category of threat
	Likelihood of occurrence for each threat category
	Financial and/or human safety estimate of impact for each threat category
Trusted dependencies	Number of levels of subcontracting in the supply chain (in other words, have the subcontractors, in turn, executed subcontracts, and what is the depth of this activity?)
	Number of suppliers by level
	Hierarchical and peer dependencies between suppliers by level
	Number of (vetted) trusted suppliers in the supply chain by level

18.11 Security Process Improvement and Maturity Models

A number of process improvement and maturity models are available for software development in general, such as the Capability Maturity Model Integration (CMMI).[13] For cyber security maturity, the CMMI Institute offers a newer product, the Cyber Capability Maturity Management platform.[14] OWASP offers the Software Assurance Maturity Model (SAMM).[15] SAMM is an open framework to help organizations formulate and implement a strategy for software security that is tailored to the specific risks facing the organization.

A comprehensive discussion of these models is beyond the scope of this book. To provide a simple overview, consider the overall objective of SAMM:

- Evaluate an organization's existing software security practices.
- Build a balanced software security assurance program in well-defined iterations.
- Demonstrate concrete improvements to a security assurance program.
- Define and measure security-related activities throughout an organization.

Perhaps the most well-known maturity model that is specifically for software security is the Building Security in Maturity Model (BSIMM). BSIMM has periodic releases, typically every year or two. The BSIMM model and its recent summarized assessment results can be downloaded from the BSIMM website.[16] According to the BSIMM developers, BSIMM is meant to be used by those who create and execute a software security initiative.

All the maturity models mentioned here (and others) have benefits, and the essential elements of the models are freely available. However, sometimes the assessment is done by external entities. It's possible to do an internal self-assessment and define the associated improvement program, but it requires dedicated resources and effort. Alternatively, some of these organizations offer assessment programs, thus providing an external view of the strengths and areas for improvement within a software organization.

18.12 Summary

All software engineers should have an understanding of what it takes to develop secure software. The steps that are needed to improve the security of your software products are relevant in each activity in the software process, regardless of which process model is used.

Although there are still many open questions, and technologies that need additional research, there are many resources available today to assist with this challenge. For every activity that normally takes place in the software process, try to incorporate

13 See https://cmmiinstitute.com/.
14 See https://cmmiinstitute.com/products/cybermaturity.
15 See https://www.owasp.org/index.php/OWASP_SAMM_Project.
16 See https://www.bsimm.com/.

security aspects. Models such as Microsoft SDL and the SQUARE model can be assessed to determine which steps you can incorporate into your development process.

Add security to a risk analysis activity, especially using detailed guidance available from NIST. Given the number of secure coding standards already available, it is certainly possible for anyone to learn how to code securely. Inspect your code for remaining vulnerabilities. Learn how to identify security holes and develop and prioritize mitigation strategies. Perform static analysis testing on your code. Visit the OWASP and BSIMM websites, among others, to learn about maturity in software security engineering.

As software becomes ever more ubiquitous, the number of vulnerabilities and successful hacks grow as well. It will take all our efforts to stem the tide, but many of the tools already exist to tackle this problem. The consequences of failing to address software security are high, and the benefits of developing secure software are huge.

PROBLEMS AND POINTS TO PONDER

18.1. What is the most important thing that a software team can do to improve software security?

18.2. If you were recommending one activity for your organization to improve software security, what would it be? If you were recommending multiple activities, what are they, and what would be the priorities, considering that it's not likely that all of them will be implemented at once?

18.3. How could you incorporate software security into your existing process model or into a new process model?

18.4. Sit down with a colleague and identify security risks on a software project that is in development. Come up with mitigation strategies and prioritize them.

18.5. Are you collecting measurement data that could be used or repurposed to help measure software security? If not, is there data that could easily be collected for this purpose?

18.6. Use the Internet to find out the details needed to create a phishing attack pattern.

18.7. Explain some of the problems that might be encountered if you wait until after the system is completed to address security risks.

18.8. Use the Internet to determine the average cost to the consumer of a single incidence of identity theft.

18.9. Consider a MobileApp that you make use of on your personal phone. List three to five security risks that developers should consider when developing apps like this one.

18.10. Determine the security requirements of a bill-paying wallet-type MobileApp.

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

19 SOFTWARE TESTING—COMPONENT LEVEL

Software component testing incorporates a strategy that describes the steps to be conducted as part of testing, when these steps are planned and then undertaken, and how much effort, time, and resources will be required. Within the testing strategy, software component testing implements a collection of component testing tactics that address test planning, test-case design, test execution, and resultant data collection and evaluation. Both component testing strategy and tactics are considered in this chapter.

KEY CONCEPTS

basis path testing 384
black-box testing 388
boundary value analysis 389
class testing 390
control structure testing 386
cyclomatic complexity 385
debugging 373
equivalence partitioning 389
independent test group 375
integration testing 375
interface testing 388
object-oriented testing 390
scaffolding 379
system testing 376
testing methods 373
testing strategies 373
unit testing 376
validation testing 376
verification 373
white-box testing 383

QUICK LOOK

What is it? Software is tested to uncover errors that were made inadvertently as it was designed and constructed. A software component testing strategy considers testing of individual components and integrating them into a working system.

Who does it? A software component testing strategy is developed by the project manager, software engineers, and testing specialists.

Why is it important? Testing often accounts for more project effort than any other software engineering action. If it is conducted haphazardly, time is wasted, unnecessary effort is expended, and even worse, errors sneak through undetected.

What are the steps? Testing begins "in the small" and progresses "to the large." By this we mean that early testing focuses on a single component or on a small group of related components and applies tests to uncover errors in the data and processing logic that have been encapsulated by the component(s). After components are tested, they must be integrated until the complete system is constructed.

What is the work product? A *test specification* documents the software team's approach to testing by defining a plan that describes an overall strategy and a procedure that defines specific testing steps and the types of test cases that will be conducted.

How do I ensure that I've done it right? An effective test plan and procedure will lead to the orderly construction of the software and the discovery of errors at each stage in the construction process.

To be effective, a component testing strategy should be flexible enough to promote a customized testing approach but rigid enough to encourage reasonable planning and management tracking as the project progresses. Component testing remains a responsibility of individual software engineers. Who does the testing, how engineers communicate their results with one another, and when testing is done is determined by the software integration approach and design philosophy adopted by the development team.

These "approaches and philosophies" are what we call *strategy and tactics*—topics to be discussed in this chapter. In Chapter 20 we discuss the integration testing techniques that often end up defining the team development strategy.

19.1 A Strategic Approach to Software Testing

Testing is a set of activities that can be planned in advance and conducted systematically. For this reason, a template for software testing—a set of steps into which we can place specific test-case design techniques and testing methods—should be defined for the software process.

A number of software testing strategies have been proposed in the literature [Jan16] [Dak14] [Gut15]. All provide you with a template for testing, and all have the following generic characteristics:

- To perform effective testing, you should conduct technical reviews (Chapter 16). By doing this, many errors will be eliminated before testing commences.
- Testing begins at the component level and works "outward" toward the integration of the entire computer-based system.
- Different testing techniques are appropriate for different software engineering approaches and at different points in time.
- Testing is conducted by the developer of the software and (for large projects) an independent test group.
- Testing and debugging are different activities, but debugging must be accommodated in any testing strategy.

A strategy for software testing incorporates a set of tactics that accommodate the low-level tests necessary to verify that a small source code segment has been correctly implemented as well as high-level tests that validate major system functions against customer requirements. A strategy should provide guidance for the practitioner and a set of milestones for the manager. Because the steps of the test strategy occur at a time when deadline pressure begins to rise, progress must be measurable and problems should surface as early as possible.

19.1.1 Verification and Validation

Software testing is one element of a broader topic that is often referred to as verification and validation (V&V). *Verification* refers to the set of tasks that ensure that software correctly implements a specific function. *Validation* refers to a different set

of tasks that ensure that the software that has been built is traceable to customer requirements. Boehm [Boe81] states this another way:

> Verification: "Are we building the product right?"
> Validation: "Are we building the right product?"

The definition of V&V encompasses many software quality assurance activities (Chapter 19).[1]

Verification and validation include a wide array of SQA activities: technical reviews, quality and configuration audits, performance monitoring, simulation, feasibility study, documentation review, database review, algorithm analysis, development testing, usability testing, qualification testing, acceptance testing, and installation testing. Although testing plays an extremely important role in V&V, many other activities are also necessary.

Testing does provide the last bastion from which quality can be assessed and, more pragmatically, errors can be uncovered. But testing should not be viewed as a safety net. As they say, "You can't test in quality. If it's not there before you begin testing, it won't be there when you're finished testing." Quality is incorporated into software throughout the process of software engineering, and testing cannot be applied as a fix at the end of the process. Proper application of methods and tools, effective technical reviews, and solid management and measurement all lead to quality that is confirmed during testing.

19.1.2 Organizing for Software Testing

For every software project, there is an inherent conflict of interest that occurs as testing begins. The people who have built the software are now asked to test the software. This seems harmless in itself; after all, who knows the program better than its developers? Unfortunately, these same developers have a vested interest in demonstrating that the program is error-free, that it works according to customer requirements, and that it will be completed on schedule and within budget. Each of these interests mitigates against thorough testing.

From a psychological point of view, software analysis and design (along with coding) are constructive tasks. The software engineer analyzes, models, and then creates a computer program and its documentation. Like any builder, the software engineer is proud of the edifice that has been built and looks askance at anyone who attempts to tear it down. When testing commences, there is a subtle, yet definite, attempt to "break" the thing that the software engineer has built. From the point of view of the builder, testing can be considered to be (psychologically) destructive. So the builder treads lightly, designing and executing tests that will demonstrate that the program works, rather than to uncover errors. Unfortunately, errors will be nevertheless present. And, if the software engineer doesn't find them, the customer will!

---

1 It should be noted that there is a strong divergence of opinion about what types of testing constitute "validation." Some people believe that *all* testing is verification and that validation is conducted when requirements are reviewed and approved, and later, by the user when the system is operational. Other people view unit and integration testing (Chapters 19 and 20) as verification and higher-order testing (Chapter 21) as validation.

There are often a number of misconceptions that you might infer from the preceding discussion: (1) that the developer of software should do no testing at all, (2) that the software should be "tossed over the wall" to strangers who will test it mercilessly, and (3) that testers get involved with the project only when the testing steps are about to begin. Each of these statements is incorrect.

The software developer is always responsible for testing the individual units (components) of the program, ensuring that each performs the function or exhibits the behavior for which it was designed. In many cases, the developer also conducts *integration testing*—a testing step that leads to the construction (and test) of the complete software architecture. Only after the software architecture is complete does an independent test group become involved.

The role of an *independent test group* (ITG) is to remove the inherent problems associated with letting the builder test the thing that has been built. Independent testing removes the conflict of interest that may otherwise be present. After all, ITG personnel are paid to find errors.

However, you don't turn the program over to ITG and walk away. The developer and the ITG work closely throughout a software project to ensure that thorough tests will be conducted. While testing is conducted, the developer must be available to correct errors that are uncovered.

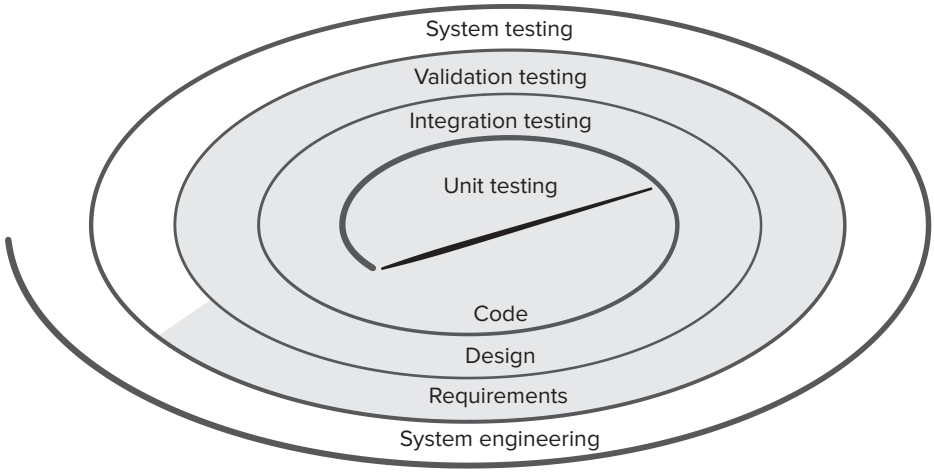
The ITG is part of the software development project team in the sense that it becomes involved during analysis and design and stays involved (planning and specifying test procedures) throughout a large project. However, in many cases the ITG reports to the software quality assurance organization, thereby achieving a degree of independence that might not be possible if it were a part of the software engineering team.

19.1.3 The Big Picture

The software process may be viewed as the spiral illustrated in Figure 19.1. Initially, system engineering defines the role of software and leads to software requirements analysis, where the information domain, function, behavior, performance,

FIGURE 19.1

Testing strategy

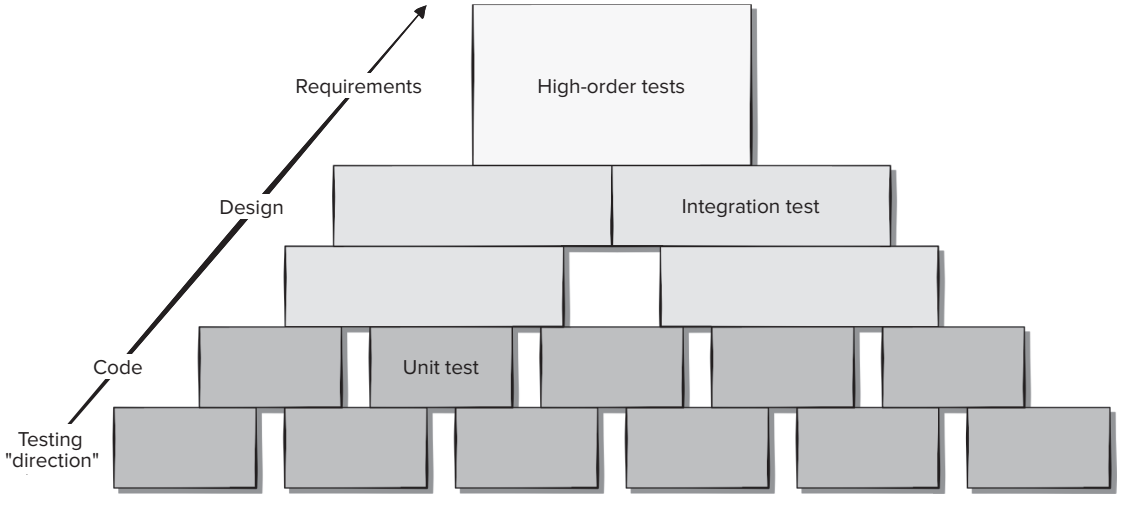


constraints, and validation criteria for software are established. Moving inward along the spiral, you come to design and finally to coding. To develop computer software, you spiral inward along streamlines that decrease the level of abstraction on each turn.

A strategy for software testing may also be viewed in the context of the spiral (Figure 19.1). *Unit testing* begins at the vortex of the spiral and concentrates on each unit (e.g., component, class, or WebApp content object) of the software as implemented in source code. Testing progresses by moving outward along the spiral to *integration testing,* where the focus is on design and the construction of the software architecture. Taking another turn outward on the spiral, you encounter *validation testing,* where requirements established as part of requirements modeling are validated against the software that has been constructed. Finally, you arrive at *system testing,* where the software and other system elements are tested as a whole. To test computer software, you spiral out along streamlines that broaden the scope of testing with each turn.

Considering the process from a procedural point of view, testing within the context of software engineering is actually a series of four steps that are implemented sequentially. The steps are shown in Figure 19.2. Initially, tests focus on each component individually, ensuring that it functions properly as a unit. Hence, the name *unit testing.* Unit testing makes heavy use of testing techniques that exercise specific paths in a component's control structure to ensure complete coverage and maximum error detection. Next, components must be assembled or integrated to form the complete software package. Integration testing addresses the issues associated with the dual problems of verification and program construction. Test-case design techniques that focus on inputs and outputs are more prevalent during integration, although techniques that exercise specific program paths may be used to ensure coverage of major control paths.

FIGURE 19.2 **Software testing steps**



After the software has been integrated (constructed), a set of *high-order tests* is conducted. Validation criteria (established during requirements analysis) must be evaluated. Validation testing provides final assurance that software meets all functional, behavioral, and performance requirements.

The last high-order testing step falls outside the boundary of software engineering and into the broader context of computer system engineering (discussed in Chapter 21). Software, once validated, must be combined with other system elements (e.g., hardware, people, databases). System testing verifies that all elements mesh properly and that overall system function and performance is achieved.

SAFEHOME



Preparing for Testing

The scene: Doug Miller's office, as component-level design continues and construction of certain components begins.

The players: Doug Miller, software engineering manager; Vinod, Jamie, Ed, and Shakira, members of the *SafeHome* software engineering team.

The conversation:

Doug: It seems to me that we haven't spent enough time talking about testing.

Vinod: True, but we've all been just a little busy. And besides, we have been thinking about it . . . in fact, more than thinking.

Doug (smiling): I know . . . we're all overloaded, but we've still got to think down the line.

Shakira: I like the idea of designing unit tests before I begin coding any of my components, so that's what I've been trying to do. I have a pretty big file of tests to run once code for my components is complete.

Doug: That's an Extreme Programming [an agile software development process, see Chapter 3] concept, no?

Ed: It is. Even though we're not using Extreme Programming per se, we decided that it'd be a good idea to design unit tests before we build the component—the design gives us all of the information we need.

Jamie: I've been doing the same thing.

Vinod: And I've taken on the role of the integrator, so every time one of the guys passes a component to me, I'll integrate it and run a series of regression tests [see Section 20.3 for a discussion on regression testing] on the partially integrated program. I've been working to design a set of appropriate tests for each function in the system.

Doug (to Vinod): How often will you run the tests?

Vinod: Every day . . . until the system is integrated . . . well, I mean until the software increment we plan to deliver is integrated.

Doug: You guys are way ahead of me!

Vinod (laughing): Anticipation is everything in the software biz, Boss.

19.1.4 Criteria for "Done"

A classic question arises every time software testing is discussed: "When are we done testing—how do we know that we've tested enough?" Sadly, there is no definitive answer to this question, but there are a few pragmatic responses and early attempts at empirical guidance.

One response to the question is: "You're never done testing; the burden simply shifts from you (the software engineer) to the end user." Every time the user executes a computer program, the program is being tested. This sobering fact underlines the importance of other software quality assurance activities. Another response (somewhat cynical but nonetheless accurate) is: "You're done testing when you run out of time or you run out of money."

Although few practitioners would argue with these responses, you need more rigorous criteria for determining when sufficient testing has been conducted. The *statistical quality assurance* approach (Section 17.6) suggests statistical use techniques [Rya11] that execute a series of tests derived from a statistical sample of all possible program executions by all users from a targeted population. By collecting metrics during software testing and making use of existing statistical models, it is possible to develop meaningful guidelines for answering the question: "When are we done testing?"

19.2 PLANNING AND RECORDKEEPING

Many strategies can be used to test software. At one extreme, you can wait until the system is fully constructed and then conduct tests on the overall system in the hope of finding errors. This approach, although appealing, simply does not work. It will result in buggy software that disappoints all stakeholders. At the other extreme, you could conduct tests on a daily basis, whenever any part of the system is constructed.

A testing strategy that is chosen by many software teams (and the one we recommend) falls between the two extremes. It takes an incremental view of testing, beginning with the testing of individual program units, moving to tests designed to facilitate the integration of the units (sometimes on a daily basis), and culminating with tests that exercise the constructed system as it evolves. The remainder of this chapter will focus on component-level testing and test-case design.

Unit testing focuses verification effort on the smallest unit of software design—the software component or module. Using the component-level design description as a guide, important control paths are tested to uncover errors within the boundary of the module. The relative complexity of tests and the errors those tests uncover is limited by the constrained scope established for unit testing. The unit test focuses on the internal processing logic and data structures within the boundaries of a component. This type of testing can be conducted in parallel for multiple components.

The best strategy will fail if a series of overriding issues are not addressed. Tom Gilb [Gil95] argues that a software testing strategy will succeed only when software testers: (1) specify product requirements in a quantifiable manner long before testing commences, (2) state testing objectives explicitly, (3) understand the users of the software and develop a profile for each user category, (4) develop a testing plan that emphasizes "rapid cycle testing,"[2] (5) build "robust" software that is designed to test

2 Gilb [Gil95] recommends that a software team "learn to test in rapid cycles (2 percent of project effort) of customer-useful, at least field 'trialable,' increments of functionality and/or quality improvement." The feedback generated from these rapid cycle tests can be used to control quality levels and the corresponding test strategies.

itself (the concept of antibugging is discussed briefly in Section 9.3), (6) use effective technical reviews as a filter prior to testing, (7) conduct technical reviews to assess the test strategy and test cases themselves, and (8) develop a continuous improvement approach (Chapter 28) for the testing process.

These principles are reflected in agile software testing as well. In agile development, the test plan needs to be established before the first sprint meeting and reviewed by stakeholders. This plan merely lays out the rough time line, standards, and tools to be used. The test cases and directions for their use are developed and reviewed by the stakeholders as the code needed to implement each user story is created. Testing results are shared with all team members as soon as practical to allow changes in both existing and future code development. For this reason, many teams choose to keep their test recordkeeping in online documents.

Test recordkeeping does not need to be burdensome. The test cases can be recorded in a Google Docs spreadsheet that briefly describes the test case, contains a pointer to the requirement being tested, contains expected output from the test case data or the criteria for success, allows testers to indicate whether the test was passed or failed and the dates the test case was run, and should have room for comments about why a test may have failed to aid in debugging. This type of online form can be viewed as needed for analysis, and it is easy to summarize at team meetings. Test-case design issues are discussed in Section 19.3.

19.2.1 Role of Scaffolding

Component testing is normally considered as an adjunct to the coding step. The design of unit tests can occur before coding begins or after source code has been generated. A review of design information provides guidance for establishing test cases that are likely to uncover errors. Each test case should be coupled with a set of expected results.

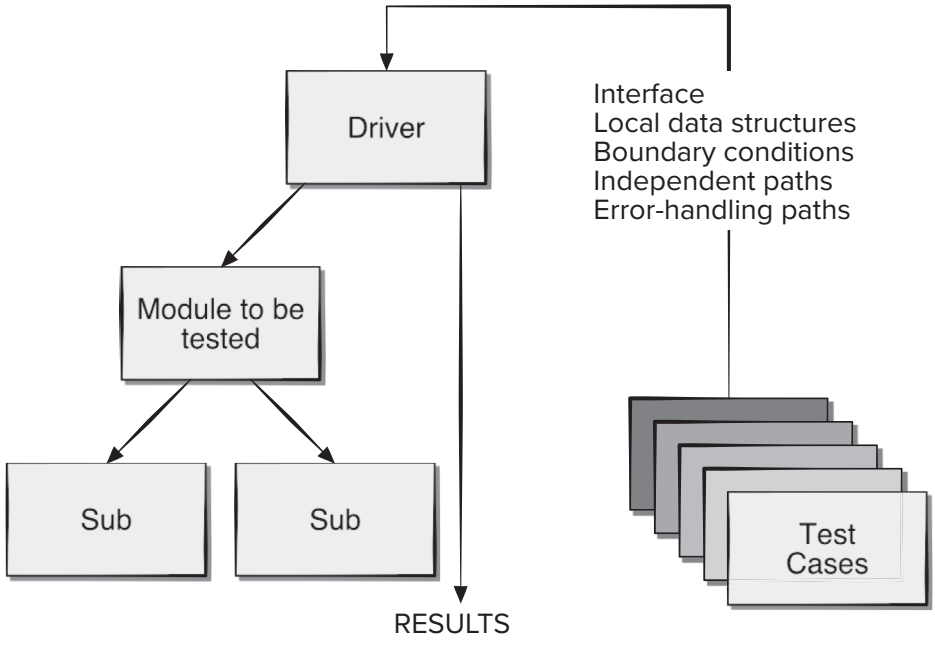
Because a component is not a stand-alone program, some type of *scaffolding* is required to create a testing framework. As part of this framework, driver and/or stub software must often be developed for each unit test. The unit-test environment is illustrated in Figure 19.3. In most applications a *driver* is nothing more than a "main program" that accepts test-case data, passes such data to the component (to be tested), and prints relevant results. *Stubs* serve to replace modules that are subordinate (invoked by) the component to be tested. A *stub* or "dummy subprogram" uses the subordinate module's interface, may do minimal data manipulation, prints verification of entry, and returns control to the module undergoing testing.

Drivers and stubs represent testing "overhead." That is, both are software that must be coded (formal design is not commonly applied) but that is not delivered with the final software product. If drivers and stubs are kept simple, actual overhead is relatively low. Unfortunately, many components cannot be adequately unit-tested with "simple" scaffolding software. In such cases, complete testing can be postponed until the integration test step (where drivers or stubs are also used).

19.2.2 Cost-Effective Testing

Exhaustive testing requires every possible combination of input values and test-case orderings be processed by the component being tested (e.g., consider the *move generator* in a computer chess game). In some cases, this would require the creation of

FIGURE 19.3
Unit-test environment



a near-infinite number of data sets. The return on exhaustive testing is often not worth the effort, since testing alone cannot be used to prove a component is correctly implemented. There are some situations in which you will not have the resources to do comprehensive unit testing. In these cases, testers should select modules crucial to the success of the project and those that are suspected to be error-prone because they have complexity metrics as the focus for your unit testing. Some techniques for minimizing the number of test cases required to do a good job testing are discussed in Sections 19.4 through 19.6.

INFO



Exhaustive Testing

Consider a 100-line program in the language C. After some basic data declaration, the program contains two nested loops that execute from 1 to 20 times each, depending on conditions specified at input. Inside the interior loop, four if-then-else constructs are required. There are approximately 10^{14} possible paths that may be executed in this program!

To put this number in perspective, we assume that a magic test processor ("magic" because no such processor exists) has been developed for exhaustive testing. The processor can develop a test case, execute it, and evaluate the results in one millisecond. Working 24 hours a day, 365 days a year, the processor would work for 3170 years to test the program. This would, undeniably, cause havoc in most development schedules.

Therefore, it is reasonable to assert that exhaustive testing is impossible for large software systems.

19.3 TEST-CASE DESIGN

It is a good idea to design unit test cases before you develop code for a component. This ensures that you'll develop code that will pass the tests or at least the tests you thought of already.

Unit tests are illustrated schematically in Figure 19.4. The module interface is tested to ensure that information properly flows into and out of the program unit under test (Section 19.5.1). Local data structures are examined to ensure that data stored temporarily maintains its integrity during all steps in an algorithm's execution. All independent paths through the control structure are exercised to ensure that all statements in a module have been executed at least once (Section 19.4.2). Boundary conditions are tested to ensure that the module operates properly at boundaries established to limit or restrict processing (Section 19.5.3). And finally, all error-handling paths are tested.

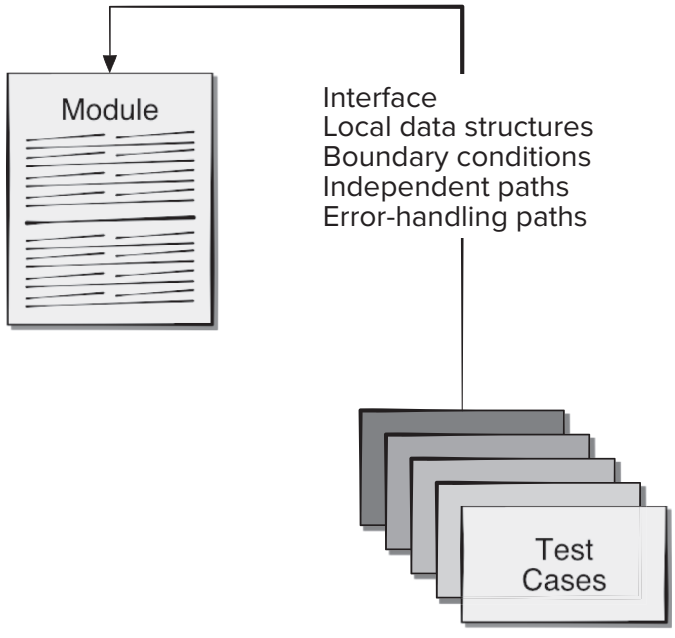
Data flow across a component interface is tested before any other testing is initiated. If data do not enter and exit properly, all other tests are moot. In addition, local data structures should be exercised and the local impact on global data should be ascertained (if possible) during unit testing.

Selective testing of execution paths is an essential task during the unit test. Test cases should be designed to uncover errors due to erroneous computations, incorrect comparisons, or improper control flow.

Boundary testing is one of the most important unit-testing tasks. Software often fails at its boundaries. That is, errors often occur when the *n*th element of an *n*-dimensional array is processed, when the *i*th repetition of a loop with *i* passes is invoked, or when the maximum or minimum allowable value is encountered. Test cases that exercise

FIGURE 19.4

Unit test



data structure, control flow, and data values just below, at, and just above maxima and minima are very likely to uncover errors.

A good design anticipates error conditions and establishes error-handling paths to reroute or cleanly terminate processing when an error does occur. Yourdon [You75] calls this approach *antibugging*. Unfortunately, there is a tendency to incorporate error handling into software and then never test the error handling. Be sure that you design tests to execute every error-handling path. If you don't, the path may fail when it is invoked, exacerbating an already dicey situation.

Among the potential errors that should be tested when error handling is evaluated are: (1) error description is unintelligible, (2) error noted does not correspond to error encountered, (3) error condition causes system intervention prior to error handling, (4) exception-condition processing is incorrect, or (5) error description does not provide enough information to assist in the location of the cause of the error.

SAFEHOME



Designing Unique Tests

The scene: Vinod's cubical.

The players: Vinod and Ed, members of the *SafeHome* software engineering team.

The conversation:

Vinod: So these are the test cases you intend to run for the *passwordValidation* operation.

Ed: Yeah, they should cover pretty much all possibilities for the kinds of passwords a user might enter.

Vinod: So let's see . . . you note that the correct password will be 8080, right?

Ed: Uh-huh.

Vinod: And you specify passwords 1234 and 6789 to test for error in recognizing invalid passwords?

Ed: Right, and I also test passwords that are close to the correct password, see . . . 8081 and 8180.

Vinod: Those are okay, but I don't see much point in running both the 1234 and 6789 inputs. They're redundant . . . test the same thing, don't they?

Ed: Well, they're different values.

Vinod: That's true, but if 1234 doesn't uncover an error . . . in other words . . . the *passwordValidation* operation notes that it's an invalid password, it's not likely that 6789 will show us anything new.

Ed: I see what you mean.

Vinod: I'm not trying to be picky here . . . it's just that we have limited time to do testing, so it's a good idea to run tests that have a high likelihood of finding new errors.

Ed: Not a problem . . . I'll give this a bit more thought.

19.3.1 Requirements and Use Cases

In requirements engineering (Chapter 7) we suggested starting the requirements gathering process by working with the customers to generate user stories that developers can refine into formal use cases and analysis models. These use cases and models can be used to guide the systematic creation of test cases that do a good job of testing the functional requirements of each software component and provide good test coverage overall [Gut15].

The analysis artifacts do not provide much insight into the creation of test cases for many nonfunctional requirements (e.g., usability or reliability). This is where the customer's acceptance statements included in the user stories can form the basis for writing test cases for the nonfunctional requirements associated with components. Test-case developers make use of additional information based on their professional experience to quantify acceptance criteria to make it testable. Testing nonfunctional requirements may require the use of integration testing methods (Chapter 20) or other specialized testing techniques (Chapter 21).

The primary purpose of testing is to help developers discover defects that were previously unknown. Executing test cases that demonstrate the component is running correctly is often not good enough. As we mentioned earlier (Section 19.3), it is important to write test cases that exercise the error-handling capabilities of a component. But if we are to uncover new defects, it is also important to write test cases to test that a component does not do things it is not supposed to do (e.g., accessing privileged data sources without proper permissions). These may be stated formally as *anti-requirements*[3] and may require specialized security testing techniques (Section 21.7) [Ale17]. These so-called *negative test cases* should be included to make sure the component behaves according to the customer's expectations.

19.3.2 Traceability

To ensure that the testing process is auditable, each test case needs to be traceable back to specific functional or nonfunctional requirements or anti-requirements. Often nonfunctional requirements need to be traceable to specific business or architectural requirements. Many agile developers resist the concept of traceability as an unnecessary burden on developers. But many test process failures can be traced to missing traceability paths, inconsistent test data, or incomplete test coverage [Rem14]. Regression testing (discussed in Section 20.3) requires retesting selected components that may be affected by changes made to other software components that it collaborates with. Although this is more often considered an issue in integration testing (Chapter 20), making sure that test cases are traceable to requirements is an important first step and needs to be done during component testing.

19.4 WHITE-BOX TESTING

White-box testing, sometimes called glass-box testing or structural testing, is a test-case design philosophy that uses the control structure described as part of component-level design to derive test cases. Using white-box testing methods, you can derive test cases that (1) guarantee that all independent paths within a module have been exercised at least once, (2) exercise all logical decisions on their true and false sides, (3) execute all loops at their boundaries and within their operational bounds, and (4) exercise internal data structures to ensure their validity.

3 Anti-requirements are sometimes described during the creation of abuse cases that describe a user story from the perspective of a malicious user and are part of threat analysis (discussed in Chapter 18).

19.4.1 Basis Path Testing

Basis path testing is a white-box testing technique first proposed by Tom McCabe [McC76]. The basis path method enables the test-case designer to derive a logical complexity measure of a procedural design and use this measure as a guide for defining a basis set of execution paths. Test cases derived to exercise the basis set are guaranteed to execute every statement in the program at least one time during testing.

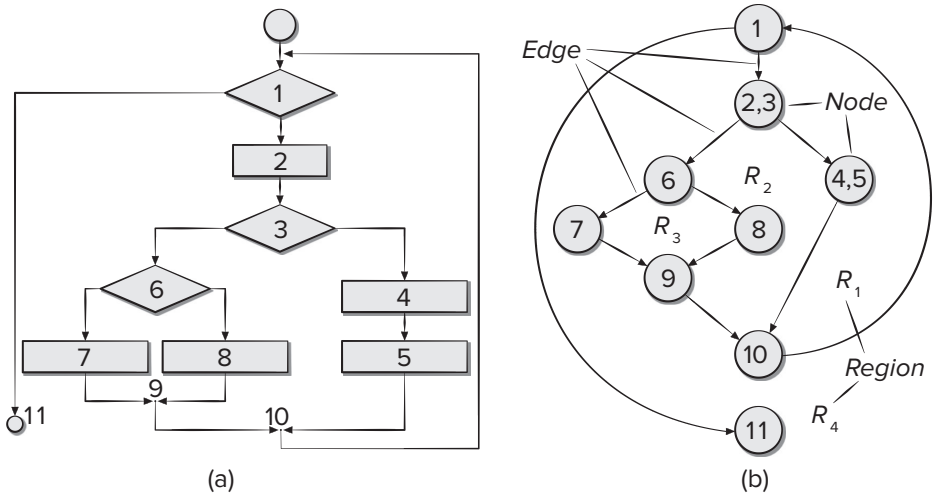
Before the basis path method can be introduced, a simple notation for the representation of control flow, called a flow graph (or program graph), must be introduced.[4] A flow graph should be drawn only when the logical structure of a component is complex. The flow graph allows you to trace program paths more readily.

To illustrate the use of a flow graph, consider the procedural design representation in Figure 19.5a. Here, a flowchart is used to depict program control structure. Figure 19.5b maps the flowchart into a corresponding flow graph (assuming that no compound conditions are contained in the decision diamonds of the flowchart). Referring to Figure 19.5b, each circle, called a *flow graph node,* represents one or more procedural statements. A sequence of process boxes and a decision diamond can map into a single node. The arrows on the flow graph, called *edges* or *links,* represent flow of control and are analogous to flowchart arrows. An edge must terminate at a node, even if the node does not represent any procedural statements (e.g., see the flow graph symbol for the if-then-else construct). Areas bounded by edges and nodes are called *regions.* When counting regions, we include the area outside the graph as a region.

An *independent path* is any path through the program that introduces at least one new set of processing statements or a new condition. When stated in terms of a flow

FIGURE 19.5

(a) Flowchart and (b) flow graph



4 In actuality, the basis path method can be conducted without the use of flow graphs. However, they serve as a useful notation for understanding control flow and illustrating the approach.

graph, an independent path must move along at least one edge that has not been traversed before the path is defined. For example, a set of independent paths for the flow graph illustrated in Figure 19.5b is

Path 1: 1-11

Path 2: 1-2-3-4-5-10-1-11

Path 3: 1-2-3-6-8-9-10-1-11

Path 4: 1-2-3-6-7-9-10-1-11

Note that each new path introduces a new edge. The path

1-2-3-4-5-10-1-2-3-6-8-9-10-1-11

is not considered to be an independent path because it is simply a combination of already specified paths and does not traverse any new edges.

Paths 1 through 4 constitute a *basis set* for the flow graph in Figure 19.5b. That is, if you can design tests to force execution of these paths (a basis set), every statement in the program will have been guaranteed to be executed at least one time and every condition will have been executed on its true and false sides. It should be noted that the basis set is not unique. In fact, a number of different basis sets can be derived for a given procedural design.

How do you know how many paths to look for? The computation of cyclomatic complexity provides the answer. *Cyclomatic complexity* is a software metric that provides a quantitative measure of the logical complexity of a program. When used in the context of the basis path testing method, the value computed for cyclomatic complexity defines the number of independent paths in the basis set of a program and provides you with an upper bound for the number of tests that must be conducted to ensure that all statements have been executed at least once.

Cyclomatic complexity has a foundation in graph theory and provides you with an extremely useful software metric. Complexity is computed in one of three ways:

1. The number of regions of the flow graph corresponds to the cyclomatic complexity.
2. Cyclomatic complexity $V(G)$ for a flow graph G is defined as

   $V(G) = E - N + 2$

   where E is the number of flow graph edges and N is the number of flow graph nodes.
3. Cyclomatic complexity $V(G)$ for a flow graph G is also defined as

   $V(G) = P + 1$

   where P is the number of predicate nodes contained in the flow graph G.

Referring once more to the flow graph in Figure 19.5b, the cyclomatic complexity can be computed using each of the algorithms just noted:

1. The flow graph has four regions.
2. $V(G) = 11$ edges $- 9$ nodes $+ 2 = 4$.
3. $V(G) = 3$ predicate nodes $+ 1 = 4$.

Therefore, the cyclomatic complexity of the flow graph in Figure 19.5b is 4.

More important, the value for $V(G)$ provides you with an upper bound for the number of independent paths that form the basis set and, by implication, an upper bound on the number of tests that must be designed and executed to guarantee coverage of all program statements. So in this case we would need to define at most four test cases to exercise each independent logic path.

SAFEHOME



Using Cyclomatic Complexity

The scene: Shakira's cubicle.

The players: Vinod and Shakira—members of the *SafeHome* software engineering team who are working on test planning for the security function.

The conversation:

Shakira: Look . . . I know that we should unit-test all the components for the security function, but there are a lot of 'em and if you consider the number of operations that have to be exercised, I don't know . . . maybe we should forget white-box testing, integrate everything, and start running black-box tests.

Vinod: You figure we don't have enough time to do component tests, exercise the operations, and then integrate?

Shakira: The deadline for the first increment is getting closer than I'd like . . . yeah, I'm concerned.

Vinod: Why don't you at least run white-box tests on the operations that are likely to be the most error-prone?

Shakira (exasperated): And exactly how do I know which are the most error-prone?

Vinod: *V* of *G*.

Shakira: Huh?

Vinod: Cyclomatic complexity—*V* of *G*. Just compute *V*(*G*) for each of the operations within each of the components and see which have the highest values for *V*(*G*). They're the ones that are most likely to be error-prone.

Shakira: And how do I compute *V* of *G*?

Vinod: It's really easy. Here's a book that describes how to do it.

Shakira (leafing through the pages): Okay, it doesn't look hard. I'll give it a try. The ops with the highest *V*(*G*) will be the candidates for white-box tests.

Vinod: Just remember that there are no guarantees. A component with a low *V*(*G*) can still be error-prone.

Shakira: Alright. But at least this'll help me to narrow down the number of components that have to undergo white-box testing.

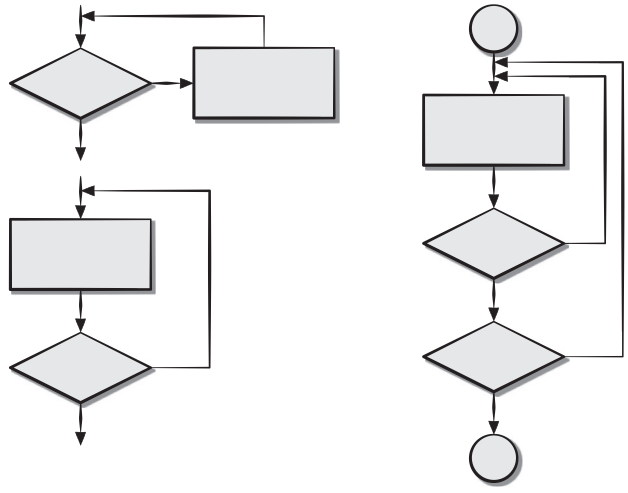
19.4.2 Control Structure Testing

The basis path testing technique described in Section 19.4.1 is one of a number of techniques for control structure testing. Although basis path testing is simple and highly effective, it is not sufficient in itself. In this section, other variations on control structure testing are discussed. These broaden testing coverage and improve the quality of white-box testing.

Condition testing [Tai89] is a test-case design method that exercises the logical conditions contained in a program module. *Data flow testing* [Fra93] selects test paths of a program according to the locations of definitions and uses of variables in the program.

FIGURE 19.6

Classes of loops



Loop testing is a white-box testing technique that focuses exclusively on the validity of loop constructs. Two different classes of loops [Bei90] can be defined: simple loops and nested loops. (Figure 19.6).

Simple Loops. The following set of tests can be applied to simple loops, where n is the maximum number of allowable passes through the loop.

1. Skip the loop entirely.
2. Only one pass through the loop.
3. Two passes through the loop.
4. m passes through the loop where $m < n$.
5. $n - 1$, n, $n + 1$ passes through the loop.

Nested Loops. If we were to extend the test approach for simple loops to nested loops, the number of possible tests would grow geometrically as the level of nesting increases. This would result in an impractical number of tests. Beizer [Bei90] suggests an approach that will help to reduce the number of tests:

1. Start at the innermost loop. Set all other loops to minimum values.
2. Conduct simple loop tests for the innermost loop while holding the outer loops at their minimum iteration parameter (e.g., loop counter) values. Add other tests for out-of-range or excluded values.
3. Work outward, conducting tests for the next loop, but keeping all other outer loops at minimum values and other nested loops to "typical" values.
4. Continue until all loops have been tested.

19.5 Black-Box Testing

Black-box testing, also called *behavioral testing* or *functional testing,* focuses on the functional requirements of the software. That is, black-box testing techniques enable you to derive sets of input conditions that will fully exercise all functional requirements for a program. Black-box testing is not an alternative to white-box techniques. Rather, it is a complementary approach that is likely to uncover a different class of errors than white-box methods.

Black-box testing attempts to find errors in the following categories: (1) incorrect or missing functions, (2) interface errors, (3) errors in data structures or external database access, (4) behavior or performance errors, and (5) initialization and termination errors.

Unlike white-box testing, which is performed early in the testing process, black-box testing tends to be applied during later stages of testing. Because black-box testing purposely disregards control structure, attention is focused on the information domain. Tests are designed to answer the following questions:

- How is functional validity tested?
- How are system behavior and performance tested?
- What classes of input will make good test cases?
- Is the system particularly sensitive to certain input values?
- How are the boundaries of a data class isolated?
- What data rates and data volume can the system tolerate?
- What effect will specific combinations of data have on system operation?

By applying black-box techniques, you derive a set of test cases that satisfy the following criteria [Mye79]: test cases that reduce, by a count that is greater than one, the number of additional test cases that must be designed to achieve reasonable testing, and test cases that tell you something about the presence or absence of classes of errors, rather than an error associated only with the specific test at hand.

19.5.1 Interface Testing

Interface testing is used to check that the program component accepts information passed to it in the proper order and data types and returns information in proper order and data format [Jan16]. Interface testing is often considered part of integration testing. Because most components are not stand-alone programs, it is important to make sure that when the component is integrated into the evolving program it will not break the build. This is where the use stubs and drivers (Section 19.2.1) become important to component testers.

Stubs and drivers sometimes incorporate test cases to be passed to the component or accessed by the component. In other cases, debugging code may need to be inserted inside the component to check that data passed was received correctly (Section 19.3). In still other cases, the testing framework should contain code to check that data returned from the component is received correctly. Some agile developers prefer to do interface testing using a copy of the production version of the evolving program with some of this debugging code added.

19.5.2 Equivalence Partitioning

Equivalence partitioning is a black-box testing method that divides the input domain of a program into classes of data from which test cases can be derived. An ideal test case single-handedly uncovers a class of errors (e.g., incorrect processing of all character data) that might otherwise require many test cases to be executed before the general error is observed.

Test-case design for equivalence partitioning is based on an evaluation of *equivalence classes* for an input condition. Using concepts introduced in the preceding section, if a set of objects can be linked by relationships that are symmetric, transitive, and reflexive, an equivalence class is present [Bei95]. An equivalence class represents a set of valid or invalid states for input conditions. Typically, an input condition is either a specific numeric value, a range of values, a set of related values, or a Boolean condition. Equivalence classes may be defined according to the following guidelines:

1. If an input condition specifies a range, one valid and two invalid equivalence classes are defined.
2. If an input condition requires a specific value, one valid and two invalid equivalence classes are defined.
3. If an input condition specifies a member of a set, one valid and one invalid equivalence class are defined.
4. If an input condition is Boolean, one valid and one invalid class are defined.

By applying the guidelines for the derivation of equivalence classes, test cases for each input domain data item can be developed and executed. Test cases are selected so that the largest number of attributes of an equivalence class are exercised at once.

19.5.3 Boundary Value Analysis

A greater number of errors occurs at the boundaries of the input domain rather than in the "center." It is for this reason that *boundary value analysis* (BVA) has been developed as a testing technique. Boundary value analysis leads to a selection of test cases that exercise bounding values.

Boundary value analysis is a test-case design technique that complements equivalence partitioning. Rather than selecting any element of an equivalence class, BVA leads to the selection of test cases at the "edges" of the class. Rather than focusing solely on input conditions, BVA derives test cases from the output domain as well [Mye79].

Guidelines for BVA are similar in many respects to those provided for equivalence partitioning:

1. If an input condition specifies a range bounded by values *a* and *b,* test cases should be designed with values *a* and *b* and just above and just below *a* and *b.*
2. If an input condition specifies a number of values, test cases should be developed that exercise the minimum and maximum numbers. Values just above and below minimum and maximum are also tested.
3. Apply guidelines 1 and 2 to output conditions. For example, assume that a temperature versus pressure table is required as output from an engineering

analysis program. Test cases should be designed to create an output report that produces the maximum (and minimum) allowable number of table entries.

4. If internal program data structures have prescribed boundaries (e.g., a table has a defined limit of 100 entries), be certain to design a test case to exercise the data structure at its boundary.

Most software engineers intuitively perform BVA to some degree. By applying these guidelines, boundary testing will be more complete, thereby having a higher likelihood for error detection.

19.6 OBJECT-ORIENTED TESTING

When object-oriented software is considered, the concept of the unit changes. Encapsulation drives the definition of classes and objects. This means that each class and each instance of a class packages attributes (data) and the operations that manipulate these data. An encapsulated class is usually the focus of unit testing. However, operations (methods) within the class are the smallest testable units. Because a class can contain a number of different operations, and a particular operation may exist as part of a number of different classes, the tactics applied to unit testing must change.

You can no longer test a single operation in isolation (the conventional view of unit testing) but rather as part of a class. To illustrate, consider a class hierarchy in which an operation X is defined for the superclass and is inherited by a number of subclasses. Each subclass uses operation X, but it is applied within the context of the private attributes and operations that have been defined for the subclass. Because the context in which operation X is used varies in subtle ways, it is necessary to test operation X in the context of each of the subclasses. This means that testing operation X in a stand-alone fashion (the conventional unit-testing approach) is usually ineffective in the object-oriented context.

19.6.1 Class Testing

Class testing for object-oriented (OO) software is the equivalent of unit testing for conventional software. Unlike unit testing of conventional software, which tends to focus on the algorithmic detail of a module and the data that flow across the module interface, class testing for OO software is driven by the operations encapsulated by the class and the state behavior of the class.

To provide brief illustrations of these methods, consider a banking application in which an **Account** class has the following operations: *open(), setup(), deposit(), withdraw(), balance(), summarize(), creditLimit(),* and *close()* [Kir94]. Each of these operations may be applied for **Account,** but certain constraints (e.g., the account must be opened before other operations can be applied and closed after all operations are completed) are implied by the nature of the problem. Even with these constraints, there are many permutations of the operations. The minimum behavioral life history of an instance of **Account** includes the following operations:

```
open•setup•deposit•withdraw•close
```

This represents the minimum test sequence for account. However, a wide variety of other behaviors may occur within this sequence:

open•setup•deposit•[deposit|withdraw|balance|summarize|creditLimit]^n• withdraw•close

A variety of different operation sequences can be generated randomly. For example:

Test case r_1*:*

open•setup•deposit•deposit•balance•summarize•withdraw•close

Test case r_2*:*

open•setup•deposit•withdraw•deposit•balance•creditLimit•withdraw•close

These and other random order tests are conducted to exercise different class instance life histories. Use of test equivalence partitioning (Section 19.5.2) can reduce the number of test cases required.

SAFEHOME



Class Testing

The scene: Shakira's cubicle.

The players: Jamie and Shakira, members of the *SafeHome* software engineering team who are working on test-case design for the security function.

The conversation:

Shakira: I've developed some tests for the **Detector** class [Figure 11.4]—you know, the one that allows access to all of the **Sensor** objects for the security function. You familiar with it?

Jamie (laughing): Sure, it's the one that allowed you to add the "doggie angst" sensor.

Shakira: The one and only. Anyway, it has an interface with four ops: *read(), enable(), disable(),* and *test().* Before a sensor can be read, it must be enabled. Once it's enabled, it can be read and tested. It can be disabled at any time, except if an alarm condition is being processed. So I defined a simple test sequence that will exercise its behavioral life history. (She shows Jamie the following sequence.)

#1: enable•test•read•disable

Jamie: That'll work, but you've got to do more testing than that!

Shakira: I know, I know. Here are some other sequences I've come up with. (She shows Jamie the following sequences.)

#2: enable•test•[read]^n•test•disable
#3: [read]^n
#4: enable•disable•[test | read]

Jamie: So let me see if I understand the intent of these. #1 goes through a normal life history, sort of a conventional usage. #2 repeats the read operation *n* times, and that's a likely scenario. #3 tries to read the sensor before it's been enabled . . . that should produce an error message of some kind, right? #4 enables and disables the sensor and then tries to read it. Isn't that the same as test #2?

Shakira: Actually no. In #4, the sensor has been enabled. What #4 really tests is whether the disable op works as it should. A *read()* or *test()* after *disable()* should generate the error message. If it doesn't, then we have an error in the disable op.

Jamie: Cool. Just remember that the four tests have to be applied for every sensor type since all the ops may be subtly different depending on the type of sensor.

Shakira: Not to worry. That's the plan.

19.6.2 Behavioral Testing

The use of the state diagram as a model that represents the dynamic behavior of a class is discussed in Chapter 8. The state diagram for a class can be used to help derive a sequence of tests that will exercise the dynamic behavior of the class (and those classes that collaborate with it). Figure 19.7 [Kir94] illustrates a state diagram for the **Account** class discussed earlier. Referring to the figure, initial transitions move through the *empty acct* and *setup acct* states. The majority of all behavior for instances of the class occurs while in the *working acct* state. A final withdrawal and account closure cause the account class to make transitions to the *nonworking acct* and *dead acct* states, respectively.

The tests to be designed should achieve coverage of every state. That is, the operation sequences should cause the **Account** class to transition through all allowable states:

Test case s_1: `open•setupAccnt•deposit (initial)•withdraw (final)•close`

Adding additional test sequences to the minimum sequence,

Test case s_2: `open•setupAccnt•deposit(initial)•deposit•balance•credit•withdraw (final)•close`

Test case s_3: `open•setupAccnt•deposit(initial)•deposit•withdraw•accntInfo•withdraw (final)•close`

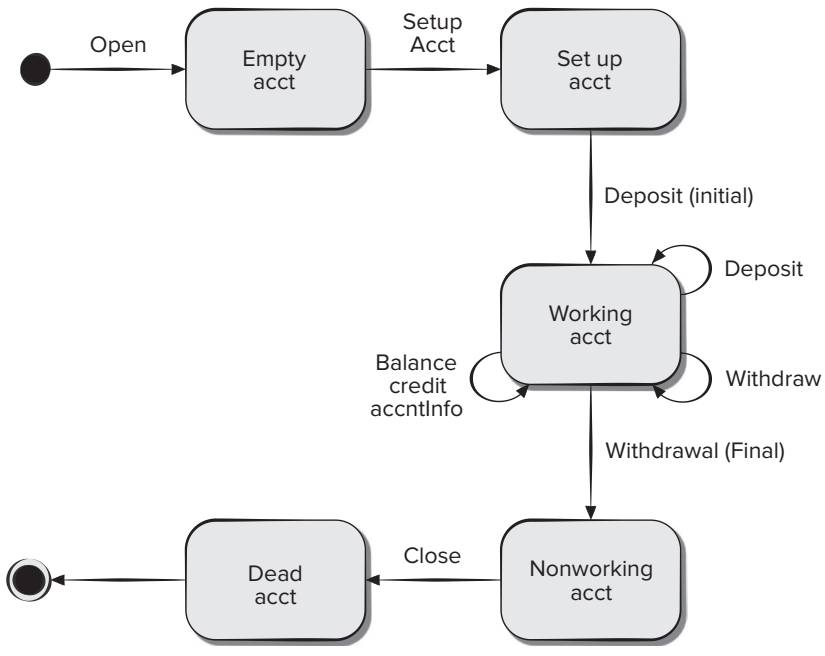


FIGURE 19.7

State diagram for the Account class
Source: Kirani, Shekhar and Tsai, W. T., "Specification and Verification of Object-Oriented Programs," Technical Report TR 94-64, University of Minnesota, December 1994, 79.

Still more test cases could be derived to ensure that all behaviors for the class have been adequately exercised. In situations in which the class behavior results in a collaboration with one or more classes, multiple state diagrams are used to track the behavioral flow of the system.

The state model can be traversed in a "breadth-first" [McG94] manner. In this context, breadth-first implies that a test case exercises a single transition and that when a new transition is to be tested, only previously tested transitions are used.

Consider a **CreditCard** object that is part of the banking system. The initial state of **CreditCard** is *undefined* (i.e., no credit card number has been provided). Upon reading the credit card during a sale, the object takes on a *defined* state; that is, the attributes `card number` and `expiration date`, along with bank-specific identifiers, are defined. The credit card is *submitted* when it is sent for authorization, and it is *approved* when authorization is received. The transition of **CreditCard** from one state to another can be tested by deriving test cases that cause the transition to occur. A breadth-first approach to this type of testing would not exercise *submitted* before it exercised *undefined* and *defined.* If it did, it would make use of transitions that had not been previously tested and would therefore violate the breadth-first criterion.

19.7 SUMMARY

Software testing accounts for the largest percentage of technical effort in the software process. Regardless of the type of software you build, a strategy for systematic test planning, execution, and control begins by considering small elements of the software and moves outward toward the program as a whole.

The objective of software testing is to uncover errors. For conventional software, this objective is achieved through a series of test steps. Unit and integration tests (discussed in Chapter 20) concentrate on functional verification of a component and incorporation of components into the software architecture. The strategy for testing object-oriented software begins with tests that exercise the operations within a class and then moves to thread-based testing for integration (discussed in Section 20.4.1). Threads are sets of classes that respond to an input or event.

Test cases should be traceable to software requirements. Each test step is accomplished through a series of systematic test techniques that assist in the design of test cases. With each testing step, the level of abstraction with which software is considered is broadened. The primary objective for test-case design is to derive a set of tests that have the highest likelihood for uncovering errors in software. To accomplish this objective, two different categories of test-case design techniques are used: white-box testing and black-box testing.

White-box tests focus on the program control structure. Test cases are derived to ensure that all statements in the program have been executed at least once during testing and that all logical conditions have been exercised. Basis path testing, a white-box technique, makes use of program graphs (or graph matrices) to derive the set of linearly independent tests that will ensure coverage. Condition and data flow testing further exercise program logic, and loop testing complements other white-box techniques by providing a procedure for exercising loops of varying degrees of complexity.

Black-box tests are designed to validate functional requirements without regard to the internal workings of a program. Black-box testing techniques focus on the information domain of the software, deriving test cases by partitioning the input and output domain of a program in a manner that provides thorough test coverage. Equivalence partitioning divides the input domain into classes of data that are likely to exercise a specific software function. Boundary value analysis probes the program's ability to handle data at the limits of acceptability.

Unlike testing (a systematic, planned activity), debugging can be viewed as an art. Beginning with a symptomatic indication of a problem, the debugging activity must track down the cause of an error. Testing can sometimes help find the root cause of the error. But often, the most valuable resource is the counsel of other members of the software engineering staff.

PROBLEMS AND POINTS TO PONDER

19.1. Using your own words, describe the difference between verification and validation. Do both make use of test-case design methods and testing strategies?

19.2. List some problems that might be associated with the creation of an independent test group. Are an ITG and an SQA group made up of the same people?

19.3. Why is a highly coupled module difficult to unit test?

19.4. Is unit testing possible or even desirable in all circumstances? Provide examples to justify your answer.

19.5. Can you think of any additional testing objectives that are not discussed in Section 19.1.1?

19.6. Select a software component that you have designed and implemented recently. Design a set of test cases that will ensure that all statements have been executed using basis path testing.

19.7. Myers [Mye79] uses the following program as a self-assessment for your ability to specify adequate testing: A program reads three integer values. The three values are interpreted as representing the lengths of the sides of a triangle. The program prints a message that states whether the triangle is scalene, isosceles, or equilateral. Develop a set of test cases that you feel will adequately test this program.

19.8. Design and implement the program (with error handling where appropriate) specified in Problem 19.7. Derive a flow graph for the program and apply basis path testing to develop test cases that will guarantee that all statements in the program have been tested. Execute the cases and show your results.

19.9. Give at least three examples in which black-box testing might give the impression that "everything's OK," while white-box tests might uncover an error. Give at least three examples in which white-box testing might give the impression that "everything's OK," while black-box tests might uncover an error.

19.10. In your own words, describe why the class is the smallest reasonable unit for testing within an OO system.

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

SOFTWARE TESTING—INTEGRATION LEVEL

20

A single developer may be able to test software components without involving other team members. This is not true for integration testing where a component must interact properly with components developed by other team members. Integration testing exposes many weaknesses of software development groups who have not gelled as a team. Integration testing presents an interesting dilemma for software engineers, who by their nature are constructive people. In fact, all testing requires that developers discard preconceived notions of the "correctness" of software just developed and instead work hard to design test cases to "break" the software. This means that team members need to be able to accept suggestions from other team members that their code is not behaving properly when it is tested as part of the latest software increment.

KEY CONCEPTS

artificial intelligence 403
black-box testing 397
bottom-up integration 399
continuous integration 400
cluster testing 404
fault-based testing 405
integration testing 398
multiple-class partition testing 405
patterns 409
regression testing 402
scenario-based testing 407
smoke testing 400
thread-based testing 404
top-down integration 398
validation testing 407
white-box testing 397



QUICK LOOK

What is it? Integration testing assembles components in a manner that allows the testing of increasingly larger software functions with the intent of finding errors as the software is assembled.

Who does it? During early stages of testing, a software engineer performs all tests. However, as the testing process progresses, testing specialists may become involved in addition to other stakeholders.

Why is it important? Test cases must be designed using disciplined techniques to ensure that the components have been integrated properly into the complete software product.

What are the steps? Internal program logic is exercised using "white-box" test-case design techniques and software requirements are exercised using "black-box" test-case design techniques.

What is the work product? A set of test cases designed to exercise internal logic, interfaces, component collaborations, and external requirements is designed and documented, expected results are defined, and actual results are recorded.

How do I ensure that I've done it right? When you begin testing, change your point of view. Try hard to "break" the software! Design test cases in a disciplined fashion, and review the test cases you do create for thoroughness.

Beizer [Bei90] describes a "software myth" that all testers face. He writes: "There's a myth that if we were really good at programming, there would be no bugs to catch . . . There are bugs, the myth says, because we are bad at what we do; and if we are bad at it, we should feel guilty about it."

Should testing instill guilt? Is testing really destructive? The answer to these questions is, No!

At the beginning of this book, we stressed the fact that software is only one element of a larger computer-based system. Ultimately, software is incorporated with other system elements (e.g., hardware, people, information), and *systems testing* (a series of system integration and validation tests) is conducted. These tests fall outside the scope of the software process and are not conducted solely by software engineers. However, steps taken during software design and testing can greatly improve the probability of successful software integration in the larger system.

In this chapter, we discuss techniques for software integration testing strategies applicable to most software applications. Specialized software testing strategies are discussed in Chapter 21.

20.1 SOFTWARE TESTING FUNDAMENTALS

The goal of testing is to find errors, and a good test is one that has a high probability of finding an error. Kaner, Falk, and Nguyen [Kan93] suggest the following attributes of a "good" test:

A good test has a high probability of finding an error. To achieve this goal, the tester must understand the software and attempt to develop a mental picture of how the software might fail.

A good test is not redundant. Testing time and resources are limited. There is no point in conducting a test that has the same purpose as another test. Every test should have a different purpose (even if it is subtly different).

A good test should be "best of breed" [Kan93]. In a group of tests that have a similar intent, time and resource limitations may dictate the execution of only those tests that have the highest likelihood of uncovering a whole class of errors.

A good test should be neither too simple nor too complex. Although it is sometimes possible to combine a series of tests into one test case, the possible side effects associated with this approach may mask errors. In general, each test should be executed separately.

Any engineered product (and most other things) can be tested in one of two ways: (1) Knowing the specified function that a product has been designed to perform, tests can be conducted that demonstrate each function is fully operational while at the same time searching for errors in each function. (2) Knowing the internal workings of a product, tests can be conducted to ensure that "all gears mesh," that is, internal operations are performed according to specifications and all internal components have been adequately exercised. The first test approach

takes an external view of testing and is called *black-box testing*. The second requires an internal view of testing and is termed *white-box testing*.[1] Both are useful in integration testing [Jan16].

20.1.1 Black-Box Testing

Black-box testing alludes to integration testing that is conducted by exercising the component interfaces with other components and with other systems. It examines some fundamental aspect of a system with little regard for the internal logical structure of the software. Instead, the focus is on ensuring the component executes correctly in the larger software build when the input data and software context specified by its preconditions is correct and behaves in the ways specified by its postconditions. It is of course important to make sure that the component behaves correctly when its preconditions are not satisfied (e.g., it can handle bad inputs without crashing).

Black-box testing is based on the requirements specified in user stories (Chapter 7). Test-case authors do not need to wait for the component implementation code to be written once the component interface is defined. Several cooperating components may need to be written to implement the functionality defined by a single user story. Validation testing (Section 20.5) often defines black-box test cases in terms of the end-user visible input actions and observable output behaviors, without any knowledge of how the components themselves were implemented.

20.1.2 White-Box Testing

White-box testing, sometimes called *glass-box testing* or *structural testing,* is an integration testing philosophy that uses implementation knowledge of the control structures described as part of component-level design to derive test cases. White-box testing of software is predicated on close examination of procedural implementation details and data structure implementation details. White-box tests can be designed only after component-level design (or source code) exists. The logical details of the program must be available. Logical paths through the software and collaborations between components are the focus of white-box integration testing.

At first glance it would seem that very thorough white-box testing would lead to "100 percent correct programs." All we need do is define all logical paths, develop test cases to exercise them, and evaluate results, that is, generate test cases to exercise program logic exhaustively. Unfortunately, exhaustive testing presents certain logistical problems. For even small programs, the number of possible logical paths can be very large. White-box testing should not, however, be dismissed as impractical. Testers should select a reasonable number of important logical paths to exercise once component integration occurs. Important data structures should also be tested for validity after component integration.

1 The terms *functional testing* and *structural testing* are sometimes used in place of black-box and white-box testing, respectively.

20.2 INTEGRATION TESTING

A neophyte in the software world might ask a seemingly legitimate question once all modules have been unit-tested: "If they all work individually, why do you doubt that they'll work when we put them together?" The problem, of course, is "putting them together"—interfacing. Data can be lost across an interface; one component can have an inadvertent, adverse effect on another; subfunctions, when combined, may not produce the desired major function; individually acceptable imprecision may be magnified to unacceptable levels; and global data structures can present problems. Sadly, the list goes on and on.

Integration testing is a systematic technique for constructing the software architecture while at the same time conducting tests to uncover errors associated with interfacing. The objective is to take unit-tested components and build a program structure that has been dictated by design.

There is often a tendency to attempt nonincremental integration, that is, to construct the program using a "big bang" approach. In the big bang approach, all components are combined in advance and the entire program is tested as a whole. Chaos usually results! Errors are encountered, but correction is difficult because isolation of causes is complicated by the vast expanse of the entire program. Taking the big bang approach to integration is a lazy strategy that is doomed to failure.

Incremental integration is the antithesis of the big bang approach. The program is constructed and tested in small increments, where errors are easier to isolate and correct; interfaces are more likely to be tested completely; and a systematic test approach may be applied. Integrate incrementally and testing as you go is a more cost-effective strategy. We discuss several common incremental integration testing strategies in the remainder of this chapter.

20.2.1 Top-Down Integration

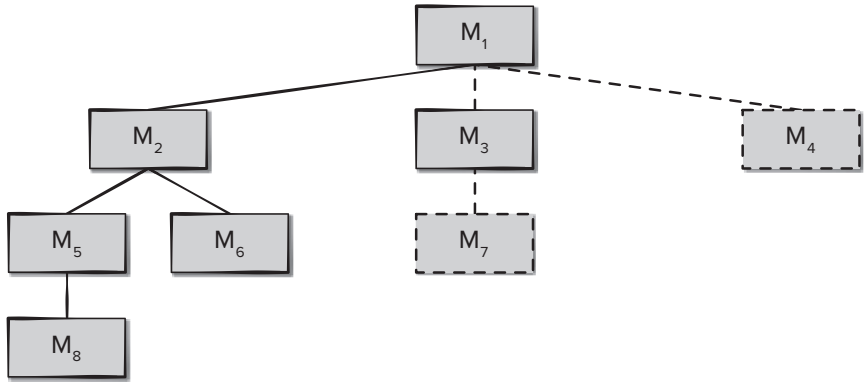
Top-down integration testing is an incremental approach to construction of the software architecture. Modules (also referred to as components in this book) are integrated by moving downward through the control hierarchy, beginning with the main control module (main program). Modules subordinate (and ultimately subordinate) to the main control module are incorporated into the structure in either a depth-first or breadth-first manner.

Referring to Figure 20.1, *depth-first integration* integrates all components on a major control path of the program structure. Selection of a major path is somewhat arbitrary and depends on application-specific characteristics (e.g., components needed to implement one use case). For example, selecting the left-hand path, components M_1, M_2, M_5 would be integrated first. Next, M_8 or (if necessary for proper functioning of M_2) M_6 would be integrated. Then, the central and right-hand control paths are built. *Breadth-first integration* incorporates all components directly subordinate at each level, moving across the structure horizontally. From the figure, components M_2, M_3, and M_4 would be integrated first. The next control level, M_5, M_6, and so on, follows. The integration process is performed in a series of five steps:

1. The main control module is used as a test driver, and stubs are substituted for all components directly subordinate to the main control module.

FIGURE 20.1

Top-down integration



2. Depending on the integration approach selected (i.e., depth or breadth first), subordinate stubs are replaced one at a time with actual components.
3. Tests are conducted as each component is integrated.
4. On completion of each set of tests, another stub is replaced with the real component.
5. Regression testing (discussed later in this section) may be conducted to ensure that new errors have not been introduced.

The process continues from step 2 until the entire program structure is built.

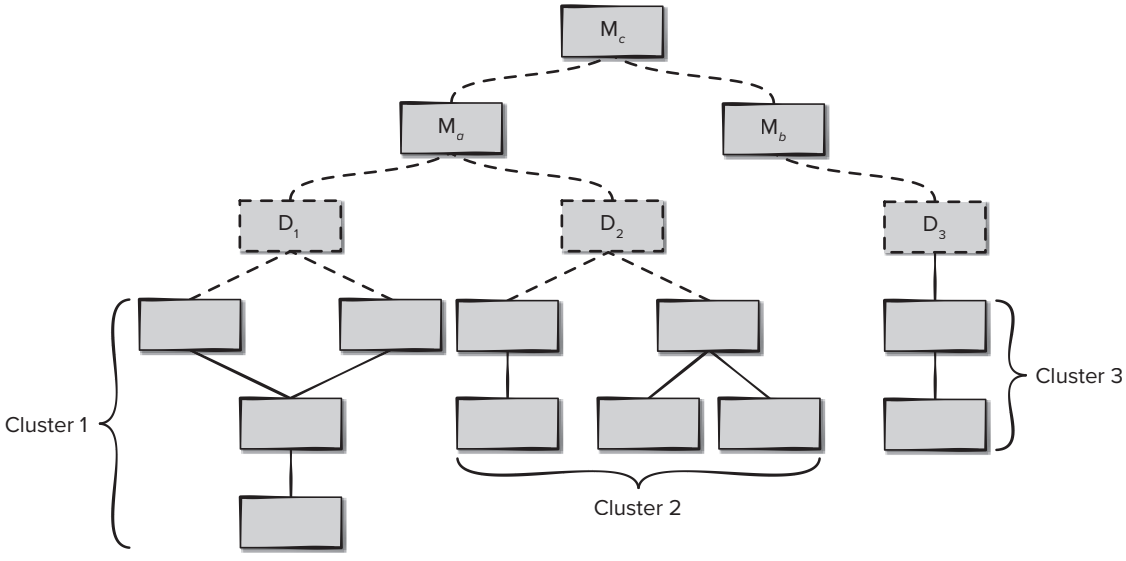
The top-down integration strategy verifies major control or decision points early in the test process. In a "well-factored" program structure, decision making occurs at upper levels in the hierarchy and is therefore encountered first. If major control problems do exist, early recognition is essential. If depth-first integration is selected, a complete function of the software may be implemented and demonstrated. Early demonstration of functional capability is a confidence builder for all stakeholders.

20.2.2 Bottom-Up Integration

Bottom-up integration testing, as its name implies, begins construction and testing with *atomic modules* (i.e., components at the lowest levels in the program structure). Bottom-up integration eliminates the need for complex stubs. Because components are integrated from the bottom up, the functionality provided by components subordinate to a given level is always available and the need for stubs is eliminated. A bottom-up integration strategy may be implemented with the following steps:

1. Low-level components are combined into clusters (sometimes called *builds*) that perform a specific software subfunction.
2. A *driver* (a control program for testing) is written to coordinate test-case input and output.
3. The cluster is tested.
4. Drivers are removed and clusters are combined, moving upward in the program structure.

FIGURE 20.2 Bottom-up integration



Integration follows the pattern illustrated in Figure 20.2. Components are combined to form clusters 1, 2, and 3. Each of the clusters is tested using a driver (shown as a dashed block). Components in clusters 1 and 2 are subordinate to M_a. Drivers D_1 and D_2 are removed, and the clusters are interfaced directly to M_a. Similarly, driver D_3 for cluster 3 is removed prior to integration with module M_b. Both M_a and M_b will ultimately be integrated with component M_c, and so forth.

As integration moves upward, the need for separate test drivers lessens. In fact, if the top two levels of program structure are integrated top down, the number of drivers can be reduced substantially and integration of clusters is greatly simplified.

20.2.3 Continuous Integration

Continuous integration is the practice of merging components into the evolving software increment once or more each day. This is a common practice for teams following agile development practices such as XP (Section 3.5.1) or DevOps (Section 3.5.2). Integration testing must take place quickly and efficiently if a team is attempting to always have a working program in place as part of continuous delivery. It is sometimes hard to maintain systems with the use of continuous integration tools [Ste18]. Maintenance and continuous integration issues are discussed in more detail in Section 22.4.

Smoke testing is an integration testing approach that can be used when product software is developed by an agile team using short increment build times. Smoke testing might be characterized as a rolling or continuous integration strategy. The software is rebuilt (with new components added) and smoke tested every day. It is designed as a pacing mechanism for time-critical projects, allowing the software team

to assess the project on a frequent basis. In essence, the smoke-testing approach encompasses the following activities:

1. Software components that have been translated into code are integrated into a *build.* A build includes all data files, libraries, reusable modules, and engineered components that are required to implement one or more product functions.
2. A series of tests is designed to expose errors that will keep the build from properly performing its function. The intent should be to uncover "show-stopper" errors that have the highest likelihood of throwing the software project behind schedule.
3. The build is integrated with other builds, and the entire product (in its current form) is smoke tested daily. The integration approach may be top down or bottom up.

The daily frequency of testing gives both managers and practitioners a realistic assessment of integration testing progress. McConnell [McC96] describes the smoke test in the following manner:

> The smoke test should exercise the entire system from end to end. It does not have to be exhaustive, but it should be capable of exposing major problems. The smoke test should be thorough enough that if the build passes, you can assume that it is stable enough to be tested more thoroughly.

Smoke testing provides a number of benefits when it is applied on complex, time-critical software projects:

- **Integration risk is minimized.** Because smoke tests are conducted daily, incompatibilities and other showstopper errors are uncovered early, thereby reducing the likelihood of serious schedule impact when errors are uncovered.
- **The quality of the end product is improved.** Because the approach is construction (integration) oriented, smoke testing is likely to uncover functional errors as well as architectural and component-level design errors. If these errors are corrected early, better product quality will result.
- **Error diagnosis and correction are simplified.** Like all integration testing approaches, errors uncovered during smoke testing are likely to be associated with "new software increments"—that is, the software that has just been added to the build(s) is a probable cause of a newly discovered error.
- **Progress is easier to assess.** With each passing day, more of the software has been integrated and more has been demonstrated to work. This improves team morale and gives managers a good indication that progress is being made.

In some ways smoke testing resembles regression testing (discussed in Section 20.3), which helps to ensure that the newly added components do not interfere with the behaviors of existing components that were previously tested. To do this, it is a good idea to rerun a subset of the test cases that executed with the existing software component before the new components were added. The effort required to rerun test cases is not trivial, and automated testing can be used to reduce the time and effort re-created to rerun these test cases [Net18]. A complete discussion of automated testing is

beyond the scope of this chapter, but links to representative tools can be found on the Web pages that supplement this book.[2]

20.2.4 Integration Test Work Products

An overall plan for integration of the software and a description of specific tests is documented in a *test specification.* This work product incorporates a test plan and a test procedure and becomes part of the software configuration. Testing is divided into phases and incremental builds that address specific functional and behavioral characteristics of the software. For example, integration testing for the *SafeHome* security system might be divided into the following test phases: user interaction, sensor processing, communications functions, and alarm processing.

Each integration test phase delineates a broad functional category within the software and generally can be related to a specific domain within the software architecture. Therefore, software increments are created to correspond to each phase.

A schedule for integration, the development of scaffolding software (Section 19.2.1), and related topics are also discussed as part of the test plan. Start and end dates for each phase are established, and "availability windows" for unit-tested modules are defined. When developing a project schedule, you'll have to consider the manner in which integration occurs so that components will be available when needed. A brief description of scaffolding software (stubs and drivers) concentrates on characteristics that might require special effort. Finally, test environment and resources are described. Unusual hardware configurations, exotic simulators, and special test tools or techniques are a few of many topics that may also be discussed.

The detailed testing procedure that is required to accomplish the test plan is described next. The order of integration and corresponding tests at each integration step are described. A listing of all test cases (annotated for subsequent reference) and expected results are also included. In the agile world, this level of test-case description occurs when code to implement the user story is being developed so the code can be tested as soon as it is ready for integration.

A history of actual test results, problems, or peculiarities is recorded in a *test report* that can be appended to the *test specification.* It is often best to implement the test report as a shared Web document to allow all stakeholders access to the latest test results and the current state of the software increment. Information contained in this online document can be vital to developers during software maintenance (Section 4.9).

20.3 Artificial Intelligence and Regression Testing

Each time a new module is added as part of integration testing, the software changes. New data flow paths are established, new input/output (I/O) may occur, and new control logic is invoked. Side effects associated with these changes may cause problems with functions that previously worked flawlessly. In the context of an integration

2 See the SEPA 9e website.

test strategy, *regression testing* is the reexecution of some subset of tests that have already been conducted to ensure that changes have not propagated unintended side effects. Regression tests should be executed every time a major change is made to the software (including the integration of new components). Regression testing helps to ensure that changes (due to testing or for other reasons) do not introduce unintended behavior or additional errors.

Regression testing may be conducted manually, by reexecuting a subset of all test cases or using automated capture/playback tools. *Capture/playback tools* enable the software engineer to capture test cases and results for subsequent playback and comparison. The *regression test suite* (the subset of tests to be executed) contains three different classes of test cases:

- A representative sample of tests that will exercise all software functions
- Additional tests that focus on software functions that are likely to be affected by the change
- Tests that focus on the software components that have been changed

As integration testing proceeds, the number of regression tests can grow quite large. Therefore, the regression test suite should be designed to include only those tests that address one or more classes of errors in each of the major program functions.

Yoo and Harman [Yoo13] write about potential uses of artificial intelligence (AI) in identifying test cases for use in regression test suites. A software tool could examine the dependencies among the components in the software increment after the new components have been added and generate test cases automatically to use for regression testing. Another possibility would be using machine learning techniques to select sets of test cases that will optimize the discovery of component collaboration errors. This work is promising, but still requires significant human interaction to review the test cases and the recommended order for executing them.

SAFEHOME



Regression Testing

The scene: Doug Miller's office, as integration testing is under way.

The players: Doug Miller, software engineering manager; Vinod, Jamie, Ed, and Shakira, members of the *SafeHome* software engineering team.

The conversation:

Doug: It seems to me that we are not spending enough time retesting software components after new components are integrated.

Vinod: I guess that is true, but isn't it good enough that we are testing the new components' interactions with the components they are supposed to collaborate with?

Doug: Not always. Sometimes components make unintended change to data used by other components. I know we are busy, but it is important to discover these problems early.

Shakira: We do have a test-case repository we have been drawing from. Perhaps we can randomly select several test cases to run using our automated testing framework.

Doug: That's a start. But maybe we should be more strategic in how we select our test cases.

Ed: I suppose we could use our test-case/requirement traceability table and check our CRC card model.

Vinod: I have been using continuous integration, meaning I integrate each component as soon as one of the developers passes it to me. I try to run a series of regression tests on the partially integrated program.

Jamie: I've been trying to design a set of appropriate tests for each function in the system. Maybe I should tag some of the more important ones for Vinod to use for regression testing.

Doug (to Vinod): How often will you run the regression test cases?

Vinod: Every day I integrate a new component I will use the regression test cases . . . until we decide the software increment is done.

Doug: Let's try using Jamie's regression test cases as they are created and see how things go.

20.4 Integration Testing in the OO Context

Object-oriented software does not have an obvious hierarchical control structure, so traditional top-down and bottom-up integration strategies (Section 20.2) have little meaning. In addition, integrating operations one at a time into a class (the conventional incremental integration approach) is often impossible because of the "direct and indirect interactions of the components that make up the class" [Ber93].

There are two different strategies for integration testing of OO systems: thread-based testing and use-based testing [Bin99]. The first, *thread-based testing*, integrates the set of classes required to respond to one input or event for the system. Each thread is integrated and tested individually. Regression testing is applied to ensure that no side effects occur. An important strategy for integration testing of OO software is thread-based testing. Threads are sets of classes that respond to an input or event.

The second integration approach, *use-based testing,* begins the construction of the system by testing those classes (called *independent classes*) that use very few (if any) *server* classes. After the independent classes are tested, the next layer of classes, called *dependent classes,* that use the independent classes are tested. This sequence of testing layers of dependent classes continues until the entire system is constructed. Use-based tests focus on classes that do not collaborate heavily with other classes.

The use of scaffolding software also changes when integration testing of OO systems is conducted. Drivers can be used to test operations at the lowest level and for the testing of whole groups of classes. A driver can also be used to replace the user interface so that tests of system functionality can be conducted prior to implementation of the interface. Stubs can be used in situations in which collaboration between classes is required but one or more of the collaborating classes has not yet been fully implemented.

Cluster testing is one step in the integration testing of OO software. Here, a cluster of collaborating classes (determined by examining the CRC and object-relationship model) is exercised by designing test cases that attempt to uncover errors in the collaborations.

20.4.1 Fault-Based Test-Case Design[3]

The object of *fault-based testing* within an OO system is to design tests that have a high likelihood of uncovering plausible faults. Because the product or system must conform to customer requirements, preliminary planning required to perform fault-based testing begins with the analysis model. The strategy for fault-based testing is to hypothesize a set of plausible faults and then derive tests to prove each hypothesis. The tester looks for plausible faults (i.e., aspects of the implementation of the system that may result in defects). To determine whether these faults exist, test cases are designed to exercise the design or code.

Of course, the effectiveness of these techniques depends on how testers perceive a plausible fault. If real faults in an OO system are perceived to be implausible, then this approach is really no better than any random testing technique. However, if the analysis and design models can provide insight into what is likely to go wrong, then fault-based testing can find significant numbers of errors with relatively low expenditures of effort.

Integration testing looks for plausible faults in operation calls or message connections. Three types of faults are encountered in this context: unexpected result, wrong operation/message used, and incorrect invocation. To determine plausible faults as functions (operations) are invoked, the behavior of the operation must be examined.

Integration testing applies to attributes as well as to operations. The "behaviors" of an object are defined by the values that its attributes are assigned. Testing should exercise the attributes to determine whether proper values occur for distinct types of object behavior.

It is important to note that integration testing attempts to find errors in the client object, not the server. Stated in conventional terms, the focus of integration testing is to determine whether errors exist in the calling code, not the called code. The operation call is used as a clue, a way to find test requirements that exercise the calling code.

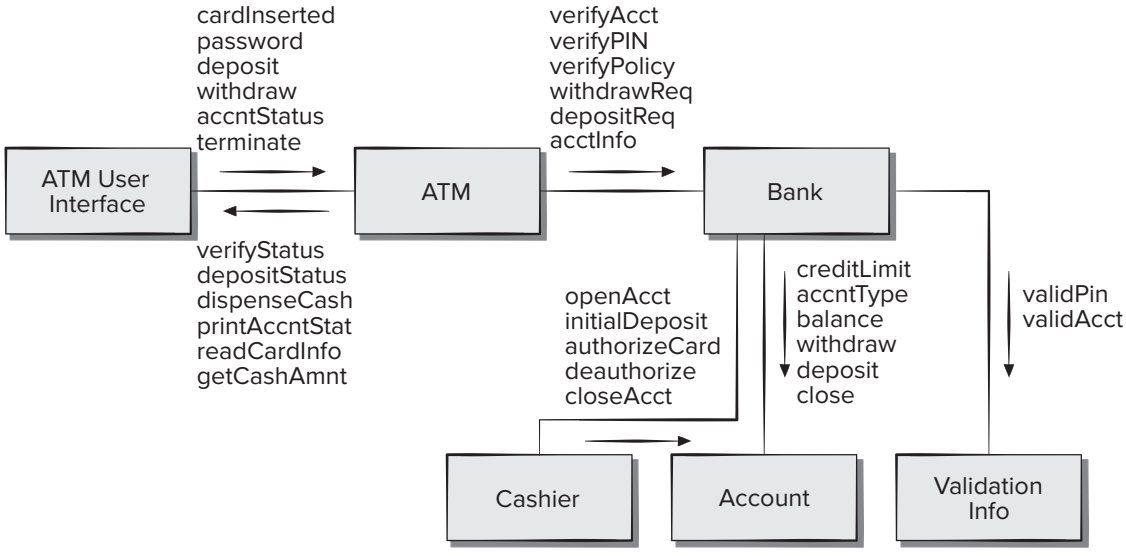
The approach for multiple-class partition testing is similar to the approach used for partition testing of individual classes. A single class is partitioned as discussed in Section 19.6.1. However, the test sequence is expanded to include those operations that are invoked via messages to collaborating classes. An alternative approach partitions tests based on the interfaces to a particular class. Referring to Figure 20.3, the **Bank** class receives messages from the **ATM** and **Cashier** classes. The methods within **Bank** can therefore be tested by partitioning them into those that serve **ATM** and those that serve **Cashier.**

Kirani and Tsai [Kir94] suggest the following sequence of steps to generate multiple-class random test cases:

1. For each client class, use the list of class operations to generate a series of random test sequences. The operations will send messages to other server classes.
2. For each message that is generated, determine the collaborator class and the corresponding operation in the server object.

3 Sections 20.4.1 and 20.4.2 have been adapted from an article by Brian Marick originally posted on the Internet newsgroup comp.testing. This adaptation is included with the permission of the author. For further information on these topics, see [Mar94]. It should be noted that the techniques discussed in Sections 20.4.1 and 20.4.2 are also applicable for conventional software.

FIGURE 20.3 Class collaboration diagram for banking application



Source: Kirani, Shekhar and Tsai, W. T., "Specification and Verification of Object-Oriented Programs," Technical Report TR 94-64, University of Minnesota, December 4, 1994, 72.

3. For each operation in the server object (that has been invoked by messages sent from the client object), determine the messages that it transmits.
4. For each of the messages, determine the next level of operations that are invoked and incorporate these into the test sequence.

To illustrate [Kir94], consider a sequence of operations for the **Bank** class relative to an **ATM** class (Figure 20.3):

```
verifyAcct•verifyPIN•[[verifyPolicy•withdrawReq]|depositReq|acctInfoREQ]^n
```

A random test case for the **Bank** class might be

Test case r_3 = `verifyAcct•verifyPIN•depositReq`

To consider the collaborators involved in this test, the messages associated with each of the operations noted in test case r_3 are considered. **Bank** must collaborate with **ValidationInfo** to execute the *verifyAcct()* and *verifyPIN()*. **Bank** must collaborate with **Account** to execute *depositReq()*. Hence, a new test case that exercises these collaborations is

Test case r_4 = `verifyAcct [Bank:validAcctValidationInfo]•verifyPIN`
`[Bank: validPinValidationInfo]•depositReq [Bank: depositaccount]`

20.4.2 Scenario-Based Test-Case Design

Fault-based testing misses two main types of errors: (1) incorrect specifications and (2) interactions among subsystems. When errors associated with an incorrect specification occur, the product doesn't do what the customer wants. It might do the wrong

thing or omit important functionality. But in either circumstance, quality (conformance to requirements) suffers. Errors associated with subsystem interaction occur when the behavior of one subsystem creates circumstances (e.g., events, data flow) that cause another subsystem to fail.

Scenario-based testing will uncover errors that occur when any actor interacts with the software. Scenario-based testing concentrates on what the user does, not what the product does. This means capturing the tasks (via use cases) that the user has to perform and then applying them and their variants as tests. This is very similar to thread testing.

Scenario testing uncovers interaction errors. But to accomplish this, test cases must be more complex and more realistic than fault-based tests. Scenario-based testing tends to exercise multiple subsystems in a single test (users do not limit themselves to the use of one subsystem at a time).

Test-case design becomes more complicated as integration of the object-oriented system begins. It is at this stage that testing of collaborations between classes must begin. To illustrate "interclass test-case generation" [Kir94], we expand the banking example introduced in Section 19.6 to include the classes and collaborations noted in Figure 20.3. The direction of the arrows in the figure indicates the direction of messages, and the labeling indicates the operations that are invoked as a consequence of the collaborations implied by the messages.

Like the testing of individual classes, class collaboration testing can be accomplished by applying random and partitioning methods, as well as scenario-based testing and behavioral testing.

20.5 VALIDATION TESTING

Like all testing steps, validation tries to uncover errors, but the focus is at the requirements level—on things that will be immediately apparent to the end user. Validation testing begins at the culmination of integration testing, when individual components have been exercised, the software is completely assembled as a package, and interfacing errors have been uncovered and corrected. At the validation or system level, the distinction between different software categories disappears. Testing focuses on user-visible actions and user-recognizable output from the system.

Validation can be defined in many ways, but a simple (albeit harsh) definition is that validation succeeds when software functions in a manner that can be reasonably expected by the customer. At this point, a battle-hardened software developer might protest: "Who or what is the arbiter of reasonable expectations?" If a *software requirements specification* has been developed, it describes each user story, all user-visible attributes, and the customer's acceptance criteria for each. The customer's acceptance criteria form the basis for a validation-testing approach.

Software validation is achieved through a series of tests that demonstrate conformity with requirements. A test plan outlines the classes of tests to be conducted, and a test procedure defines specific test cases that are designed to ensure that all functional requirements are satisfied, all behavioral characteristics are achieved, all content is accurate and properly presented, all performance requirements are attained, documentation is correct, and usability and other requirements are met (e.g., transportability,

compatibility, error recovery, maintainability). If a deviation from specification is uncovered, a *deficiency list* is created. A method for resolving deficiencies (acceptable to stakeholders) must be established. Specialized testing methods for these nonfunctional requirements are discussed in Chapter 21.

An important element of the validation process is a *configuration review.* The intent of the review is to ensure that all elements of the software configuration have been properly developed, are cataloged, and have the necessary detail to bolster the support activities. The configuration review, sometimes called an audit, is discussed in more detail in Chapter 22.

SAFEHOME



Preparing for Validation

The scene: Doug Miller's office, as component-level design continues and construction of certain components continues.

The players: Doug Miller, software engineering manager; Vinod, Jamie, Ed, and Shakira, members of the *SafeHome* software engineering team.

The conversation:

Doug: The first increment will be ready for validation in what . . . about three weeks?

Vinod: That's about right. Integration is going well. We're smoke testing daily, finding some bugs, but nothing we can't handle. So far, so good.

Doug: Talk to me about validation.

Shakira: Well, we'll use all of the use cases as the basis for our test design. I haven't started yet, but I'll be developing tests for all of the use cases that I've been responsible for.

Ed: Same here.

Jamie: Me too, but we've got to get our act together for acceptance testing and also for alpha and beta testing, no?

Doug: Yes. In fact I've been thinking; we could bring in an outside contractor to help us with validation. I have the money in the budget . . . and it'd give us a new point of view.

Vinod: I think we've got it under control.

Doug: I'm sure you do, but an ITG gives us an independent look at the software.

Jamie: We're tight on time here, Doug. I for one don't have the time to babysit anybody you bring in to do the job.

Doug: I know, I know. But if an ITG works from requirements and use cases, not too much babysitting will be required.

Vinod: I still think we've got it under control.

Doug: I hear you, Vinod, but I am going to overrule on this one. Let's plan to meet with the ITG rep later this week. Get 'em started and see what they come up with.

Vinod: Okay, maybe it'll lighten the load a bit.

At the validation or system level, the details of class connections disappear. The validation of OO software focuses on user-visible actions and user-recognizable outputs from the system. To assist in the derivation of validation tests, the tester should draw upon use cases (Chapters 7 and 8) that are part of the requirements model. The use case provides a scenario that has a high likelihood of uncovered errors in user-interaction requirements. Conventional black-box testing methods (Chapter 19) can be used to drive validation tests. In addition, you may choose to derive test cases from the object-behavior model created as part of object-oriented analysis (OOA).

20.6 TESTING PATTERNS

The use of patterns as a mechanism for describing solutions to specific design problems was discussed in Chapter 15. But patterns can also be used to propose solutions to other software engineering situations—in this case, software testing. *Testing patterns* describe common testing problems and solutions that can assist you in dealing with them.

Much of software testing, even during the past decade, has been an ad hoc activity. If testing patterns can help a software team to communicate about testing more effectively, to understand the motivating forces that lead to a specific approach to testing, and to approach the design of tests as an evolutionary activity in which each iteration results in a more complete suite of test cases, then patterns have accomplished much.

Testing patterns are described in much the same way as design patterns (Chapter 15). Dozens of testing patterns have been proposed in the literature (e.g., [Mar02]). The following three testing patterns (presented in abstract form only) provide representative examples:

Pattern name: **PairTesting**

Abstract: A process-oriented pattern, **pair testing** describes a technique that is analogous to pair programming [Chapter 3] in which two testers work together to design and execute a series of tests that can be applied to unit, integration or validation testing activities.

Pattern name: **SeparateTestInterface**

Abstract: There is a need to test every class in an object-oriented system, including "internal classes" (i.e., classes that do not expose any interface outside of the component that used them). The **SeparateTestInterface** pattern describes how to create "a test interface that can be used to describe specific tests on classes that are visible only internally to a component" [Lan01].

Pattern name: **ScenarioTesting**

Abstract: Once unit and integration tests have been conducted, there is a need to determine whether the software will perform in a manner that satisfies users. The **ScenarioTesting** pattern describes a technique for exercising the software from the user's point of view. A failure at this level indicates that the software has failed to meet a user visible requirement [Kan01].

A comprehensive discussion of testing patterns is beyond the scope of this book. If you have further interest, see [Bin99], [Mar02], [Tho04], [Mac10], and [Gon17] for additional information on this important topic.

20.7 SUMMARY

Integration testing builds the software architecture while at the same time conducting tests to uncover errors associated with interfacing between software components. The objective is to take unit-tested components and build a program structure that has been dictated by design.

Experienced software developers often say, "Testing never ends; it just gets transferred from you [the software engineer] to your customer. Every time your customer uses the program, a test is being conducted." By applying test-case design, you can

achieve more complete testing and thereby uncover and correct the highest number of errors before the "customer's tests" begin.

Hetzel [Het84] describes white-box testing as "testing in the small." His implication is that the white-box tests that have been considered in this chapter are typically applied to small program components (e.g., modules or small groups of modules). Black-box testing, on the other hand, broadens your focus and might be called "testing in the large."

Black-box integration testing is based on the requirements specified in the user stories or some other analysis modeling representation. Test-case authors do not need to wait for the component implementation code to be written, as long as they understand the required functionality of the components undergoing testing. Validation testing is often accomplished with black-box test cases that produce end-user visible input actions and observable output behaviors.

White-box testing requires a close examination of procedural implementation details and data structure implementation details for the components undergoing testing. White-box tests can be designed only after component-level design (or source code) exists. Logical paths through the software and collaborations between components are the focus of white-box integration testing.

Integration testing of OO software can be accomplished using a thread-based or use-based strategy. Thread-based testing integrates the set of classes that collaborate to respond to one input or event. Use-based testing constructs the system in layers, beginning with those classes that do not make use of server classes. Integration test-case design methods can also make use of random and partition tests. In addition, scenario-based testing and tests derived from behavioral models can be used to test a class and its collaborators. A test sequence tracks the flow of operations across class collaborations.

OO system validation testing is black-box oriented and can be accomplished by applying the same black-box methods discussed for conventional software. However, scenario-based testing dominates the validation of OO systems, making the use case a primary driver for validation testing.

Regression testing is the process of reexecuting a selected test case following any change made to a software system. Regression tests should be executed whenever new components or changes are added to a software increment. Regression testing helps to ensure that changes do not introduce unintended behavior or additional errors.

Testing patterns describe common testing problems and solutions that can assist you in dealing with them. If testing patterns can help a software team to communicate about testing more effectively, to understand the motivating forces that lead to a specific approach to testing, and to approach the design of test cases as an evolutionary activity in which each iteration results in a more complete suite of test cases, then patterns have accomplished much.

PROBLEMS AND POINTS TO PONDER

20.1. How can project scheduling affect integration testing?

20.2. Who should perform validation testing—the software developer or the software user? Justify your answer.

20.3. Will exhaustive testing (even if it is possible for very small programs) guarantee that a program is 100 percent correct?

20.4. Why should "testing" begin with requirements analysis and design?

20.5. Should nonfunctional requirements (e.g., security or performance) be tested as part of integration testing?

20.6. Why do we have to retest subclasses that are instantiated from an existing class, if the existing class has already been thoroughly tested?

20.7. What is the difference between thread-based and use-based strategies for integration testing?

20.8. Develop a complete test strategy for the *SafeHome* system discussed earlier in this book. Document it in a *test specification.*

20.9. Pick one of the *SafeHome* system user stories to use as the basis of scenarios-based testing, and construct a set of integration test cases needed to do integration testing for that user story.

20.10. For the test cases you wrote in Problem 20.9, identify a subset of test cases you will use for regression testing software components that are added to the program.

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

21 SOFTWARE TESTING—SPECIALIZED TESTING FOR MOBILITY

The same sense of urgency that drives MobileApp projects also pervades all mobility projects. Stakeholders are worried that they will miss a market window and press to introduce the MobileApp to its intended market. Technical activities that often occur late in the process, such as performance and security testing, are sometimes given short shrift. Usability testing that should occur during the design phase may end up being deferred until just before delivery. These can be catastrophic mistakes. To avoid this situation, you and other team members must ensure that each work product exhibits high quality, or users will move to a competing product [Soa11].

KEY CONCEPTS

accessibility testing 432
alpha test 430
beta test 430
compatibility testing 414
content testing 420
documentation testing 434
internationalization 423
load testing 425
model-based testing 429
navigation testing 421
performance testing 424
real-time testing 426
recovery testing 417
security testing 423
stress testing 425
testing AI systems 428
testing guidelines 413
test strategies for MobileApps 413
test strategies for WebApps 418
usability testing 430

QUICK LOOK

What is it? Mobility testing is a collection of related activities with a single goal: to uncover errors in MobileApp content, function, usability, navigability, performance, capacity, and security.

Who does it? Software engineers and other project stakeholders (managers, customers, end users) all participate in mobility testing.

Why is it important? If end users encounter errors or difficulties within the MobileApp, they will go elsewhere for the personalized content and function they need.

What are the steps? The mobility testing process begins by focusing on user-visible aspects of the MobileApp and proceeds to tests that exercise technology and infrastructure.

What is the work product? A MobileApp test plan is often produced. A suite of test cases is developed for each testing step, and an archive of test results is maintained for future use.

How do I ensure that I've done it right? Although you can never be sure that you've performed every test that is needed, you can be certain that testing has uncovered errors (and that those errors have been corrected). In addition, if you've established a test plan, you can check to ensure that all planned tests have been conducted.

MobileApp requirements and design models cannot be tested solely with executable test cases. You and your team should conduct technical reviews (Chapter 16) that examine usability (Chapter 12) as well as MobileApp performance and security.

There are several important questions to ask when creating a mobility testing strategy [Sch09]:

- Do you have to build a fully functional prototype before you test with users?
- Should you test with the user's device or provide a device for testing?
- What devices and user groups should you include in testing?
- What are the trade-offs associated with lab testing versus remote testing?

We address each of these questions throughout this chapter.

21.1 Mobile Testing Guidelines

MobileApps that run entirely on a mobile device can be tested using traditional software testing methods (Chapters 19 and 20). Alternatively, they can be tested using emulators running on personal computers. Things become more complicated when thin-client MobileApps[1] are to be tested. They exhibit many of the same testing challenges found in WebApps (Section 20.2), but thin-client MobileApps have the additional concerns associated with transmission of data through Internet gateways and telephone networks [Was10].

In general, users expect MobileApps to be context aware and deliver personalized user experiences based on the physical location of a device in relation to available network features. Testing MobileApps in dynamic ad hoc networks for every possible device and network configuration is difficult, if not impossible.

MobileApps are expected to deliver much of the complex functionality and reliability found in desktop applications, but they are resident on mobile platforms with relatively limited resources. The following guidelines provide a basis for mobile application testing [Kea07]:

- Understand the network and device landscape before testing to identify bottlenecks (Section 21.6).
- Conduct tests in uncontrolled real-world test conditions (field-based testing, Section 21.8).
- Select the right automation test tool (Section 21.11).
- Use the Weighted Device Platform Matrix method to identify the most critical hardware/platform combination to test (Section 21.8).
- Check the end-to-end functional flow in all possible platforms at least once (Section 21.10).

1 Thin-client apps typically have software for the user interface running on the mobile device (or Web browser software) and use a network interface to an Internet-based application or cloud-based data storage.

- Conduct performance testing, GUI testing, and compatibility testing using actual devices (Sections 21.8 and 21.11).
- Measure performance only in realistic conditions of wireless traffic and user load (Section 21.8).

21.2 The Testing Strategies

The strategy for testing mobile applications adopts the basic principles for all software testing. However, the unique nature of MobileApps demands the consideration of a number of specialized issues:

- **User-experience testing.** Users are involved early in the development process to ensure that the MobileApp lives up to the usability and accessibility expectations of the stakeholders on all supported devices (Section 21.3).
- **Device compatibility testing.** Testers verify that the MobileApp works correctly on all required hardware and software combinations (Section 21.9).
- **Performance testing.** Testers check nonfunctional requirements unique to mobile devices (e.g., download times, processor speed, storage capacity, power availability) (Section 21.8).
- **Connectivity testing.** Testers ensure that the MobileApp can access any needed networks or Web services and can tolerate weak or interrupted network access (Section 21.6).
- **Security testing.** Testers ensure that the MobileApp does not compromise the privacy or security requirements of its users (Section 21.7).
- **Testing in the wild.** The app is tested under realistic conditions on actual user devices in a variety of networking environments around the globe (Section 21.9).
- **Certification testing.** Testers ensure that the MobileApp meets the standards established by the app stores that will distribute it.

Technology alone is not sufficient to guarantee commercial success of a MobileApp. Users abandon MobileApps quickly if they do not work well or fail to meet expectations. It is important to recall that testing has two important goals: (1) to create test cases that uncover defects early in the development cycle and (2) to verify the presence of important quality attributes. The quality attributes for MobileApps are based on those set forth in ISO 2050:2011 [ISO17] and encompass functionality, reliability, usability, efficiency, maintainability, and portability (Chapter 17).

Developing a MobileApp testing strategy requires an understanding of both software testing and the challenges that make mobile devices and their network infrastructure unique [Kho12a]. In addition to a thorough knowledge of conventional software testing approaches (Chapters 19 and 20), a MobileApp tester should have a good understanding of telecommunications principles and an awareness of the differences and capabilities of mobile operating systems platforms. This basic knowledge must be complemented with a thorough understanding of the different types of mobile testing (e.g., MobileApp testing, mobile handset testing, mobile website testing), the use

of simulators, test automation tools, and remote data access services (RDA). Each of these topics is discussed later in this chapter.

21.3 User Experience Testing Issues

In a crowded marketplace in which products provide the same functionality, users will choose the MobileApp that is easiest to use. The user interface and its interaction mechanisms are visible to the MobileApp users. It is important to test the quality of the user experience provided by the MobileApp to ensure that it meets the expectations of its users.

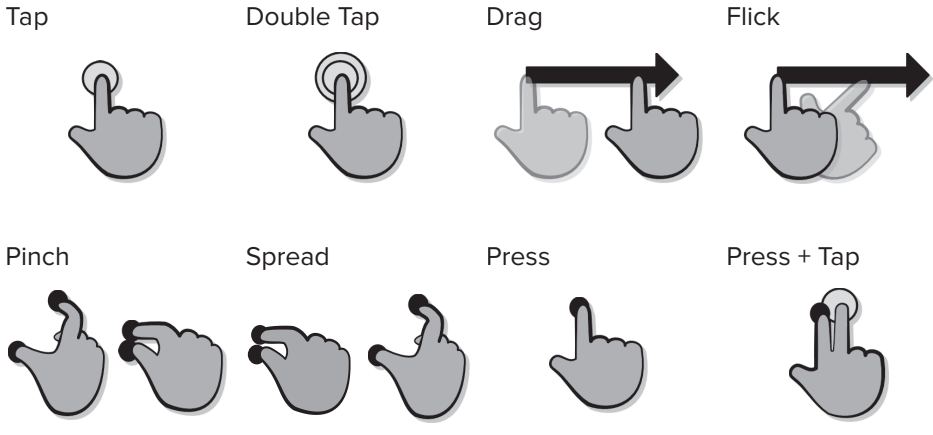
What characteristics of MobileApp usability become the focus of testing, and what specific objectives are addressed? Many of the procedures for assessing the usability of software user interfaces discussed in Chapters 12 and 13 can be used to assess MobileApps. Similarly, many of the strategies used to assess the quality of WebApps (Section 21.5) may be used to test the user interface portion of the MobileApp. There is more to building a good MobileApp user interface than simply shrinking the size of a user interface from an existing desktop application.

21.3.1 Gesture Testing

Touch screens are ubiquitous on mobile devices, and, as a consequence, developers have added multitouch gestures (e.g., swiping, zooming, scrolling, selection) as a means of augmenting the user interaction possibilities without losing screen real estate. Figure 21.1 shows several commonly found gestures on MobileApps. Unfortunately, gesture-intensive interfaces present a number of review and testing challenges.

Paper prototypes, sometimes developed as part of the design, cannot be used to adequately review the adequacy or efficacy of gestures. When testing is initiated, it's difficult to use automated tools to test touch or gesture interface actions. The location

FIGURE 21.1
Mobile app gestures



of screen objects is affected by screen size and resolution, as well as previous user actions, making accurate gesture testing difficult. And even as testing is conducted, gestures are hard to log accurately for replay.

Instead, testers need to create test framework programs that make calls to functions that simulate gesture events. All of this is expensive and time consuming.

Accessibility testing for visually impaired users is challenging because gesture interfaces typically do not provide either tactile or auditory feedback. Usability and accessibility testing for gestures become very important for ubiquitous devices like smartphones. It may be important to test the operation of the device when gesture operations are not available.

Ideally, user stories or use cases are written in sufficient detail to allow their use as the basis for test scripts. It is important to recruit representative users and include all targeted devices to take screen differences into account when testing gestures with a MobileApp. Finally, testers should ensure that the gestures conform to the standards and contexts set for the mobile device or platform.

21.3.2 Virtual Keyboard Input

Because a virtual keyboard may obscure part of the display screen when activated, it is important to test the MobileApp to ensure that important screen information is not hidden from the user while typing. If the screen information must be hidden, it is important to test the ability of the MobileApp to allow page flipping by the user without losing typed information [Sch09].

Virtual keyboards are typically smaller than personal computer keyboards, and therefore, it is difficult to type with 10 fingers. Because the keys themselves are smaller and harder to hit and provide no tactile feedback, the MobileApp must be tested to ensure that it allows easy error correction and can manage mistyped words without crashing.

Predictive technologies (i.e., autocompletion of partially typed words) are often used with virtual keyboards to help expedite user input. It is important to test the correctness of the word completions for the natural language chosen by the user, if the MobileApp is designed for a global market. It is also important to test the usability of any mechanism that allows the user to override a suggested completion.

Virtual keyboard testing is often conducted in the usability laboratory, but some should be conducted in the wild. If virtual keyboard tests uncover significant problems, the only alternative may be to ensure that the MobileApp can accept input from devices other than a virtual keyboard (e.g., a physical keyboard or voice input).

21.3.3 Voice Input and Recognition

Voice input has become an increasingly common method for providing input and commands in hands-busy, eyes-busy situations. Voice input may take several forms with different levels of programming complexity required to process each. Voicemail input occurs when a message is simply recorded for playback later. Discrete word recognition can be used to allow users to verbally select items from a menu with a small number of choices. Continuous speech recognition translates dictated speech into meaningful text strings. Each type of voice input has its own testing challenges.

According to Shneiderman [Shn09], all forms of voice input and processing are hindered by interference from noisy environments. Using voice commands to control a device impresses a greater cognitive load on the user, as compared to pointing to a screen object or pressing a key. The user must think of the correct word or words to get the MobileApp to perform the desired action. However, the breadth and accuracy of speech recognition systems are evolving rapidly, and it is likely the voice recognition will become the dominant form of communication in many MobileApps.

Testing the quality and reliability of voice input and recognition should take environmental conditions and individual voice variation into account. Errors will be made by users of the MobileApp and by the portions of the system processing the input. The MobileApp should be tested to ensure that bad input does not crash the MobileApp or the device. Large numbers of users and environments should be involved to be sure the MobileApp is working with an acceptable error rate. It is important to log errors to help developers improve the ability of the MobileApp to process speech input.

21.3.4 Alerts and Extraordinary Conditions

When a MobileApp runs in a real-time environment, there are factors that may impact its behavior. For example, a Wi-Fi signal may be lost, or an incoming text message, phone call, or calendar alert may be received while the user is working with the MobileApp.

These factors can disrupt the MobileApp user's work flow, yet most users opt to allow alerts and other interruptions as they work. A MobileApp test environment must be able to simulate these alerts and conditions. In addition, you should test the MobileApp's ability to handle alerts and conditions in a production environment on actual devices (Section 21.9).

Part of MobileApp testing should focus on the usability issues relating to alerts and pop-up messages. Testing should examine the clarity and context of alerts, the appropriateness of their location on the device display screen, and when foreign languages are involved, verification that the translation from one language to another is correct.

Many alerts and conditions may be triggered differently on various mobile devices or by network or context changes. Although many of the exception-handling processes can be simulated with a software test harness, you should not rely solely on testing in the development environment. This again emphasizes the importance of testing the MobileApp in the wild on actual devices.

Many computer-based systems must recover from faults and resume processing with little or no downtime. In some cases, a system must be fault tolerant; that is, processing faults must not cause overall system function to cease. In other cases, a system failure must be corrected within a specified period of time or severe economic damage will occur.

Recovery testing is a system test that forces the software to fail in a variety of ways and verifies that recovery is properly performed. If recovery is automatic (performed by the system itself), reinitialization, checkpointing mechanisms, data recovery, and restart are evaluated for correctness. If recovery requires human intervention, the mean time to repair (MTTR) is evaluated to determine whether it is within acceptable limits.

21.4 Web Application Testing

Many Web testing practices are also appropriate for testing thin-client MobileApps and interactive simulations. The strategy for WebApp testing adopts the basic principles for all software testing and applies a strategy and tactics that are used for object-oriented systems. The following steps summarize the approach:

1. The content model for the WebApp is reviewed to uncover errors.
2. The interface model is reviewed to ensure that all use cases can be accommodated.
3. The design model for the WebApp is reviewed to uncover navigation errors.
4. The user interface is tested to uncover errors in presentation and/or navigation mechanics.
5. Each functional component is unit tested.
6. Navigation throughout the architecture is tested.
7. The WebApp is implemented in a variety of different environmental configurations and is tested for compatibility with each configuration.
8. Security tests are conducted in an attempt to exploit vulnerabilities in the WebApp or within its environment.
9. Performance tests are conducted.
10. The WebApp is tested by a controlled and monitored population of end users. The results of their interaction with the system are evaluated for errors.

Because many WebApps evolve continuously, the testing process is an ongoing activity, conducted by support staff who use regression tests derived from the tests developed when the WebApp was first engineered. Methods for WebApp testing are considered in Section 21.5.

21.5 Web Testing Strategies

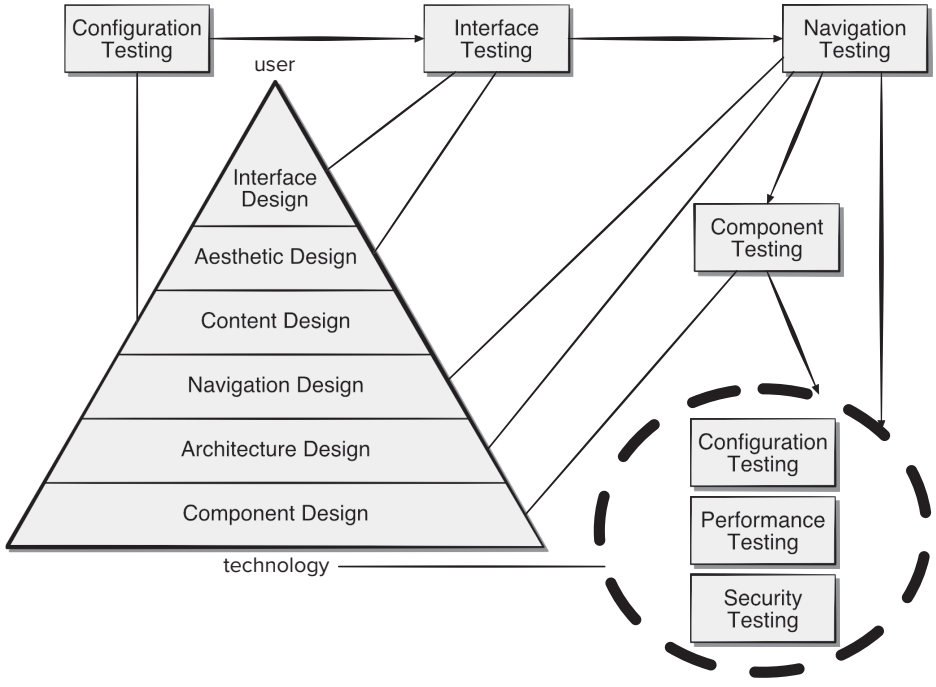
Testing is the process of exercising software with the intent of finding (and ultimately correcting) errors. This fundamental philosophy, first presented in Chapter 20, does not change for WebApps. In fact, because Web-based systems and applications reside on a network and interoperate with many different operating systems, browsers (or other personal communication devices), hardware platforms, communications protocols, and "backroom" applications, the search for errors represents a significant challenge.

Figure 21.2 juxtaposes the mobility testing process with the design pyramid for WebApps (Chapter 13). Note that as the testing flow proceeds from left to right and top to bottom, user-visible elements of the WebApp design (top elements of the pyramid) are tested first, followed by infrastructure design elements.

Because many apps evolve continuously, the testing process is an ongoing activity, conducted by app support staff who use regression tests derived from the tests developed when the app was first engineered.

FIGURE 21.2

The testing process



SAFEHOME



WebApp Testing

The scene: Doug Miller's office.

The players: Doug Miller, manager of the *SafeHome* software engineering group, and Vinod Raman, a member of the product software engineering team.

The conversation:

Doug: What do you think of the **SafeHome Assured.com** e-commerce WebApp V0.0?

Vinod: The outsourcing vendor has done a good job. Sharon [development manager for the vendor] tells me they're testing as we speak.

Doug: I'd like you and the rest of the team to do a little informal testing on the e-commerce site.

Vinod (grimacing): I thought we were going to hire a third-party testing company to validate the WebApp. We're still killing ourselves trying to get the product software out the door.

Doug: We're going to hire a testing vendor for performance and security testing, and our outsourcing vendor is already testing. Just thought another point of view would be helpful, and besides, we'd like to keep costs in line, so . . .

Vinod (sighs): What are you looking for?

Doug: I want to be sure that the interface and all navigation are solid.

Vinod: I suppose we can start with the use cases for each of the major interface functions:

Learn about *SafeHome*.
Specify the *SafeHome* system you need.
Purchase a *SafeHome* system.
Get technical support.

Doug: Good. But take the navigation paths all the way to their conclusion.

Vinod (looking through a notebook of use cases): Yeah, when you select **Specify the SafeHome system you need**, that'll take you to:

Select SafeHome components.
Get *SafeHome* component recommendations.

We can exercise the semantics of each path.

Doug: While you're there, check out the content that appears at each navigation node.

Vinod: Of course . . . and the functional elements as well. Who's testing usability?

Doug: Oh . . . the testing vendor will coordinate usability testing. We've hired a market research firm to line up 20 typical users for the usability study, but if you guys uncover any usability issues . . .

Vinod: I know, pass them along.

Doug: Thanks, Vinod.

21.5.1 Content Testing

Errors in WebApp content can be as trivial as minor typographical mistakes or as significant as incorrect information, improper organization, or violation of intellectual property laws. *Content testing* attempts to uncover these and many other problems before the user encounters them.

Content testing has three important objectives: (1) to uncover syntactic errors (e.g., typos, grammar mistakes) in text-based documents, graphical representations, and other media; (2) to uncover semantic errors (i.e., errors in the accuracy or completeness of information) in any content object presented as navigation occurs; and (3) to find errors in the organization or structure of content that is presented to the end user.

Content testing combines both reviews and the generation of executable test cases. Although technical reviews are not a part of testing, content review should be performed to ensure that content has quality and to uncover semantic errors. Executable testing is used to uncover content errors that can be traced to dynamically derived content that is driven by data acquired from one or more databases.

To accomplish the first objective, automated spelling and grammar checkers may be used. However, many syntactic errors evade detection by such tools and must be discovered by a human reviewer (tester). In fact, a large website might enlist the services of a professional copy editor to uncover typographical errors, grammatical mistakes, errors in content consistency, errors in graphical representations, and cross-referencing errors.

Semantic testing focuses on the information presented within each content object. The reviewer (tester) must answer the following questions:

- Is the information factually accurate?
- Is the information concise and to the point?
- Is the layout of the content object easy for the user to understand?
- Can information embedded within a content object be found easily?
- Have proper references been provided for all information derived from other sources?
- Is the information presented consistent internally and consistent with information presented in other content objects?

- Is the content offensive, misleading, or does it open the door to litigation?
- Does the content infringe on existing copyrights or trademarks?
- Does the content contain internal links that supplement existing content? Are the links correct?
- Does the aesthetic style of the content conflict with the aesthetic style of the interface?

Obtaining answers to each of these questions for a large WebApp (containing hundreds of content objects) can be a daunting task. However, failure to uncover semantic errors will shake the user's faith in the WebApp and can lead to failure of the Web-based application.

21.5.2 Interface Testing

Interface testing exercises interaction mechanisms and validates aesthetic aspects of the user interface. The overall strategy for interface testing is to (1) uncover errors related to specific interface mechanisms (e.g., errors in the proper execution of a menu link or the way data are entered in a form) and (2) uncover errors in the way the interface implements the semantics of navigation, WebApp functionality, or content display. With the exception of WebApp-oriented specifics, the interface strategy noted here is applicable to all types of client-server software. To accomplish this strategy, a number of tactical steps are initiated:

- Interface features are tested to ensure that design rules, aesthetics, and related visual content are available for the user without error.
- Individual interface mechanisms are tested in a manner that is analogous to unit testing. For example, tests are designed to exercise all forms, client-side scripting, dynamic HTML, scripts, streaming content, and application-specific interface mechanisms (e.g., a shopping cart for an e-commerce application).
- Each interface mechanism is tested within the context of a use case or network semantic unit (NSU) (Chapter 13) for a specific user category.
- The complete interface is tested against selected use cases and NSUs to uncover errors in the semantics of the interface. It is at this stage that a series of usability tests are conducted.
- The interface is tested within a variety of environments (e.g., browsers) to ensure that it will be compatible.

21.5.3 Navigation Testing

A user travels through a WebApp in much the same way as a visitor walks through a store or museum. There are many pathways to take, stops to make, things to learn and look at, activities to initiate, and decisions to make. This navigation process is predictable in the sense that every visitor has a set of objectives when he arrives. At the same time, the navigation process can be unpredictable because the visitor, influenced by something she sees or learns, may choose a path or initiate an action that is not typical for the original objective. The job of navigation testing is (1) to ensure that the mechanisms that allow the WebApp user to travel through the

WebApp are all functional and (2) to validate that each NSU can be achieved by the appropriate user category.

The first phase of navigation testing actually begins during interface testing. Navigation mechanisms (links and anchors of all types, redirects,[2] bookmarks, frames and frame sets, site maps, and the accuracy of internal search facilities) are tested to ensure that each performs its intended function. Some of the tests noted can be performed by automated tools (e.g., link checking), while others are designed and executed manually. The intent throughout is to ensure that errors in navigation mechanics are found before the WebApp goes online.

Each NSU (Chapter 13) is defined by a set of navigation paths (called "the user journey") that connect navigation nodes (e.g., Web pages, content objects, or functionality). Taken as a whole, each NSU allows a user to achieve specific requirements defined by one or more use cases for a user category. Navigation testing exercises each NSU to ensure that these requirements can be achieved. If NSUs have not been created as part of WebApp analysis or design, you can apply use cases for the design of navigation test cases. You should answer the following questions as each NSU or use case is tested:

- Is the NSU achieved in its entirety without error?
- Is every navigation node (defined for an NSU) reachable within the context of the navigation paths defined for the NSU?
- If the NSU can be achieved using more than one navigation path, has every relevant path been tested?
- If guidance is provided by the user interface to assist in navigation, are directions correct and understandable as navigation proceeds?
- Is there a mechanism (other than the browser "back" arrow) for returning to the preceding navigation node and to the beginning of the navigation path?
- Do mechanisms for navigation within a large navigation node (i.e., a long Web page) work properly?
- If a function is to be executed at a node and the user chooses not to provide input, can the remainder of the NSU be completed?
- If a function is executed at a node and an error in function processing occurs, can the NSU be completed?
- Is there a way to discontinue the navigation before all nodes have been reached, but then return to where the navigation was discontinued and proceed from there?
- Is every node reachable from the site map? Are node names meaningful to end users?
- If a node within an NSU is reached from some external source, is it possible to process to the next node on the navigation path? Is it possible to return to the previous node on the navigation path?
- Does the user understand his location within the content architecture as the NSU is executed?

2 When a server request is forwarded to a nonexistent URL.

Navigation testing, like interface and usability testing, should be conducted by as many different constituencies as possible. You have responsibility for early stages of navigation testing, but later stages should be conducted by other project stakeholders, an independent testing team, and ultimately, by nontechnical users. The intent is to exercise WebApp navigation thoroughly.

21.6 INTERNATIONALIZATION

Internationalization is the process of creating a software product so that it can be used in several countries and with various languages without requiring any engineering changes. *Localization* is the process of adapting a software application for use in targeted global regions by adding locale-specific requirements and translating text elements to appropriate languages. Localization effort may involve taking each country's currency, culture, taxes, and standards (both technical and legal) into account in addition to differences in languages [Sla12]. Launching a MobileApp in many parts of the world without testing it there would be very foolish.

Because it can be very costly to build an in-house testing facility in each country for which localization is planned, outsourcing testing to local vendors in each country is often more cost effective [Reu12]. However, using an outsourcing approach risks a degradation of communication between the MobileApp development team and those who are performing localization tests.

Crowdsourcing has become popular in many online communities.[3] Reuveni [Reu12] suggests that crowdsourcing could be used to engage localization testers dispersed around the globe outside of the development environment. To accomplish this, it is important to find a community that prides itself on its reputation and has a track record of successes. An easy-to-use real-time platform allows community members to communicate with the project decision makers. To protect intellectual property, only trustworthy community members who are willing to sign nondisclosure agreements are allowed to participate.

21.7 SECURITY TESTING

Any computer-based system that manages sensitive information or causes actions that can improperly harm (or benefit) individuals is a target for improper or illegal penetration. Penetration spans a broad range of activities: hackers who attempt to penetrate systems for sport, disgruntled employees who attempt to penetrate for revenge, dishonest individuals who attempt to penetrate for illicit personal gain.

Security testing attempts to verify that protection mechanisms built into a system will, in fact, protect it from improper penetration. Given enough time and resources, thorough security testing will ultimately penetrate a system. The role of the system designer is to make penetration cost more than the value of the information that will

3 *Crowdsourcing* is a distributed problem-solving model where community members work on solutions to problems posted to the group.

be obtained. Security assurance and security engineering are discussed in more detail in Chapter 18.

Mobile security is a complex subject that must be fully understood before effective security testing can be accomplished.[4] MobileApps and the client-side and server-side environments in which they are housed represent an attractive target for external hackers, disgruntled employees, dishonest competitors, and anyone else who wishes to steal sensitive information, maliciously modify content, degrade performance, disable functionality, or embarrass a person, organization, or business.

Security tests are designed to probe vulnerabilities of the client-side environment, the network communications that occur as data are passed from client to server and back again, and the server-side environment. Each of these domains can be attacked, and it is the job of the security tester to uncover weaknesses that can be exploited by those with the intent to do so.

On the client side, vulnerabilities can often be traced to preexisting bugs in browsers, e-mail programs, or communication software. On the server side, vulnerabilities include denial-of-service attacks and malicious scripts that can be passed along to the client side or used to disable server operations. In addition, server-side databases can be accessed without authorization (data theft).

To protect against these (and many other) vulnerabilities, firewalls, authentication, encryption, and authorization techniques can be used. Security tests should be designed to probe each of these security technologies in an effort to uncover security holes.

The actual design of security tests requires in-depth knowledge of the inner workings of each security element and a comprehensive understanding of a full range of networking technologies. If the MobileApp or WebApp is business critical, maintains sensitive data, or is a likely target of hackers, it's a good idea to outsource security testing to a vendor who specializes in it.

21.8 PERFORMANCE TESTING

For real-time and embedded systems, software that provides required functionality but does not conform to performance requirements is unacceptable. Performance testing is designed to test the run-time performance of software within the context of an integrated system. Performance testing occurs throughout all steps in the testing process. Even at the unit level, the performance of an individual module may be assessed as tests are conducted. However, it is not until all system elements are fully integrated that the true performance of a system can be ascertained.

Nothing is more frustrating than a MobileApp that takes minutes to load content when competitive apps download similar content in seconds. Nothing is more aggravating than trying to log on to a WebApp and receiving a "server-busy" message, with the suggestion that you try again later. Nothing is more disconcerting than a MobileApp or WebApp that responds instantly in some situations and then seems to go into an infinite wait state in other situations. All of these occurrences happen on the Web every day, and all of them are performance related.

4 Books by Bell et al. [Bel17], Sullivan and Liu [Sul11], and Cross [Cro07] provide useful information about the subject.

Performance testing is used to uncover performance problems that can result from a lack of server-side resources, inappropriate network bandwidth, inadequate database capabilities, faulty or weak operating system capabilities, poorly designed WebApp functionality, and other hardware or software issues that can lead to degraded client-server performance. The intent is twofold: (1) to understand how the system responds as *loading* (i.e., number of users, number of transactions, or overall data volume), and (2) to collect metrics that will lead to design modifications to improve performance.

Performance tests are often coupled with stress testing and usually require both hardware and software instrumentation. That is, it is often necessary to measure resource utilization (e.g., processor cycles) in an exacting fashion. External instrumentation can monitor execution intervals, log events (e.g., interrupts) as they occur, and sample machine states on a regular basis. By instrumenting a system, the tester can uncover situations that lead to degradation and possible system failure.

Some aspects of MobileApp performance, at least as the end user perceives it, are difficult to test. Network loading, the vagaries of network interfacing hardware, and similar issues are not easily tested at the client or browser level. Mobile performance tests are designed to simulate real-world loading situations. As the number of simultaneous app users grows, or the number of online transactions increases, or the amount of data (downloaded or uploaded) increases, performance testing will help answer the following questions:

- Does the server response time degrade to a point where it is noticeable and unacceptable?
- At what point (in terms of users, transactions, or data loading) does performance become unacceptable?
- What system components are responsible for performance degradation?
- What is the average response time for users under a variety of loading conditions?
- Does performance degradation have an impact on system security?
- Is app reliability or accuracy affected as the load on the system grows?
- What happens when loads that are greater than maximum server capacity are applied?
- Does performance degradation have an impact on company revenues?

To develop answers to these questions, two different performance tests are conducted: (1) *load testing* examines real-world loading at a variety of load levels and in a variety of combinations, and (2) *stress testing* forces loading to be increased to the breaking point to determine how much capacity the app environment can handle.

The intent of load testing is to determine how the WebApp and its server-side environment will respond to various loading conditions. As testing proceeds, permutations to the following variables define a set of test conditions:

N, number of concurrent users

T, number of online transactions per unit of time

D, data load processed by the server per transaction

In every case, these variables are defined within normal operating bounds of the system. As each test condition is run, one or more of the following measures are collected: average user response, average time to download a standardized unit of data, or average time to process a transaction. You should examine these measures to determine whether a precipitous decrease in performance can be traced to a specific combination of *N, T,* and *D*.

Load testing can also be used to assess recommended connection speeds for users of the WebApp. Overall throughput, *P,* is computed in the following manner:

$$P = N \times T \times D$$

As an example, consider a popular sports news site. At a given moment, 20,000 concurrent users submit a request (a transaction, *T*) once every 2 minutes on average. Each transaction requires the WebApp to download a new article that averages 3K bytes in length. Therefore, throughput can be calculated as:

$$P = \frac{20{,}000 \times 0.5 \times 3\text{kb}}{60} = 500 \text{ Kbytes/sec}$$

$$= 4 \text{ megabits per second}$$

The network connection for the server would therefore have to support this data rate and should be tested to ensure that it does.

Stress testing for mobile apps attempts to find errors that will occur under extreme operating conditions. In addition, it provides a mechanism for determining whether the MobileApp will degrade gracefully without compromising security. Among the many actions that might create extreme conditions are: (1) running several mobile apps on the same device, (2) infecting system software with viruses or malware, (3) attempting to take over a device and use it to spread spam, (4) forcing the mobile app to process inordinately large numbers of transactions, and (5) storing inordinately large quantities of data on the device. As these conditions are encountered, the MobileApp is checked to ensure that resource-intensive services (e.g., streaming media) are handled properly.

21.9 REAL-TIME TESTING

The time-dependent, asynchronous nature of many mobile and real-time applications adds a new and potentially difficult element to the testing mix—time. Not only does the test-case designer have to consider conventional test cases but also event handling (i.e., interrupt processing), the timing of the data, and the parallelism of the tasks (processes) that handle the data. In many situations, test data provided when a real-time system is in one state will result in proper processing, while the same data provided when the system is in a different state may lead to error. In addition, the intimate relationship that exists between real-time software and its hardware environment can also cause testing problems. Software tests must consider the impact of hardware faults on software processing. Such faults can be extremely difficult to simulate realistically.

Many MobileApp developers advocate *testing in the wild*, or testing in the users' native environments with the production release versions of the MobileApp resources

TABLE 21.1

Weighted device platform matrix

		OS1	OS2	OS3
	Ranking	3	4	7
Device1	7	N/A	28	49
Device2	3	9	N/A	N/A
Device3	4	12	N/A	N/A
Device4	9	N/A	36	63

[Soa11]. Testing in the wild is designed to be agile and respond to changes as the MobileApp evolves [Ute12].

Some of the characteristics of testing in the wild include adverse and unpredictable environments, outdated browsers and plug-ins, unique hardware, and imperfect connectivity (both Wi-Fi and mobile carrier). To mirror real-world conditions, the demographic characteristics of testers should match those of targeted users, as well as those of their devices. In addition, you should include use cases involving small numbers of users, less-popular browsers, as well as a diverse set of mobile devices. Testing in the wild is always somewhat unpredictable, and test plans must be adapted as testing progresses. For further information, Rooksby and his colleagues have identified themes that are present in successful strategies for testing in the wild [Roo09].

Because MobileApps are often developed for multiple devices and designed to be used in many different contexts and locations, a *weighted device platform matrix* (WDPM) helps ensure that test coverage includes each combination of mobile device and context variables. The WDPM can also be used to help prioritize the device/context combinations so that the most important are tested first.

The steps to build the WDPM (Table 21.1) for several devices and operating systems are: (1) list the important operating system variants as the matrix column labels, (2) list the targeted devices as the matrix row labels, (3) assign a ranking (e.g., 0 to 10) to indicate the relative importance of each operating system and each device, and (4) compute the product of each pair of rankings and enter each product as the cell entry in the matrix (use NA for combinations that are not available).

Testing effort should be adjusted so that the device/platform combinations with the highest ratings receive the most attention for each context variable under consideration.[5] In Table 21.1, **Device4** and **OS3** have the highest rating and would receive high-priority attention during testing.

Actual mobile devices have inherent limitations precipitated by the combination of hardware and firmware delivered in the device. If the range of potential device platforms is large, it is expensive and time consuming to perform MobileApp testing.

5 *Context variables* are variables that are associated with either the current connection or the current transaction that the MobileApp will use to direct its visible-user behavior.

Mobile devices are not designed with testing in mind. Limited processing power and storage capacity may not allow loading of the diagnostic software needed to record the test-case performance. Emulated devices are often easier to manage and allow easier acquisition of test data. Each mobile network (there are hundreds, worldwide) uses its own unique infrastructure to support the mobile Web. Emulators often cannot emulate the effects and timing of network services, and you may not see problems that users will have when the MobileApp runs on an actual device.

Creating test environments in-house is an expensive and error-prone process. Cloud-based testing can offer a standardized infrastructure and preconfigured software images, freeing the MobileApp team from the need to worry about finding servers or purchasing their own licenses for software and testing tools [Goa14]. Cloud service providers give testers access to scalable, ready-to-use virtual laboratories with a library of operating systems, test and execution management tools, and storage necessary for creating a test environment that closely mirrors the real world [Tao17].

Cloud-based testing is not without potential problems: lack of standards, potential security issues, data location and integrity issues, incomplete infrastructure support, improper usages of services, and performance issues are only some of the common challenges that face development teams that use the cloud approach.

Last, it is important to monitor power consumption specifically associated with the use of the MobileApp on a mobile device. Transmitting information from mobile devices consumes more power than monitoring a network for a signal. Processing streaming media consumes more power than loading a Web page or sending a text message. Assessing power consumption accurately must be done in real time on the actual device and in the wild.

21.10 Testing AI Systems

As we discussed in Chapter 13, mobile users expect products like MobileApps, VR systems, and video games to be context aware. Whether the software product is reacting to the user's environment [Abd16], automatically adapting the user interface based on past user behaviors [Par15], or providing a realistic nonplaying character (NPC) in a game situation [Ste16], artificial intelligence (AI) techniques are involved. Often these techniques make use of things like machine learning, data mining, statistics, heuristic programming, or rule-based systems that are outside the scope of this book. There are several problems common to testing these systems that can be addressed with the techniques we have discussed.

AI techniques make use of information that has been obtained from human experts or summarized from large numbers of observations saved in a data store of some kind. These data need to be organized in some way so that they can be accessed and updated efficiently if the software product is to be context aware or self-adaptive. The heuristics for making use of these data to assist decision making in the software are usually described by humans in use cases or formulas obtained from statistical data analysis. Part of what makes these systems hard to test is the large number of data interactions that need to be accounted for by the software, but whose occurrence is hard to predict. Software engineers often need to rely on simulation and model-based techniques to test AI systems.

21.10.1 Static and Dynamic Testing

Static testing is a software verification technique that focuses on review rather than executable testing. It is important to ensure that human experts (stakeholders who understand the application domain) agree with the ways in which the developers have represented the information and its use in the AI system. Like all software verification techniques, it is important to ensure that the program code represents the AI specifications, which means mapping between use case inputs and outputs is reflected in the code.

Dynamic testing for AI systems is a validation technique that exercises the source code with test cases. The intent is to show that the AI system conforms to the behaviors specified by the human experts. In the case of knowledge discovery or data mining, the program may have been designed to discover new relationships unknown to human experts. Human experts must validate these new relationships before they are used in safety-critical software products [Abd16] [Par15].

Many of the real-time testing issues discussed in Section 21.9 apply in dynamic testing of AI systems. Even if automatically generated simulated test cases are used, it is not possible to test every combination of events the software will encounter in the wild. It is often desirable to build in mechanisms to allow the users to specify when they are not happy with the decisions made by the program and collect information on the program state for future corrective action by developers.

21.10.2 Model-Based Testing

Model-based testing (MBT) is a black-box testing technique that uses information contained in the requirements model (in particular the user stories) as the basis for the generation of test cases [DAC03]. In many cases, the model-based testing technique uses formalism like UML state diagrams, an element of the behavioral model (Chapter 8), as the basis for the design of test cases.[6] The MBT technique requires five steps:

1. **Analyze an existing behavioral model for the software, or create one.** Recall that a *behavioral model* indicates how software will respond to external events or stimuli. To create the model, you should perform the steps discussed in Chapter 8: (1) evaluate all use cases to fully understand the sequence of interaction within the system, (2) identify events that drive the interaction sequence and understand how these events relate to specific objects, (3) create a sequence for each use case, (4) build a UML state diagram for the system (e.g., see Figure 8.8), and (5) review the behavioral model to verify accuracy and consistency.
2. **Traverse the behavioral model and specify the inputs that will force the software to make the transition from state to state.** The inputs will trigger events that will cause the transition to occur.
3. **Review the behavioral model, and note the expected outputs as the software makes the transition from state to state.** Recall that each state transition is triggered by an event and that as a consequence of the transition

6 Model-based testing can also be used when software requirements are represented with decision tables, grammars, or Markov chains [DAC03].

some function is invoked and outputs are created. For each set of inputs (test cases) you specified in step 2, specify the expected outputs as they are characterized in the behavioral model.

4. **Execute the test cases.** Test cases can be executed manually, or a test script can be created for use by an automated testing tool.
5. **Compare actual and expected results and take corrective action as required.**

MBT helps to uncover errors in software behavior, and as a consequence, it is extremely useful when testing event-driven applications such as context-aware MobileApps.

21.11 Testing Virtual Environments

It is virtually impossible for a software developer to foresee how the customer will actually use a program. Instructions for use may be misinterpreted; strange combinations of input actions may be used; feedback that seemed clear to the tester may be unintelligible to a user in the field. User experience designers are very aware of the importance of getting feedback from actual users early in the prototyping process to avoid creating software that users dislike.

Acceptance tests are a series of specific tests conducted by the customer in an attempt to uncover product errors before accepting the software from the developer. Conducted by the end user rather than software engineers, acceptance testing can range from an informal "test drive" to a planned and systematically executed series of scripted tests.

When a software product is built for one customer, it is reasonable for that person to conduct a series to validate all requirements. If software is a virtual simulation or game developed as a product to be used by many customers, it is impractical to allow each user to perform formal acceptance tests. Most software product builders use a process called alpha and beta testing to uncover errors that only end users seem able to find.

The *alpha test* is conducted at the developer's site by a representative group of end users. The software is used in a natural setting with the developer "looking over the shoulder" of the users and recording errors and usage problems. Alpha tests are conducted in a controlled environment.

The *beta test* is conducted at one or more end-user sites. Unlike alpha testing, the developer generally is not present. Therefore, the beta test is a "live" application of the software in an environment that cannot be controlled by the developer. The customer records all problems (real or imagined) that are encountered during beta testing and reports these at regular intervals. As a result of problems reported during beta tests, the developer makes modifications and then prepares for release of the software product to the entire customer base.

21.11.1 Usability Testing

Usability testing evaluates the degree to which users can interact effectively with the app and the degree to which the app guides users' actions, provides meaningful

feedback, and enforces a consistent interaction approach. Rather than focusing intently on the semantics of some interactive objective, usability reviews and tests are designed to determine the degree to which the app interface makes the user's life easy.[7]

Developers contribute to the design of usability tests, but in general the tests are conducted by end users. Usability testing can occur at a variety of different levels of abstraction: (1) the usability of a specific interface mechanism (e.g., a form) can be assessed, (2) the usability of a complete virtual interface (encompassing interface mechanisms, data objects, and related functions) can be evaluated, or (3) the usability of the complete virtual-world application can be considered.

The first step in usability testing is to identify a set of usability categories and establish testing objectives for each category. The following test categories and objectives (written in the form of a question) illustrate this approach:[8]

Interactivity. Are interaction mechanisms (e.g., pull-down menus, buttons, widgets, inputs) easy to understand and use?

Layout. Are navigation mechanisms, content, and functions placed in a manner that allows the user to find them quickly?

Readability. Is text well written and understandable?[9] Are graphic representations easy to understand?

Aesthetics. Do layout, color, typeface, and related characteristics lead to ease of use? Do users "feel comfortable" with the look and feel of the app?

Display characteristics. Does the app make optimal use of screen size and resolution?

Time sensitivity. Can important features, functions, and content be used or acquired in a timely manner?

Feedback. Do users receive meaningful feedback to their actions? Is the user's work interruptible and recoverable when a system message is displayed?

Personalization. Does the app tailor itself to the specific needs of different user categories or individual users?

Help. Is it easy for users to access help and other support options?

Accessibility. Is the app accessible to people who have disabilities?

Trustworthiness. Are users able to control how personal information is shared? Does the app make use of personal information without user permission?

A series of tests is designed within each of these categories. In some cases, the "test" may be a visual review of the app screen displays. In other cases interface semantics tests may be executed again, but in this instance usability concerns are paramount.

---

7 The term *user-friendliness* has been used in this context. The problem, of course, is that one user's perception of a "friendly" interface may be radically different from another's.

8 For additional information on usability, see Chapter 12.

9 The FOG Readability Index and others may be used to provide a quantitative assessment of readability.

FIGURE 21.3
Qualitative assessment of usability



As an example, we consider usability assessment for interaction and interface mechanisms. The following is a list of interface features that might be reviewed and tested for usability: animations, buttons, color, control, graphics, labels, menus, messages, navigation, selection mechanisms, text, and HUDs[10] (heads-up user displays). As each feature is assessed, it is graded on a qualitative scale by the users who are doing the testing. Figure 21.3 shows a possible set of assessment "grades" that can be selected by users. These grades are applied to each feature individually, to a complete app screen display, or to the app as a whole.

21.11.2 Accessibility Testing

Accessibility testing is the process of verifying the degree to which all people can use a computer system regardless of any user's special need. The special needs most commonly considered for computer system accessibility are: visual, hearing, movement, and cognitive impairments [Zan18]. Many of these special needs evolve as people get older. As a profession, virtual environment development has not done a good job of providing access systems with rich graphical interfaces that rely heavily on touch interactions [Dia14]. The problems merely shift with a switch to voice-activated

10 Mobile apps and games frequently provide graphical displays containing user status, system messages, navigation date, and menu choices as part of the device screen display or HUD.

personal assistants like Alexa® or Siri®. Just imagine trying to operate your smartphone without using all these senses: sight, hearing, touch, or speech.

We discussed guidelines[11] for designing accessible software products in Chapter 13. An effective design strategy should ensure that all important interactions with the user be presented using more than one information channel. A few examples of the areas of focus for accessibility testing follow [Zan18] [Dia14]:

- Ensure that all nontext screen objects are also represented by a text-based description.
- Verify that color is not used exclusively to convey information to the user.
- Demonstrate that high contrast and magnification options are available for elderly or visually challenged users.
- Ensure that speech input alternatives have been implemented to accommodate users that may not be able to manipulate a keyboard, keypad, or mouse.
- Demonstrate that blinking, scrolling, or auto content updating is avoided to accommodate users with reading difficulties.

It is likely that mobile, cloud-based software will come to dominate many things that users need to accomplish on a day-to-day basis (e.g., banking, tax preparation, restaurant reservations, trip planning) and as a consequence, the need for accessible software products will only grow. Along with expert review and automated tools to assess accessibility, a thorough accessibility testing strategy will help to ensure that every user, no matter their challenges, will be accommodated.

21.11.3 Playability Testing

Playability is the degree to which a game or simulation is fun to play and usable by the user/player and was originally conceived as part of the development of video games. Game playability is affected by the quality of the game: usability, storyline, strategy, mechanics, realism, graphics, and sound. With the advent of virtual/augmented reality simulations whose intention is to provide entertainment or learning opportunities (e.g., such as simulated troubleshooting), it makes sense to use playability testing as part of the usability testing for a virtual environment created by MobileApp [Vel16].

Expert review can be used as part of the playability testing, but unless expert users are your target user group you may not get the feedback you need to have your MobileApp succeed in the marketplace. Expert review should be supplemented by playability tests conducted by representative end users, as you might do for a beta or acceptance test. In a typical play test, the user might be given general instructions on using the app and the developers would then step back and observe players use of the game without interruption. The players may be asked to complete a survey on their experience once they are done with the play test [Hus15].

The developers might record the play session or simply take notes on what they observe. The developers are looking for places in the play session where the player does

11 Here is an example of a software accessibility checklist used by the United States Department of Justice: https://www.justice.gov/crt/software-accessibility-checklist.

not know what to do next (this is usually marked by a sudden halt in the player's actions). Developers should note where the player is in the app work flow when this event happens. When the play test has ended, the developers might discuss why the player got stuck and how the player got herself unstuck (it she did). This suggests that playability testing might be helpful in assessing the accessibility of a virtual environment as well.

21.12 Testing Documentation and Help Facilities

The term *usability testing* conjures up images of large numbers of test cases prepared to exercise computer programs and the data that they manipulate. But errors in help facilities or online program documentation can be as devastating to the acceptance of the program as errors in data or source code. Nothing is more frustrating than following a user guide or help facility exactly and getting results or behaviors that do not coincide with those predicted by the documentation. It is for this reason that documentation testing should be an important part of every software test plan.

Documentation testing can be approached in two phases. The first phase, technical review (Chapter 16), examines the document for editorial clarity. The second phase, live test, uses the documentation in conjunction with the actual program.

Surprisingly, a live test for documentation can be approached using techniques that are analogous to many of the black-box testing methods discussed earlier. Graph-based testing can be used to describe the use of the program; equivalence partitioning and boundary value analysis can be used to define various classes of input and associated interactions. MBT can be used to ensure that documented behavior and actual behavior coincide. Program usage is then tracked through the documentation.

INFO



Documentation Testing

The following questions should be answered during documentation and/or help facility testing:

- Does the documentation accurately describe how to accomplish each mode of use?
- Is the description of each interaction sequence accurate?
- Are examples accurate?
- Are terminology, menu descriptions, and system responses consistent with the actual program?
- Is it relatively easy to locate guidance within the documentation?
- Can troubleshooting be accomplished easily with the documentation?
- Are the document's table of contents and index robust, accurate, and complete?
- Is the design of the document (layout, typefaces, indentation, graphics) conducive to understanding and quick assimilation of information?
- Are all software error messages displayed for the user described in more detail in the document? Are actions to be taken as a consequence of an error message clearly delineated?
- If hypertext links are used, are they accurate and complete?
- If hypertext is used, is the navigation design appropriate for the information required?

The only viable way to answer these questions is to have an independent third party (e.g., selected users) test the documentation in the context of program usage. All discrepancies are noted, and areas of document ambiguity or weakness are defined for potential rewrite.

21.13 SUMMARY

The goal of MobileApp testing is to exercise each of the many dimensions of software quality for mobile applications with the intent of finding errors or uncovering issues that may lead to quality failures. Testing focuses on the quality elements such as content, function, structure, usability, use of context, navigability, performance, power management, compatibility, interoperability, capacity, and security. It incorporates reviews and usability assessments that occur as the MobileApp is designed, and tests that are conducted once the MobileApp has been implemented and deployed on an actual device.

The MobileApp testing strategy exercises each quality dimension by initially examining "units" of content, functionality, or navigation. Once individual units have been validated, the focus shifts to tests that exercise the MobileApp as a whole. To accomplish this, many tests are derived from the user's perspective and are driven by information contained in use cases. A MobileApp test plan is developed and identifies testing steps, work products (e.g., test cases), and mechanisms for the evaluation of test results. The testing process encompasses several different types of testing.

Content testing (and reviews) focus on various categories of content. The intent is to examine errors that affect the presentation of the content to the end user. The content needs to be examined for performance issues imposed by the mobile device constraints. Interface testing exercises the interaction mechanisms that define the user experience provided by the MobileApp. The intent is to uncover errors that result when the MobileApp does not take device, user, or location context into account.

Navigation testing is based on use cases, derived as part of the modeling activity. The test cases are designed to exercise each usage scenario against the navigation design within the architectural framework used to deploy the MobileApp. Component testing exercises content and functional units within the MobileApp.

Performance testing encompasses a series of tests that are designed to assess MobileApp response time and reliability as demands on server-side resource capacity increase.

Security testing incorporates a series of tests designed to exploit vulnerabilities in the MobileApp or its environment. The intent is to find security holes in either the device operating environment or the Web services being accessed.

Finally, MobileApp testing should address performance issues such as power usage, processing speed, memory limitations, ability to recover from failures, and connectivity issues.

Navigation testing applies use cases, derived as part of the modeling activity, in the design of test cases that exercise each usage scenario against the navigation design. Navigation mechanisms are tested to ensure that any errors impeding completion of a use case are identified and corrected. Component testing exercises content and functional units within the MobileApp.

PROBLEMS AND POINTS TO PONDER

21.1. Are there any situations in which MobileApp testing on actual devices can be disregarded?

21.2. Is it fair to say that the overall mobility testing strategy begins with user-visible elements and moves toward technology elements? Are there exceptions to this strategy?

21.3. Describe the steps associated with user experience testing for an app.

21.4. What is the objective of security testing? Who performs this testing activity?

21.5. Assume that you are developing a MobileApp to access an online pharmacy (`YourCornerPharmacy.com`) that caters to senior citizens. The pharmacy provides typical functions but also maintains a database for each customer so that it can provide drug information and warn of potential drug interactions. Discuss any special usability or accessibility tests for this MobileApp.

21.6. Assume that you have implemented a Web service that provides a drug interaction–checking function for `YourCornerPharmacy.com` (see Problem 21.5). Discuss the types of component-level tests that would have to be conducted on the mobile device to ensure that the MobileApp accesses this function properly.

21.7. Is it possible to test every configuration that a MobileApp is likely to encounter in the production environment? If it is not, how do you select a meaningful set of configuration tests?

21.8. Describe a security test that might need to be conducted for the YourCornerPharmacy MobileApp (Problem 21.5). Who should perform this test?

21.9. What is the difference between testing that is associated with interface mechanisms and testing that addresses interface semantics?

21.10. What is the difference between testing for navigation syntax and navigation semantics?

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

22 SOFTWARE CONFIGURATION MANAGEMENT

Change is inevitable when computer software is built and can lead to confusion when you and other members of a software team are working on a project. Confusion arises when changes are not analyzed before they are made, recorded before they are implemented, reported to those with a need to know, or controlled in a manner that will improve quality and reduce error. Babich [Bab86] suggests an approach that will minimize confusion, improve productivity, and reduce the number of mistakes when he writes: "Configuration management is the art of identifying, organizing, and controlling modifications to the software being built by a programming team. The goal is to maximize productivity by minimizing mistakes."

KEY CONCEPTS

baselines 441
change control 448
change management, mobility and agile . . . 453
change management process 447
configuration audit 452
configuration management, elements of . . . 440
configuration objects 441
content management 455
continuous integration 446
identification 452
integration and publishing 455
repository. 453
SCM process 448
software configuration items 438
status reporting. 452
version control. 445



QUICK LOOK

What is it? When you build computer software, change happens. And because it happens, you need to manage it effectively. Software configuration management (SCM), also called change management, is a set of activities designed to manage change.

Who does it? Everyone involved in the software process is involved with change management to some extent, but specialized support positions are sometimes created to manage the SCM process.

Why is it important? If you don't control change, it controls you. And that's never good. It's very easy for a stream of uncontrolled changes to turn a well-run software project into chaos. As a consequence, software quality suffers and delivery is delayed.

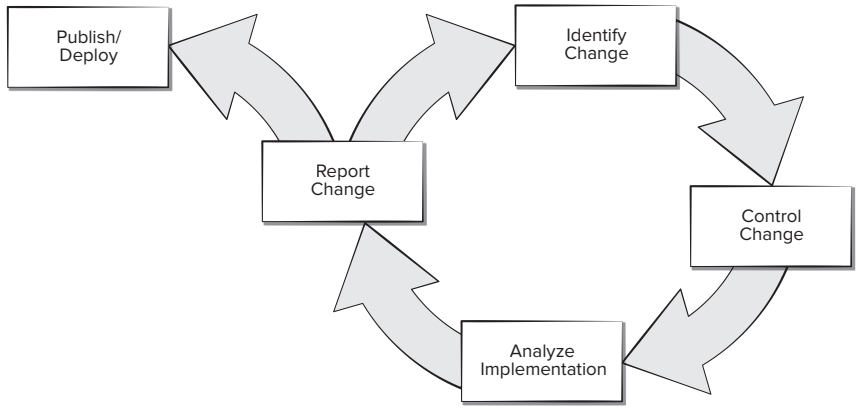
What are the steps? Because many work products are produced when software is built, each must be uniquely identified. Once this is accomplished, mechanisms for version and change control can be established.

What is the work product? A software configuration management plan defines the project strategy for change management. Changes result in updated software products that must be retested and documented, without breaking the project schedule or the production versions of the software products.

How do I ensure that I've done it right? When every work product can be accounted for, traced, controlled, tracked, and analyzed; when everyone who needs to know about a change has been informed—you've done it right.

FIGURE 22.1

Software configuration management work flow



Software configuration management (SCM) is an umbrella activity that is applied throughout the software process. Typical SCM work flow is shown in Figure 22.1. Because change can occur at any time, SCM activities are developed to (1) identify change, (2) control change, (3) ensure that change is being properly implemented, and (4) report changes to others who may have an interest.

It is important to make a clear distinction between software support and software configuration management. Support (Chapter 27) is a set of software engineering activities that occur after software has been delivered to the customer and put into operation. Software configuration management is a set of tracking and control activities that are initiated when a software engineering project begins and terminates only when the software is taken out of operation.

A primary goal of software engineering is to improve the ease with which changes can be accommodated and reduce the amount of effort expended when changes must be made. In this chapter, we discuss the specific activities that enable you to manage change.

22.1 SOFTWARE CONFIGURATION MANAGEMENT

The output of the software process is information that may be divided into three broad categories: (1) computer programs (both source level and executable forms), (2) work products that describe the computer programs (targeted at various stakeholders), and (3) data or content (contained within the program or external to it). In Web design or game development, managing changes to the multimedia content items can be more demanding than managing the changes to the software or documentation. The items that comprise all information produced as part of the software process are collectively called a *software configuration*.

As software engineering work progresses, a hierarchy of *software configuration items* (SCIs)—a named element of information that can be as small as a single UML diagram or as large as the complete design document—is created. If each SCI simply led to other SCIs, little confusion would result. Unfortunately, another variable enters the process—*change*. Change may occur at any time, for any reason. In fact, the *first law of system*

engineering [Ber80] states: "No matter where you are in the system life cycle, the system will change, and the desire to change it will persist throughout the life cycle."

What is the origin of these changes? The answer to this question is as varied as the changes themselves. However, there are four fundamental sources of change:

- New business or market conditions dictate changes in product requirements or business rules.
- New stakeholder needs demand modification of data produced by information systems, functionality delivered by products, or services delivered by a computer-based system.
- Reorganization or business growth or downsizing causes changes in project priorities or software engineering team structure.
- Budgetary or scheduling constraints cause a redefinition of the system or product.

Software configuration management is a set of activities that have been developed to manage change throughout the life cycle of computer software. SCM can be viewed as a software quality assurance activity that is applied throughout the software process. In the sections that follow, we describe major SCM tasks and important concepts that help us to manage change.

22.1.1 An SCM Scenario

This section is extracted from [Dar01].[1]

> A typical configuration management (CM) operational scenario involves several stakeholders: a project manager who is in charge of a software group, a configuration manager who is in charge of the CM procedures and policies, the software engineers who are responsible for developing and maintaining the software product, and the customer who uses the product. In the scenario, assume that the product is a small one involving about 15,000 lines of code being developed by an agile team with four developers. (Note that other scenarios of smaller or larger teams are possible, but, in essence, there are generic issues that each of these projects face concerning CM.)

At the operational level, the scenario involves various roles and tasks. For the project manager or team leader, the goal is to ensure that the product is developed within a certain time frame. Hence, the manager monitors the progress of development and recognizes and reacts to problems. This is done by generating and analyzing reports about the status of the software system and by performing reviews on the system.

The goals of the configuration manager (who on a small team may be the project manager) are to ensure that procedures and policies for creating, changing, and testing of code are followed, as well as to make information about the project accessible. To implement techniques for maintaining control over code changes, this manager introduces mechanisms for making official requests for changes, for evaluating proposed changes with the development team, and ensuring the changes are acceptable to the product owner. Also, the manager collects statistics about components in the software system, such as information determining which components in the system are problematic.

---

1 Special permission to reproduce "Spectrum of Functionality in CM Systems" by Susan Dart [Dar01], © 2001 by Carnegie Mellon University is granted by the Software Engineering Institute.

For the software engineers, the goal is to work effectively. There must be a mechanism to ensure that simultaneous changes to the same component are properly tracked, managed, and executed. This means engineers do not unnecessarily interfere with each other in the creation and testing of code and in the production of supporting work products. But, at the same time, they try to communicate and coordinate efficiently. Specifically, engineers use tools that help build a consistent software product. They communicate and coordinate by notifying one another about tasks required and tasks completed. Changes are propagated across each other's work by merging files. Mechanisms exist to ensure that, for components that undergo simultaneous changes, there is some way of resolving conflicts and merging changes. A history is kept of the evolution of all components of the system along with a log with reasons for changes and a record of what actually changed. The engineers have their own workspace for creating, changing, testing, and integrating code. At a certain point, the code is made into a baseline from which further development continues and from which variants for other target machines are made.

The customer uses the product. Because the product is under CM control, the customer follows formal procedures for requesting changes and for indicating bugs in the product.

Ideally, a CM system used in this scenario should support all these roles and tasks; that is, the roles determine the functionality required of a CM system. The project manager sees CM as an auditing mechanism; the configuration manager sees it as a controlling, tracking, and policy-making mechanism; the software engineer sees it as a changing, building, and access control mechanism; and the customer sees it as a quality assurance mechanism.

22.1.2 Elements of a Configuration Management System

In her comprehensive white paper on software configuration management, Susan Dart [Dar01] identifies four important elements that should exist when a configuration management system is developed:

- **Component elements.** A set of tools coupled within a file management system (e.g., a database) that enables access to and management of each software configuration item.
- **Process elements.** A collection of procedures and tasks that define an effective approach to change management (and related activities) for all constituencies involved in the management, engineering, and use of computer software.
- **Construction elements.** A set of tools that automate the construction of software by ensuring that the proper set of validated components (i.e., the correct version) have been assembled.
- **Human elements.** A set of tools and process features (encompassing other CM elements) used by the software team to implement effective SCM.

These elements (to be discussed in more detail in later sections) are not mutually exclusive. For example, component elements work in conjunction with construction elements as the software process evolves. Process elements guide many human activities that are related to SCM and might therefore be considered human elements as well.

22.1.3 Baselines

Change is a fact of life in software development. Customers want to modify requirements. Developers want to modify the technical approach. Managers want to modify the project strategy. Why all this modification? The answer is really quite simple. As time passes, all constituencies know more (about what they need, which approach would be best, and how to get it done and still make money). Most software changes are justified, so there's no point in complaining about them. Rather, be certain that you have mechanisms in place to handle them.

A *baseline* is a software configuration management concept that helps you to control change without seriously impeding justifiable change. The IEEE [IEE17] defines a baseline as:

> A specification or product that has been formally reviewed and agreed upon, that thereafter serves as the basis for further development, and that can be changed only through formal change control procedures.

Before a software configuration item becomes a baseline, change may be made quickly and informally. However, once a baseline is established, changes can be made, but a specific, formal procedure must be applied to evaluate and verify each change.

In the context of software engineering, a baseline is a milestone in the development of software. A baseline is marked by the delivery of one or more software configuration items that have been approved as a consequence of a technical review (Chapter 16). For example, the elements of a design model have been documented and reviewed. Errors are found and corrected. Once all parts of the model have been reviewed, corrected, and then approved, the design model becomes a baseline. Further changes to the program architecture (documented in the design model) can be made only after each has been evaluated and approved. Although baselines can be defined at any level of detail, the most common software baselines are shown in Figure 22.2.

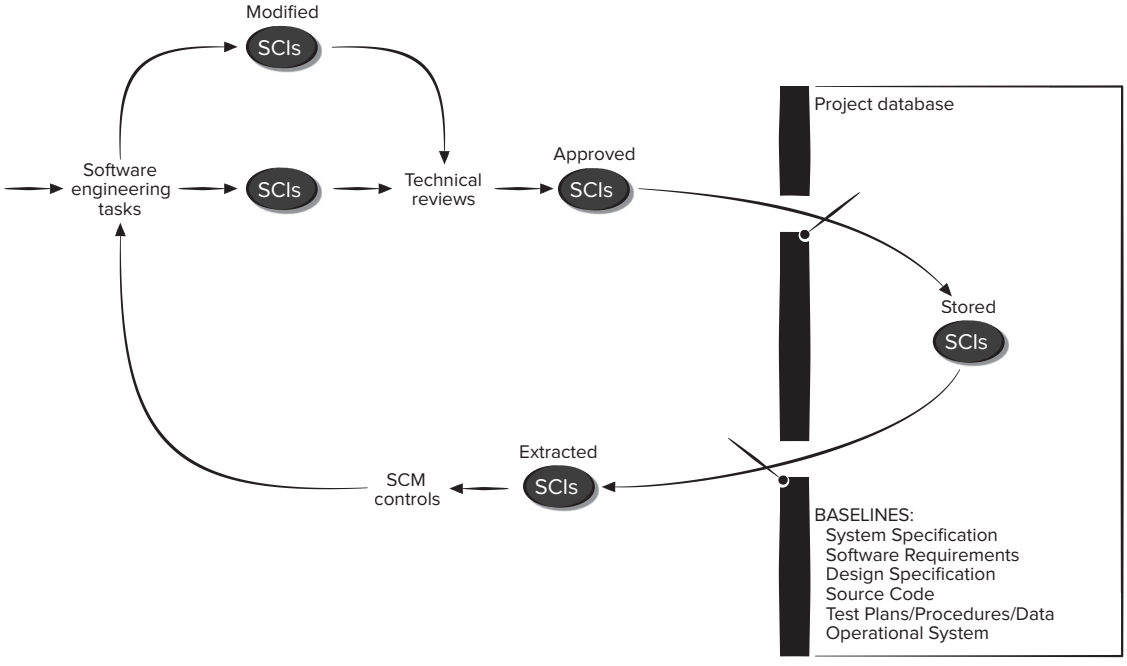
The progression of events that lead to a baseline is also illustrated in Figure 22.2. Software engineering tasks produce one or more SCIs. After SCIs are reviewed and approved, they are placed in a *project database* (also called a *project library* or *software repository* and discussed in Section 22.5). Be sure that the project database is maintained in a centralized, controlled location. When a member of a software engineering team wants to make a modification to a baselined SCI, it is copied from the project database into the engineer's private workspace. However, this extracted SCI can be modified only if SCM controls (discussed later in this chapter) are followed. The arrows in Figure 22.2 illustrate the modification path for a baselined SCI.

22.1.4 Software Configuration Items

We have already defined a software configuration item as information that is created as part of the software engineering process. In the extreme, an SCI could be considered to be a single section of a large specification or one test case in a large suite of tests. More realistically, an SCI is all or part of a work product (e.g., a document, an entire suite of test cases, a named program component, a multimedia content asset, or a software tool).

In reality, SCIs are organized to form configuration objects that may be cataloged in the project database with a single name. A *configuration object* has a

FIGURE 22.2 Baselined SCIs and the project database



name, attributes, and is "connected" to other objects by relationships. Referring to Figure 22.3, the configuration objects, **DesignSpecification, DataModel, ComponentN, SourceCode,** and **TestSpecification** are each defined separately. However, each of the objects is related to the others as shown by the arrows. A curved arrow indicates a compositional relation. That is, **DataModel** and **ComponentN** are part of the object **DesignSpecification.** A double-headed straight arrow indicates an interrelationship. If a change were made to the **SourceCode** object, the interrelationships enable you to determine what other objects (and SCIs) might be affected.[2]

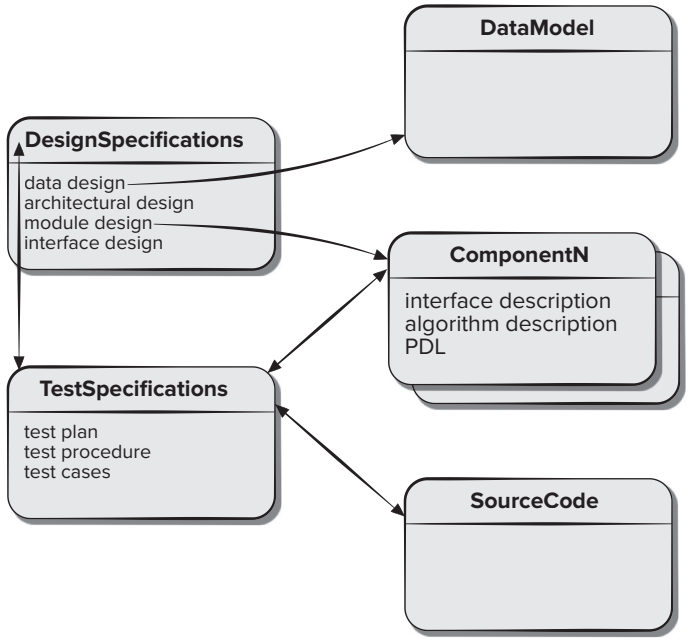
22.1.5 Management of Dependencies and Changes

We introduced the concept of traceability and the use of traceability matrices in Section 7.2.6. The traceability matrix is one way to document dependencies among requirements, architectural decisions (Section 10.5), and defect causes (Section 17.6). These dependencies need to be considered when determining the impact of a proposed change and guiding the selection test cases that should be used for regression testing (Section 20.3). de Sousa and Redmiles write that viewing dependency management as impact management[3] helps developers to focus on how changes made affect their work [Sou08].

2 These relationships are defined within the database. The structure of the database (repository) is discussed in greater detail in Section 22.2.

3 Impact management is discussed further in Section 22.5.2.

FIGURE 22.3
Configuration objects



Impact analysis focuses on organizational behavior as well as individual actions. Impact management involves two complementary aspects: (1) ensuring that software developers employ strategies to minimize the impact of their colleagues' actions on their own work, and (2) encouraging software developers to use practices that minimize the impact of their own work on that of their colleagues. It is important to note that when a developer tries to minimize the impact of her work on others, she is also reducing the work others need to do to minimize the impact of her work on theirs [Sou08].

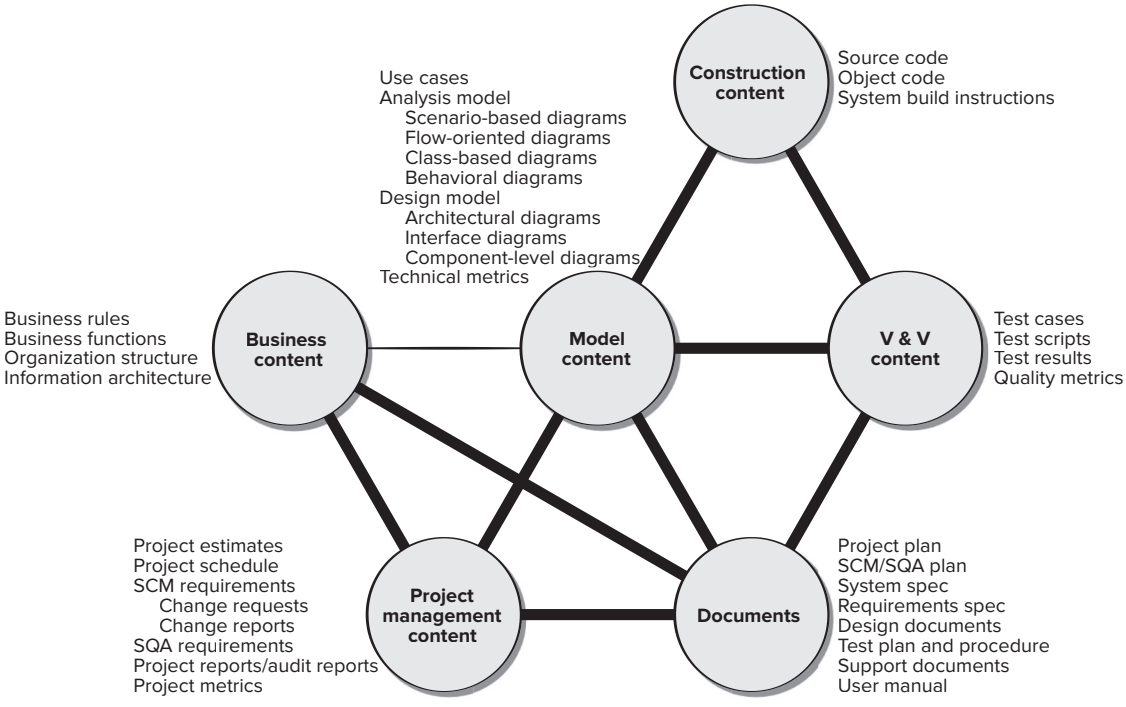
It is important to maintain software work products to ensure that developers are aware of the dependencies among the SCIs. Developers must establish discipline when checking items in and out of the SCM repository and when making approved changes, as discussed in Section 22.2.

22.2 THE SCM REPOSITORY

The SCM repository is the set of mechanisms and data structures that allow a software team to manage change in an effective manner. It provides the obvious functions of a modern database management system by ensuring data integrity, sharing, and integration. In addition, the SCM repository provides a hub for the integration of software tools, is central to the flow of the software process, and can enforce uniform structure and format for software engineering work products.

To achieve these capabilities, the repository is defined in terms of a meta-model. The *meta-model* determines how information is stored in the repository, how data

FIGURE 22.4 Content of the repository



can be accessed by tools and viewed by software engineers, how well data security and integrity can be maintained, and how easily the existing model can be extended to accommodate new needs.

22.2.1 General Features and Content

The features and content of the repository are best understood by looking at it from two perspectives: what is to be stored in the repository and what specific services are provided by the repository. A detailed breakdown of types of representations, documents, and other work products that are stored in the repository is presented in Figure 22.4.

A robust repository provides two different classes of services: (1) the same types of services that might be expected from any sophisticated database management system and (2) services that are specific to the software engineering environment.

A repository that serves a software engineering team should also (1) integrate with or directly support process management functions, (2) support specific rules that govern the SCM function and the data maintained within the repository, (3) provide an interface to other software engineering tools, and (4) accommodate storage of sophisticated data objects (e.g., text, graphics, video, audio).

22.2.2 SCM Features

To support SCM, the repository must be capable of maintaining SCIs related to many different versions of the software. More important, it must provide the mechanisms

for assembling these SCIs into a version-specific configuration. The repository tool set needs to provide support for the following features.

Versioning. As a project progresses, many versions (Section 22.5.2) of individual work products will be created. The repository must be able to save all these versions to enable effective management of product releases and to permit developers to go back to previous versions during testing and debugging.

The repository must be able to control a wide variety of object types, including text, graphics, bit maps, complex documents, and unique objects such as screen and report definitions, object files, test data, and results. A mature repository tracks versions of objects with arbitrary levels of granularity; for example, a single data definition or a cluster of modules can be tracked.

Dependency Tracking and Change Management. The repository manages a wide variety of relationships among the data elements stored in it. These include relationships between enterprise entities and processes, among the parts of an application design, between design components and the enterprise information architecture, between design elements and deliverables, and so on. Some of these relationships are merely associations, and some are dependencies or mandatory relationships.

The ability to keep track of all these relationships is crucial to the integrity of the information stored in the repository and to the generation of deliverables based on it, and it is one of the most important contributions of the repository concept to the improvement of the software development process. For example, if a UML class diagram is modified, the repository can detect whether related classes, interface descriptions, and code components also require modification and can bring affected SCIs to the developer's attention.

Requirements Tracing. This special function depends on link management and provides the ability to track all the design and construction components and deliverables that result from a specific requirements specification (forward tracing). In addition, it provides the ability to identify which requirement generated any given work product (backward tracing).

Configuration Management. A configuration management facility keeps track of a series of configurations representing specific project milestones or production releases.

Audit Trails. An audit trail establishes additional information about when, why, and by whom changes are made. Information about the source of changes can be entered as attributes of specific objects in the repository. A repository trigger mechanism is helpful for prompting the developer or the tool that is being used to initiate entry of audit information (such as the reason for a change) whenever a design element is modified.

22.3 VERSION CONTROL SYSTEMS

Version control combines procedures and tools to manage different versions of configuration objects that are created during the software process. A version control system implements or is directly integrated with four major capabilities: (1) a project

database (*repository*) that stores all relevant configuration objects, (2) a *version management* capability that stores all versions of a configuration object (or enables any version to be constructed using differences from past versions), (3) a *make facility* that enables you to collect all relevant configuration objects and construct a specific version of the software. In addition, version control and change control systems often implement (4) an *issues tracking* (also called *bug tracking*) capability that enables the team to record and track the status of all outstanding issues associated with each configuration object.

A number of version control systems establish a *change set*—a collection of all changes (to some baseline configuration) that are required to create a specific version of the software. Dart [Dar91] notes that a change set "captures all changes to all files in the configuration along with the reason for changes and details of who made the changes and when."

A number of named change sets can be identified for an application or system. This enables you to construct a version of the software by specifying the change sets (by name) that must be applied to the baseline configuration. To accomplish this, a *system modeling* approach is applied. The system model contains: (1) a *template* that includes a component hierarchy and a "build order" for the components that describes how the system must be constructed, (2) construction rules, and (3) verification rules.[4]

A number of different automated approaches to version control have been proposed over the years.[5] The primary difference in approaches is the sophistication of the attributes that are used to construct specific versions and variants of a system and the mechanics of the process for construction.

22.4 CONTINUOUS INTEGRATION

Best practices for SCM include: (1) keeping the number of code variants small, (2) test early and often, (3) integrate early and often, and (4) tool use to automate testing, building, and code integration. *Continuous integration* (CI) is important to agile developers following the DevOps workflow (Section 3.5.3). CI also adds value to SCM by ensuring that each change is promptly integrated into the project source code, compiled, and tested automatically. CI offers development teams several concrete advantages [Mol12]:

Accelerated feedback. Notifying developers immediately when integration fails allows fixes to be made while the number of performed changes is small.

Increased quality. Building and integrating software whenever necessary provides confidence into the quality of the developed product.

4 It is also possible to query the system model to assess how a change in one component will impact other components.

5 Github (https://github.com/), Perforce (https://www.perforce.com/), and Apache Subversion also known as SVN (http://subversion.apache.org/) are popular version control systems.

Reduced risk. Integrating components early avoids risking a long integration phase because design failures are discovered and fixed early.

Improved reporting. Providing additional information (e.g., code analysis metrics) allows for more accurate configuration status accounting.

CI is becoming a key technology as software organizations begin their shift to more agile software development processes. CI is best done using specialized tools.[6] CI allows project managers, quality assurance managers, and software engineers to improve software quality by reducing the likelihood of defects escaping outside the development team. Early defect capture always reduces the development costs by allowing cheaper fixes earlier in the software project time line.

22.5 THE CHANGE MANAGEMENT PROCESS

The software change management process defines a series of tasks that have four primary objectives: (1) to identify all items that collectively define the software configuration, (2) to manage changes to one or more of these items, (3) to facilitate the construction of different versions of an application, and (4) to ensure that software quality is maintained as the configuration evolves over time.

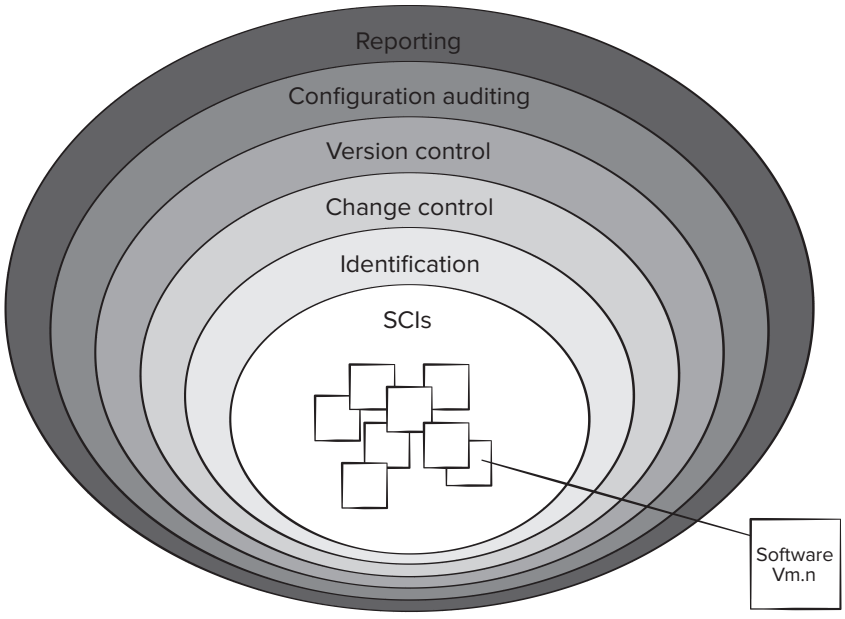
A process that achieves these objectives need not be bureaucratic and ponderous, but it must be characterized in a manner that enables a software team to develop answers to a set of complex questions:

- How does a software team identify the discrete elements of a software configuration?
- How does an organization manage the many existing versions of a program (and its documentation) in a manner that will enable change to be accommodated efficiently?
- How does an organization control changes before and after software is released to a customer?
- How does an organization assess the impact of change and manage the impact effectively?
- Who has responsibility for approving and ranking requested changes?
- How can we ensure that changes have been made properly?
- What mechanism is used to apprise others of changes that are made?

These questions lead to the definition of five SCM tasks—identification, version control, change control, configuration auditing, and reporting—illustrated in Figure 22.5.

6 Puppet (https://puppet.com/), Jenkins (https://jenkins.io/), and Hudson (http://hudson-ci.org/) are examples of CI tools. Travis-CI (https://travis-ci.org/) is a CI tool designed for sync projects residing on Github.

FIGURE 22.5
Layers of the SCM process



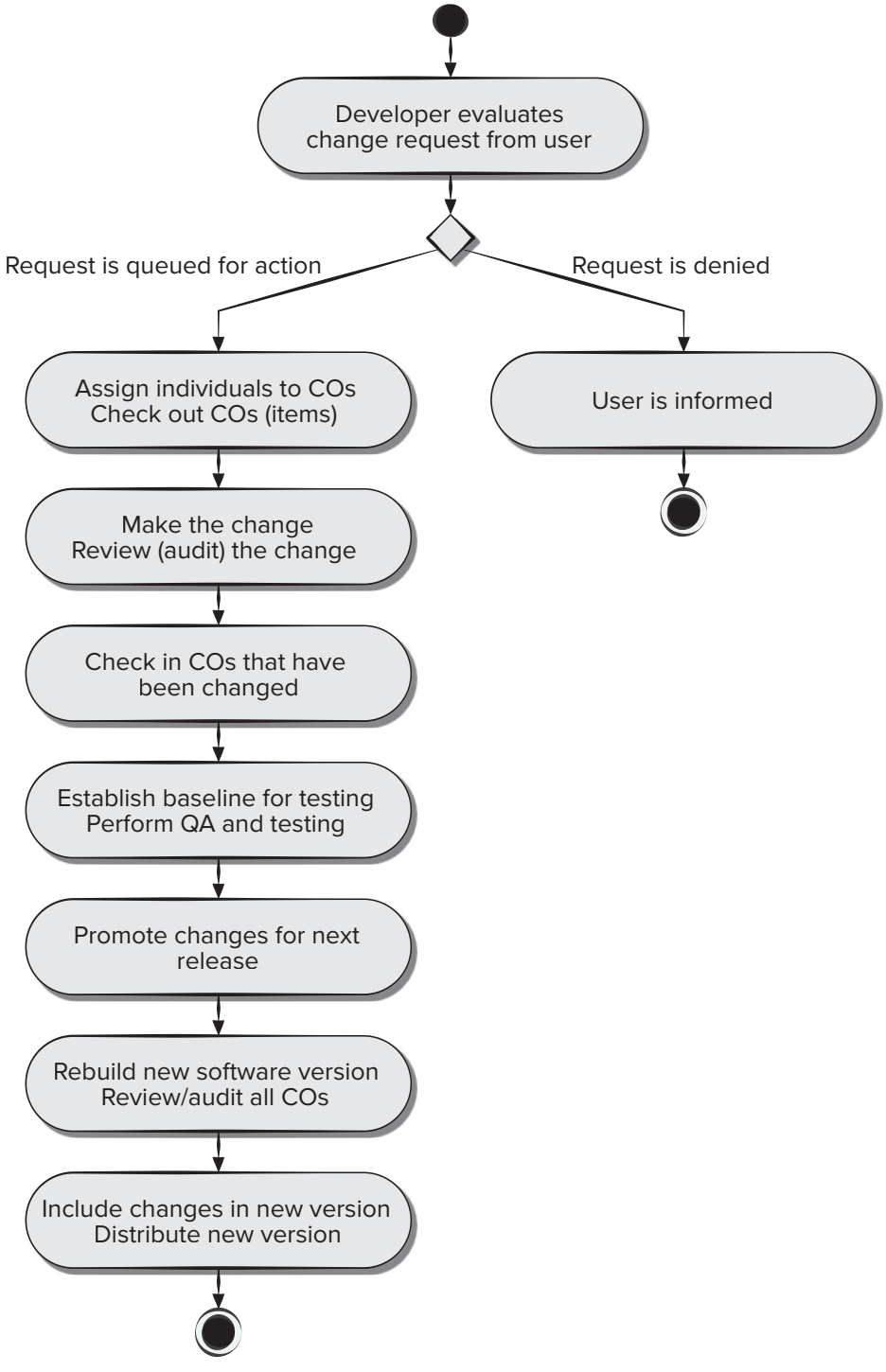
Referring to the figure, SCM tasks can be viewed as concentric layers. SCIs flow outward through these layers throughout their useful life, ultimately becoming part of the software configuration of one or more versions of an application or system. As an SCI moves through a layer, the actions implied by each SCM task may or may not be applicable. For example, when a new SCI is created, it must be identified. However, if no changes are requested for the SCI, the change control layer does not apply. The SCI is assigned to a specific version of the software (version control mechanisms come into play). A record of the SCI (its name, creation date, version designation, etc.) is maintained for configuration auditing purposes and reported to those with a need to know. In the sections that follow, we examine each of these SCM process layers in more detail.

22.5.1 Change Control

For a large software project, uncontrolled change rapidly leads to chaos. For such projects, change control combines human procedures and automated tools to provide a mechanism for the control of change. The change control process is illustrated schematically in Figure 22.6. A *change request* is submitted and evaluated to assess technical merit, potential side effects, overall impact on other configuration objects and system functions, and the projected cost of the change. The results of the evaluation are presented as a *change report,* which is used by a *change control authority* (CCA)—a person or group that makes a final decision on the status and priority of the change. An *engineering change order* (ECO) is generated for each approved change. The ECO describes the change to be made, the constraints that must be respected, and the criteria for review and audit.

FIGURE 22.6

The change control process



The object(s) to be changed can be placed in a directory that is controlled solely by the software engineer making the change. A version control system (see the CVS sidebar) updates the original file once the change has been made. As an alternative, the object(s) to be changed can be "checked out" of the project database (repository), the change is made, and appropriate SQA activities are applied. The object(s) is (are) then "checked in" to the database, and appropriate version control mechanisms (Section 22.3) are used to create the next version of the software.

These version control mechanisms, integrated within the change control process, implement two important elements of change management—access control and synchronization control. *Access control* governs which software engineers have the authority to access and modify a particular configuration object. *Synchronization control* helps to ensure that parallel changes, performed by two different people, don't overwrite one another.

You may feel uncomfortable with the level of bureaucracy implied by the change control process description shown in Figure 22.6. This feeling is not uncommon. Without proper safeguards, change control can retard progress and create unnecessary red tape. Most software developers who have change control mechanisms (unfortunately, many have none) have created a number of layers of control to help avoid the problems alluded to here.

Prior to an SCI becoming a baseline, only *informal change control* need be applied. The developer of the configuration object (SCI) in question may make whatever changes are justified by project and technical requirements (as long as changes do not affect broader system requirements that lie outside the developer's scope of work). Once the object has undergone technical review and has been approved, a baseline can be created.[7] Once an SCI becomes a baseline, *project level change control* is implemented. Now, to make a change, the developer must gain approval from the project manager (if the change is "local") or from the CCA if the change affects other SCIs. In some cases, the developer dispenses with the formal generation of change requests, change reports, and ECOs. However, assessment of each change is conducted and all changes are tracked and reviewed.

When the software product is released to customers, *formal change control* is instituted. The formal change control procedure has been outlined in Figure 22.6.

The change control authority plays an active role in the second and third layers of control. Depending on the size and character of a software project, the CCA may be composed of one person—the project manager—or a number of people (e.g., representatives from software, hardware, database engineering, support, marketing). The role of the CCA is to take a global view, that is, to assess the impact of change beyond the SCI in question. How will the change affect hardware? How will the change affect performance? How will the change modify customers' perception of the product? How will the change affect product quality and reliability? These and many other questions are addressed by the CCA.

7 A baseline can be created for other reasons as well. For example, when "daily builds" are created, all components checked in by a given time become the baseline for the next day's work.

SAFEHOME



SCM Issues

The scene: Doug Miller's office as the *SafeHome* software project begins.

The players: Doug Miller, manager of the *SafeHome* software engineering team, and Vinod Raman, Jamie Lazar, and other members of the product software engineering team.

The conversation:

Doug: I know it's early, but we've got to talk about change management.

Vinod (laughing): Hardly. Marketing called this morning with a few "second thoughts." Nothing major, but it's just the beginning.

Jamie: We've been pretty informal about change management on past projects.

Doug: I know, but this is bigger and more visible, and as I recall . . .

Vinod (nodding): We got killed by uncontrolled changes on the home lighting control project . . . remember the delays that . . .

Doug (frowning): A nightmare that I'd prefer not to relive.

Jamie: So what do we do?

Doug: As I see it, three things. First we have to develop—or borrow—a change control process.

Jamie: You mean how people request changes?

Vinod: Yeah, but also how we evaluate the change, decide when to do it (if that's what we decide), and how we keep records of what's affected by the change.

Doug: Second, we've got to get a really good SCM tool for change and version control.

Jamie: We can build a database for all of our work products.

Vinod: They're called SCIs in this context, and most good tools provide some support for that.

Doug: That's a good start, now we have to . . .

Jamie: Uh, Doug, you said there were three things . . .

Doug (smiling): Third—we've all got to commit to follow the change management process and use the tools—no matter what, okay?

22.5.2 Impact Management

A web of software work product interdependencies must be considered every time a change is made. Impact management encompasses the work required to properly understand these interdependencies and control their effects on other SCIs (and the people who are responsible for them).

Impact management is accomplished with three actions [Sou08]. First, an impact network identifies the members of a software team (and other stakeholders) who might effect or be affected by changes that are made to the software. A clear definition of the software architecture (Chapter 10) assists greatly in the creation of an impact network. Next, forward impact management assesses the impact of your own changes on the members of the impact network and then informs members of the impact of those changes. Finally, backward impact management examines changes that are made by other team members and their impact on your work and incorporates mechanisms to mitigate the impact.

22.5.3 Configuration Audit

Identification, version control, and change control help you to maintain order in what would otherwise be a chaotic and fluid situation. However, even the most successful control mechanisms track a change only until an ECO is generated. How can a software team ensure that the change has been properly implemented? The answer is twofold: (1) technical reviews and (2) the software configuration audit.

The technical review (Chapter 16) focuses on the technical correctness of the configuration object that has been modified. The reviewers assess the SCI to determine consistency with other SCIs, omissions, or potential side effects. A technical review should be conducted for all but the most trivial changes.

A *software configuration audit* complements the technical review by assessing a configuration object for characteristics that are generally not considered during review. The audit asks and answers the following questions:

1. Has the change specified in the ECO been made? Have any additional modifications been incorporated?
2. Has a technical review been conducted to assess technical correctness?
3. Has the software process been followed, and have software engineering standards been properly applied?
4. Has the change been "highlighted" in the SCI? Have the change date and change author been specified? Do the attributes of the configuration object reflect the change?
5. Have SCM procedures for noting the change, recording it, and reporting it been followed?
6. Have all related SCIs been properly updated?

In some cases, the audit questions are asked as part of a technical review. However, when SCM is a formal activity, the configuration audit is conducted separately by the quality assurance group. Such formal configuration audits also ensure that the correct SCIs (by version) have been incorporated into a specific build and that all documentation is up to date and consistent with the version that has been built.

22.5.4 Status Reporting

Configuration status reporting (sometimes called *status accounting*) is an SCM task that answers the following questions: (1) What happened? (2) Who did it? (3) When did it happen? (4) What else will be affected?

The flow of information for configuration status reporting (CSR) is illustrated in Figure 22.6. At the very least, develop a "need to know" list for every configuration object and keep it up to date. When a change is made, be sure that everyone on the list is notified. Each time an SCI is assigned new or updated identification, a CSR entry is made. Each time a change is approved by the CCA (i.e., an ECO is issued), a CSR entry is made. Each time a configuration audit is conducted, the results are reported as part of the CSR task. Output from CSR may be placed in an online database or website, so that software developers or support staff can access change information by keyword category. In addition, a CSR report is generated on a regular basis and is intended to keep management and practitioners apprised of important changes.

22.6 MOBILITY AND AGILE CHANGE MANAGEMENT

Earlier in this book, we discussed the special nature of WebApps and MobileApps and the specialized methods[8] that are required to build them. Game developers face similar challenges, as do all agile development teams. Among the many characteristics that differentiate these applications from traditional software is the ubiquitous nature of change.

Mobile developers and game developers often use an iterative, incremental process model that applies many principles derived from agile software development (Chapter 4). Using this approach, an engineering team often develops an increment in a very short time period using a customer-driven approach. Subsequent increments add additional content and functionality, and each is likely to implement changes that lead to enhanced content, better usability, improved aesthetics, better navigation, enhanced performance, and stronger security. Therefore, in the agile world of app and game development, change is viewed somewhat differently.

If you're a member of a software team that builds apps or games, you must embrace change. And yet, a typical agile team eschews all things that appear to be process-heavy, bureaucratic, and formal. Software configuration management is often viewed (albeit incorrectly) to have these characteristics. This seeming contradiction is remedied not by rejecting SCM principles, practices, and tools, but rather by molding them to meet the special needs of mobile projects.

22.6.1 e-Change Control

The work flow associated with change control for conventional software (Section 22.5.1) is generally too ponderous for WebApp and mobile software development. It is unlikely that the change request, change report, and engineering change order sequence can be achieved in an agile manner that is acceptable for many game and app development projects. How then do we manage a continuous stream of changes requested for content and functionality?

To implement effective change management within the "code and go" philosophy that continues to dominate much of game and mobile development, the conventional change control process can be modified. Each change should be categorized into one of four classes:

Class 1. A content or function change that corrects an error or enhances local content or functionality.

Class 2. A content or function change that has an impact on other content objects or functional components.

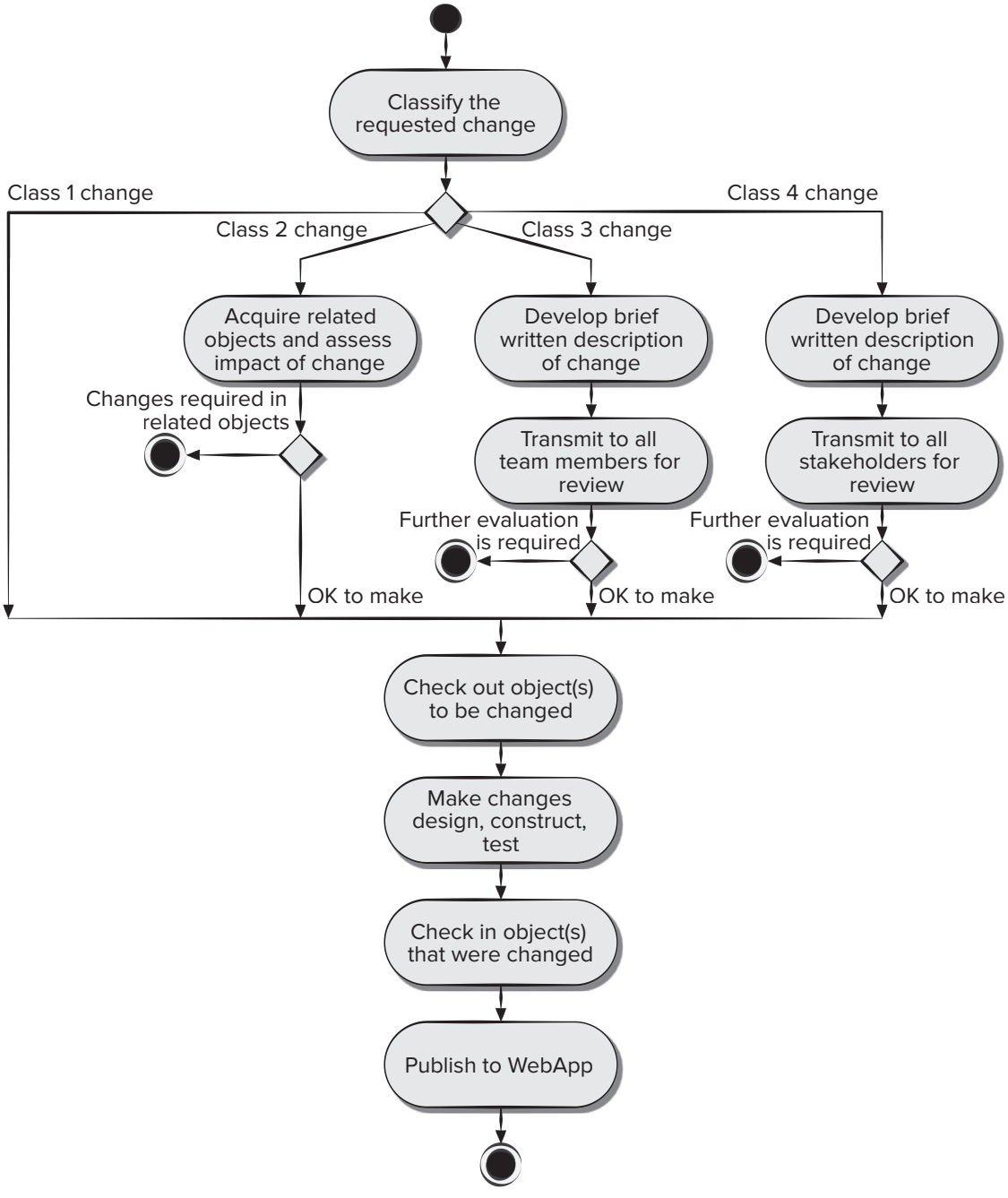
Class 3. A content or function change that has a broad impact across an app (e.g., major extension of functionality, significant enhancement or reduction in content, major required changes in navigation).

Class 4. A major design change (e.g., a change in interface design or navigation approach) that will be immediately noticeable to one or more categories of user.

Once the requested change has been categorized, it can be processed according to the algorithm shown in Figure 22.7 for WebApps but is equally applicable for apps and games.

8 See [Pre08] for a comprehensive discussion of Web engineering methods.

FIGURE 22.7 Managing changes for WebApps



Referring to the figure, class 1 and 2 changes are treated informally and are handled in an agile manner. For a class 1 change, you would evaluate the impact of the change, but no external review or documentation is required. As the change is made, standard check-in and check-out procedures are enforced by configuration repository tools. For class 2 changes, you should review the impact of the change on related objects (or ask other developers responsible for those objects to do so). If the change can be made without requiring significant changes to other objects, modification occurs without additional review or documentation. If substantive changes are required, further evaluation and planning are necessary.

Class 3 and 4 changes are also treated in an agile manner, but some descriptive documentation and more formal review procedures are required. A *change description*—describing the change and providing a brief assessment of the impact of the change—is developed for class 3 changes. The description is distributed to all members of the team who review it to better assess its impact. A change description is also developed for class 4 changes, but in this case all stakeholders conduct the review.

22.6.2 Content Management

Content management is related to configuration management in the sense that a content management system (CMS) establishes a process (supported by appropriate tools) that acquires existing content (from a broad array of app and/or game configuration objects), structures it in a way that enables it to be presented to an end user, and then provides it to the client-side environment for display.

The most common use of a content management system occurs when a dynamic application is built. Apps and games create screen displays "on the fly." That is, the user typically performs an action that the software responds to by changing the information displayed on the screen. The user action may cause the app to query a server-side database; it then formats the information accordingly and presents it to the user.

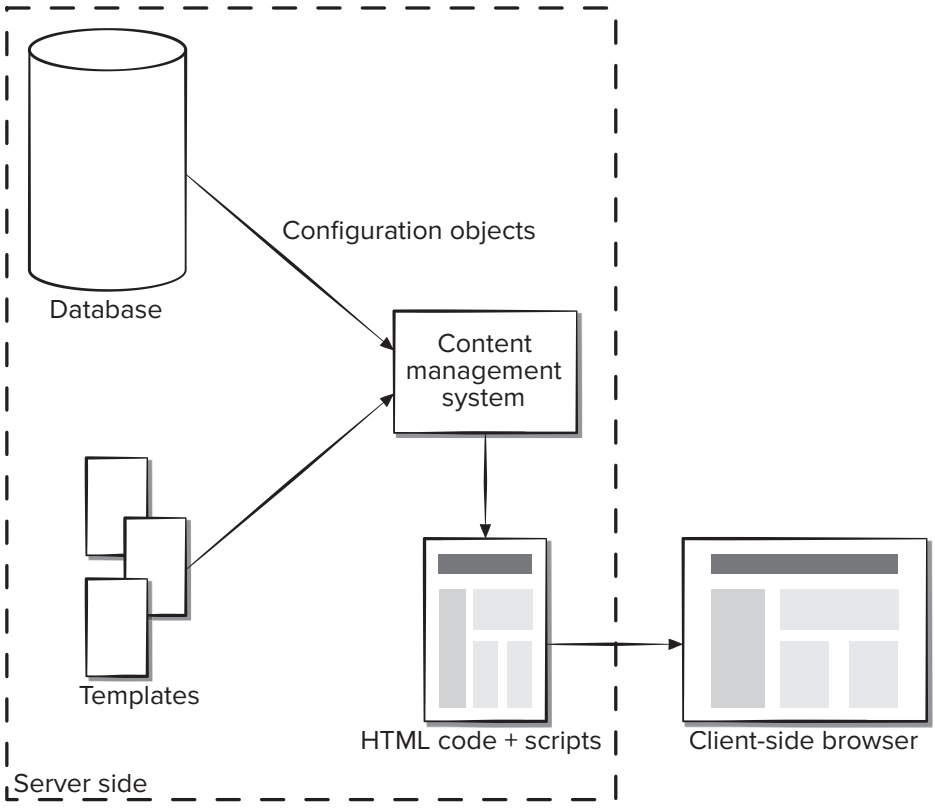
For example, a music store (e.g., Apple iTunes) provides hundreds of thousands of tracks for sale. When a user requests a music track, a database is queried and a variety of information about the artist, the CD (e.g., its cover image or graphics), the musical content, and sample audio are all downloaded and configured into a standard content template. The resultant page is built on the server side and passed to the client side for examination by the end user. A generic representation for WebApps is shown in Figure 22.8.

22.6.3 Integration and Publishing

Content management systems are useful for composing Web services to create context-aware MobileApps and updating game-level scenes at run time, as well as building dynamic Web pages. In the most general sense, a CMS "configures" content for the end user by invoking three integrated subsystems: a collection subsystem, a management subsystem, and a publishing subsystem [Boi04].

The Collection Subsystem. Content is derived from data and information that must be created or acquired by a content developer. The *collection subsystem* encompasses all actions required to create and/or acquire content, and the technical functions that are necessary to (1) convert content into a form that can be represented by a mark-up

FIGURE 22.8
Content management system



language (e.g., HTML, XML), and (2) organize content into screens that can be displayed efficiently on the client side.

Content creation and acquisition (often called *authoring* or *level design* for games) commonly occurs in parallel with other development activities and is often conducted by nontechnical content developers. This activity combines elements of creativity and research and is supported by tools that enable the content author to characterize content in a manner that can be standardized for use within the app or game.

Once content exists, it must be converted to conform to the requirements of a CMS. This implies stripping raw content of any unnecessary information (e.g., redundant graphical representations), formatting the content to conform to the requirements of the CMS, and mapping the results into an information structure that will enable it to be managed and published.

The Management Subsystem. Once content exists, it must be stored in a repository, cataloged for subsequent acquisition and use, and labeled to define (1) current status (e.g., is the content object complete or in development?), (2) the appropriate version of the content object, and (3) related content objects. Configuration management is

performed within this subsystem. Therefore, the *management subsystem* implements a repository that encompasses the following elements:

- **Content database.** The information structure that has been established to store all content objects.
- **Database capabilities.** Functions that enable the CMS to search for specific content objects (or categories of objects), store and retrieve objects, and manage the file structure that has been established for the content.
- **Configuration management functions.** The functional elements and associated workflow that support content object identification, version control, change management, change auditing, and reporting.

In addition to these elements, the management subsystem implements an administration function that encompasses the metadata and rules that control the overall structure of the content and the manner in which it is supported.

The Publishing Subsystem. Content must be extracted from the repository, converted to a form that is amenable to publication and formatted so that it can be transmitted to client-side screen displays. The publishing subsystem accomplishes these tasks using a series of templates. Each *template* is a function that builds a publication using one of three different components [Boi04]:

- **Static elements.** Text, graphics, media, and scripts that require no further processing are transmitted directly to the client side.
- **Publication services.** Function calls to specific retrieval and formatting services that personalize content (using predefined rules), perform data conversion, and build appropriate navigation links.
- **External services.** Access to external corporate information infrastructure such as enterprise data or "backroom" applications.

A content management system that encompasses each of these subsystems is applicable for major Web and mobile projects. However, the basic philosophy and functionality associated with a CMS are applicable to all dynamic applications.

22.6.4 Version Control

As apps and games evolve through a series of increments, a number of different versions may exist at the same time. One version (the current operational app) is available via the Internet for end users; another version (the next app increment) may be in the final stages of testing prior to deployment; a third version is in development and represents a major update in content, interface aesthetics, and functionality. Configuration objects must be clearly defined so that each can be associated with the appropriate version. Without some type of control, developers and content creators may end up overwriting each other's changes.

It's likely that you've experienced a similar situation. To avoid it, a version control process is required.

1. **A central repository for the app or game project should be established.** The repository will hold current versions of all configuration objects (content, functional components, and others).

2. **Each developer creates his own working folder.** The folder contains those objects that are being created or changed at any given time.
3. **The clocks on all developer workstations should be synchronized.** This is done to avoid overwriting conflicts when two developers make updates that are very close to one another in time.
4. **As new configuration objects are developed or existing objects are changed, they are imported into the central repository.** The version control tool will manage all check-in and check-out functions from the working folders of each developer. The tool should also provide automatic e-mail updates to all interested parties when changes to the repository are made.
5. **As objects are imported or exported from the repository, an automatic, time-stamped log message is made.** This provides useful information for auditing and can become part of an effective reporting scheme.

The version control tool maintains different versions of the app and can revert to an older version if required.

22.6.5 Auditing and Reporting

In the interest of agility, the auditing and reporting functions are deemphasized during the development of games or apps.[9] However, they are not eliminated altogether. All objects that are checked into or out of the repository are recorded in a log that can be reviewed at any point in time. A complete log report can be created so that all members of the team have a chronology of changes over a defined period of time. In addition, an automated e-mail notification (addressed to those developers and stakeholders who have interest) can be sent every time an object is checked in or out of the repository.

22.7 SUMMARY

Software configuration management is an umbrella activity that is applied throughout the software process. SCM identifies, controls, audits, and reports modifications that invariably occur while software is being developed and after it has been released to a customer. All work products created as part of software engineering become part of a software configuration. The configuration is organized in a manner that enables orderly control of change.

The software configuration is composed of a set of interrelated objects, also called software configuration items, that are produced as a result of some software engineering activity. In addition to software engineering work products, the development environment that is used to create software can also be placed under configuration control. All SCIs are stored within a repository that implements a set of mechanisms and data

9 This is beginning to change. There is an increasing emphasis on SCM as one element of application security [Fug14]. By providing a mechanism for tracking and reporting every change made to every application object, a change management tool can provide valuable protection against malicious changes.

structures to ensure data integrity, provide integration support for other software tools, support information sharing among all members of the software team, and implement functions in support of version and change control.

Once a configuration object has been developed and reviewed, it becomes a baseline. Changes to a baselined object result in the creation of a new version of that object. The evolution of a program can be tracked by examining the revision history of all configuration objects. Version control is the set of procedures and tools for managing the use of these objects.

Change control is a procedural activity that ensures quality and consistency as changes are made to a configuration object. The change control process begins with a change request, leads to a decision to make or reject the request for change, and culminates with a controlled update of the SCI that is to be changed.

The configuration audit is an SQA activity that helps to ensure that quality is maintained as changes are made. Status reporting provides information about each change to those with a need to know.

Configuration management for apps or games is similar in most respects to SCM for conventional software. However, each of the core SCM tasks should be streamlined to make it as lean as possible, and special provisions for content management must be implemented.

PROBLEMS AND POINTS TO PONDER

22.1. Why is the first law of system engineering true? Provide specific examples for each of the four fundamental reasons for change.

22.2. What are the four elements that exist when an effective SCM system is implemented? Discuss each briefly.

22.3. Assume that you're the manager of a small project. What baselines would you define for the project, and how would you control them?

22.4. Design a project database (repository) system that would enable a software engineer to store, cross-reference, trace, update, change, and so forth all important software configuration items. How would the database handle different versions of the same program? Would source code be handled differently than documentation? How will two developers be precluded from making different changes to the same SCI at the same time?

22.5. Research an existing SCM tool, and describe how it implements control for versions, variants, and configuration objects in general.

22.6. Research an existing SCM tool, and describe how it implements the mechanics of version control. Alternatively, read two or three papers on SCM and describe the different data structures and referencing mechanisms that are used for version control.

22.7. Develop a checklist for use during configuration audits.

22.8. What is the difference between an SCM audit and a technical review? Can their function be folded into one review? What are the pros and cons?

22.9. Briefly describe the differences between SCM for conventional software and SCM for WebApps or MobileApps.

22.10. Describe the value of continuous integration tools to agile software developers.

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

23 SOFTWARE METRICS AND ANALYTICS

A key element of any engineering process is measurement. Measurement can be applied to the software process with the intent of improving it on a continuous basis. Measurement can be used throughout a software project to assist in estimation, quality control, productivity assessment, and project control. You can use measures to better understand the attributes of the models that you create and to assess the quality of the engineered products or systems that you build.

KEY CONCEPTS

data science 461
defect removal efficiency (DRE) 482
goal 485
indicator 462
measure 461
measurement 462
metrics 461
  arguments for 481
  attributes of 462
  design metrics 466
  establishing a program 485
function-oriented 481
LOC-based metrics 481
private and public 480
process 476
productivity 481
project 476
size-oriented 480
software quality 482
source code 473
testing 474
software analytics 462



QUICK LOOK

What is it? Software process and project metrics are quantitative measures that enable you to gain insight into the efficacy of the software process and the projects that are conducted using the process as a framework. Product metrics help software engineers gain insight into the design and construction of the software they build.

Who does it? Software metrics are analyzed and assessed by software managers. Software engineers use product metrics to help them build higher-quality software.

Why is it important? If you don't measure, judgment can only be based on subjective evaluation. You need objective criteria to help guide the design of data, architecture, interfaces, and components. If you measure, trends (either good or bad) can be spotted, better estimates can be made, and true improvement can be accomplished over time.

What are the steps? Derive the process, project, and product measures and metrics that you intend to use. Collect the metrics and then analyze them against historical data. Use the analysis results to gain insight into the process, project, and product.

What is the work product? A set of software metrics that provides insight into the process and understanding of the project.

How do I ensure that I've done it right? Define only a few metrics, and then use them to gain insight into the quality of a software process, project, and product. Apply a consistent, yet simple measurement scheme that is never to be used to assess, reward, or punish individual performance.

Measurement can be used by software engineers to help assess the quality of work products and to assist in tactical decision making as a project proceeds. But unlike other engineering disciplines, software engineering is not grounded in the basic quantitative laws of physics. Direct measures, such as voltage, mass, velocity, or temperature, are uncommon in the software world. Because software measures and metrics are often indirect, they are open to debate.

Within the context of the software process and the projects that are conducted using the process, a software team is concerned primarily with productivity and quality metrics—measures of software development "output" as a function of effort and time applied and measures of the "fitness for use" of the work products that are produced. For planning and estimating purposes, our interest is historical. What was software development productivity on past projects? What was the quality of the software that was produced? How can past productivity and quality data be extrapolated to the present? How can it help us plan and estimate more accurately?

Measurement is a management and technical tool. If conducted properly, it provides you with insight. And as a result, it assists the project manager and the software team in making decisions that will lead to a successful project.

In this chapter, we present measures that can be used to assess the quality of the product as it is being engineered. We also present measures that can be used to help manage software projects. These measures provide you with a real-time indication of the effectiveness of your software processes (analysis, design, testing) and the overall quality of the software as it is being built.

23.1 SOFTWARE MEASUREMENT

Data science[1] is concerned with measurement, machine learning, and prediction of future events based on these measures. Measurement assigns numbers or symbols to attributes of entities in the real word. To accomplish this, a measurement model encompassing a consistent set of rules is required. Although the theory of measurement (e.g., [Kyb84]) and its application to computer software (e.g., [Zus97]) are topics that are beyond the scope of this book, it is worthwhile to establish a fundamental framework and a set of basic principles that guide the definition of metrics for software development.

23.1.1 Measures, Metrics, and Indicators

Although the terms *measure, measurement,* and *metrics* are often used interchangeably, it is important to note the subtle differences between them. When a single data point has been collected (e.g., the number of errors uncovered within a single software component), a *measure* has been established. *Measurement* occurs as the result of the collection of one or more data points (e.g., a number of component reviews and unit tests are investigated to collect measures of the number of errors for each). A *software*

1 Appendix 2 in the book contains an introduction to data science geared toward software engineers.

metric relates the individual measures in some way (e.g., the average number of errors found per review or the average number of errors found per unit test).

A software engineer collects measures and develops metrics so that indicators will be obtained. An *indicator* is a metric or combination of metrics that provides insight into the software process, a software project, or the product itself.

23.1.2 Attributes of Effective Software Metrics

Hundreds of metrics have been proposed for computer software, but not all provide practical support to the software engineer. Some demand measurement that is too complex; others are so esoteric that few real-world professionals have any hope of understanding them, and others violate the basic intuitive notions of what high-quality software really is. Experience indicates that a metric will be used only if it is intuitive and easy to compute. If dozens of "counts" have to be made, and complex computations are required, it is unlikely that the metric will be widely adopted.

Ejiogu [Eji91] defines a set of attributes that should be encompassed by effective software metrics. It should be relatively easy to learn how to derive the metric, and its computation should not demand inordinate effort or time. The metric should satisfy the engineer's intuitive notions about the product attribute under consideration (e.g., a metric that measures module cohesion should increase in value as the level of cohesion increases). The metric should always yield results that are unambiguous. The mathematical computation of the metric should use measures that do not lead to bizarre combinations of units. For example, multiplying people on the project teams by programming language variables in the program results in a suspicious mix of units that are not intuitively persuasive. Metrics should be based on the requirements model, the design model, or the structure of the program itself. They should not be dependent on the vagaries of programming language syntax or semantics. Finally, the metric should provide you with information that can lead to a higher-quality end product.

23.2 Software Analytics

There is some confusion about the differences between software metrics and software analytics. Software metrics are used to gauge the quality or performance of a product or process. *Key performance indicators* (KPIs) are metrics that are used to track performance and trigger remedial actions when their values fall in a predetermined range. But how do you know that metrics are meaningful in the first place?

Software analytics is the systematic computational analysis of software engineering data or statistics to provide managers and software engineers with meaningful insights and empower their teams to make better decisions [Bus12]. It is important that the insights provide timely, actionable advice to developers. For example, knowing the number of defects in a software product today is not as important as knowing the number of defects is 5 percent higher than last month. Analytics can help developers predict the number of defects to expect, where to test for them, and how much time it will take to fix them. This allows managers and developers to create incremental schedules that use these predictions to determine expected completion times. The use

of automated tools capable of processing large, dynamic data sets of engineering metrics and measures [Men13] is required to provide real-time insight into large project and product data sets.

Buse and Zimmermann [Bus12] suggest that analytics can help developers make decisions regarding:

- **Targeted testing.** To help focus regression testing and integration testing resources
- **Targeted refactoring.** To help make strategic decisions on how to avoid large technical debt costs
- **Release planning.** To help ensure that market needs as well as technical features in software product are taken into account
- **Understanding customers.** To help developers get actionable information on product use by customers in the field during product engineering
- **Judging stability.** To help managers and developers monitor the state of the evolving prototype and anticipate future maintenance needs
- **Targeting inspection.** To help teams determine the value of individual inspection activities, their frequency, and their scope

The statistical techniques (data mining, machine learning, statistical modeling) required to do software analytic work is beyond the scope of this book. Some of these techniques are discussed briefly in Appendix 2. We will focus on the use of software metrics in the remainder of this chapter.

23.3 PRODUCT METRICS

Over the past four decades, many researchers have attempted to develop a single metric that provides a comprehensive measure of software complexity. Fenton [Fen94] characterizes this research as a search for "the impossible holy grail." Although dozens of complexity measures have been proposed [Zus90], each takes a somewhat different view of what complexity is and what attributes of a system lead to complexity. By analogy, consider a metric for evaluating an attractive car. Some observers might emphasize body design; others might consider mechanical characteristics; still others might tout cost, or performance, or the use of alternative fuels, or the ability to recycle when the car is junked. Because any one of these characteristics may be at odds with others, it is difficult to derive a single value for "attractiveness." The same problem occurs with computer software.

Yet there is a need to measure and control software complexity. And if a single value of this quality metric is difficult to derive, it should be possible to develop measures of different internal program attributes (e.g., effective modularity, functional independence, and other attributes discussed in Chapter 9). These measures and the metrics derived from them can be used as independent indicators of the quality of requirements and design models. But here again, problems arise. Fenton [Fen94] notes this when he states: "The danger of attempting to find measures which characterize so many different attributes is that inevitably the measures have to satisfy conflicting

aims. This is counter to the representational theory of measurement." Although Fenton's statement is correct, many people argue that product measurement conducted during the early stages of the software process provides software engineers with a consistent and objective mechanism for assessing quality.[2]

SAFEHOME



Debating Product Metrics

The scene: Vinod's cubicle.

The players: Vinod, Jamie, and Ed, members of the *SafeHome* software engineering team who are continuing work of component-level design and test-case design.

The conversation:

Vinod: Doug [Doug Miller, software engineering manager] told me that we should all use product metrics, but he was kind of vague. He also said that he wouldn't push the matter . . . that using them was up to us.

Jamie: That's good, 'cause there's no way I have time to start measuring stuff. We're fighting to maintain the schedule as it is.

Ed: I agree with Jamie. We're up against it, here . . . no time.

Vinod: Yeah, I know, but there's probably some merit to using them.

Jamie: I'm not arguing that, Vinod, it's a time thing . . . and I for one don't have any to spare.

Vinod: But what if measuring saves you time?

Ed: Wrong, it takes time and like Jamie said . . .

Vinod: No, wait . . . what if it saves us is time?

Jamie: How?

Vinod: Rework . . . that's how. If a measure we use helps us to avoid one major or even moderate problem, and that saves us from having to rework a part of the system, we save time. No?

Ed: It's possible, I suppose, but can you guarantee that some product metric will help us find a problem?

Vinod: Can you guarantee that it won't?

Jamie: So what are you proposing?

Vinod: I think we should select a few design metrics, probably class-oriented, and use them as part of our review process for every component we develop.

Ed: I'm not real familiar with class-oriented metrics.

Vinod: I'll spend some time checking them out and make a recommendation . . . okay with you guys?

(Ed and Jamie nod without much enthusiasm.)

23.3.1 Metrics for the Requirements Model

Technical work in software engineering begins with the creation of the requirements model. It is at this stage that requirements are derived and a foundation for design is established. Therefore, product metrics that provide insight into the quality of the analysis model are desirable.

Although relatively few analysis and specification metrics have appeared in the literature, it is possible to adapt metrics (e.g., use case points or function points)

2 Although criticism of specific metrics is common in the literature, many critiques focus on esoteric issues and miss the primary objective of metrics in the real world: to help software engineers establish a systematic and objective way to gain insight into their work and to improve product quality as a result.

that are often used for project estimation (Section 25.6) and apply them in this context. These estimation metrics examine the requirements model with the intent of predicting the "size" of the resultant system. Size is sometimes (but not always) an indicator of design complexity and is almost always an indicator of increased coding, integration, and testing effort. By measuring characteristics of the requirements model, it is possible to gain quantitative insight into its specificity and completeness.

Conventional Software. Davis and his colleagues [Dav93] propose a list of characteristics that can be used to assess the quality of the requirements model and the corresponding requirements specification: *specificity* (lack of ambiguity), *completeness, correctness, understandability, verifiability, internal and external consistency, achievability, concision, traceability, modifiability, precision,* and *reusability.* In addition, the authors note that high-quality specifications are electronically stored; executable or at least interpretable; annotated by relative importance; and stable, versioned, organized, cross-referenced, and specified at the right level of detail.

Although many of these characteristics appear to be qualitative in nature, each can be represented using one or more metrics [Dav93]. For example, we assume that there are n_r requirements in a specification, such that

$$n_r = n_f + n_{\text{nf}}$$

where n_f is the number of functional requirements and n_{nf} is the number of nonfunctional (e.g., performance) requirements.

To determine the *specificity* (lack of ambiguity) of requirements, Davis and colleagues suggest a metric that is based on the consistency of the reviewers' interpretation of each requirement:

$$Q_1 = \frac{n_{ui}}{n_r}$$

where n_{ui} is the number of requirements for which all reviewers had identical interpretations. The closer the value of Q to 1, the lower is the ambiguity of the specification. Other characteristics are computed in a similar manner.

Mobile Software. The objective of all mobile projects is to deliver a combination of content and functionality to the end user. Measures and metrics used for traditional software engineering projects are difficult to translate directly to MobileApps. Yet, it is possible to develop measures that can be determined during the requirements gathering activities that can serve as the basis for creating MobileApp metrics. Among the measures that can be collected are the following:

Number of static screen displays. These pages represent low relative complexity and generally require less effort to construct than dynamic pages. This measure provides an indication of the overall size of the application and the effort required to develop it.

Number of dynamic screen displays. These pages represent higher relative complexity and require more effort to construct than static pages. This measure provides an indication of the overall size of the application and the effort required to develop it.

Number of persistent data objects. As the number of persistent data objects (e.g., a database or data file) grows, the complexity of the MobileApp also grows and the effort to implement it increases proportionally.

Number of external systems interfaced. As the requirement for interfacing grows, system complexity and development effort also increase.

Number of static content objects. These objects represent low relative complexity and generally require less effort to construct than dynamic pages.

Number of dynamic content objects. These objects represent higher relative complexity and require more effort to construct than static pages.

Number of executable functions. As the number of executable functions (e.g., a script or applet) increases, modeling and construction effort also increase.

For example with these measures, you can define a metric that reflects the degree of end-user customization that is required for the MobileApp and correlate it to the effort expended on the project and/or the errors uncovered as reviews and testing are conducted. To accomplish this, you define

N_{sp} = number of static screen displays
N_{dp} = number of dynamic screen displays

Then,

$$\text{Customization index, } C = \frac{N_{dp}}{N_{dp} + N_{sp}}$$

The value of C ranges from 0 to 1. As C grows larger, the level of app customization becomes a significant technical issue.

Similar metrics can be computed and correlated with project measures such as effort expended, errors and defects uncovered, and models or documentation pages produced. If the values of these metrics are stored in a database with project measures (after a number of projects have been completed), the relationships between the app requirement measures and project measures will provide indicators that can aid in project assessment tasks.

23.3.2 Design Metrics for Conventional Software

It is inconceivable that the design of a new aircraft, a new computer chip, or a new office building would be conducted without defining design measures, determining metrics for various aspects of design quality, and using them as indicators to guide the manner in which the design evolves. And yet, the design of complex software-based systems often proceeds with virtually no measurement. The irony of this is that design metrics for software are available, but the vast majority of software engineers continue to be unaware of their existence.

Architectural design metrics focus on characteristics of the program architecture (Chapter 10) with an emphasis on the architectural structure and the effectiveness of modules or components within the architecture. These metrics are "black box" in the sense that they do not require any knowledge of the inner workings of a particular software component. Metrics can provide insight into structural data and system complexity associated with architectural design.

Card and Glass [Car90] define three software design complexity measures: structural complexity, data complexity, and system complexity.

For hierarchical architectures (e.g., call-and-return architectures), *structural complexity* of a module i is defined in the following manner:

$$S(i) = f_{\text{out}}^2(i)$$

where $f_{\text{out}}(i)$ is the fan-out[3] of module *i.*

Data complexity provides an indication of the complexity in the internal interface for a module i and is defined as

$$D(i) = \frac{v(i)}{f_{\text{out}}(i) + 1}$$

where $v(i)$ is the number of input and output variables that are passed to and from module *i.*

Finally, *system complexity* is defined as the sum of structural and data complexity, specified as

$$C(i) = S(i) + D(i)$$

As each of these complexity values increases, the overall architectural complexity of the system also increases. This leads to a greater likelihood that integration and testing effort will also increase.

Fenton [Fen91] suggests a number of simple morphology (i.e., shape) metrics that enable different program architectures to be compared using a set of straightforward dimensions. Referring to the call-and-return architecture in Figure 23.1, the following metrics can be defined:

$$\text{Size} = n + a$$

where n is the number of nodes and a is the number of arcs. For the architecture shown in Figure 23.1,

$$\text{Size} = 17 + 18 = 35$$

Depth = longest path from the root (top) node to a leaf node. For the architecture shown in Figure 23.1, depth = 4.

Width = maximum number of nodes at any one level of the architecture. For the architecture shown in Figure 23.1, width = 6.

The arc-to-node ratio, $r = a/n$, measures the connectivity density of the architecture and may provide a simple indication of the coupling of the architecture. For the architecture shown in Figure 23.1, $r = 18/17 = 1.06$.

The U.S. Air Force Systems Command [USA87] has developed a number of software quality indicators that are based on measurable design characteristics of a computer program. Using concepts similar to those proposed in IEEE Std. 982.1-2005 [IEE05], the Air Force uses information obtained from data and architectural design

3 *Fan-out* is defined as the number of modules immediately subordinate to module *I,* that is, the number of modules that are directly invoked by module *i.*